

INTELIGENCIA ARTIFICIAL PARA LA GESTIÓN DE INVERSIONES FINANCIERAS



TRABAJO FIN DE GRADO
CURSO 2025-2026

AUTORES:

VASIL GEORGIEV KYUCHUKOV

DAVID RAMOS DE LUCAS

FRANCISCO JOSÉ SÁNCHEZ DE LEÓN ACEVEDO

SERGIO VINAGRE MARTÍNEZ

DIRECTOR

IVÁN GARCÍA-MAGARIÑO GARCÍA

GRADO EN INGENIERÍA INFORMÁTICA
FACULTAD DE INFORMÁTICA
UNIVERSIDAD COMPLUTENSE DE MADRID

JUNIO 2026

DEDICATORIA

Dedicado a todos los profesores que nos han aportado los conocimientos y motivación para la realización de este TFG, así como a la universidad por dotarnos de los medios necesarios para ello.

AGRADECIMIENTOS

En primer lugar, queremos agradecer a nuestros familiares y amigos que han aguantado nuestros altibajos en periodos de tanto estrés y trabajo, sin ellos este proyecto no hubiera salido adelante.

Por otro lado, también queremos agradecerle su esfuerzo, comprensión y ayuda a Iván García-Magariño García, nuestro tutor del TFG, que ha estado siempre disponible y abierto a guiarnos en el proceso de desarrollo del trabajo. Sin su ayuda inestimable, el alcance de este proyecto no sería nada comparable con el alcanzado en su versión final.

RESUMEN

Inteligencia Artificial para la Gestión de Inversiones Financieras

La creciente accesibilidad a los mercados financieros no ha venido acompañada de una democratización equivalente del conocimiento financiero, lo que genera una brecha de riesgo significativa para el inversor minorista no profesional. Horizon es una plataforma web orientada a la predicción de tendencias bursátiles dirigida precisamente a este perfil de usuario, aportando módulos de formación en finanzas que acompañan al sistema de predicción de tendencias. El sistema integra un motor predictivo basado en XGBoost, entrenado por activo y validado mediante *walk-forward*, con un módulo de Inteligencia Artificial Explicable que emplea valores SHAP justificando y ponderando los elementos que conforman cada predicción. Se complementa con un asistente conversacional basado en un Modelo de Lenguaje de Gran Escala que actúa como tutor financiero contextualizado. Este enfoque permite a la plataforma aportar valor tanto financiero como educativo a particulares que no disponen de información perfecta de los mercados financieros ni de los conceptos que los forman.

Palabras clave

Inteligencia Artificial, Aprendizaje Automático, Predicción Bursátil, XGBoost, Inteligencia Artificial Explicable, SHAP, Modelos de Lenguaje, FinTech, Educación Financiera

ABSTRACT

HORIZON - Artificial Intelligence for the Management of Financial Investments

The growing accessibility to financial markets has not been accompanied by an equivalent democratization of financial knowledge, creating a significant risk gap for the non-professional retail investor. Horizon is a web platform oriented towards stock market trend prediction, aimed precisely at this user profile, providing financial education modules that complement the trend prediction system. The system integrates a predictive engine based on XGBoost, trained per asset and validated through walk-forward methodology, alongside an Explainable Artificial Intelligence module that employs SHAP values to justify and weight the elements that comprise each prediction. It is further complemented by a conversational assistant based on a Large Language Model that acts as a contextualized financial tutor. This approach enables the platform to deliver both financial and educational value to individuals who lack perfect information about financial markets and the concepts that underpin them.

Keywords

Artificial Intelligence, Machine Learning, Stock Market Forecasting, XGBoost, Explainable Artificial Intelligence, SHAP, Large Language Models, FinTech, Financial Education

ÍNDICE DE CONTENIDOS

Dedicatoria	2
Agradecimientos	3
Resumen	4
Abstract.....	5
Índice de Figuras	10
Índice de Tablas	14
Capítulo 1 - Introducción	16
1.1 Motivación	18
1.2 Objetivos.....	18
1.3 Alcance y Limitaciones	20
1.4 Plan de Trabajo	21
Capítulo 2 - Estado del Arte	25
2.1 Evolución de las Plataformas de Inversión y Robo-Advisors	25
2.2 Predicción de Series Temporales Mediante Deep Learning.....	26
2.3 Modelos de Lenguaje (LLM) en el Ecosistema FinTech	27
2.4 Análisis de Alternativas y Justificación de la Propuesta.	28
Capítulo 3 - Especificación de Requisitos Software	31
3.1 Visión General del Sistema.....	31
3.2 Requisitos Funcionales	32
3.3 Diagrama de Casos de Uso	34
3.4 Requisitos No Funcionales	58
3.5 Restricciones y Dependencias	58
3.6 Referencia a Apéndices	59

Capítulo 4 - Diseño de Interfaz de Usuario	60
4.1 Metodología de Diseño	60
4.2 Investigación y Entrevistas Exploratorias	60
4.3 Modelado de Persona.....	61
4.3.1 Arquetipo de persona primaria.....	61
4.4 Wireframe de Diseños	63
4.5 Diseños de Alta Fidelidad.....	69
Capítulo 5 - Tecnologías y Herramientas	74
5.1 Lenguajes de Programación	74
5.2 Módulo de Inteligencia Artificial y Procesamiento de Datos	75
5.3 Tecnología del Servidor (Backend)	77
5.4 Base de Datos, Autenticación y Almacenamiento	78
5.5 Entorno de Desarrollo y Despliegue	79
Capítulo 6 - Desarrollo/Implementación	81
6.1 Estructura y Arquitectura del Proyecto	81
6.1.1 Organización de Directorios	81
6.1.2 Patrón de Diseño Empleado.....	84
6.2 Implementación del Frontend.....	86
6.2.1 Arquitectura y Organización del Frontend.....	86
6.2.2 Maquetación Semántica y Diseño Adaptativo	87
6.2.3 Sistema de Estilos y Personalización Dinámica	88
6.2.4 Lógica de Interfaz y Manejo de Eventos (Vanilla JS)	89
6.2.5 Comunicación con el Backend y Seguridad	89
6.2.6 Vistas Principales de Usuario	90

6.3 Implementación del Backend	100
6.3.1 Servidor FastAPI y Configuración	100
6.3.2 Endpoints	102
6.3.3 Servicios	104
6.3.4 DAOs	105
6.4 Implementación BBDD	108
6.4.1 Estructura de Tablas Principales	108
6.4.2 Operaciones en Supabase	113
6.4.3 Restricciones de Integridad	114
6.5 Funcionalidades Principales	115
6.5.1 Gestión de Usuarios	115
6.5.2 Gestión de Activos	127
6.5.3 Gestión de Portfolio	134
6.5.4 AcademIA	142
6.5.5 Dashboard General	146
6.6 Pipeline ETL y Módulo de Inteligencia Artificial	150
6.6.1 Arquitectura General del Pipeline de Datos	150
6.6.2 Etapa de Extracción de Datos	151
6.6.3 Etapa de Transformación e Ingeniería de Variables	152
6.6.4 Etiquetado y Validación Temporal	153
6.6.5 Etapa de Carga y Persistencia	153
6.6.6 Modelo Predictivo: eXtreme Gradient Boosting (XGBoost)	155
6.6.7 Sistema de Etiquetado y Calibración de Umbrales	157
6.6.8 Metodología de Validación y Resultados	157

6.6.9 Inteligencia Artificial Explicable (XAI)	158
6.6.10 Log de Predicciones y Backfill	160
6.6.11 Limitaciones del Modelo y Líneas Futuras.....	162
6.7 Testing pre-commit.....	163
6.8 Gestión de Dependencias del Proyecto	164
6.8.1 Organización del Archivo requirements.txt	164
6.8.2 Reproducibilidad del Ambiente.....	165
Capítulo 7 - Evaluación en Usuarios	166
7.1 Metodología y Usuarios	166
7.2 Variables Evaluadas.....	167
7.3 Tareas Analizadas.....	168
7.4 Conclusiones de los Usuarios	169
Capítulo 8 - Manual de Usuario.....	172
Capítulo 9 - Conclusiones y Trabajo Futuro	173
9.1 Conclusiones	173
9.2 Trabajo Futuro	174
Contribuciones Personales	187
Bibliografía	198
Apéndices	202

ÍNDICE DE FIGURAS

Figura 1 - Logo Horizon.....	18
Figura 2 - Diagrama de Casos de Uso	35
Figura 3 - Mario Gómez, persona primaria	61
Figura 4 - Dashboard baja fidelidad.....	63
Figura 5 - Artículo reflexión baja fidelidad	64
Figura 6 - Análisis baja fidelidad	65
Figura 7 - Portfolios baja fidelidad.....	66
Figura 8 - Login y Registro baja fidelidad	66
Figura 9 - AcademIA baja fidelidad	67
Figura 10 - Mi perfil baja fidelidad.....	68
Figura 11 - Dashboard alta fidelidad	69
Figura 12 - AcademIA alta fidelidad	70
Figura 13 - Análisis alta fidelidad	71
Figura 14 - Sidebar alta fidelidad	72
Figura 15 - Mi perfil alta fidelidad	73
Figura 16 - Raíz del proyecto	81
Figura 17- Diagrama de Clases de Portfolio	85
Figura 18 - Organización del directorio /frontend	87
Figura 19 - Vista de Dashboard Principal (index.html)	90
Figura 20 - Vista de AcademIA Horizon (academia.html)	92
Figura 21 - Vista de Análisis de Activos (analysis.html)	94
Figura 22 – Explicabilidad IA Activos con Tendencia Lateral	95

Figura 23 - Vista de Gestión de Portfolios (portfolio.html)	96
Figura 24 - Vista de Perfil de Usuario (profile.html)	98
Figura 25 - Imports main.py	100
Figura 26 - Import de excepciones	101
Figura 27 - Middleware CORS	101
Figura 28 - Wiring de dependencias.....	102
Figura 29 - Endpoint Post	103
Figura 30 - chat_dto.py	104
Figura 31 - portfolio_dao_crear	105
Figura 32 – portfolio_dao_obtenerporid.....	106
Figura 33 - portfolio_dao_update	106
Figura 34 - portfolio_dao_delete	107
Figura 35- Diagrama Entidad Relacion BBDD.....	108
Figura 36 – Diagrama UML de Iniciar Sesión	115
Figura 37 - Vista Iniciar Sesión	116
Figura 38 - Diagrama UML de Cerrar Sesión	117
Figura 39 - Vista Cerrar Sesión	118
Figura 40 – Diagrama UML de Registro	119
Figura 41 - Vista Registrarse	120
Figura 42 - Diagrama UML de Eliminar Perfil	121
Figura 43 - Vista de Eliminar Perfil	122
Figura 44 - Diagrama UML de Editar Perfil	123
Figura 45 - Vista de Editar Perfil.....	124
Figura 46 - Diagrama UML de Editar Foto de Perfil	125

Figura 47 – Vista de Editar Foto Perfil	126
Figura 48 - Diagrama UML añadir activo	127
Figura 49 - Añadir activo	128
Figura 50 - Diagrama UML buscar activo.....	129
Figura 51 - Buscador de activos	129
Figura 52 - Diagrama UML de Noticias	129
Figura 53 - Vista Noticias	130
Figura 54 - Diagrama UML rentabilidad activo	131
Figura 55 - Rentabilidad Activo	132
Figura 56 - Diagrama UML insights IA	133
Figura 57 - Diagrama UML de Añadir Activo.....	134
Figura 58 - Diagrama UML de Eliminar Activo	136
Figura 59 - Diagrama UML de Crear Portfolio	137
Figura 60 - Diagrama UML de Eliminar Portfolio	138
Figura 61 - Diagrama UML Rentabilidad Portfolio	139
Figura 62 - Rentabilidad Portfolio	140
Figura 63 - Diagrama UML Avisos	140
Figura 64 - Avisos.....	141
Figura 65 - Diagrama UML de Cursos en Academia	142
Figura 66 - Vista Chat Academia.....	144
Figura 67 - Vista Reflexiones	146
Figura 68 - Diagrama de clases de Reflexiones	147
Figura 69 - Vista Dashboard Modo Oscuro	147
Figura 70 - Vista Ayuda	149

Figura 71 - Diagrama de arquitectura del pipeline	151
Figura 72 – Diagrama del Flujo de Carga	155
Figura 73- Implementación IA explicable (XAI)	159
Figura 74- Uso de la Funcionalidad del Log de Predicciones	161
Figura 75 - Utilidad promedio por tarea	171
Figura 76 – Horizon Logo	177

ÍNDICE DE TABLAS

Tabla 1 - Plan de trabajo	22
Tabla 2 - Requisitos Funcionales	34
Tabla 3 - Caso de Uso Número 1	36
Tabla 4 - Caso de Uso Número 2	37
Tabla 5 - Caso de Uso Número 3	38
Tabla 6 - Caso de Uso Número 4	39
Tabla 7 - Caso de Uso Número 5	41
Tabla 8 - Caso de Uso Número 6	42
Tabla 9 - Caso de Uso Número 7	43
Tabla 10 - Caso de Uso Número 8	45
Tabla 11 - Caso de Uso Número 9	47
Tabla 12 - Caso de Uso Número 10	48
Tabla 13 - Caso de Uso Número 11	49
Tabla 14 - Caso de Uso Número 12	50
Tabla 15 - Caso de Uso Número 13	51
Tabla 16 - Caso de Uso Número 14	52
Tabla 17 - Caso de Uso Número 15	53
Tabla 18 - Caso de Uso Número 16	54
Tabla 19 - Caso de Uso Número 17	55
Tabla 20 - Caso de Uso Número 18	56
Tabla 21 - Caso de Uso Número 19	57
Tabla 22 - Work Plan	181

Tabla 23 - Desarrollador 1	204
Tabla 24 - Desarrollador 2	204
Tabla 25 - Desarrollador 3	205
Tabla 26 - Desarrollador 4	205
Tabla 27 - Tutor	206
Tabla 28 - Características de Usuario Registrado	208
Tabla 29 - Características Usuarios No Registrados	208

Capítulo 1 - Introducción

En la última década, los mercados financieros han experimentado una transformación estructural impulsada por la digitalización. Lo que históricamente constituía un dominio reservado a instituciones bancarias y operadores profesionales es hoy un ecosistema abierto en el que cualquier usuario con acceso a internet puede participar en la compraventa de activos en tiempo real. Esta democratización del acceso no ha venido acompañada de una democratización equivalente del conocimiento, la brecha entre la facilidad de operar y la comprensión real del riesgo asumido representa uno de los principales factores de pérdida y desventaja para el inversor minorista no formado (Brenner & Meyll, 2020).

En este contexto, la Inteligencia Artificial (IA) y el aprendizaje automático (*machine learning*, ML) ofrecen una oportunidad sin precedentes para cerrar dicha brecha. Los algoritmos de ensamblaje basados en árboles de decisión con optimización del gradiente, en particular XGBoost (Chen & Guestrin, 2016), han demostrado una capacidad destacable para identificar patrones en series temporales financieras de naturaleza tabular, superando en robustez y eficiencia computacional a alternativas más complejas en escenarios de alta dimensionalidad y ruido estocástico (Sezer, Gudelek & Ozbayoglu, 2020).

La predicción por sí sola es insuficiente si el usuario no comprende los fundamentos que la sustentan. En este sentido, la Inteligencia Artificial Explicable (Explainable AI, XAI) surge como una disciplina crítica en el dominio financiero. Mediante técnicas como los valores SHAP (SHapley Additive exPlanations), basados en teoría de juegos cooperativos (Lundberg & Lee, 2017), es posible cuantificar la contribución marginal de cada variable sobre una predicción concreta, transformando la salida opaca de un modelo de caja negra en una explicación interpretable y auditable. La intersección entre XAI y finanzas constituye una línea de investigación activa, motivada tanto por la necesidad regulatoria de transparencia algorítmica como por su potencial pedagógico para el inversor no experto (Arrieta et al., 2020).

El presente Trabajo Fin de Grado surge de la convergencia de tres áreas de la ingeniería informática: el desarrollo de aplicaciones web de alto rendimiento, el análisis predictivo mediante ML y XAI, y el diseño de interfaces conversacionales inteligentes basadas en Modelos de Lenguaje de Gran Escala (Large Language Models, LLM). El resultado es Horizon, una plataforma educativa de predicción financiera estructurada en cuatro capas: presentación, controladores (FastAPI), servicios y acceso a datos (DAOs), que se articula en torno a seis módulos principales. El módulo de predicción implementa un *pipeline* ETL completo que alimenta un motor basado en XGBoost, entrenado mediante validación *walk-forward*. El módulo de XAI aplica valores SHAP con TreeExplainer para generar explicaciones interpretables de cada predicción, exponiendo qué indicadores técnicos han determinado la señal emitida. El módulo de AcademIA integra un asistente conversacional basado en LLM con contexto enriquecido del usuario, actuando como tutor financiero interactivo. Todo ello se sustenta sobre un sistema de autenticación centralizado mediante Supabase.

El documento se estructura de la siguiente manera. El Capítulo 2 revisa el estado del arte en los dominios convergentes del proyecto. El Capítulo 3 recoge la especificación de requisitos software. El Capítulo 4 detalla el proceso de diseño de interfaz de usuario. El Capítulo 5 describe las tecnologías y herramientas seleccionadas. El Capítulo 6 desarrolla la implementación completa del sistema. El Capítulo 7 presenta la evaluación con usuarios reales. El Capítulo 8 contiene el manual de usuario. El Capítulo 9 recoge las conclusiones y las líneas de trabajo futuro.

Nuestro logo y nombre, "Horizon", pretenden resumir el objetivo del proyecto, dibujando un horizonte como meta del inversor principiante, ascendente para demostrar su camino de aprendizaje, con colores azul y blanco denotando el rigor técnico propio de la aplicación.



Figura 1 - Logo Horizon

1.1 Motivación

El acceso a los mercados financieros se ha facilitado significativamente gracias a plataformas de tecnología financiera y neobancos como Revolut o Trade Republic. Sin embargo, esta accesibilidad conlleva una sobrecarga de información que deteriora la calidad de las decisiones del inversor minorista no formado (Agnew & Szykman, 2005). A ello se suma la creciente preocupación por la pérdida de poder adquisitivo: los episodios inflacionarios recientes han erosionado el valor real del ahorro tradicional (D'Acunto, Prabhala & Rossi, 2019), impulsando el interés por activos de valor refugio como el oro (Baur & Lucey, 2010) y por la inversión en general, sin que la mayoría de los usuarios cuente con la formación necesaria para operar con criterio.

La falta de herramientas que expliquen estos fenómenos de forma didáctica motiva la creación de Horizon, un asistente inteligente que personaliza la experiencia en función del perfil de riesgo del usuario y mitigando la barrera de entrada al mundo de la inversión.

1.2 Objetivos

El Objetivo General de este trabajo es diseñar, implementar y desplegar una aplicación web que integre modelos de Inteligencia Artificial para ofrecer asesoramiento financiero personalizado y predicción de tendencias bursátiles en tiempo real, o actualizado periódicamente. La meta final de dicha aplicación web es lograr ser un entorno orientado al análisis predictivo y la educación financiera, de tal forma que

permita a los usuarios comprender de mejor manera cómo funcionan los mercados financieros y les ayude a monitorizar los activos financieros deseados mediante técnicas de IA. A su vez, la IA implementada deberá seguir pautas éticas y no invitar a los usuarios a invertir dinero real ni activos concretos, sino orientar al inversor principiante en el conocimiento de las buenas prácticas de inversión, garantizando la seguridad de los datos en todo momento.

En cuanto a los Objetivos Específicos, se pretende lograr la consecución del objetivo general mediante los descritos a continuación:

1. Estudio del Estado del Arte: Analizar las técnicas actuales de predicción financiera, arquitecturas de ML (como Gradient Boosting Trees y XGBoost), la tecnología de los Transformers para procesamiento de lenguaje y algoritmos de clasificación multiclase. Estudiar aplicaciones alternativas para identificar puntos diferenciales.
2. Desarrollo del Modelo Predictivo: Entrenar y validar un modelo basado en XGBoost capaz de clasificar y predecir la tendencia a corto plazo de distintos *tickers* bursátiles (acciones, criptomonedas y materias primas) empleando características estadísticas extraídas sobre ventanas temporales multiescala. Implementar a su vez un *pipeline* de ingesta continua de datos históricos.
3. Agente IA: Implementar un agente conversacional de IA, basado en modelos de lenguaje LLM, en la aplicación web capaz de resolver dudas conceptuales y explicar métricas financieras de forma didáctica y ética, conociendo datos concretos del usuario para ofrecer una experiencia personalizada y adaptable.
4. Desarrollo de la Plataforma Web: Construir una interfaz de usuario intuitiva y usable para el usuario final, así como un *backend* robusto que integre el modelo de IA y gestione la base de datos a una velocidad apta para su usabilidad concurrente.
5. Módulo Educativo: Construir un módulo de gestión de contenidos educativos que permitan al usuario progresar en su aprendizaje, aumentar sus

conocimientos, y construir una base relativamente sólida de inversiones y riesgos.

6. Validación: Evaluar la precisión técnica del modelo (mediante metodologías *walk-forward* y métricas como el *balanced accuracy*) y la usabilidad de la herramienta mediante pruebas finales.
7. Seguridad de los datos: El objetivo base consistirá en asegurar la confidencialidad de los datos sensibles de los usuarios, utilizando algoritmos de cifrado para contraseñas, y cuidando la exposición de datos al *frontend*.

1.3 Alcance y Limitaciones

El alcance del presente trabajo se limita al diseño y desarrollo de una plataforma web *full-stack* orientada al análisis financiero de activos y la formación del usuario. En más detalle, el alcance se especifica a partir de las siguientes funcionalidades y restricciones que delimitan el trabajo:

- Módulo de Gestión de Usuarios y Perfiles: Implementación de un sistema de autenticación seguro y gestión de perfiles donde se almacenarán los datos del usuario y su historial de aprendizaje, limitados por la capacidad de la base de datos empleada.
- Módulo de Ingesta y Visualización de Datos: Desarrollo de un *pipeline* de datos que conecte con APIs financieras externas para la obtención de series temporales en tiempo real y datos históricos. Se incluirá una página principal interactiva con gráficas dinámicas. El alcance se limitará a las capacidades de las APIs gratuitas que usaremos, tratando de mitigar ese límite con técnicas de almacenamiento en caché.
- Módulo de Análisis Predictivo (ML): Diseño, entrenamiento y despliegue de un motor predictivo estricto basado en modelos de ensamble de árboles (XGBoost), optimizado para problemas de clasificación de tendencias en un universo seleccionado de activos estables y volátiles.

- Módulo de Academia: Creación de un repositorio estructurado de contenidos formativos, siendo esto un apartado específico en la web (Academia). Además, se integra un asistente basado en LLM, a través de una API de modelos ya establecidos, configurado mediante ingeniería de *prompts* y preprocesamiento del mensaje para actuar como resolutor de dudas o explicación de conceptos, teniendo en cuenta aspectos éticos relacionados con el dinero. Igual que antes, el alcance se limita a las capacidades del modelo gratuito empleado.
- Limitación a Entorno de Simulación: Implementación de la aplicación web con vistas informativas y analíticas, sin implicar una inversión real en cualquier activo, ni tampoco un asesoramiento financiero legal. La principal finalidad es que el usuario pueda realizar el seguimiento de sus inversiones basadas en las predicciones y datos del sistema, sin riesgo de capital real y con una precisión sujeta a la volatilidad intrínseca del mercado.
- Limitaciones de Datos: Dado el uso de APIs gratuitas y los recursos de procesamiento limitados, la frecuencia de actualización de datos podría presentar un retardo mayor que el óptimo y el histórico de los mismos podría estar limitado.

1.4 Plan de Trabajo

El plan de trabajo seguido para la consecución de los objetivos descritos en el apartado anterior se ha fundamentado en la utilización de una metodología de desarrollo ágil. Esta aproximación ha permitido una construcción incremental de la plataforma, facilitando la detección temprana de errores y la integración continua de los módulos de IA con la interfaz web. El cronograma se divide en seis fases principales, distribuidas a lo largo del periodo de ejecución del trabajo:

Fase	Duración
Especificación de Requisitos	23 de septiembre a 1 de noviembre
Diseño de Interfaz de Usuario	1 de noviembre a 22 de diciembre
Tecnologías, Herramientas y Base de Datos	22 de diciembre a 20 de enero
Setup Base y Maquetación UI	20 de enero a 20 de febrero
Arquitectura Backend y Datos	20 de febrero a 5 de marzo
Funcionalidades Principales	5 de marzo a 27 de abril
IA y ML	20 de enero a 29 de abril
Testing y Validación	27 de febrero a 15 de mayo

Tabla 1 - Plan de trabajo

El proyecto comienza con la Especificación de Requisitos, una etapa que se alarga durante el primer mes y medio del curso, donde se definen las necesidades funcionales y no funcionales del sistema. Durante este periodo, se investigó sobre aplicaciones alternativas, artículos académicos del mismo ámbito, y soluciones posibles. Se identifican los casos de uso y los perfiles de usuario, estableciendo las bases que guiarán el resto del desarrollo para asegurar que la solución final resuelva el problema planteado.

Posteriormente, se aborda el Diseño de Interfaz de Usuario (UX/UI), fase en la que se traslada la teoría a la experiencia visual. Continuando en el primer cuatrimestre, realizamos esta etapa mientras cursamos la asignatura de Diseño de Sistemas Interactivos. Aquí se construyen los *wireframes* y prototipos de baja y alta fidelidad, centrando el esfuerzo en la arquitectura de la información y la usabilidad. El objetivo es, siguiendo con los objetivos fundamentales del proyecto, garantizar que la aplicación

sea intuitiva, usable para principiantes, y que la estética sea coherente con los estándares actuales de diseño digital.

Una vez definido el diseño, se procede a la selección de Tecnologías, Herramientas y Base de Datos. En este bloque, se elige el *stack* tecnológico más eficiente y se diseña el esquema de la base de datos, en base a las necesidades de los casos de uso. Consideramos importante dedicar más tiempo a la elección de las tecnologías para asegurarnos de ser capaces de desarrollar en esos lenguajes y que sea escalable al menos a los objetivos iniciales.

Esta planificación técnica permite que el *Setup* Base y Maquetación UI se ejecute con éxito, configurando el entorno de desarrollo básico y transformando los diseños previos en componentes de código front-end listos para interactuar con la lógica del sistema.

El núcleo técnico se desarrolla en las fases de Arquitectura *Backend* y Datos y Funcionalidades Principales. En la primera, se construye la estructura del servidor y las APIs que permiten la comunicación entre el cliente y el almacenamiento, desarrollando todos los *endpoints* previstos para consumir y entidades DAO y DTO para asegurar la correcta transferencia de datos y arquitectura del *backend*.

En la segunda, se implementa la lógica principal. Se desarrollan las funcionalidades de la aplicación en *backend* a través de la definición de los servicios, que establecen la comunicación entre entidades de datos y otros servicios para manejar los datos. Además, se implementan las llamadas al *backend* desde los archivos dinámicos del *frontend*, modificando las directrices para consumir los datos desde los *endpoints* para mostrarlos, si ha cambiado algo durante la definición de los servicios.

En paralelo a las fases técnicas previas se realiza la integración de la Inteligencia Artificial y ML. Mientras se desarrollan tanto *backend* como *frontend*, se va perfeccionando el modelo para predecir las tendencias de los activos, investigando a la vez mejores soluciones. Además, se implementa el asistente conversacional inteligente, iterando desde un asistente sin límite de consultas, hasta poder preprocesar el mensaje, enviar el contexto del usuario, y reducir el consumo de *tokens*.

El plan de trabajo finaliza con el periodo de *Testing* y Validación. Se añaden pruebas unitarias y de integración que verifican la ausencia de errores, la seguridad del sistema y el cumplimiento total de los requisitos iniciales definidos al inicio del cronograma, con pruebas en usuarios reales. Se realiza en paralelo al desarrollo técnico de la aplicación como medida de contención de errores, junto con una evaluación con usuarios reales para validar la aplicación y detectar puntos de mejora.

Capítulo 2 - Estado del Arte

El desarrollo de una plataforma de predicción financiera asistida por IA requiere fundamentarse en una revisión crítica de la literatura existente en tres ámbitos convergentes: la evolución de las herramientas de gestión patrimonial automatizada, las técnicas de aprendizaje profundo aplicadas a la predicción de series temporales bursátiles, y la integración de modelos de lenguaje de gran escala en el ecosistema financiero. El presente capítulo analiza el estado actual de cada uno de estos dominios, identifica las carencias y limitaciones de las soluciones existentes, y justifica las decisiones de diseño adoptadas en Horizon como respuesta a dichas limitaciones.

2.1 Evolución de las Plataformas de Inversión y Robo-Advisors

La irrupción de la tecnología financiera ha reducido drásticamente las barreras de entrada al mercado mediante la simplificación de interfaces y la disminución del capital mínimo requerido (D'Acunto, Prabhala & Rossi, 2019). Esta accesibilidad ha fomentado la proliferación de los denominados *robo-advisors*, definidos formalmente como intermediarios de software basados en algoritmos que proveen servicios de gestión patrimonial automatizada. Tradicionalmente, estas plataformas fundamentan su asignación de activos en la Teoría Moderna de Carteras (Markowitz, 1952), optimizando la rentabilidad esperada en función de perfiles de riesgo estáticos predefinidos.

La evolución del sector ha derivado hacia modelos de mayor autonomía, donde el usuario define su exposición al riesgo de forma dinámica ante entornos macroeconómicos cambiantes. No obstante, a pesar de facilitar la inserción de inversores noveles, la literatura académica advierte sobre las limitaciones pedagógicas de los *robo-advisors* tradicionales. Estas herramientas operan con frecuencia como cajas negras que ejecutan órdenes sin fomentar una verdadera alfabetización financiera, lo que puede derivar en una dependencia excesiva del sistema y en una falsa sensación de seguridad por parte del usuario (Brenner & Meyll, 2020). Desde una perspectiva regulatoria, Baker & Dellaert (2018) subrayan además la necesidad de

marcos normativos que garanticen la transparencia algorítmica en el asesoramiento automatizado, una exigencia que los sistemas actuales satisfacen de forma desigual.

El presente trabajo aborda esta carencia integrando un componente pedagógico que trasciende la gestión pasiva, combinando un motor predictivo con herramientas de XAI y un asistente conversacional que traduce la operativa algorítmica a un lenguaje comprensible para el inversor no experto.

2.2 Predicción de Series Temporales Mediante Deep Learning

El pronóstico de series temporales financieras constituye un problema de elevada complejidad debido a la baja relación señal-ruido y la no estacionariedad inherente a los mercados bursátiles (Makridakis, Spiliotis & Assimakopoulos, 2018). Históricamente, la academia se apoyó en modelos estadísticos lineales como ARIMA, sin embargo, su incapacidad para capturar dependencias no lineales impulsó la adopción de arquitecturas de aprendizaje profundo (*deep learning*, DL) durante la última década.

Las redes neuronales recurrentes (RNN), y en particular las unidades recurrentes con compuerta (Gated Recurrent Unit, GRU), propuestas por Cho et al. (2014), han demostrado una notable capacidad para modelar dependencias temporales en secuencias financieras. Su variante bidireccional (BiGRU), derivada del trabajo de Schuster & Paliwal (1997), extiende esta capacidad procesando la secuencia en ambas direcciones, enriqueciendo la representación contextual de cada instante temporal. La incorporación de mecanismos de atención (Bahdanau, Cho & Bengio, 2015) permite además ponderar de forma diferenciada la contribución de cada paso temporal a la predicción final, dotando al modelo de una interpretabilidad estructural que resulta especialmente valiosa en el dominio financiero. Fischer & Krauss (2018) demostraron empíricamente la superioridad de estas arquitecturas recurrentes sobre métodos estadísticos clásicos en tareas de predicción bursátil, aunque advirtieron sobre su tendencia al sobreajuste cuando no se aplican esquemas de validación temporalmente coherentes.

En paralelo, los algoritmos basados en el ensamblaje de árboles de decisión mediante optimización del gradiente han emergido como una alternativa robusta para datos financieros de naturaleza tabular. En particular, XGBoost (Chen & Guestrin, 2016) se ha consolidado como un estándar analítico por su eficiencia computacional y su estabilidad frente al ruido estocástico, sin requerir el elevado coste propio del aprendizaje profundo (Sezer, Gudelek & Ozbayoglu, 2020). La combinación de ambos paradigmas mediante modelos de ensamblaje (ensemble) constituye una línea de investigación activa que busca aprovechar las fortalezas complementarias de cada enfoque.

Finalmente, la validación rigurosa en series temporales exige el uso de esquemas de validación *walk-forward* en lugar de la validación cruzada clásica, que introduce sesgo temporal (*look-ahead bias*) al mezclar observaciones futuras en el entrenamiento (Cerqueira, Torgo & Mozetič, 2020). Horizon adopta este enfoque para garantizar la validez fuera de muestra de sus métricas.

2.3 Modelos de Lenguaje (LLM) en el Ecosistema FinTech

La implementación de LLM ha catalizado un cambio de paradigma en el sector financiero, superando ampliamente las limitaciones de los sistemas conversacionales basados en árboles de decisión predefinidos. Cimentados en la arquitectura Transformer y en mecanismos de auto-atención (Vaswani et al., 2017), los LLM poseen una capacidad excepcional para capturar dependencias semánticas a largo plazo y procesar volúmenes de información heterogénea, lo que los convierte en herramientas especialmente adecuadas para la generación de respuestas contextualizadas en dominios especializados (Zhao et al., 2023).

En el ámbito financiero, su aplicación se ha extendido desde el análisis de sentimiento de noticias y documentos regulatorios hasta la asistencia conversacional personalizada al inversor. Trabajos como FinGPT (Yang, Liu & Wang, 2023) han demostrado la viabilidad de adaptar modelos de lenguaje de propósito general a tareas financieras específicas mediante técnicas de ajuste fino (*fine-tuning*), obteniendo

resultados competitivos con un coste computacional significativamente inferior al de entrenar modelos desde cero. En una revisión sistemática reciente, Li et al. (2023) identificaron la personalización del asesoramiento y la explicación de conceptos financieros en lenguaje natural como los casos de uso de mayor impacto potencial para los LLM en este sector.

Desde una perspectiva educativa, estas herramientas ejercen como tutores virtuales adaptativos capaces de procesar datos técnicos del mercado y traducir dinámicas complejas a un lenguaje comprensible, mitigando la asimetría de información que históricamente ha perjudicado al inversor minorista. Sin embargo, su despliegue plantea consideraciones éticas relevantes: la tendencia a generar respuestas plausibles pero incorrectas (alucinaciones) y la ausencia de responsabilidad regulatoria clara exigen que estos sistemas operen bajo restricciones explícitas que delimiten su ámbito de actuación (Li et al., 2023). Horizon integra un asistente conversacional basado en LLM con un sistema de *prompts* enriquecidos con contexto real del usuario, garantizando que las respuestas se circunscriban al ámbito educativo y no constituyan asesoramiento financiero vinculante.

2.4 Análisis de Alternativas y Justificación de la Propuesta.

El panorama tecnológico actual revela tres categorías de soluciones previas que se alinean con los objetivos de Horizon: las plataformas comerciales de gestión automatizada, los simuladores educativos de bolsa y los trabajos académicos universitarios de predicción bursátil mediante aprendizaje automático. El análisis de cada categoría permite identificar las carencias que motivan y justifican la propuesta del presente proyecto.

En el segmento de plataformas comerciales, herramientas como Indexa Capital o getquin destacan por la integración de IA para la auditoría de carteras y la personalización de recomendaciones. Sin embargo, su operatividad está estrictamente condicionada a la asunción de riesgo financiero real, lo que constituye una barrera de entrada para el inversor principiante. Sistemas como AlgosOne han democratizado el

ML para el *trading*, pero operan desde un enfoque centrado de forma exclusiva en la rentabilidad, careciendo de un marco pedagógico transparente que favorezca la alfabetización financiera del usuario. En el extremo opuesto se ubican plataformas como el Simulador de Latibex, que cumplen una valiosa función didáctica mediante *paper trading*, pero cuentan con interfaces anticuadas, sobrecargadas, y carecen de motores analíticos predictivos que guíen activamente al usuario en su proceso de toma de decisiones (Latibex, 2026).

Una tercera categoría de referencia la constituyen los trabajos de investigación universitaria, los cuales abordan la predicción bursátil mediante aprendizaje profundo, pero presentan de forma sistemática una limitación estructural. El alcance queda restringido al experimento de modelado, sin que ninguno de ellos articule dicho experimento dentro de una plataforma educativa completa orientada al usuario. Alcaraz López (2022), en su trabajo desarrollado en la Universidad Politécnica de Madrid, presenta dos modelos Long Short-Term Memory (LSTM) para la predicción a un día del precio de cierre en el NASDAQ-100. El propio autor reconoce que el sistema *"cuenta con limitaciones dado que en un entorno tan complejo como la inversión bursátil se debe realizar también un análisis fundamental de las empresas"* y señala la ausencia de variables macroeconómicas como una restricción explícita. El trabajo no contempla interfaz de usuario, gestión de cartera ni componente pedagógica. En la misma línea, Traseira Linares (2022), desde la Universidad Pontificia Comillas, construye un modelo de predicción del S&P 500 con redes neuronales artificiales y evalúa una estrategia de inversión basada en sus señales, pero el trabajo adopta una orientación de investigación cuantitativa pura, sin que exista plataforma desplegable ni módulo de explicabilidad que justifique las decisiones del modelo ante el usuario final. Hernández Rodríguez (2020), desde la Universidad de Valladolid, emplea arquitecturas LSTM, GRU y SimpleRNN sobre series del NASDAQ y desarrolla una clasificación de señales de compraventa. Pese a la solidez técnica de la experimentación, el trabajo permanece igualmente circunscrito al ámbito del modelo, sin contemplar despliegue ni formación del usuario. En todos los casos, la evaluación se basa en métricas de error de regresión (RMSE) o en validaciones estáticas que no garantizan la coherencia temporal requerida

en series financieras, lo que introduce el riesgo de *look-ahead bias* documentado por Cerqueira, Torgo y Mozetič (2020).

Horizon se justifica como un ecosistema digital que cubre el vacío identificado en las tres categorías previas. Al combinar un motor predictivo basado en XGBoost, técnicas XAI para transparentar el razonamiento del modelo mediante valores SHAP, y un tutor virtual LLM con contexto enriquecido del usuario, la plataforma supera tanto la simulación estática como el experimento académico aislado, para proporcionar un entorno de aprendizaje empírico, seguro e impulsado por datos. Asimismo, la adopción de un esquema de validación *walk-forward* garantiza la integridad temporal de las métricas y elimina el sesgo de los enfoques de partición estática empleados en los trabajos precedentes.

Capítulo 3 - Especificación de Requisitos Software

A continuación, se expone la Especificación de Requisitos Software (SRS) que se ha elaborado durante el desarrollo del sistema de gestión de inversiones financieras Horizon y cuyo objetivo ha sido definir de forma exhaustiva los requisitos funcionales y no funcionales para el desarrollo de este. Esta especificación ha sido realizada en base a las directrices contenidas en el estándar IEEE Práctica Recomendada para Especificaciones de Requisitos Software ANSI/IEEE, 1998. Se dispone del documento completo adjunto en el [Apéndice A](#).

3.1 Visión General del Sistema

El sistema Horizon se ha concebido como una aplicación web basada en una arquitectura cliente-servidor. Su funcionamiento se apoya en la integración de servicios externos para la obtención de datos de mercado, así como en un *backend* encargado del procesamiento de información y la ejecución de modelos de predicción.

El sistema permite al usuario consultar información bursátil mediante gráficos interactivos, llevar seguimiento de carteras de inversión simuladas y compuestas por activos que el usuario elija y obtener predicciones sobre la evolución futura de dichos activos. Estas predicciones se complementan con explicabilidad IA, a través de la cual se describe la contribución de cada factor a la predicción, lo que contribuye a mejorar la transparencia y el entendimiento del sistema.

Además de todo ello, Horizon incorpora un módulo formativo “AcademIA” que facilita el aprendizaje del usuario mediante contenidos educativos (cursos de aprendizaje integrados.) y un *chatbot* basado en Inteligencia Artificial. Con ello, el usuario es capaz de adquirir o ampliar conocimientos financieros a medida y autónoma.

3.2 Requisitos Funcionales

Los requisitos funcionales definen las capacidades principales del sistema. En Horizon, estas funcionalidades se organizan en distintos bloques: gestión de usuarios, visualización de información financiera, gestión de carteras, módulo de Inteligencia Artificial y módulo educativo.

En primer lugar, el sistema permite a los usuarios registrarse, iniciar sesión y gestionar su perfil, garantizando así un acceso personalizado, así como eliminar su cuenta si así lo desean. Incluye también personalización de la interfaz en cuanto a tema oscuro/claro, edición de perfil, pudiendo cambiar datos, foto de perfil y contraseña, y actualización de la membresía de plan gratuito a plan pro.

En segundo lugar, proporciona herramientas para la búsqueda y visualización de activos financieros mediante gráficos interactivos, que constituyen la base del análisis del usuario. Incluye gráficos de velas de 7, 30 y 90 días, dando a conocer también datos como la fiabilidad del modelo, el precio actual y enlaces a una selección de noticias recientes sobre el activo que se busca.

Además, el sistema permite la creación y gestión de carteras de inversión simuladas, facilitando el seguimiento de activos seleccionados. A esto se añade un módulo de Inteligencia Artificial capaz de generar predicciones sobre la evolución de los activos, acompañado de explicaciones interpretables.

Finalmente, el módulo de AcademIA complementa la funcionalidad del sistema mediante cursos formativos integrados y una asistencia interactiva a través de un *chatbot* disponible para el usuario.

Con el objetivo de sintetizar los requisitos más relevantes, en la siguiente tabla se presentan algunos de los principales requisitos funcionales identificados:

ID	Requisito
RF1	El sistema debe permitir el registro de nuevos usuarios

RF2	El sistema debe permitir el inicio y cierre de sesión en la cuenta de un usuario
RF3	El sistema debe permitir la búsqueda de activos financieros
RF4	El sistema debe mostrar gráficos interactivos de los activos financieros
RF5	El sistema debe permitir la gestión de <i>portfolios</i> (creación de tantos como se desea, compuestos por los activos que se desea)
RF6	El sistema debe permitir añadir y eliminar activos de un <i>portfolio</i>
RF7	El sistema debe generar predicciones mediante IA
RF8	El sistema debe proporcionar explicaciones (XAI) de las predicciones
RF9	El sistema debe incluir un <i>chatbot</i> para asistencia al usuario
RF10	El sistema debe permitir la edición del perfil del usuario, incluyendo foto de perfil, contraseña, nombre, suscripción, e incluso eliminación de su cuenta.

RF11

El sistema debe permitir la visualización de cursos educativos en función de su suscripción.

Tabla 2 - Requisitos Funcionales

3.3 Diagrama de Casos de Uso

Con el fin de representar de forma visual las funcionalidades del sistema y la interacción con los distintos tipos de usuarios, se ha elaborado el siguiente diagrama de casos de uso. Dicho diagrama muestra cómo el actor principal (usuario registrado) interactúa con el sistema para realizar las acciones disponibles dentro del sistema Horizon.

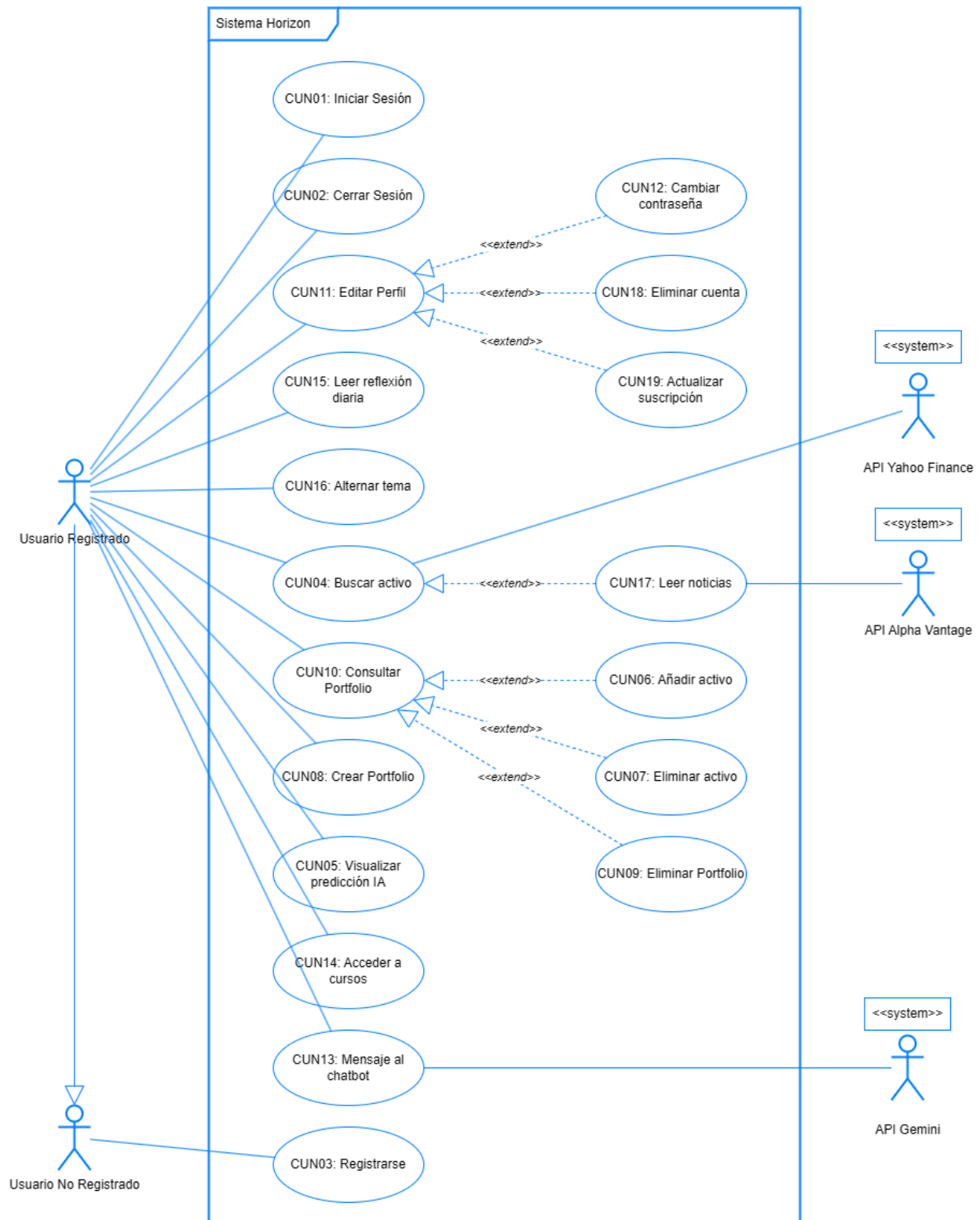


Figura 2 - Diagrama de Casos de Uso

Adicionalmente, se detallan los distintos casos de uso a continuación:

Caso de Uso: CUN01	Iniciar Sesión
Objetivo en contexto	Autenticar al usuario para que pueda acceder a la aplicación web
Entradas	Correo electrónico y contraseña
Precondiciones	El usuario debe estar registrado previamente
Salidas	Token de sesión y redirección al <i>dashboard</i>
Postcondición si éxito	El usuario accede a su cuenta y puede utilizar el sistema
Postcondición si fallo	El sistema deniega el acceso y muestra un mensaje de error
Actores	Usuario Registrado
Secuencia normal	1. El usuario introduce sus credenciales 2. El sistema verifica los datos 3. Se confirma el inicio de sesión y se redirige al <i>dashboard</i>
Secuencia alternativa	S-1: Credenciales incorrectas. El sistema pide al usuario que las reintroduzca

Tabla 3 - Caso de Uso Número 1

Caso de Uso: CUN02	Cerrar Sesión
Objetivo en contexto	Cerrar la sesión en curso del usuario
Entradas	N/A
Precondiciones	Que el usuario haya iniciado sesión y esta no haya expirado
Salidas	Cierre de sesión en curso del usuario en la aplicación
Postcondición si éxito	El usuario cierra su sesión en curso en la aplicación y se le redirige a la pantalla de inicio de sesión o registro
Postcondición si fallo	N/A
Actores	Usuario Registrado
Secuencia normal	1. Desde cualquier pantalla, el usuario dispondrá del botón de cerrar sesión, ubicado en el menú lateral. 2. El usuario hace clic sobre el botón de "Cerrar sesión" 3. Se cierra la sesión actual
Secuencia alternativa	N/A

Tabla 4 - Caso de Uso Número 2

Caso de Uso: CUN03	Registrarse (Crear Cuenta)
Objetivo en contexto	El usuario consigue crear una nueva cuenta en el sistema
Entradas	Nombre, Apellidos, Correo, Contraseña, Foto de perfil (opcional)
Precondiciones	Que no exista ninguna cuenta ya creada con este correo electrónico.
Salidas	<i>Token</i> de sesión y redirección al <i>dashboard</i>
Postcondición si éxito	El usuario accede a su nueva cuenta y puede utilizar el sistema
Postcondición si fallo	El sistema deniega el acceso y muestra un mensaje de error, bien por cuenta ya existente o bien por datos que faltan por rellenar
Actores	Usuario No Registrado
Secuencia normal	1. El usuario rellena los datos del formulario 2. El sistema verifica los datos, que estén rellenos y que no existe ninguna otra cuenta con el mismo correo electrónico. 3. Se confirma la creación de la nueva cuenta y se redirige al <i>dashboard</i>
Secuencia alternativa	S-1: Credenciales incompletas. El sistema fuerza a rellenar los datos faltantes para crear la cuenta S-2: Cuenta ya existente. El sistema muestra mensaje de error y pide introducir un correo diferente

Tabla 5 - Caso de Uso Número 3

Caso de Uso: CUN04	Buscar y visualizar activo financiero
Objetivo en contexto	Buscar un <i>ticker</i> específico y renderizar su gráfico de evolución
Entradas	Nombre de la empresa o <i>ticker</i> (ej. AAPL)
Precondiciones	El usuario registrado ha de haber iniciado sesión en la aplicación
Salidas	Gráfico interactivo del activo
Postcondición si éxito	El gráfico se visualiza correctamente en pantalla, así como otras variables clave como la tendencia, la fiabilidad, precio y noticias relacionadas
Postcondición si fallo	Se muestra un mensaje indicando que el activo no existe o la API falla
Actores	Usuario Registrado
Secuencia normal	1. El usuario introduce el <i>ticker</i> en el buscador 2. El sistema consulta la API externa (ej. Yahoo Finance) 3. El sistema renderiza el gráfico en la interfaz
Secuencia alternativa	S-1: Fallo en la recogida de los datos. El sistema muestra mensaje de error e incapacidad para visualizar el gráfico

Tabla 6 - Caso de Uso Número 4

Caso de Uso: CUN05	Visualizar predicción de IA y Explicabilidad (XAI)
Objetivo en contexto	Obtener la tendencia futura de un activo y la justificación textual del modelo
Entradas	<i>Ticker</i> o nombre del activo que se desea consultar
Precondiciones	El usuario debe ser Usuario Registrado y haber iniciado sesión en la aplicación. El modelo de ML debe estar entrenado y operativo
Salidas	Gráfico de precios e historial de señales IA disponible para 7, 30 y 90 días, así como Explicabilidad IA, justificando la contribución de cada factor de la predicción
Postcondición si éxito	Se muestra con éxito el gráfico de precios e historial de señales IA, así como el gráfico y la explicación de la contribución de cada factor a la predicción
Postcondición si fallo	La inferencia falla por indisponibilidad de datos o pierde precisión por falta de cantidad de datos
Actores	Usuario Registrado
Secuencia normal	1. El usuario accede a la pestaña de análisis de <i>ticker</i> 2. El usuario busca el activo deseado bien por su nombre, o bien por su <i>ticker</i> 3. Al abrirse la pestaña de análisis del activo, el usuario encontrará una sección donde aparecerá el gráfico

	de precio e historial de señales IA, y otra sección para la Explicabilidad IA, relacionadas con dicho activo 4. La información se basará en los datos más recientes disponibles para dichos activos y el usuario la tendrá lista para la consulta
Secuencia alternativa	S-1: Que no se pueda realizar el análisis por falta de datos en la base de datos

Tabla 7 - Caso de Uso Número 5

Caso de Uso: CUN06	Añadir Activo a <i>Portfolio</i>
Objetivo en contexto	Guardar un activo financiero en la lista de seguimiento del usuario
Entradas	<i>Ticker</i> del activo
Precondiciones	El usuario registrado debe haber iniciado sesión en la aplicación
Salidas	Actualización del <i>portfolio</i> en la base de datos
Postcondición si éxito	El activo aparece en la lista de seguimiento del usuario
Postcondición si fallo	El activo no se guarda debido a un error de red o base de datos
Actores	Usuario Registrado
Secuencia normal	1a. El usuario busca y visualiza un activo 2a. Pulsa el botón "Añadir a <i>portfolio</i> " y selecciona el <i>portfolio</i> al que desea añadir el activo 3a. El usuario confirma la operación y el sistema actualiza la lista de seguimiento 1b. Desde la vista de <i>portfolio</i> , el usuario hace clic sobre el botón "Añadir Inversión" 2b. El usuario busca el activo deseado. 3b. El usuario confirma la operación con el botón "Añadir Activo" y aplica los cambios.
Secuencia alternativa	S-1: El activo ya está en el <i>portfolio</i> . El sistema avisa mediante un <i>popup</i> .

Tabla 8 - Caso de Uso Número 6

Caso de Uso: CUN07	Eliminar Activo de <i>Portfolio</i>
Objetivo en contexto	Eliminar un activo financiero del <i>portfolio</i> usuario
Entradas	<i>Ticker</i> del activo
Precondiciones	El usuario debe haber iniciado sesión en la aplicación y el activo seleccionado debe estar en el <i>portfolio</i> objetivo de la acción
Salidas	Actualización del <i>portfolio</i> en la base de datos
Postcondición si éxito	El activo desaparece del <i>portfolio</i> del usuario
Postcondición si fallo	El activo no desaparece de la lista debido a un error de red o base de datos
Actores	Usuario Registrado
Secuencia normal	1. El usuario selecciona un activo de su lista de seguimiento 2. Pulsa el botón "Quitar" de la Lista 3. El sistema actualiza la lista de seguimiento, eliminando dicho activo
Secuencia alternativa	S-1: El activo NO está en el <i>portfolio</i> objetivo, el sistema simplemente no mostraría dicho activo en la lista del <i>portfolio</i> y no saldría el botón asociado para eliminar de la lista

Tabla 9 - Caso de Uso Número 7

Caso de Uso: CUN08	Crear <i>Portfolio</i>
Objetivo en contexto	El usuario crea un <i>Portfolio</i> (lista de seguimiento), dándole un nombre, descripción y nivel de riesgo que se desea asumir
Entradas	Nombre para el <i>portfolio</i> , descripción (opcional) y nivel de riesgo a asumir
Precondiciones	El usuario debe haber iniciado sesión en la aplicación y el <i>portfolio</i> que se pretende crear no puede ser uno ya existente
Salidas	<i>Portfolio</i> creado para el usuario
Postcondición si éxito	El <i>portfolio</i> se crea con éxito y está disponible para la visualización por parte del usuario.
Postcondición si fallo	El <i>portfolio</i> no se puede crear y el sistema muestra el mensaje de error pertinente.
Actores	Usuario Registrado
Secuencia normal	1. Desde la vista de <i>Portfolio</i> , el usuario hace clic sobre el botón “+Nuevo” para empezar a crear un nuevo <i>portfolio</i>

	2. El usuario rellena el formulario, insertando un nombre para el <i>portfolio</i>, una descripción y un nivel de riesgo. 3. El usuario confirma los datos con el botón "Crear" 4. Se crea el <i>portfolio</i> en la base de datos y está disponible desde la vista de <i>Portfolio</i> para ser consultado por el usuario.
Secuencia alternativa	S-1: El usuario no introduce el nombre. El sistema avisa con un mensaje de error, incitando al usuario que lo introduzca.

Tabla 10 - Caso de Uso Número 8

Caso de Uso: CUN09	Eliminar <i>Portfolio</i>
Objetivo en contexto	El usuario elimina un <i>portfolio</i> ya existente desde la lista de sus <i>portfolios</i>
Entradas	El <i>portfolio</i> , ya existente
Precondiciones	El usuario debe haber iniciado sesión en la aplicación y el <i>portfolio</i> que se pretende eliminar ha de ser un <i>portfolio</i> ya existente
Salidas	N/A
Postcondición si éxito	El <i>portfolio</i> se elimina con éxito y deja de estar disponible para su visualización por parte del usuario
Postcondición si fallo	El <i>portfolio</i> no se puede eliminar por error en el sistema y el este muestra el mensaje de error pertinente
Actores	Usuario Registrado
Secuencia normal	1. Desde la vista de Potrfolio, el usuario selecciona, de la lista de <i>portfolios</i>, aquel que desea eliminar 2. Una vez en su vista, el usuario hace clic sobre el botón "Eliminar <i>portfolio</i>" 3. El usuario confirma la acción pulsando el botón "Eliminar" en el modal emergente

	4. El <i>portfolio</i> queda eliminado de la base de datos y ya no se encuentra en la lista de <i>portfolios</i> del usuario, ni está disponible para su visualización
Secuencia alternativa	N/A

Tabla 11 - Caso de Uso Número 9

Caso de Uso: CUN10	Consultar <i>Portfolio</i>
Objetivo en contexto	El usuario consulta un <i>portfolio</i> ya existente desde la lista de sus <i>portfolios</i>
Entradas	El <i>portfolio</i> , ya existente
Precondiciones	El usuario debe haber iniciado sesión en la aplicación y el <i>portfolio</i> que se pretende consultar ha de ser un <i>portfolio</i> ya existente
Salidas	Acceso a la vista de <i>portfolio</i>
Postcondición si éxito	El usuario consulta el <i>portfolio</i> con éxito en la vista de <i>portfolio</i>
Postcondición si fallo	El <i>portfolio</i> no se puede visualizar por error en el sistema o porque no existe
Actores	Usuario Registrado
Secuencia normal	1. Desde la vista de <i>Potrfolio</i>, el usuario selecciona, de la lista de <i>portfolios</i>, aquel que desea consultar 2. Al seleccionarlo, la vista de <i>portfolio</i> automáticamente muestra el <i>portfolio</i> seleccionado
Secuencia alternativa	N/A

Tabla 12 - Caso de Uso Número 10

Caso de Uso: CUN11	Editar Perfil
Objetivo en contexto	El usuario puede editar la información de su perfil y cambiarla
Entradas	N/A
Precondiciones	El usuario debe estar registrado en la aplicación
Salidas	El perfil de usuario queda modificado por los nuevos datos introducidos
Postcondición si éxito	Los datos que aparecen en el perfil del usuario han de ser los nuevos, ya habiéndose modificado los anteriores
Postcondición si fallo	Los datos del usuario no sufren ninguna modificación
Actores	Usuario Registrado
Secuencia normal	1. El usuario accede a su perfil 2. Pulsa el botón "Editar Perfil" 3. Obtiene un formulario con sus atributos que puede modificar 4. El usuario pulsa el botón de guardar y se procede a modificar los datos que ha cambiado 5. Sale mensaje de información que confirma que los datos se hayan modificado con éxito.
Secuencia alternativa	N/A

Tabla 13 - Caso de Uso Número 11

Caso de Uso: CUN12	Cambiar contraseña
Objetivo en contexto	El usuario cambia la contraseña para el acceso al sistema Horizon
Entradas	Contraseña antigua
Precondiciones	El usuario debe estar registrado en la aplicación y haber iniciado sesión en su perfil
Salidas	El perfil de usuario modificado, de modo que la contraseña de acceso ahora es nueva
Postcondición si éxito	La contraseña de acceso se cambia con éxito
Postcondición si fallo	La contraseña de acceso para el perfil de el usuario no sufre ninguna modificación
Actores	Usuario Registrado
Secuencia normal	1. El usuario accede a su perfil 2. Pulsa el botón "Cambiar contraseña" 3. El usuario introduce su contraseña antigua y confirma dos veces la contraseña nueva 4. El usuario pulsa el botón de confirmación y se aplican los cambios
Secuencia alternativa	S-1: El usuario falla en introducir la contraseña antigua S-2: El usuario falla en confirmar la contraseña nueva

Tabla 14 - Caso de Uso Número 12

Caso de Uso: CUN13	Mensaje al <i>chatbot</i>
Objetivo en contexto	El usuario envía un mensaje al <i>chatbot</i> que tiene disponible en AcademIA
Entradas	La petición del usuario
Precondiciones	N/A
Salidas	Respuesta a la petición del usuario según la capacidad que tiene la IA de hacerlo
Postcondición si éxito	El <i>chatbot</i> (API de Gemini) devolverá una respuesta a la petición del usuario
Postcondición si fallo	El <i>chatbot</i> no es capaz de ofrecer una respuesta válida o que solucione la petición problemática del usuario
Actores	Usuario Registrado
Secuencia normal	1. El usuario accede al <i>chatbot</i> en la parte inferior derecha de la página de AcademIA. 2. El usuario introduce su petición en el área de texto. 3. Pulsa el botón de enviar y recibe una respuesta a su petición
Secuencia alternativa	S-1: Fallo en la API de Gemini, de disponibilización general, que hace que el usuario no pueda utilizar el <i>chatbot</i>

Tabla 15 - Caso de Uso Número 13

Caso de Uso: CUN14	Acceder a curso didáctico
Objetivo en contexto	El usuario accede abre un curso didáctico, de la lista de cursos disponibles en el módulo de Academia
Entradas	N/A
Precondiciones	N/A
Salidas	Diapositivas del curso didáctico seleccionado
Postcondición si éxito	El usuario abre el curso didáctico en caso de que sea disponible acorde a su membresía
Postcondición si fallo	El usuario no puede abrir el curso didáctico, bien indisponibilidad de la base de datos, o bien por no tener la membresía adecuada
Actores	Usuario Registrado
Secuencia normal	1. El usuario accede al módulo de Academia 2. El usuario elige un curso de los mostrados en la lista de cursos disponibles en Academia 3. Se abre la vista de curso, mostrando las diapositivas en el centro y permitiendo al usuario poder navegar con dos botones para avanzar y retroceder
Secuencia alternativa	S-1: Fallo en la base de datos que causa indisponibilidad de los recursos (diapositivas) para el curso que se desea consultar S-2: El usuario intenta abrir un curso que es dedicado a membresías "Pro", mientras que su membresía vigente es "Gratis"

Tabla 16 - Caso de Uso Número 14

Caso de Uso: CUN15	Leer reflexión del día
Objetivo en contexto	Desde la página principal <i>dashboard</i> , el usuario hace clic sobre la reflexión del día para leer el artículo
Entradas	N/A
Precondiciones	Que el usuario haya iniciado sesión con su cuenta en el sistema Horizon
Salidas	Artículo de reflexión del día, disponible para lectura
Postcondición si éxito	Se abre con éxito la página del artículo del día donde el usuario podrá leerlo
Postcondición si fallo	No se consigue abrir la página del artículo del día y el usuario no llega a leerlo
Actores	Usuario registrado
Secuencia normal	1. El usuario accede al a la página principal del sistema, "<i>dashboard</i>" 2. El usuario hace clic sobre el recuadro de "Reflexión del día" 3. Se abre la vista de la reflexión, donde el usuario puede leer el artículo completo
Secuencia alternativa	S-1: El sistema no logra cargar los artículos de reflexión de la base de datos, mostrando el error correspondiente

Tabla 17 - Caso de Uso Número 15

Caso de Uso: CUN16	Alternar entre tema oscuro y claro
Objetivo en contexto	El usuario cambia de tema el aspecto visual del sistema Horizon, pudiendo alternarlo entre oscuro y claro
Entradas	N/A
Precondiciones	Que el usuario haya iniciado sesión con su cuenta en el sistema Horizon
Salidas	Tema de colores cambiado
Postcondición si éxito	El usuario consigue exitosamente invertir el tema de colores, pasando de claro a oscuro, o viceversa.
Postcondición si fallo	N/A
Actores	Usuario registrado
Secuencia normal	1A. El usuario dispone de un botón de acción rápida en el panel izquierdo, cuya pulsación completaría la acción de inmediato 1B. De no elegir la opción anterior, el usuario accede a la sección de Ajustes, donde podrá seleccionar el tema visual 2B. El usuario hace clic al botón de guardar ajustes para que los cambios se hagan efectivos y se realice el cambio de tema
Secuencia alternativa	N/A

Tabla 18 - Caso de Uso Número 16

Caso de Uso: CUN17	Leer noticias relacionadas con un activo
Objetivo en contexto	El usuario accede a noticias recientes relacionadas con un activo, disponibles para la lectura
Entradas	<i>Ticker</i> o nombre del activo con el que se relaciona la noticia
Precondiciones	Que el usuario haya iniciado sesión con su cuenta en el sistema Horizon
Salidas	Vista de análisis de activo, donde se encuentra disponible una sección de noticias relacionadas
Postcondición si éxito	El usuario logra acceder a la pestaña de análisis del activo seleccionado, donde puede encontrar una sección de noticias recientes relacionadas con dicho activo
Postcondición si fallo	N/A
Actores	Usuario registrado
Secuencia normal	1. El usuario accede a la pestaña de análisis de <i>ticker</i> 2. El usuario busca el activo deseado bien por su nombre, o bien por su <i>ticker</i> 3. En la vista de activo, el usuario accede a la sección de noticias relacionadas con dicho activo 4. El usuario hace clic sobre la noticia que le interese y el sistema la abrirá en una nueva pestaña
Secuencia alternativa	S-1: A falta de datos en la base de datos sobre un activo determinado, no se podrá realizar la acción

Tabla 19 - Caso de Uso Número 17

Caso de Uso: CUN18	Eliminar Cuenta
Objetivo en contexto	El usuario elimina su cuenta del sistema Horizon
Entradas	N/A
Precondiciones	El usuario tiene que estar registrado y haber iniciado sesión en su cuenta en el sistema Horizon
Salidas	N/A
Postcondición si éxito	La cuenta del usuario se borra con éxito del sistema Horizon y deja de existir, junto con todos los datos asociados en la base de datos del sistema
Postcondición si fallo	N/A
Actores	Usuario Registrado
Secuencia normal	1. El usuario accede a la vista de Perfil 2. El usuario hace clic sobre el botón "Eliminar" 3. El usuario confirma que desea eliminar su cuenta, escribiendo lo que se pide por pantalla y pulsando el botón "Eliminar cuenta" 4. La cuenta se elimina del sistema Horizon y ya no es posible volver a acceder a ella porque no existe.
Secuencia alternativa	S-1: El usuario no introduce correctamente lo que se pide por pantalla (acción de confirmación) y el sistema no permite finalizar la acción de borrado.

Tabla 20 - Caso de Uso Número 18

Caso de Uso: CUN19	Actualizar suscripción
Objetivo en contexto	El usuario actualiza su perfil, bien mejorándolo desde gratuito a pro, o bien cancelando la suscripción pro y volviendo a la gratuita
Entradas	N/A
Precondiciones	El usuario ha de tener una cuenta en el sistema Horizon y haber iniciado sesión para realizar la acción
Salidas	N/A
Postcondición si éxito	El plan cambia con éxito
Postcondición si fallo	N/A
Actores	Usuario Registrado
Secuencia normal	1. Desde la vista de su perfil el usuario hace clic sobre el botón "Actualizar Plan" 2a. Si el plan es "Gratis", será preguntado si quiere actualizar al plan "Pro" y se simulará la acción de compra con solo pulsar un botón. 2b. Si el plan es "Pro", será preguntado si quiere darse de baja y volver al plan "Gratis" y se simulará el proceso de baja con solo pulsar un botón. 3. El usuario ha conseguido cambiar de plan con éxito y sus privilegios en el sistema Horizon serán actualizados acorde al nuevo plan
Secuencia alternativa	N/A

Tabla 21 - Caso de Uso Número 19

3.4 Requisitos No Funcionales

Además de las funcionalidades descritas anteriormente, el sistema debe cumplir una serie de requisitos no funcionales que garantizan su calidad.

En términos de rendimiento, el sistema debe responder de forma ágil, estableciéndose como requisito disponer de una latencia mínima. El objetivo es que el módulo de IA se desarrolle de tal forma que no afecte al funcionamiento del sistema, no reduciendo bajo ningún concepto su tiempo de respuesta. Asimismo, se debe optimizar el uso de APIs externas mediante mecanismos de caché.

Desde el punto de vista de la seguridad, se garantiza la protección de los datos de los usuarios mediante el uso de técnicas de *hashing* para contraseñas y cifrado de claves de acceso a servicios externos. Adicionalmente, el sistema ha de ser robusto frente a intentos de ataques y soportar múltiples conexiones concurrentes.

En cuanto a la fiabilidad, el sistema debe proporcionar resultados consistentes y basados en datos verificables, especialmente en el módulo de Inteligencia Artificial. Respecto a la disponibilidad, se busca un funcionamiento continuo del sistema, gestionando de forma controlada posibles fallos en servicios externos.

Por otro lado, la mantenibilidad se garantiza mediante una arquitectura modular que separa *frontend* y *backend*, facilitando la evolución del sistema y continuidad en el desarrollo. Finalmente, la portabilidad se logra al tratarse de una aplicación web accesible desde distintos navegadores y dispositivos, adaptada para ser *responsive* ante cambios del tamaño de la pantalla.

3.5 Restricciones y Dependencias

El sistema presenta diversas restricciones derivadas principalmente de la dependencia de servicios externos, como APIs de datos financieros y plataformas de Inteligencia Artificial, que imponen limitaciones en el número de peticiones y recursos disponibles.

Asimismo, el correcto funcionamiento del sistema requiere una conexión a internet estable. Por otro lado, el diseño sigue un enfoque orientado a escritorio, aunque se ha tenido en cuenta la adaptabilidad a dispositivos con distintos tamaños.

3.6 Referencia a Apéndices

La especificación completa de requisitos software, incluyendo el detalle de los casos de uso, las interfaces del sistema y el análisis WAVE, se encuentra disponible en el [Apéndice A](#).

Capítulo 4 - Diseño de Interfaz de Usuario

4.1 Metodología de Diseño

Para el desarrollo de Horizon, hemos seleccionado la metodología de diseño centrada en el usuario, inspirada en los principios del Diseño Guiado por Objetivos (DGO). Dado que la aplicación busca reducir la barrera de entrada al mundo de la inversión y traducir conceptos financieros complejos a un lenguaje sencillo, es fundamental no basar el diseño en la implementación tecnológica, es decir, en cómo funciona el sistema por dentro, sino en las necesidades y expectativas de los nuevos inversores. Esta metodología nos permite poner a las personas en el centro de todas las decisiones, empatizando con sus frustraciones actuales para diseñar interacciones que ofrezcan una experiencia útil y comprensible.

A continuación, se detalla el proceso de diseño iterativo que hemos seguido. Todo este proceso se ha desarrollado a la par que los conocimientos adquiridos en la asignatura de Desarrollo de Sistemas Interactivos, lo que nos motivó a aplicar un ciclo de trabajo aproximado al del DGO y a utilizar herramientas de prototipados profesionales.

4.2 Investigación y Entrevistas Exploratorias

La primera etapa del proceso consistió en conocer a nuestros usuarios de primera mano. Para ello, realizamos entrevistas y tuvimos conversaciones con el propósito de obtener datos cualitativos sobre lo que busca un inversor principiante y los problemas a los que se enfrenta. Gracias a estas entrevistas, descubrimos hallazgos clave (factoides):

- Las aplicaciones similares actuales muestran demasiados datos que resultan incomprensibles para un principiante.
- Los usuarios valoran un asistente virtual accesible siempre para resolver dudas.

- Los usuarios valoran muy positivamente la estética simple, limpia y de alto rendimiento característica de los neobancos.
- Existe un gran miedo a la pérdida de poder adquisitivo y dinero real.
- Los usuarios requieren mucha atención y tiempo para consumir contenidos académicos.

4.3 Modelado de Persona

A partir de la información extraída en la investigación, procedimos a la fase de modelado creando un arquetipo de persona. Este modelo ficticio, pero muy específico, nos ayudó a representar a nuestro grupo de usuarios objetivo, encapsulando sus miedos, metas y contexto de uso.

Al analizar su contexto, identificamos una doble necesidad. Por un lado, el aprendizaje financiero requiere consumir mucha información, por lo que definimos que la aplicación debía tener un enfoque *desktop-first* (escritorio) por la cantidad de información que se comparte al usuario, los módulos de cursos y las figuras.

Por otro lado, la persona también necesita realizar consultas rápidas a lo largo del día, como comprobar la rentabilidad de sus simulaciones, por lo que determinamos que era imprescindible desarrollar también una interfaz móvil (postura temporal).

4.3.1 Arquetipo de persona primaria



Figura 3 - Mario Gómez, persona primaria

Nombre: Mario Gómez, el inversor principiante cauteloso.
(imagen generada por inteligencia artificial)

Edad y sexo: 22 años, hombre

Breve descripción: Joven profesional con capacidad de ahorro que quiere empezar a rentabilizar su dinero, pero se siente intimidado por la complejidad del mundo financiero.

Profesión: Consultor junior en consultoría tecnológica.

Dónde vive: Madrid, piso de alquiler compartido.

Salario: 24.000€ brutos anuales.

Objetivo Final: Construir una base teórica financiera sólida para su futuro y tomar mejores decisiones con su dinero sintiendo seguridad y respaldo.

Objetivos:

- Aprender paso a paso conceptos financieros complejos traducidos a un lenguaje sencillo.
- Probar qué pasaría con su dinero en un entorno de simulación sin el riesgo de perder dinero real con precios reales a lo largo de los días.
- Poder consultar sus dudas en contexto a través de un asistente de inteligencia artificial.

Frase: "Quiero invertir mis ahorros, pero las aplicaciones actuales tienen demasiados datos que no entiendo y me da miedo dar un paso en falso."

Conocimientos y habilidades:

- Nivel tecnológico alto: Usuario intensivo de móvil y ordenador. Está muy acostumbrado a interfaces modernas.
- Nivel de conocimientos financieros nulo/básico.

Motivaciones: Reducir su barrera de entrada al mundo de las inversiones, sentirse acompañado en el proceso de aprendizaje y aplicar esos conocimientos en su trabajo.

Contexto de uso:

Postura soberana: Utiliza su ordenador en casa, en un ambiente tranquilo, para concentrarse y consumir los contenidos formativos y académicos.

Postura temporal: Utiliza el móvil en momentos puntuales del día (como trayectos en transporte público) para consultas rápidas sobre la rentabilidad diaria de su simulación y ver noticias del mercado.

4.4 Wireframe de Diseños

En la fase de definición del *framework* de interacción, comenzamos trasladando las necesidades detectadas a *wireframes* de baja fidelización utilizando la aplicación web de Balsamiq.

El objetivo en esta etapa fue centrarnos exclusivamente en la estructura general, el flujo de las tareas y la disposición lógica de la información, eliminando cualquier detalle gráfico, como pueden ser los colores o las tipografías concretas. En Balsamiq estructuramos visualmente cómo se separarían los contenidos de las vistas a desarrollar, principalmente con enfoque de escritorio. Al ser prototipos rápidos y de bajo coste, nos permitieron iterar y explorar distintas alternativas sobre la arquitectura de la aplicación sin comprometernos con un desarrollo complejo.

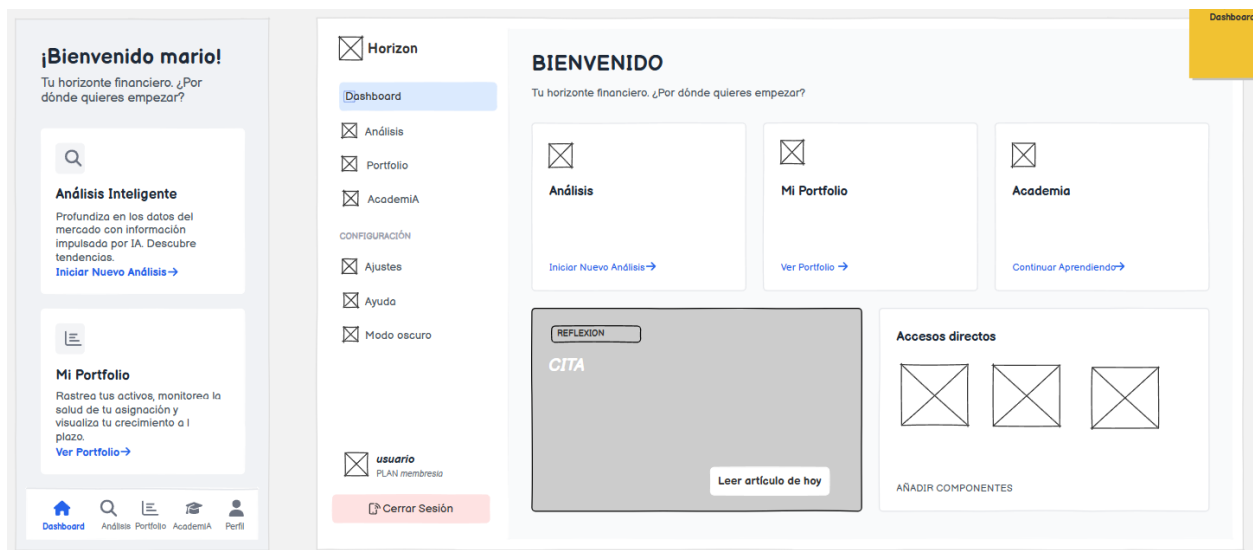


Figura 4 - Dashboard baja fidelidad

Esta primera vista, en la Figura 4, refleja la página principal de la aplicación, con diferentes accesos directos al resto de páginas como *Análisis*, *Portfolio* o *Academia*, dejando espacio para futuras páginas complementarias como *Ajustes* o *Ayuda* en la parte inferior derecha. Junto a la página se detalla la barra lateral propia de la aplicación de escritorio, donde el usuario puede cerrar sesión, acceder a su perfil, y

navegar a través de las funcionalidades, además de aplicar rápidamente un cambio entre modo oscuro y modo claro.

Se presenta también la funcionalidad de las reflexiones. Uno de los contenedores se pinta de un color distintivo, a definir en la fase de alta fidelidad, para destacar una cita aleatoria de un inversor reconocido, siguiendo esa dinámica educativa de la aplicación.

Por último, se define la vista de móvil. Optamos por una barra inferior en lugar de la barra lateral por la visibilidad que esta quitaba en pantallas tan pequeñas, incluso estando recogida. Además, era poco accesible con una mano los botones en el lado izquierdo y en orientación vertical, para tareas de postura temporal como ver rápidamente el precio de una acción hoy. Por ello decidimos incluir todo en la barra inferior, inspirándonos en aplicaciones reconocidas de móvil como X (antes Twitter), Instagram o TikTok. Para no saturar la barra, algunos accesos se eliminan, accediendo a ellos a través de la página principal (como Ajustes) y otros a través de otras páginas (como el Modo oscuro desde Ajustes).

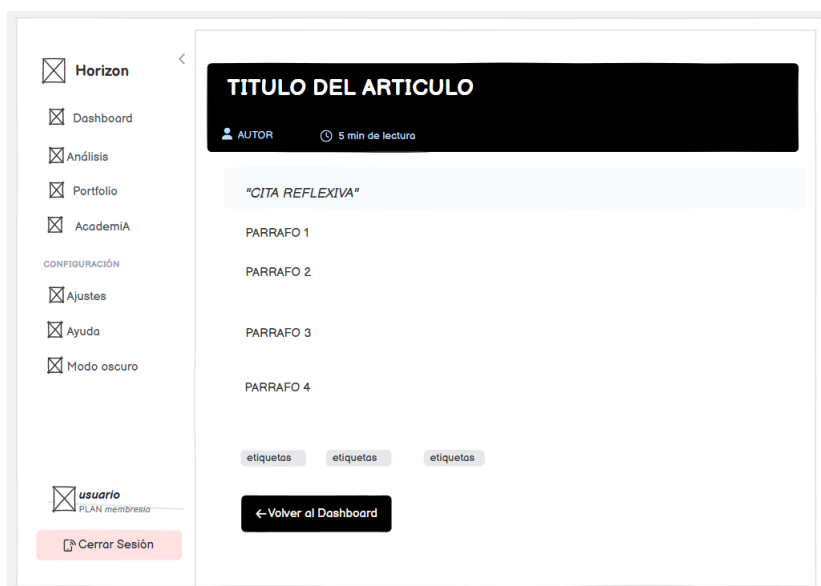


Figura 5 - Artículo reflexión baja fidelidad

Al hacer clic en “Leer artículo de hoy” se despliega la vista del artículo de reflexión, que se detalla en la Figura 5. Este diseño de baja fidelidad ubica el *banner* del

título con datos como el autor y el tiempo estimado de lectura. Después, iniciando con la cita a modo de gancho, se continúa con los párrafos del artículo. Finaliza con diversas etiquetas que pueden servir para resumir el artículo, y un botón de volver.

Dada la lectura vertical del usuario, de arriba a abajo, el botón de volver concluye el flujo de la vista y permite continuar la navegación de forma orgánica.

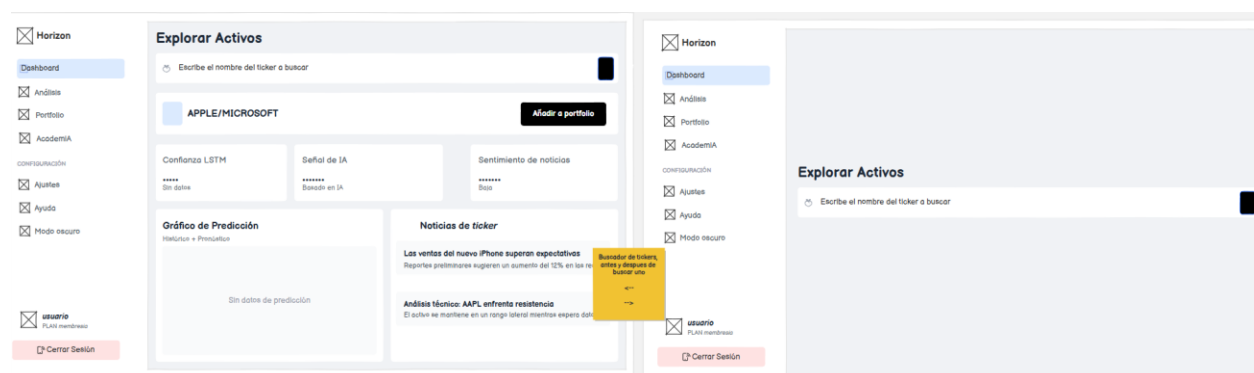


Figura 6 - Análisis baja fidelidad

Se define la vista del análisis de activo en la Figura 6. A la derecha se muestra la vista inicial, con una barra central indicando que *ticker* se quiere buscar, centrando la atención del usuario únicamente en la búsqueda. Para ello nos inspiramos en las interfaces de los chats de inteligencia artificial *online*, como ChatGPT [1], donde al comienzo de la conversación se propone una barra central que se desplaza cuando se envía el primer mensaje. De una manera similar, cuando se envía el nombre del *ticker*, la barra se desplaza para arriba, dando lugar a la vista de la izquierda, con la información relativa al activo.

La información se lee en orden, de arriba a abajo, es por ello que la información más detallada, como la XAI, se deja para el usuario más interesado que decide llegar al final.

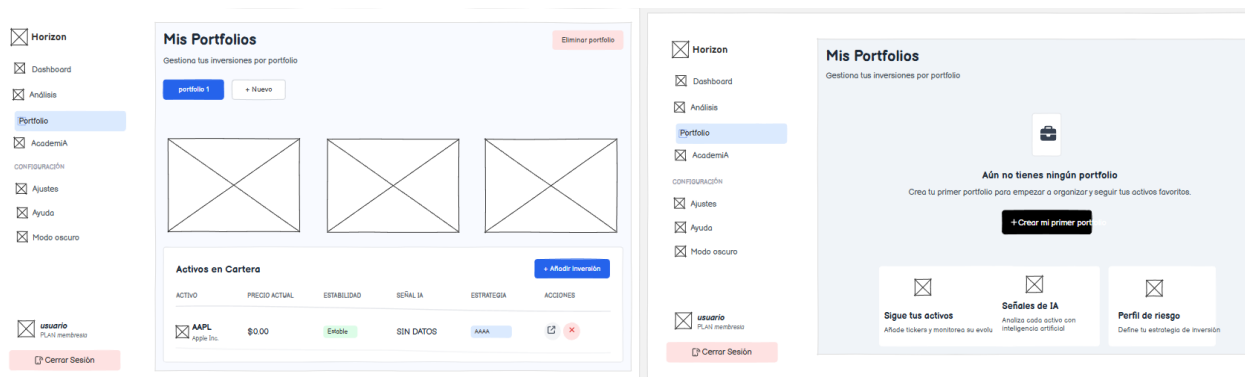


Figura 7 - Portfolios baja fidelidad

La vista de *portfolios* tiene dos representaciones, ambas en la Figura 7, una para el usuario que ya tiene *portfolios* y otra para el que no tiene ninguno todavía. Hemos decidido separarlas pensando sobre todo en el usuario nuevo, que, al no tener conocimientos previos de la aplicación, recibe unas instrucciones de uso de esta pestaña, destacando el botón de crear el primer *portfolio*.

Para el usuario ya acostumbrado, la vista es muy diferente. Las *tabs* superiores le permiten cambiar entre sus *portfolios*. Los contenedores grandes muestran información resumida del *portfolio* en el que se encuentra, seguido de una tabla con cada uno de los activos que tiene añadidos, pudiendo acceder a la vista de análisis de ese activo o eliminarlo de su *portfolio* rápidamente con dos botones.

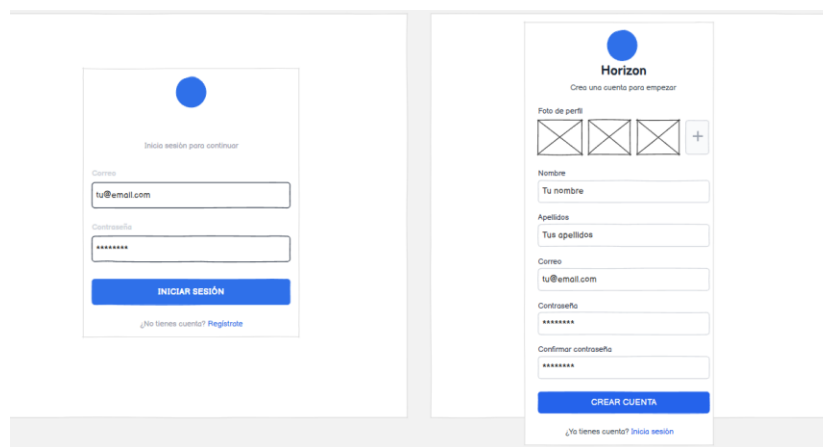


Figura 8 - Login y Registro baja fidelidad

Las páginas de inicio de sesión y registro permiten dar un primer vistazo a los formularios que deberá rellenar el usuario final. En ambos casos se incluye un texto

inferior que redirige a la otra pestaña por si el usuario se ha equivocado, pulsando iniciar sesión cuando ya tiene cuenta y viceversa. Detallado en la Figura 8.

Para el caso del registro se añaden algunos campos más, como la confirmación de la contraseña, el nombre y apellidos, o la foto de perfil, dando 3 opciones predeterminadas al usuario.

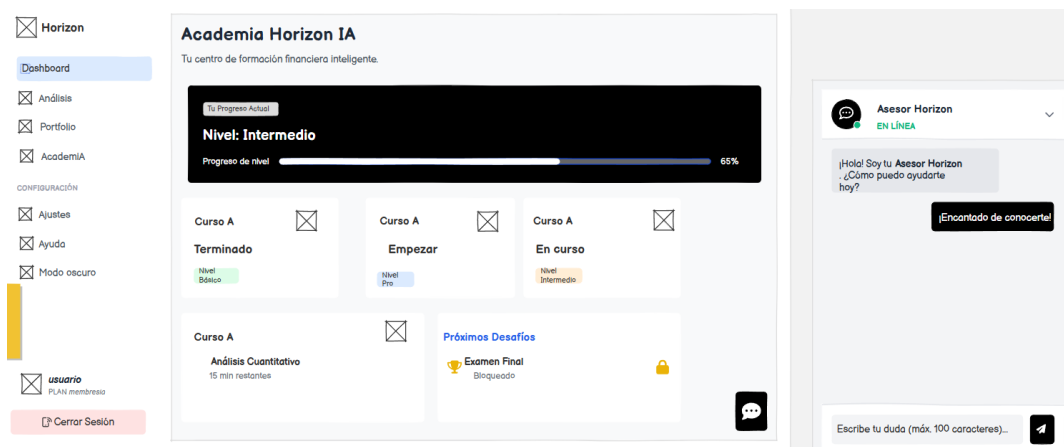


Figura 9 - Academia baja fidelidad

El apartado de la Academia tiene un comportamiento un poco diferente, con el *chatbot* desplegable, pudiéndose ver en la Figura 9. Dejando la definición de los colores para más adelante, la Academia ofrece en primer lugar un repaso del progreso del usuario, seguido de varios contenedores pulsables para acceder a los cursos.

El botón inferior despliega el chat, que se superpone en un espacio reducido para poder escribir y recibir las respuestas. Se muestran dos mensajes ya enviados para sugerir la funcionalidad del chat al usuario e introducir al asistente.

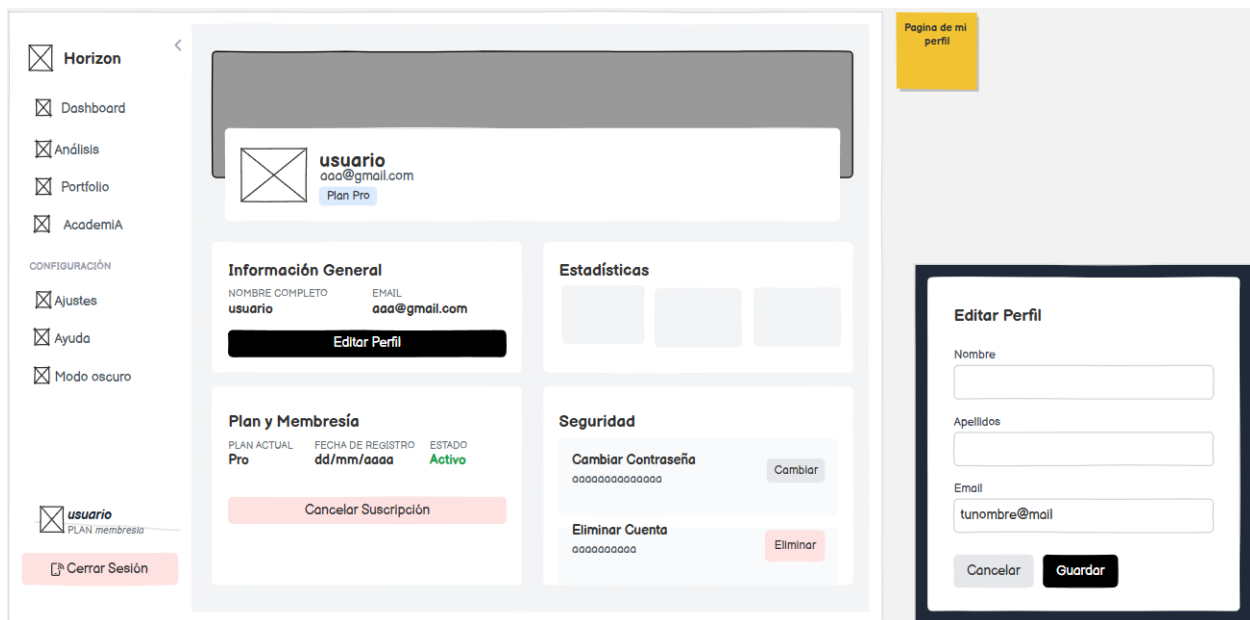


Figura 10 - Mi perfil baja fidelidad

El perfil del usuario permite ver detalles de la cuenta. En primer lugar, el nombre del usuario junto a su email y foto, pudiendo pulsar en la foto para cambiarla por otra de las predeterminadas o una personalizada, desplegando un modal. Detrás aparece un *banner*, dando un toque de personalización al perfil que permite ver rápidamente si el usuario tiene la suscripción pro o gratuita.

Más adelante puede editar su información general, como el nombre y apellidos, ver sus estadísticas, a definir en próximas iteraciones, plan de membresía y fecha de registro en la aplicación, y cuestiones de seguridad como el cambio de contraseña o eliminación completa de la cuenta.

A la derecha se muestra la vista de un modal para editar el perfil, que se usará como referencia para construir todos los modales (eliminar cuenta, editar contraseña, cambiar foto de perfil, entre otros), con detalles como los botones alineados a la izquierda, un título, y los campos rellenables con la misma forma que en inicio de sesión y registro.

4.5 Diseños de Alta Fidelidad

Una vez que la estructura y la navegación básica fueron establecidas con Balsamiq, avanzamos a la construcción del *framework* visual empleando Figma. En este apartado pasamos de la baja a la alta fidelidad.

En esta iteración nos enfocamos en detalles más concretos, introduciendo iconos más cercanos a la versión final, colores característicos de la aplicación. Tomando en cuenta los resultados de las entrevistas, aplicamos un diseño minimalista, limpio y moderno, fuertemente inspirado en el estilo de los bancos digitales que generaba tanta confianza en nuestros usuarios encuestados. Diseñamos las vistas principales introduciendo la paleta de colores, tipografías claras, y el espacio en blanco necesario para que la información financiera no resultara abrumadora. Estos prototipos interactivos en Figma nos permitieron acercarnos fielmente a lo que será el producto final y disponer de un material guía para el desarrollo del software.

Para la realización de estos diseños nos hemos centrado en las vistas principales, que incluyen la barra lateral, la página principal, mi perfil, análisis de activos, y *portfolios*. Esto es debido a que el resto de las vistas más secundarias se pueden desarrollar durante el proceso de implementación utilizando los diseños de baja fidelidad planteados y los detalles de tipografía y color de aquellas vistas de alta fidelidad.

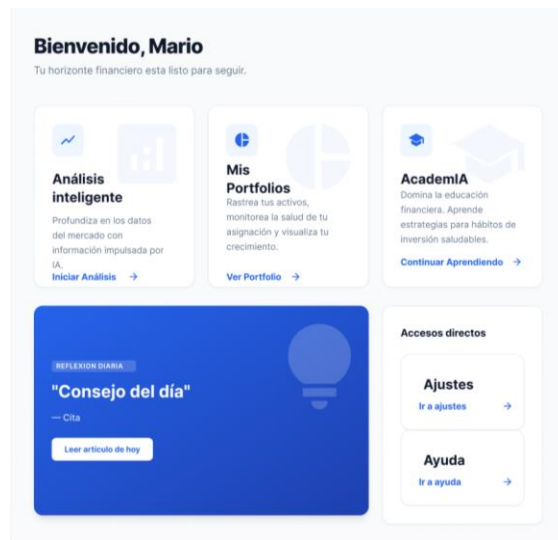


Figura 11 - Dashboard alta fidelidad

La vista de la página principal de la Figura 11 completa al diseño de baja fidelidad añadiendo los colores principales de la aplicación, iconos de fondo reactivos al cursor, iconos más acercados al diseño final y unos accesos directos completados con contenido de Ajustes y Ayuda. Se incluye una descripción breve de las páginas a visitar, salvo en estos dos últimos casos por la simplicidad de sus funcionalidades.

La funcionalidad de las citas reflexivas toma protagonismo a diferencia del resto de contenedores, con un color distinto y mayor tamaño, dado que es el único componente que no tiene una función de redirección sino una funcionalidad propia. Este texto cambia aleatoriamente al recargar la página principal.

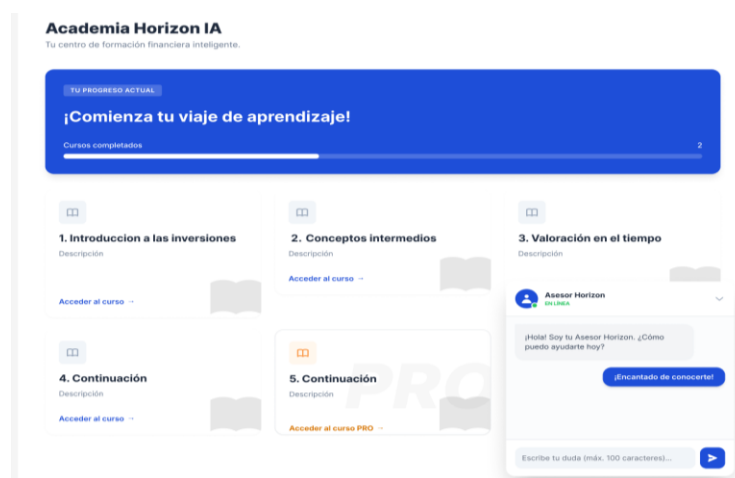


Figura 12 - AcademIA alta fidelidad

Para la AcademIA se aplican ahora los colores principales de la aplicación. Se detalla la vista en conjunto del *chatbot* superpuesto a los cursos disponibles. El componente principal indica el progreso de cursos, seguido de los distintos contenidos académicos disponibles. Se comienza a definir la identidad de la suscripción Pro, con colores naranjas, a través de los cursos de pago como el número 5 en la Figura 12.

Al igual que en el diseño de baja fidelidad, se concreta con qué mensajes inicia el asistente virtual, invitando al usuario a seguir conversando, viendo las burbujas de texto de plantilla. Se indica una limitación técnica del máximo de caracteres por mensaje.

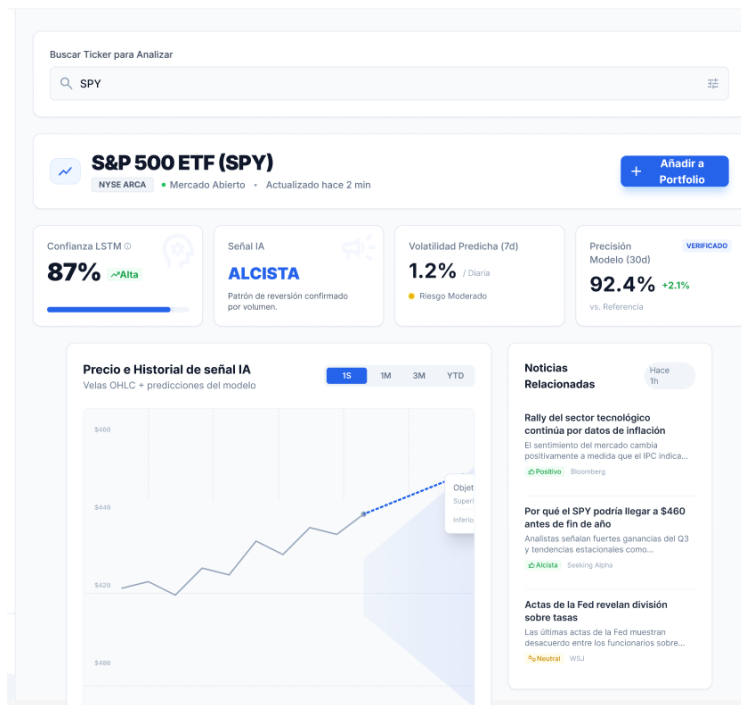


Figura 13 - Análisis alta fidelidad

La búsqueda de activos individuales en la Figura 13 muestra el activo buscado en la barra superior, confirmándose en el contenedor alargado de debajo, donde se puede añadir el activo actual a un *portfolio* ya creado desde la misma vista. Más abajo se detalla información de forma resumida, como la confianza en el modelo empleado, la señal resumida de la IA, y demás detalles susceptibles de cambio.

Aún más abajo encontramos contenido más detallado, como la gráfica de predicciones, para la que se dibuja una gráfica genérica, pero el diseño final podría cambiar por las limitaciones de librerías de dibujo de gráficas gratuitas. Idealmente se incluyen botones para personalizar los límites de dibujo de la gráfica, cuyos periodos son susceptibles de cambio.

A su derecha encontraríamos un conjunto con tres noticias recientes del activo en cuestión, con su título y un pequeño resumen, listas para acceder y redirigir a la fuente. Se indica también hace cuanto se han refrescado las noticias.

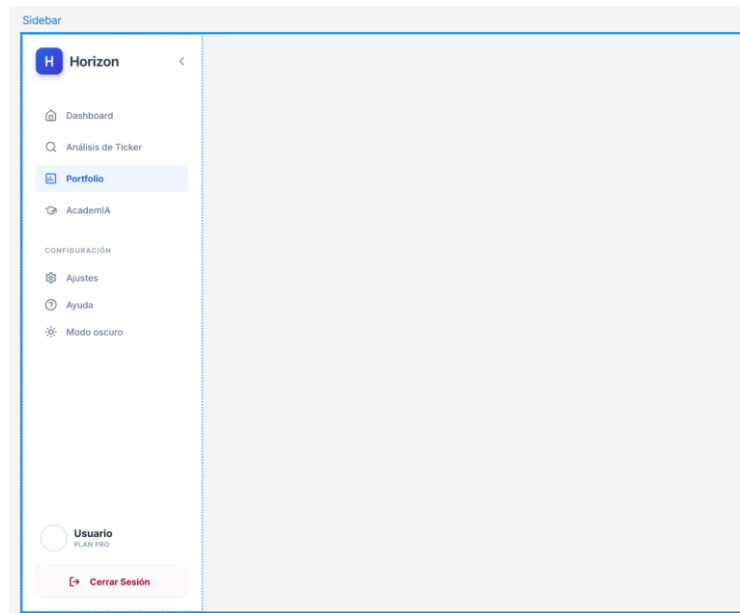


Figura 14 - Sidebar alta fidelidad

La estructura general incluye la barra lateral, en la versión de escritorio, descrita en la Figura 14. La barra lateral resume las vistas accesibles, con iconos identificativos al lado. Al estar en una de las páginas se aprecia un contenedor relleno de color azul claro, junto con el título de la página y su icono en el color primario azul de la aplicación.

Se respeta en su mayoría el *wireframe* diseñado con Balsamiq inicialmente, pero se detalla cómo se verá el logo (versión azul para el modo claro), botón de cerrar sesión más fiel y personalizado con icono y color rojo, e iconos para todas las vistas definidos. Permite aproximarse mucho más al diseño final y guiar con mayor fidelidad la creación de la barra lateral.

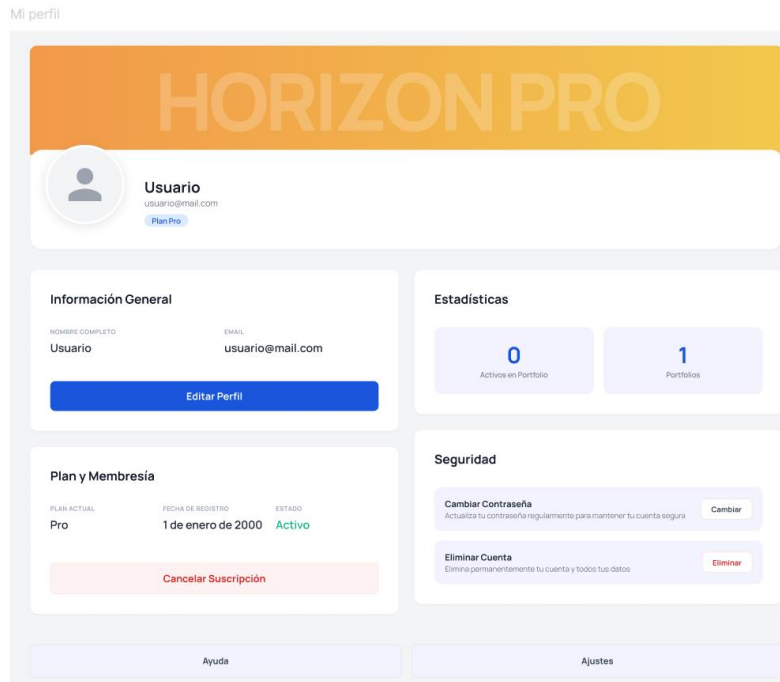


Figura 15 - Mi perfil alta fidelidad

Por último, se detalla la vista de alta fidelidad de mi perfil en la Figura 15. Se muestra, como se indicó en la iteración de baja fidelidad, información relativa al perfil del usuario, nombre y apellidos, correo electrónico, estadísticas de activos y membresía.

En primer lugar, se continúa definiendo esa diferencia visual entre la suscripción Pro y Gratis, a través de una personalización en el *banner* del perfil.

Un diseño muy continuista respecto al de baja fidelidad, que plantea el uso de dos botones inferiores para acceder a funcionalidades de configuración y ayuda, y aplicando los colores de la aplicación reutilizados en otras vistas.

Capítulo 5 - Tecnologías y Herramientas

La consecución de los objetivos definidos en el proyecto Horizon requiere la selección de tecnologías y herramientas que permita una integración fluida entre los modelos de DL y la interfaz de usuario, garantizando al mismo tiempo un entorno de ejecución eficiente y escalable. A continuación, se detallan y justifican las herramientas, lenguajes y librerías que conforman la arquitectura del sistema.

5.1 Lenguajes de Programación

El núcleo del proyecto se sustenta en lenguajes estándar que aseguran un alto rendimiento y una amplia compatibilidad:

- Python: Lenguaje de propósito general seleccionado como el pilar fundamental del *backend* y del módulo de IA. Su elección se fundamenta en su indiscutible liderazgo en el ecosistema de *Data Science* y ML, ofreciendo soporte nativo para el cálculo y la manipulación de datos requeridos por los modelos predictivos.
- HTML5, CSS3 y JavaScript (Vanilla): Para el desarrollo del *frontend*, se ha optado por prescindir de *frameworks* pesados (como React o Angular), utilizando tecnologías web estándar (Vanilla JS). Esta decisión arquitectónica prioriza el control absoluto sobre el *Document Object Model* (DOM), la optimización del rendimiento en la carga de vistas dinámicas y la construcción de un sistema de diseño propio y ligero.

A través de estos lenguajes se nos permite poner en práctica lo aprendido en asignaturas del Grado de Ingeniería Informática, como Aplicaciones Web o Tecnologías de la Programación, usando lenguajes aprendidos como los dedicados al *frontend*, y pivotando hacia lenguajes más actuales y relacionados con el mundo de los datos como Python, buscando documentación y sustentando nuestro entendimiento en lenguajes estudiados en la carrera como Java o C++.

5.2 Módulo de Inteligencia Artificial y Procesamiento de Datos

Para dar soporte a todo el ciclo de vida del dato, la arquitectura predictiva se ha consolidado empleando el siguiente ecosistema de librerías especializadas en Python:

NumPy y Pandas (numpy, pandas): Constituyen la base matemática y estructural del sistema. NumPy aporta la eficiencia del cálculo matricial y vectorial subyacente, mientras que Pandas provee la estructura de *DataFrames*, esencial para el manejo, alineación temporal y limpieza de las series históricas financieras antes de su paso por el modelo.

XGBoost (xgboost>=2.0.0): Constituye el motor predictivo principal de la arquitectura. Se ha seleccionado este algoritmo de ensamblaje basado en árboles de decisión (Gradient Boosting) por su alta eficiencia computacional y su robustez frente al ruido inherente a los mercados financieros. Para capturar el contexto temporal sin necesidad de recurrir a complejas redes neuronales recurrentes, XGBoost se alimenta de características multiescala (último valor, media, desviación típica y tendencia lineal) extraídas estadísticamente sobre subventanas de datos cortas, medias y completas.

SHAP (SHapley Additive exPlanations): Librería crítica para el componente de XAI de Horizon. Dado que los algoritmos de ensamblaje operan a menudo como "cajas negras", SHAP aplica teoría de juegos cooperativos para calcular la contribución marginal de cada variable en una predicción concreta. Esto permite al sistema generar explicaciones interpretables para el usuario (identificando qué indicadores técnicos han impulsado una tendencia alcista o bajista), fomentando la transparencia y el carácter educativo de la plataforma.

yfinance (yfinance>=0.2.28): Herramienta integrada en la etapa de extracción del *pipeline* de datos. Garantiza la descarga automatizada y estructurada de las series temporales de precios y volúmenes de negociación directamente desde la API de Yahoo Finance, segmentando la profundidad del histórico según la naturaleza estable o volátil del activo.

pandas_ta (pandas_ta>=0.3.14b): Extensión dedicada al Análisis Técnico cuantitativo. Se emplea para el cálculo determinista de la batería de indicadores y

osciladores financieros (tales como el RSI, MACD o las Bandas de Bollinger), los cuales actúan como variables de entrada base antes de aplicar las agregaciones multiescala.

scikit-learn (scikit-learn>=1.3.0): Proporciona la infraestructura necesaria para el preprocesamiento de datos (mediante escaladores para normalizar distribuciones) y la validación del sistema. Resulta vital para la implementación de metodologías de evaluación estrictas, como la validación secuencial *walk-forward*, y el cálculo de la Precisión Equilibrada (*Balanced Accuracy*), métrica de referencia del proyecto

Joblib (joblib): Herramienta empleada para la serialización, compresión y persistencia en disco de los artefactos generados tras el entrenamiento (tales como el propio modelo XGBoost, los escaladores de variables y los diccionarios de umbrales estáticos). Garantiza una carga eficiente de los modelos en memoria durante las peticiones de inferencia en tiempo real desde el *backend*.

Google-genai: Librería oficial de Google desarrollada para interactuar de manera nativa con sus modelos de lenguaje de última generación, como es la familia Gemini. En el contexto de la arquitectura, se utiliza en el *backend* para gestionar las peticiones hacia la API de Google AI Studio, permitiendo integrar capacidades de IA generativa en el asistente conversacional.

Estrategia de Etiquetado y Segmentación: La lógica de entrenamiento se sustenta en un sistema de etiquetado por umbrales calibrados individualmente para cada activo financiero (*frozen thresholds*). Estos umbrales determinan la clase del dato según el retorno porcentual esperado: retornos inferiores al límite bajo se etiquetan como tendencia Bajista (0), retornos superiores al límite alto como Alcista (2), y el rango intermedio como Lateral (1).

Asimismo, el procesamiento diferencia los activos según su naturaleza operativa: para *tickers* estables (como AAPL o KO) se emplean ventanas de observación de 30 días, mientras que para activos de alta volatilidad (como TSLA o criptoactivos) la ventana se extiende a 60 días para capturar mejor la variabilidad del mercado.

5.3 Tecnología del Servidor (Backend)

Se ha seleccionado FastAPI como el *framework* principal para el desarrollo de la API REST del servidor. Esta decisión se justifica por su capacidad de situarse, en términos de rendimiento y *benchmarks* independientes, al nivel de frameworks tradicionalmente asociados a lenguajes compilados como Node.js o Go.

A diferencia de alternativas síncronas como Flask o Django, el diseño de FastAPI se basa de forma nativa en el estándar ASGI, permitiendo aprovechar íntegramente las capacidades *async/await* de las nuevas versiones de Python (3.10+). Esta propiedad asíncrona es un punto importante para gestionar múltiples peticiones concurrentes, como solicitudes de predicción de activos o consultas masivas de datos de mercado sin bloquear el hilo de ejecución principal del servidor.

Para la ejecución de la aplicación, se ha optado por Uvicorn, el servidor ASGI de referencia en la actualidad y el recomendado oficialmente por la documentación de FastAPI para entornos de producción. Su superioridad frente a servidores WSGI tradicionales, como Gunicorn o Waitress, se basa en su implementación técnica sobre uvloop y httptools. Uvloop es una reimplementación de alto rendimiento del bucle de eventos asíncronos basada en libuv (la misma librería usada por Node.js). En cuanto a httptools, consiste en un *parser* HTTP extremadamente eficiente escrito en C.

Uvicorn soporta varios procesos en paralelo y la recarga automática durante el desarrollo (`--reload`), lo que agiliza el ciclo de iteración. Su correcta implementación del protocolo ASGI garantiza la compatibilidad total con todas las características asíncronas de FastAPI.

La integridad de la información en el sistema se gestiona a través de Pydantic, librería que constituye el núcleo del sistema de validación y serialización de datos del servidor. Dado que Python es un lenguaje de tipado dinámico, la ausencia de una capa de validación explícita podría comprometer la lógica de negocio o la estabilidad de los modelos de predicción ante datos malformados.

La elección de Pydantic se fundamenta en su capacidad para forzar los tipos declarados en tiempo de ejecución. Cada modelo definido bajo esta librería actúa

como un esquema de datos auto validado que desplaza la responsabilidad de la verificación al borde del sistema (la capa de entrada). Si una petición entrante no cumple estrictamente con el contrato definido, el sistema rechaza automáticamente la solicitud devolviendo un error HTTP 422 (*Unprocessable Entity*), protegiendo así las capas internas de datos inconsistentes. Asimismo, esta herramienta elimina el código repetitivo (*boilerplate*) al gestionar automáticamente la conversión bidireccional entre objetos Python y formato JSON, e integra validaciones específicas de formato, como correos electrónicos, de manera nativa.

5.4 Base de Datos, Autenticación y Almacenamiento

La selección de Supabase como plataforma de base de datos y autenticación se debe a su característica *Backend-as-a-Service* (BaaS) construido sobre PostgreSQL. En comparación a alternativas como Firebase (Google) o AWS Amplify, Supabase ofrece una ventaja para este proyecto como es la transparencia total del modelo de datos al trabajar directamente sobre una base de datos relacional estándar, sin abstracciones propietarias que dificulten la portabilidad del sistema. Esta característica ha sido determinante para el desarrollo del proyecto, dado que el sistema maneja un esquema de datos complejo que incluye múltiples tablas interrelacionadas mediante claves foráneas y restricciones de integridad.

A diferencia de las metodologías tradicionales enfocadas en la utilización de herramientas de administración local como phpMyAdmin, que requieren la exportación de forma manual de *scripts* SQL para sincronizar los datos alojados en la base de datos entre los diferentes desarrolladores de una forma muy ineficiente y arriesgada, Supabase aporta una instancia de PostgreSQL en la nube que permite que todos los miembros del proyecto puedan trabajar a la vez asegurando que cualquier modificación de la estructura de las tablas de los datos de prueba sea accesible inmediatamente para el resto de miembros del proyecto.

Para la gestión de la identidad y el control de acceso en la aplicación, se ha delegado íntegramente estas funciones en el módulo Supabase Auth. Esta decisión se fundamenta en la necesidad de contar con un servicio de autenticación centralizado, seguro y escalable que se integre de forma nativa con el motor de base de datos relacional de la plataforma. Esta herramienta también da fuerza a la arquitectura del sistema de acuerdo con el principio de mínimo privilegio, así como una clara segregación de permisos que asegure la integridad de los *portfolios* simulados y los datos sensibles de los usuarios. Las especificaciones técnicas orientadas se concretarán con mayor detalle en los capítulos posteriores.

Para complementar la infraestructura de datos, se ha incorporado Supabase Storage, un servicio de almacenamiento de objetos escalable e integrado de forma nativa en la plataforma. Este módulo permite al sistema gestionar archivos no estructurados, como las imágenes de perfil de los usuarios, dentro del mismo ecosistema de seguridad que la base de datos. Su elección se justifica por la capacidad de centralizar la administración de recursos multimedia bajo políticas de acceso unificadas, asegurando una gestión eficiente de los activos digitales sin necesidad de recurrir a proveedores externos adicionales.

Se ha delegado pequeñas partes del procesamiento directamente al motor de base de datos mediante el uso de funciones almacenadas de PostgreSQL, para funciones muy básicas con el objetivo de probar nuevas herramientas BaaS. Al ejecutar la lógica en el lado del servidor de datos, se reduce la latencia derivada del tráfico de red entre el *backend* y la base de datos, permitiendo que el motor de PostgreSQL gestione de manera nativa tareas de manipulación y validación de registros.

5.5 Entorno de Desarrollo y Despliegue

La gestión del código fuente se realiza mediante el sistema de control de versiones Git, utilizando GitHub como plataforma de alojamiento centralizado. Se ha adoptado el flujo de trabajo GitFlow, el cual organiza la evolución del software en dos

ramas principales con responsabilidades diferenciadas siendo estas la rama *developer* y la rama *master*. La rama *developer* actúa como el entorno de integración donde se realiza el desarrollo activo y se consolidan las nuevas funcionalidades. La rama *master* se reserva exclusivamente para el código estable, verificado y relativamente listo para producción.

La integración de código desde *developer* hacia *master* solo se produce tras comprobar el correcto funcionamiento de las implementaciones. Para el seguimiento de este progreso, se emplea el estándar de versionado semántico (MAJOR.MINOR.PATCH). Bajo este esquema, el proyecto inició en la versión 0.0.0 y avanzó mediante incrementos de parche (0.0.1, 0.0.2) para correcciones menores, mientras que la incorporación de nuevas funcionalidades funcionales elevó el número de versión menor (0.1.0, 0.2.0).

Para garantizar la seguridad del proyecto, se han implementado medidas de protección de datos. Se emplean archivos de configuración de entorno para la administración de credenciales sensibles, como las claves de acceso a Supabase y servicios externos. Estos ficheros se mantienen fuera del control de versiones mediante reglas de exclusión, evitando la exposición de datos críticos en el repositorio. A través del archivo `.gitignore`, se excluyen deliberadamente los archivos de gran tamaño generados durante el entrenamiento de los modelos de inteligencia artificial, específicamente los pesos en formatos `.pth` y `.pkl`.

Para asegurar la calidad, el flujo de trabajo se complementa con herramientas de análisis estático como Pylint y la configuración de *hooks* de *pre-commit*. Estos automatismos verifican el cumplimiento de los estándares de codificación y la ejecución de pruebas antes de cada integración, garantizando la integridad de los flujos del sistema antes de su subida al repositorio.

Capítulo 6 - Desarrollo/Implementación

6.1 Estructura y Arquitectura del Proyecto

Horizon se ha desarrollado siguiendo una arquitectura de cuatro capas: *frontend* (presentación), *backend* (controlador y servicios) y datos. A continuación, se describe la organización de los directorios y el patrón de diseño empleado.

6.1.1 Organización de Directorios

A continuación, se muestra una captura de pantalla de la raíz del proyecto desde Visual Studio Code (VS Code), junto a la razón de ser de cada uno de los archivos y carpetas.

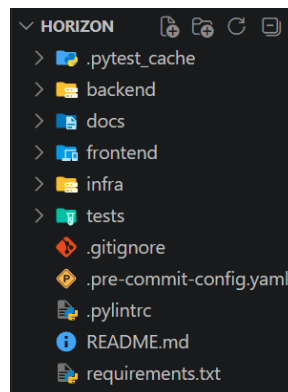


Figura 16 - Raíz del proyecto

- backend/

Núcleo del servidor de Horizon, implementado con FastAPI. Expone la API REST consumida por el *frontend*. Residen los *endpoints* y la lógica que integra la autenticación de sesión, los servicios de negocio (predicciones, *portfolios*, chat IA, cursos), el acceso a datos, y el soporte para el *pipeline* de ML/XAI cuando aplica.

- frontend/

Interfaz de usuario que contiene las páginas HTML de la plataforma (*Dashboard*, *Análisis de activos*, *Portfolios*, *AcademIA*, *Perfil*, *Login/Registro*, etc.), los estilos CSS que definen los componentes que se introducen, y los *scripts* JavaScript que gestionan los datos dinámicos, almacenan/usan el *token* de sesión, y realizan llamadas a los *endpoints* del *backend*.

- docs/

Contiene la documentación del proyecto. Define la arquitectura de la aplicación, flujos principales (por ejemplo, *login*, consulta de activos, gestión de *portfolios*, chat IA), y notas técnicas relevantes como la definición de la base de datos. Es la referencia para mantener consistencia en la evolución del sistema.

- infra/

La carpeta /infra concentra la capa de despliegue y la infraestructura técnica de Horizon, separándola del código de aplicación para mantener claramente diferenciadas las decisiones funcionales de las relativas a su puesta en marcha.

Su objetivo es facilitar una ejecución portable del proyecto, definiendo tanto la construcción y ejecución del *backend* como la publicación del *frontend* y el enrutado de las peticiones hacia la API. Para ello, integra la documentación de uso y los recursos necesarios para levantar el entorno en Docker, con un *proxy* inverso Nginx que centraliza el acceso, sirve la interfaz y redirige las llamadas a la API. La configuración está además orientada a conservar compatibilidad con el desarrollo local y a permitir un despliegue local accesible por IP dentro de la misma red, de modo que la aplicación pueda ejecutarse en el ordenador anfitrión y consumirse desde otros equipos sin cambios funcionales.

No se prevé una entrega con dominio público dada la indisponibilidad total que requeriría sobre los equipos informáticos de los integrantes, pero se plantea la implementación básica. La puesta en marcha se detalla en el [Capítulo 7 – Manual de Usuario](#).

- tests/

Pruebas automatizadas para validar el comportamiento del sistema. Contiene la estructura de distintos tipos de *tests* y documentación de soporte sobre cómo ejecutarlas e interpretar resultados.

Contempla varias dimensiones: pruebas unitarias, que validan los *endpoints* aislados, pruebas de integración, que validan el funcionamiento conjunto de módulos y flujos, y tareas de linting/validación estática para mantener consistencia y calidad del código, que salta cuando se modifica algo de código Python en *backend/*. Se tomó esta decisión técnica por la fragilidad del lenguaje Python para detectar errores de tipado y mantener la legibilidad. Además, incluye material complementario orientado a diagnósticos y experimentación controlada de los modelos ML, así como *scripts* auxiliares para facilitar la ejecución de estas verificaciones en entornos de desarrollo.

- .git/

Metadatos del repositorio Git, que incluye el historial, las ramas, y referencias, entre otros. No forma parte del producto desplegable, pero es clave para el control de versiones. Es una carpeta oculta.

- .pytest_cache/

Carpeta generada por pytest para acelerar la ejecución de *tests*. No es código fuente y se incluye en *.gitignore* para no subirla al repositorio.

- README.md

Documento de entrada al proyecto. Explica qué es Horizon, cómo instalar dependencias y cómo ejecutar *backend/frontend*. Es el manual de usuario básico.

- requirements.txt

Dependencias Python necesarias para ejecutar el *backend* (FastAPI + librerías asociadas a servicios, datos y/o ML). Permite reproducir el entorno con `pip install -r requirements.txt`.

- .gitignore

Reglas para excluir del repositorio ficheros generados o sensibles (como .env, cachés, salidas de *tests*, etc.).

- .pylintrc

Configuración del *linter* pylint usado en los *test* de lintado. Mantiene consistencia y calidad del código Python del *backend*.

- .pre-commit-config.yaml

Configuración de *pre-commit hooks* para automatizar controles antes de cada *commit*. Ejecuta las pruebas definidas antes de subir cualquier cambio al repositorio, para asegurar que los *endpoints* y flujos siguen funcionando correctamente.

6.1.2 Patrón de Diseño Empleado

Se ha empleado un enfoque basado en arquitectura por capas, donde el *backend* se estructura siguiendo el patrón Controller–Service–DAO, y el *frontend* se mantiene desacoplado consumiendo los servicios mediante una API REST. Este diseño permite encapsular el funcionamiento de cualquiera de las capas sin que afecte al resto. Por ejemplo, cambiar la tecnología de la base de datos solo afectaría al funcionamiento de la capa DAO, y por tanto solo requeriría cambiar el acceso a los datos.

La capa de controladores (Controllers) corresponde a los *endpoints* expuestos por el *backend*. Su función es actuar como frontera de entrada, gestionando el enrutado HTTP, la validación básica de peticiones y la construcción de respuestas, delegando el trabajo real en la capa de servicios.

La capa de servicios (Services) agrupa la lógica de negocio. Orquesta los casos de uso del sistema (por ejemplo, autenticación, gestión de *portfolios*, predicciones, chat, etc.) y coordina las operaciones necesarias, manteniendo la coherencia funcional entre distintos *endpoints*.

La capa de acceso a datos (DAOs) encapsula el acceso a la información (consultas y operaciones CRUD), evitando que controladores o servicios dependan de detalles concretos de la base de datos o del proveedor.

Con el objetivo de ilustrar de manera tangible la aplicación práctica del patrón de diseño arquitectónico descrito anteriormente, en la siguiente figura se presenta el diagrama de clases correspondiente al módulo central de Gestión de *Portfolios*.

Figura 17- Diagrama de Clases de Portfolio

6.2 Implementación del *Frontend*

6.2.1 Arquitectura y Organización del *Frontend*

El *frontend* se organiza bajo un patrón modular de carpetas, orientado a la separación de responsabilidades:

- Vistas (./ raíz): En el directorio raíz se ubican los documentos HTML independientes para cada flujo de usuario (como index.html, analysis.html o portfolio.html). Esta separación física simplifica el enrutamiento y asegura que cada página sea semánticamente independiente, cargando sus dependencias justas antes del cierre de la etiqueta <body>.
- Estilos (/css): La carpeta /css contiene la hoja de estilos global (styles.css), encargada de definir el sistema de diseño nativo y las directivas de responsividad.
- Lógica (/js): La carpeta /js alberga *scripts* fuertemente desacoplados y orientados a eventos, diferenciando claramente entre núcleos de configuración (config.js), controladores globales de la interfaz compartida (layout.js) y controladores específicos que encapsulan la lógica estricta de cada vista particular activa.

Dicha organización se puede observar en la siguiente Figura:

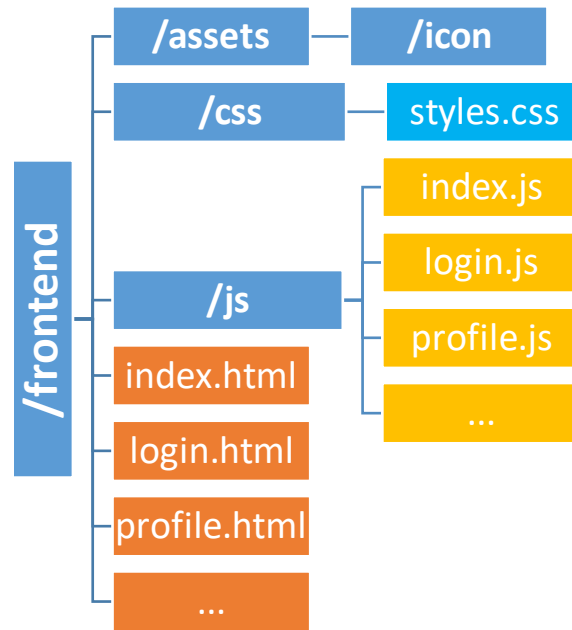


Figura 18 - Organización del directorio `/frontend`

6.2.2 Maquetación Semántica y Diseño Adaptativo

La estructura base de todas las vistas se apoya en el uso estricto de etiquetas semánticas de HTML5 (tales como `<header>`, `<nav>`, `<main>` y `<aside>`). Esta práctica no solo facilita la indexación en motores de búsqueda, sino que mejora la accesibilidad (cumplimiento de estándares para lectores de pantalla) y optimiza la estructuración del árbol DOM para su posterior manipulación transversal con JavaScript.

Para resolver la adaptabilidad a diferentes dispositivos (*Responsive Design*), se ha implementado una estrategia *desktop-first* fuertemente apoyada en los módulos de diseño CSS Grid y Flexbox. Se ha prestado especial atención, además, para cuidar malas prácticas de programación en CSS como, por ejemplo, uso indebido de `!important` en componentes. Esto permite resolver interfaces complejas sin recurrir a posicionamientos absolutos frágiles. Un ejemplo crítico de esta implementación se encuentra en la navegación principal del sistema (`#sidebar`). En resoluciones de escritorio, este componente actúa como un panel lateral de altura completa. Sin embargo, mediante la declaración de la *media query* `@media (max-width: 768px)`, las propiedades CSS

hacen posible la adaptación visual: se oculta el panel lateral (sidebar) y aparece menú inferior de navegación (bottom-navbar), optimizando al máximo el espacio superior disponible en las pantallas de los dispositivos móviles.

6.2.3 Sistema de Estilos y Personalización Dinámica

La gestión de la identidad visual de Horizon y su consistencia a través de las distintas vistas se organiza mediante el uso de variables CSS. Estas variables se definen globalmente en la pseudoclase `:root` del archivo `styles.css`, encapsulando la paleta de colores corporativa (por ejemplo, `--color-primary: #2563EB`), las tipografías principales (`-font-sans: 'Inter'`) y las escalas de espaciado. Este enfoque arquitectónico evita la duplicación masiva de código hexadecimal y permite alterar el aspecto del sistema modificando un único punto de origen.

El detalle técnico más destacable en este módulo es la implementación nativa y altamente optimizada del Modo Oscuro. En lugar de emplear hojas de estilo `.css` alternativas, la lógica (centralizada en el archivo `config.js`) se resuelve en el lado del cliente utilizando la función `window.applyThemePreference()`. Este método inyecta dinámicamente el atributo `data-theme="dark"` directamente en el nodo raíz de la página (`document.documentElement`). Al aplicar este atributo en el nivel más alto del DOM, los selectores CSS específicos reasignan localmente los valores de las variables (haciendo, por ejemplo, que `--color-surface` tome el valor `#111827`), invirtiendo la interfaz de forma instantánea gracias al motor de renderizado del navegador. Además, para notificar a librerías externas o gráficos complejos sobre este cambio, el *script* emite un evento personalizado a nivel de ventana invocando `window.dispatchEvent(new CustomEvent('horizon:theme-changed'))`, mientras que la preferencia del usuario se persiste a través de la API `localStorage` haciendo uso de la constante centralizada `window.STORAGE_KEYS.THEME`.

6.2.4 Lógica de Interfaz y Manejo de Eventos (Vanilla JS)

Para evitar la duplicación de código en la gestión de aquellos elementos que persisten a lo largo de toda la aplicación, el archivo `layout.js` asume el rol de controlador principal del entorno. Su ejecución se condiciona a la validación `document.readyState === 'loading'` y a la escucha del evento `DOMContentLoaded`, lo que garantiza que el árbol DOM esté completamente construido antes de intentar acceder a cualquier nodo o inicializar los escuchadores de eventos.

La interacción dinámica se gestiona mediante la asignación directa de escuchadores de eventos e inyección de Template Strings de ES6. Por ejemplo, el controlador inyecta el menú lateral y sincroniza los datos visuales del usuario autenticado llamando a `window.getCurrentUserData()`, para posteriormente modificar la propiedad `textContent` del nodo `#user-name`. El comportamiento colapsable del menú se controla asignando un escuchador de eventos (`addEventListener`) al botón `#sidebarToggle` que alterna de forma binaria la clase `.collapsed` sobre el contenedor principal, delegando en CSS el cálculo de las transiciones fluidas. Asimismo, el *script* identifica la página actual analizando el objeto `window.location.pathname` y aplica la clase `.active` al nodo de navegación correspondiente, brindando feedback visual sin requerir la recarga del componente.

6.2.5 Comunicación con el Backend y Seguridad

La interacción entre el cliente y el servidor se basa en una API REST, cuyo flujo de seguridad se gestiona íntegramente en la capa cliente. Para dotar al ecosistema de escalabilidad en sus despliegues, el archivo `config.js` expone el objeto global `window.API_BASE`, el cual evalúa en tiempo real la propiedad `window.location.hostname`. Si detecta que la ejecución proviene de un entorno local (como `localhost` o `127.0.0.1`), enruta automáticamente las peticiones hacia el servidor de desarrollo, y en caso contrario, apunta hacia el clúster de producción.

Toda la comunicación de datos se implementa utilizando la API nativa `fetch` bajo el patrón de concurrencia `async/await`, garantizando que la ejecución de peticiones complejas no bloquee el hilo principal (`main thread`) del navegador. El núcleo de seguridad de este proceso reside en la función envoltorio `window.fetchWithAuth()`. Este método intercepta todas las peticiones a rutas protegidas, recupera el JSON Web Token almacenado en `localStorage` mediante `window.getAccessToken()` e inyecta la cabecera `Authorization, Bearer`, reconstruyendo el objeto de opciones de red mediante el operador `spread` (`options.headers`). Adicionalmente, esta función centraliza el manejo de errores: si el servidor retorna un estado HTTP 401 Unauthorized, el bloque `catch` intercepta el error globalmente, purga los datos locales invocando `window.clearAuthSession()` y fuerza una redirección inmediata llamando a `window.location.replace('login.html')`. El uso del método `replace` es una decisión de seguridad deliberada, ya que sobrescribe el historial del navegador e impide que un usuario no autenticado pueda intentar volver a una vista protegida pulsando el botón de retroceso.

6.2.6 Vistas Principales de Usuario

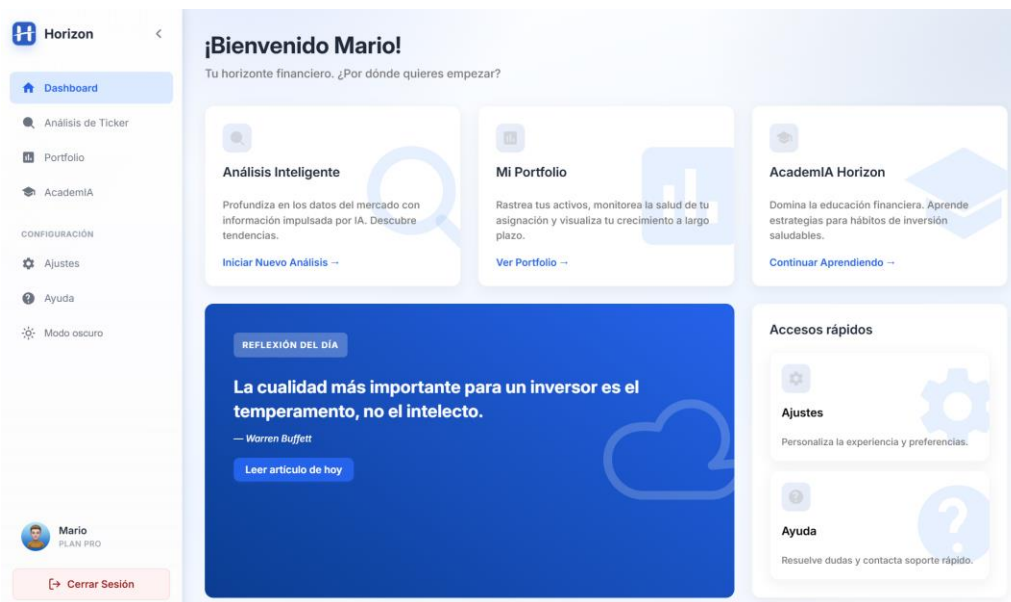


Figura 19 - Vista de Dashboard Principal (`index.html`)

El panel de control actúa como el nodo central y punto de entrada al sistema Horizon tras la autenticación. Su arquitectura visual combina la navegación global inyectada asincrónicamente por el *script* layout.js, el cual construye el #sidebar mediante la función initSidebar(), con una estructura de rejilla adaptativa definida en styles.css.

A nivel de implementación, la sección de opciones principales (.opts-section) emplea un sistema de CSS Grid configurado con la propiedad grid-template-columns: repeat(3, 1fr) y un gap: var(--spacing-xl). Esta configuración técnica es la que permite que las tarjetas de acceso rápido (Análisis, Portfolio, Academia) se reorganicen de forma fluida y automática según el ancho del contenedor #dashboard-contenido, con ajustes responsivos posteriores a través de *media queries*.

Los elementos dinámicos de la vista funcionan como nodos receptores que se hidratan tras la carga del DOM. Se tienen los siguientes:

- Mensaje de Bienvenida: el sistema valida el token y carga la sesión del usuario mediante `window.validateAuthToken()` y `window.setCurrentUserData(result.user)`. Técnicamente, se localiza el nodo con el identificador #mensaje-bienvenida y se manipula su propiedad `textContent` utilizando una template string de ES6 para inyectar el nombre del usuario (ej. Bienvenido, `${user.nombre}`).
- Componente de Reflexión Diaria: Este módulo educativo realiza una llamada asíncrona mediante la función envoltorio `window.fetchWithAuth()` al *endpoint* /reflexion/aleatoria del *backend*. Una vez resuelta la promesa, el *script* de la vista actualiza `textContent` de los nodos correspondientes para renderizar la cita y el autor asociados.
- Interactividad y Feedback: Todas las tarjetas (.card) incluyen directivas de bajo nivel en CSS para mejorar la experiencia de usuario. Se utilizan pseudo-clases :hover que aplican transiciones de `transform: translateY(-4px)` y modifican la sombra mediante `box-shadow: 0 8px 24px var(--color-black-a12)`, proporcionando una respuesta visual inmediata al usuario.

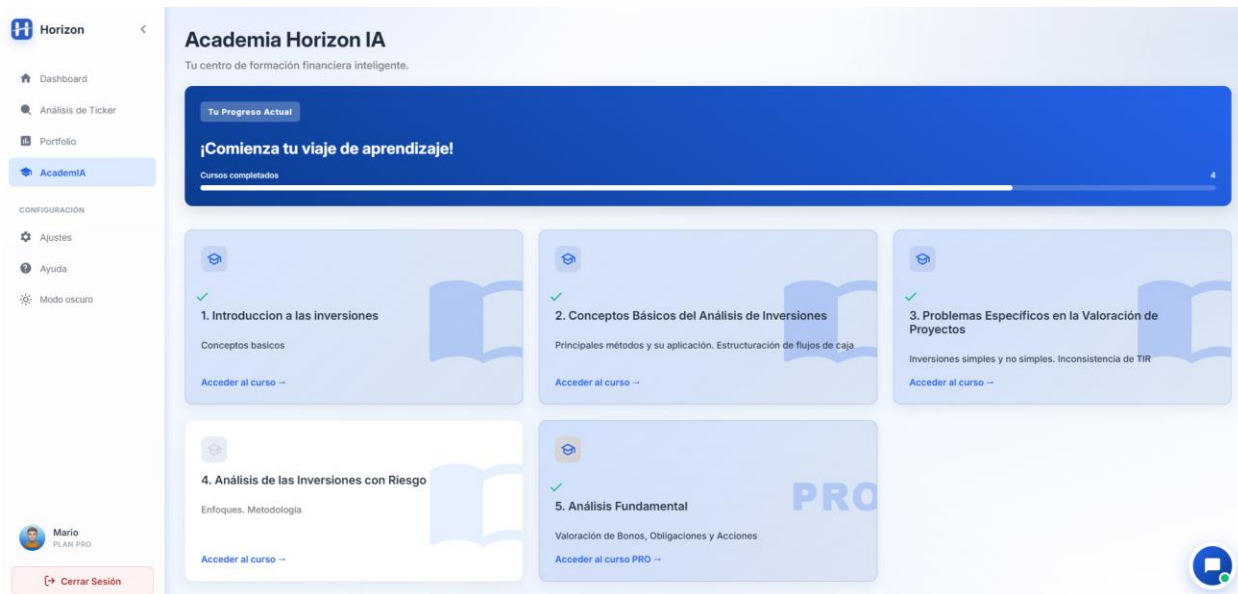


Figura 20 - Vista de Academia Horizon (academia.html)

Esta vista fusiona el aprendizaje estructurado con la asistencia inteligente, permitiendo a los usuarios avanzar en su formación financiera mediante un sistema de carrusel interactivo. La lógica de la página está centralizada en academia.js y comienza con una validación de seguridad inmediata a través de una función autoejecutable (IIFE) que invoca a `window.validateAuthToken()`.

A nivel de implementación, el componente de progreso global utiliza el nodo `#progress-fill-academy` para visualizar el avance del usuario. Técnicamente, el cliente calcula el porcentaje a partir de `cursosCompletados` y `totalCursos` y actualiza dinámicamente la propiedad CSS `width` del elemento, permitiendo una transición visual fluida que refleja los cursos terminados frente al total disponible. El servidor devuelve el conteo de cursos completados mediante `/cursos/resumen/progreso`, pero el porcentaje final se ensambla del lado del cliente.

El núcleo técnico de esta sección reside en la gestión del carrusel de diapositivas y el seguimiento del progreso:

Se realiza una Gestión de Estado y Unicidad: para controlar las diapositivas visualizadas, se emplea un objeto global `estadoCurso` que utiliza una estructura de datos de tipo Set para la propiedad `visitadas`. El uso de un Set es una decisión de ingeniería

clave, ya que garantiza por definición que cada identificador de diapositiva sea único, evitando redundancias al registrar el progreso.

Se realiza una Validación de Lectura Asíncrona: para certificar que el usuario consume el contenido, se ha implementado un mecanismo de retardo mediante `setTimeout()`. Al mostrar una diapositiva nueva, el sistema inicializa un temporizador de 5 segundos y, tras cumplirse este intervalo, marca la diapositiva como visitada. Por otro lado, la función `guardarProgreso()` no se dispara directamente desde ese contador, sino que el guardado en *backend* ocurre más adelante cuando el usuario avanza de diapositiva o completa el curso, usando `window.fetchWithAuth()` al endpoint `/cursos/{id_curso}/progreso`.

Se realiza el Procesamiento de Contenido (*Markdown Parser*, solo reglas específicas): el contenido de la diapositiva se procesa en la función `formatearContenidoDiapositiva()`. Esta función detecta sintaxis Markdown de imágenes con la expresión regular `/!\s*([^\s]*)\s*\s*([^\s]*)\s*/g` y la convierte en etiquetas `<img>` HTML con estilos en línea como `max-width: 100%; border-radius: 8px; margin: 15px 0;`. Para URLs directas de imagen también aplica un renderizado enriquecido con `object-fit: contain` y sombras.

Y, por último, se implementa un Control de Acceso PRO: la lógica de apertura de cursos incluye una validación de membresía en el lado del cliente. Antes de renderizar el modal, el *script* evalúa el atributo `plan_pro` del objeto curso y lo contrasta con `usuario.membresia`. De esta forma, si un usuario con plan "Gratis" intenta acceder a contenido restringido, el sistema bloquea la ejecución de `abrirCurso()` y despliega una notificación de advertencia.

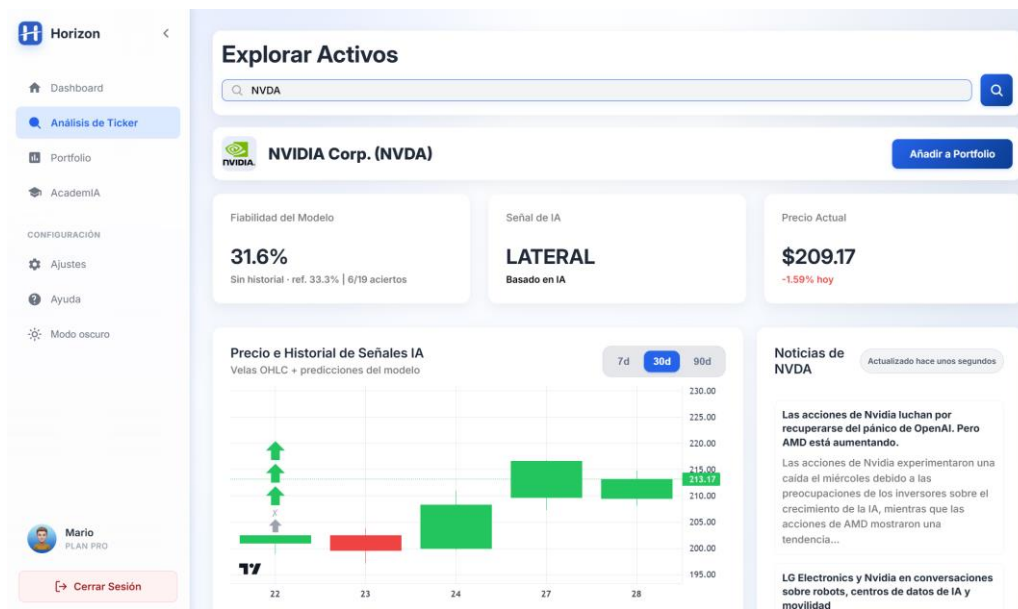


Figura 21 - Vista de Análisis de Activos (analysis.html)

Esta vista representa el módulo con mayor densidad computacional del *frontend*, encargada de la búsqueda, visualización y análisis predictivo de instrumentos financieros. La implementación orquesta múltiples librerías externas y *scripts* especializados (como `priceSignalChart.js` y `shapWaterfall.js`) para transformar datos crudos de mercado en interfaces interactivas, mientras que `analysis.js` valida la autenticación del usuario.

La arquitectura de esta página se fundamenta en la integración de dos motores gráficos de alto rendimiento: la Visualización de Mercado y la Explicabilidad de IA.

En cuanto a la Visualización de Mercado (Lightweight Charts), utiliza la librería de TradingView instanciando un gráfico con `LightweightCharts.createChart(container, { ... })` sobre un contenedor específico del DOM. Técnicamente, el *script* `priceSignalChart.js` configura una serie de velas japonesas con `addCandlestickSeries()` y procesa arrays de objetos OHLC a partir de la respuesta del *endpoint* `/activos/{ticker}/price-history?days=....`. Para optimizar el renderizado, se aplican opciones de escala como `rightPriceScale` y se ajusta el eje temporal con `timeScale().fitContent()` tras recibir la respuesta de la API.

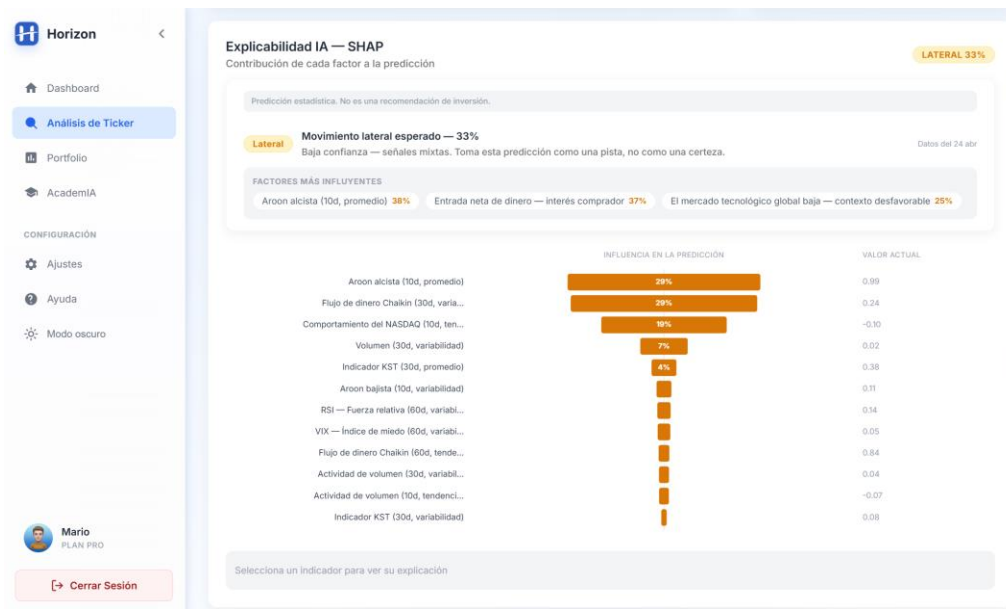


Figura 22 – Explicabilidad IA Activos con Tendencia Lateral

En cuanto a la XAI, el hito técnico de esta vista es el renderizado del diagrama de cascada para la explicabilidad de las predicciones. Se emplea la librería D3.js v7 para realizar una manipulación programática de elementos SVG. El script `shapWaterfall.js` procesa los vectores de contribución devueltos por el backend desde `/activos/{ticker}/xai/latest-shap`. Utiliza funciones como `d3.scaleLinear()` para mapear los valores SHAP a coordenadas y `d3.select().join('rect')` para dibujar las barras dinámicamente. Cada barra se etiqueta y colorea en función de su signo o tipo de señal, permitiendo al usuario identificar visualmente qué indicadores han tenido un peso determinante en la predicción.

A nivel de lógica de red y búsqueda, la vista implementa también mecanismos como el Buscador Reactivo, Sincronización de Señales y Persistencia de Contexto.

En cuanto al Buscador Reactivo: la lógica de búsqueda, delegada en `search.js`, intercepta las entradas del usuario y realiza peticiones GET asíncronas al endpoint `GET /activos?q=...`. Para evitar saturar el servidor, se aplica un `debounce` de entrada y se muestra feedback visual con clases como `.is-loading` mientras se resuelve la promesa.

Por otro lado, la Sincronización de Señales: la vista consume concurrentemente datos de precios y señales desde el mismo *endpoint* `/activos/{ticker}/price-history`, luego `priceSignalChart.js` superpone `markers` en el gráfico, utilizándose para la carga `fetch(...)`.

Por último, la Persistencia de Contexto: el *ticker* seleccionado se mantiene en la variable `selectedTicker` dentro de `search.js` y se usa internamente para actualizar la vista y el modal.

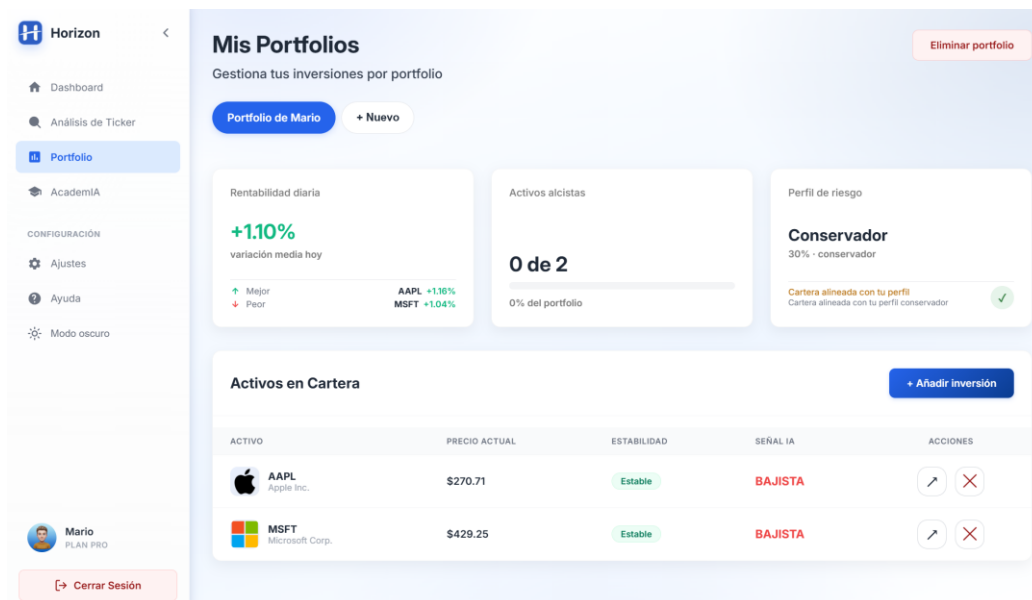


Figura 23 - Vista de Gestión de Portfolios (*portfolio.html*)

Esta vista constituye el núcleo de gestión de activos de Horizon, operando como una interfaz dinámica que permite la administración de múltiples carteras en una arquitectura de aplicación de página única (SPA). La lógica está orquestada principalmente por `portfolio.js`, apoyándose en `portfolio_modal.js` para la creación de nuevas entidades.

Podríamos desglosar la implementación técnica técnica en los siguientes procesos descritos a continuación.

Navegación por Pestañas Dinámicas: al inicializar la página, el sistema recupera el listado de carteras del usuario. La función `renderPortfolioSelector()` genera dinámicamente los botones HTML correspondientes a cada *portfolio*, aplicando clases CSS diferenciadas como `'btn-primary portfolio-tab active'` para el *portfolio* seleccionado

y 'btn-outline portfolio-tab' para el resto. Cada pestaña incorpora un *event listener* que actualiza la variable `currentPortfolioId` y desencadena la ejecución de `loadPortfolioDetails(id)`, permitiendo la carga dinámica de datos sin recargar la página. Adicionalmente, se incluye una pestaña especial "+ Nuevo" que invoca la función `createNewPortfolio()` para abrir el modal de creación.

Renderizado de Activos y Manipulación del DOM: La visualización de las posiciones financieras se realiza mediante la función `renderAssetsTable(acciones)`, que construye dinámicamente las filas de la tabla utilizando `innerHTML`. Cada fila incluye información relevante como el *ticker*, precio, estabilidad y señal de IA, así como botones de acción que permiten acceder al análisis detallado (`goToTickerAnalysis()`) o eliminar el activo (`deleteAsset()`). Además, se emplea la función `window.renderAssetIcon()` para renderizar iconos personalizados de cada activo. En cuanto al formateo condicional, este se aplica exclusivamente a las señales de IA mediante clases como `text-success`, `text-danger` y `text-muted`, no a los rendimientos directamente en la tabla.

Cálculo de Métricas de *Portfolio*: la interfaz calcula indicadores clave a partir de los datos proporcionados por la API. La rentabilidad diaria se obtiene como una media ponderada de las variaciones de los activos (`variaciones.media`), mostrándose con formato condicional según su signo. Asimismo, se calcula el número de activos con señal alcista, expresado en términos absolutos y porcentuales. El perfil de riesgo se determina a partir del valor `portfolio.riesgo`, clasificándose en conservador, moderado o agresivo según umbrales predefinidos. Finalmente, se identifican el mejor y peor activo en función de sus variaciones individuales (`variaciones.porTicker`), aplicando también estilos visuales diferenciados.

Sistema de Notificaciones (Toasts): Para proporcionar feedback inmediato sobre acciones CRUD (como añadir o eliminar activos), se ha implementado la función `showAlert(message, type)`. Técnicamente, esta función crea un elemento `div`, le asigna dinámicamente la clase `alert alert-${type}` y lo inyecta al final del `body`. El ciclo de vida de la notificación se gestiona mediante un `setTimeout` de 3 segundos, tras el cual se

añade la clase `.is-closing-toast` para disparar una transición CSS de desvanecimiento antes de eliminar físicamente el nodo del DOM mediante el método `remove()`.

Gestión de Modales y Creación: la creación de carteras se delega a `portfolio_modal.js`, que controla el elemento `#portfolioModal`. Esta lógica desacoplada permite que la función `createPortfolio()` recoja los valores de entrada mediante `document.getElementById().value`, realice una limpieza de cadenas con `trim()` y ejecute una petición POST asíncrona. Tras recibir un código 201 Created, se invoca la función callback `loadPortfolios()` para refrescar la lista de pestañas sin interrumpir el flujo de navegación del usuario.

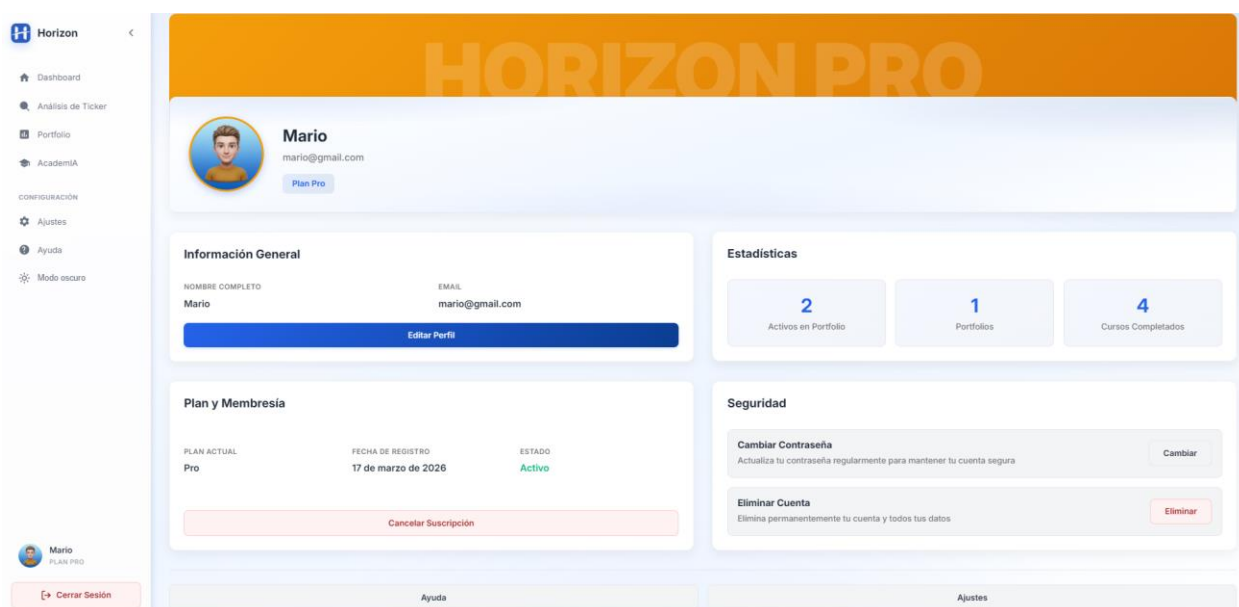


Figura 24 - Vista de Perfil de Usuario (`profile.html`)

Esta vista actúa como el centro de control de identidad, configuración y seguridad del usuario. La arquitectura de esta página se aleja de los envíos de formulario tradicionales, apostando por un modelo de Aplicación de Página Única (SPA) donde `profile.js` gestiona el estado y las mutaciones del DOM de forma asíncrona.

La implementación técnica de esta vista abarca flujos críticos de gestión de datos, que se describen a continuación.

Edición Parcial de Identidad (Hidratación del DOM): La actualización de datos personales (nombre, apellidos) se realiza a través del formulario `#formEditProfile`. El script

intercepta el evento de envío mediante `addEventListener('submit')` y evita la recarga de la página con `e.preventDefault()`. A continuación, se extraen los valores de los campos utilizando `document.getElementById().value.trim()` para limpiar espacios en blanco. A diferencia de un modelo de edición selectiva, el sistema construye siempre un objeto JSON con los tres campos (nombre, apellidos y email, este último deshabilitado ya que el email no se puede cambiar) y lo envía en su totalidad mediante una petición PATCH a `/usuarios/me`, utilizando la función nativa `fetch` junto con la cabecera de autorización correspondiente. Tras una respuesta satisfactoria (`response.ok`), no se actualiza únicamente un elemento aislado del DOM, sino que se invoca la función `loadProfileData()`, que recarga y sincroniza toda la información visible del perfil, incluyendo nombre, email, plan de suscripción, avatar y otros elementos relacionados, sin necesidad de recargar la página completa.

Renderizado de Estado de Suscripción: la vista adapta parcialmente su presentación en función del nivel de suscripción del usuario. El elemento `#profilePlan` se limita a mostrar el texto correspondiente al plan activo (por ejemplo, "Plan Pro" o "Plan Gratis"). Sin embargo, la diferenciación visual principal no se aplica directamente sobre este nodo, sino sobre el contenedor `.profile-header`, al que se le añade dinámicamente la clase `is-pro` en caso de que el usuario disponga de una suscripción avanzada. Adicionalmente, la función `updateUpgradeButton()` modifica el comportamiento, texto y estilo del botón de mejora de plan, reforzando la coherencia visual y funcional de la interfaz.

Seguridad en Acciones Destructivas (Eliminar Cuenta): para la funcionalidad crítica de borrado de perfil, se implementa una barrera de seguridad interactiva para prevenir eliminaciones accidentales. Al hacer clic en "Eliminar Cuenta", la función `openDeleteAccountModal()` localiza el contenedor del modal y manipula su objeto `classList` llamando a `remove('is-hidden')` para renderizarlo. El núcleo de la seguridad reside en la validación en tiempo real: el botón de confirmación (`#deleteAccountConfirmBtn`) se inicializa con el atributo `disabled = true`. Mediante el evento `input`, la función `validateDeleteAccountInput()` evalúa si la cadena de texto introducida por el usuario coincide estrictamente con la variable temporal

expectedDeleteAccountName. Solo cuando la validación booleana es verdadera, se rehabilita el botón.

Purga de Sesión Local: si el usuario confirma la eliminación, el sistema envía una petición DELETE al *endpoint* correspondiente. Tras una respuesta satisfactoria (response.ok), se procede a eliminar el contexto de sesión local mediante la función `window.clearAuthSession()`, que limpia las credenciales almacenadas (como el token JWT) del `localStorage`. Finalmente, se redirige al usuario a la página de inicio de sesión mediante `window.location.replace('login.html')`, evitando así que el estado previo pueda recuperarse mediante la navegación del navegador.

6.3 Implementación del Backend

6.3.1 Servidor FastAPI y Configuración

El archivo `main.py` concentra en su inicio toda la configuración global de la aplicación. Sus importaciones se organizan en tres bloques: los componentes del *framework* (FastAPI, HTTPException, Depends, CORSMiddleware), los DTOs que validan y serializan las peticiones y respuestas mediante esquemas Pydantic, y los DAOs y servicios que encapsulan respectivamente el acceso a datos y la lógica de negocio de cada dominio funcional.

```
1 from fastapi import FastAPI, HTTPException, Depends, UploadFile, File
2 from fastapi.middleware.cors import CORSMiddleware
3 from .database import supabase
4
5 # Entidades
6 from .dtos import *
7 from .daos import usuario_dao, ActivoDAO, ReflexionDAO, ChatDAO, PortfolioDAO, CursoDAO
8 from .services import (
9     AuthService,
10    GeminiService,
11    ChatService,
12    ActivoService,
13    TickerNewsService,
14    ReflexionService,
15    PortfolioService,
16    CursoService,
17    UserService,
18    storage_service,
19    PriceHistoryService,
20 )
```

Figura 25 - Imports main.py

Se importan los componentes principales del framework FastAPI para instanciar la app, HTTPException para emitir errores HTTP semánticos, Depends para el sistema de inyección de dependencias declarativo de FastAPI, UploadFile y File para gestionar la subida de archivos binarios. CORSMiddleware para la configuración del acceso entre orígenes y adicionalmente, se importa la instancia supabase, que actúa como cliente compartido de Supabase y se inyectará en los servicios que requieran acceso a datos.

Los DTOs se importan con wildcard, definen los esquemas Pydantic que validan y serializan los cuerpos de las peticiones y respuestas HTTP. Los DAOs encapsulan el acceso a la base de datos. Los Servicios contienen la lógica de negocio de cada dominio funcional y se describen en detalle en la sección correspondiente.

```
# Exception handlers
from .exceptions import register_exception_handlers
```

Figura 26 - Import de excepciones

Se importa una función centralizadora que registra en la app todos los manejadores de error personalizados, manteniendo el main.py limpio de lógica de error.

```
26 app = FastAPI()
27
28 # Configurar CORS
29 app.add_middleware(
30     CORSMiddleware,
31     allow_origins=["*"], # En producción cambiamos "*" por el dominio real (ej: "https://miweb.com")
32     allow_credentials=True,
33     allow_methods=["*"],
34     allow_headers=["*"],
35 )
36
37 # Registrar exception handlers globales
38 register_exception_handlers(app)
39
```

Figura 27 - Middleware CORS

Se crea la instancia principal de FastAPI y se le aplica el *middleware* CORS mediante `add_middleware`. Este *middleware* controla qué orígenes externos tienen permiso para realizar peticiones a la API, resolviendo las restricciones de seguridad que los navegadores imponen por defecto entre dominios distintos. Durante el desarrollo se permiten todos los orígenes (`allow_credentials=True`, `allow_methods=["*"]` y `allow_headers=["*"]`) habilitan el envío de credenciales y no restringen métodos ni

cabeceras, configuración adecuada para una API RESTful de uso interno. El despliegue del servidor en producción no forma parte del alcance de esta versión del proyecto, quedando la configuración preparada para ese escenario mediante el comentario incluido en el propio código.

```
37 # Registrar exception handlers globales
38 register_exception_handlers(app)
39
40
41 # Wiring explícito de dependencias (evita singletons innecesarios)
42 auth_service = AuthService(supabase, usuario_dao, storage_service)
43 gemini_service = GeminiService()
44 chat_dao = ChatDAO()
45 chat_service = ChatService(
46     usuario_dao_instance=usuario_dao,
47     chat_dao_instance=chat_dao,
48     gemini_service_instance=gemini_service,
49     portfolio_dao_cls=PortfolioDAO,
50     curso_dao_cls=CursoDAO,
51     activo_dao_cls=ActivoDAO,
52 )
53 activo_service = ActivoService(ActivoDAO)
54 ticker_news_service = TickerNewsService()
55 reflexion_service = ReflexionService(ReflexionDAO)
56 portfolio_service = PortfolioService(portfolio_dao=PortfolioDAO)
57 curso_service = CursoService(CursoDAO)
58 user_service = UserService(usuario_dao, storage_service)
```

Figura 28 - Wiring de dependencias

Se invoca la función importada desde el módulo de excepciones, pasándole la instancia `app`. Internamente esa función llama a `app.add_exception_handler(...)` para cada tipo de error personalizado del dominio, logrando que cualquier excepción lanzada desde cualquier capa (DAO, servicio o router) sea interceptada y convertida en una respuesta HTTP coherente sin duplicar código de manejo en cada *endpoint*.

El *wiring* de dependencias hace que, en lugar de usar un contenedor de inyección automático, cada servicio recibe sus colaboradores a través del constructor. Gracias a esto se evita la creación de *singletons* implícitos difíciles de rastrear

6.3.2 Endpoints

El resto de `main.py` está compuesto íntegramente por *endpoints*. En total hay 28 *endpoints* distribuidos en cinco dominios funcionales: activos, usuarios/autenticación,

portfolios, cursos y reflexiones. Los métodos HTTP utilizados son GET (el más frecuente), POST, PATCH, PUT y DELETE.

- *Endpoint* GET: Se usa para obtener información sin modificar nada en el servidor.
- *Endpoint* POST: Se usa para crear un nuevo recurso. El cliente envía datos en el cuerpo de la petición y el servidor lo crea.
- *Endpoint* PUT: Se usa para sustituir completamente un recurso existente. El cliente debe enviar todos los campos, aunque no hayan cambiado.
- *Endpoint* PATCH: Se usa para actualizar solo algunos campos de un recurso, sin necesidad de enviar el objeto entero.
- *Endpoint* DELETE: Se usa para borrar un recurso.

```
178 @app.post("/chat", response_model=ChatMessageResponse)
179 def chat(mensaje: ChatMessageRequest, user_id: str = Depends(auth_service.get_current_user)):
180     # El servicio lanza excepciones personalizadas que los handlers convierten en respuestas HTTP
181     return chat_service.procesar_mensaje(
182         id_usuario=user_id,
183         mensaje=mensaje.message
184     )
185
```

Figura 29 - *Endpoint* Post

Todos los *endpoints* siguen el mismo patrón: no contienen lógica propia. Actúan como un controller que recibe la petición HTTP, extrae los parámetros y los pasa directamente al servicio correspondiente. En este ejemplo se observa la estructura de un *endpoint*:

- Validar la entrada. FastAPI usa el tipo `ChatMessageRequest` (un schema Pydantic de `chat_dto.py`) para validar el cuerpo de la petición antes de que el *endpoint* se ejecute. Si el payload no cumple el schema, FastAPI rechaza la petición sin llegar a llamar a la función.
- Delegación total al servicio. El cuerpo del *endpoint* es una sola línea que llama al servicio y devuelve su resultado. No hay condicionales, no hay acceso a la base de datos, no hay transformación de datos; todo eso ocurre dentro de `chat_service.procesar_mensaje`

6.3.3 Servicios

Los *endpoints* de `main.py` son intencionalmente simples ya que solo reciben la petición HTTP, delegan el trabajo al servicio correspondiente y devuelven el resultado. Toda la lógica real vive en la carpeta `backend/services/`. El proyecto tiene 20 servicios.

Volviendo a la figura anterior, podemos comprobar que el *endpoint* tiene 3 acciones: Validar que el cuerpo encaje con el DTO de entrada (`ChatMessageRequest`), extraer el `user_id` via `auth_service` y llamar a `chat_service.procesar_mensaje(...)` y devuelve el resultado como `ChatMessageResponse` (DTO de salida).

Los DTOs (Data Transfer Objects) son clases Pydantic que definen exactamente qué datos entran y salen de cada operación. Actúan como un contrato: si el dato no encaja con el esquema, FastAPI rechaza la petición automáticamente antes de que llegue al servicio. Para continuar con los del chat, estos se encuentran en `backend/dtos/chat_dto.py`

```
1  from pydantic import BaseModel, Field
2
3
4  # DTO para recibir un mensaje del chatbot desde el frontend
5  class ChatMessageRequest(BaseModel):
6      message: str = Field(..., min_length=1, max_length=100, description="Mensaje del usuario (máximo 100 caracteres)")
7
8
9  # DTO para la respuesta del modelo IA
10 class ChatMessageResponse(BaseModel):
11     usuario_mensaje: str
12     respuesta_ia: str
```

Figura 30 - `chat_dto.py`

`ChatMessageRequest` garantiza que nadie mande un mensaje vacío o demasiado largo antes de que el código lo procese. `ChatMessageResponse` garantiza que la respuesta siempre tenga la misma forma, independientemente de lo que haga el servicio internamente.

6.3.4 DAOs

DAO (Data Access Object) es el único punto del código que tiene acceso directamente a la base de datos. Ningún servicio ni *endpoint* accede a Supabase directamente; siempre pasan por un DAO. Esto mantiene la lógica de negocio (servicios) completamente desacoplada de la gestión de la base de datos. El proyecto tiene 12 DAOs, uno por cada área. (chat_dao, usuario_dao, portoflio_dao, activo_dao, historico_dao, curso_dao, reflexión_dao, predicción_dao, prediction_log_dao, xai_shap_dao, explicación_xai_dao)

Los DAOs encapsulan las operaciones CRUD (*Create, Read, Update, Delete*) sobre la base de datos. Cada método del DAO corresponde a una operación. En el portolfio_dao se encuentran reflejadas todas las operaciones CRUD

- *Create*: Inserta en *portfolios* e inserta en la tabla *usuario_portoflio* para llevar el recuento de *portfolios* del usuario. Una sola acción de negocio genera dos inserciones en tablas distintas. Si la segunda falla, el *portfolio* ya fue creado.

```
26 @staticmethod
27 def crear(portfolio_data: Dict) -> Optional[int]:
28
29     try:
30         # Asegurar timestamps
31         if "created_at" not in portfolio_data:
32             portfolio_data["created_at"] = datetime.now().isoformat()
33         if "updated_at" not in portfolio_data:
34             portfolio_data["updated_at"] = datetime.now().isoformat()
35
36         # INSERT para crear nuevo portfolio (no UPSERT, porque id_portfolio es autoincrement)
37         response = {
38             supabase.table(PortfolioDAO.PORTFOLIOS_TABLE)
39             .insert(portfolio_data)
40             .execute()
41         }
42
43         print(f"[DB] INSERT Response: {response.data}")
44
45         if response.data:
46             portfolio_id = response.data[0].get('id_portfolio')
47             id_usuario = response.data[0].get('id_usuario')
48
49             print(f"[DB] Intentando guardar relacion: usuario={id_usuario}, portfolio={portfolio_id}")
50
51             # Guardar la relación en usuarios_portfolios
52             try:
53                 relacion = {
54                     "id_usuario": id_usuario,
55                     "id_portfolio": portfolio_id,
56                     "created_at": datetime.now().isoformat()
57                 }
58
59                 relacion_response = {
60                     supabase.table("usuario_portfolio")
61                     .insert(relacion)
62                     .execute()
63                 }
```

Figura 31 - portfolio_dao_crear

- *Read*: lee de la base de datos todos los *portfolios* correspondientes al usuario, filtrando por su id: `obtener_por_id(portfolio_id)`

```
@staticmethod
def obtener_por_id(portfolio_id: int) -> Optional[Dict]:
    """
    Obtiene un portfolio por ID.

    Args:
        portfolio_id: ID del portfolio

    Returns:
        Dict con datos del portfolio o None
    """
    try:
        response = (
            supabase.table(PortfolioDAO.PORTFOLIOS_TABLE)
            .select("*")
            .eq("id_portfolio", portfolio_id)
            .execute()
        )

        print(f"🔍 SELECT portfolios WHERE id_portfolio={portfolio_id}")
        print(f"📦 Response: {response.data}")

        portfolio = response.data[0] if response.data else None

        if portfolio:
            print(f"✅ Portfolio encontrado: {portfolio}")
        else:
            print(f"❌ Portfolio {portfolio_id} no encontrado en BD")

        return portfolio

    except Exception as e:
        print(f"❌ Error obteniendo portfolio: {e}")
        return None
```

Figura 32 – `portfolio_dao_obtenerporid`

- *Update*: Modifica los registros existentes del *portfolio*

```
@staticmethod
def actualizar(portfolio_id: int, update_data: Dict) -> bool:
    """
    Actualiza un portfolio existente usando UPDATE.

    Args:
        portfolio_id: ID del portfolio
        update_data: Dict con campos a actualizar

    Returns:
        bool: True si exitoso
    """
    try:
        update_data['updated_at'] = datetime.now().isoformat()

        # UPDATE para modificar un portfolio existente
        response = (
            supabase.table(PortfolioDAO.PORTFOLIOS_TABLE)
            .update(update_data)
            .eq("id_portfolio", portfolio_id)
            .execute()
        )

        print(f"✅ Portfolio {portfolio_id} actualizado")
        return response.data and len(response.data) > 0

    except Exception as e:
        print(f"❌ Error actualizando portfolio: {e}")
        return False
```

Figura 33 - `portfolio_dao_update`

- *Delete*: Elimina de la base de datos el portoflio deseado. En caso de que el portoflio tenga activos, estos se desasocian automáticamente por cascada de la base de datos.

```

195 @staticmethod
196 def eliminar(portfolio_id: int) -> bool:
197     """
198     Elimina un portfolio y todas sus acciones.
199
200     Nota: Las acciones se eliminan automáticamente por cascade.
201
202     Args:
203         portfolio_id: ID del portfolio
204
205     Returns:
206         bool: True si exitoso
207     """
208     try:
209         supabase.table(PortfolioDAO.PORTFOLIOS_TABLE).delete().eq(
210             "id_portfolio", portfolio_id
211         ).execute()
212
213         print(f"✅ Portfolio {portfolio_id} eliminado")
214         return True
215
216     except Exception as e:
217         print(f"❌ Error eliminando portfolio: {e}")
218         return False
219

```

Figura 34 - portfolio_dao_delete

El PortfolioDAO nunca decide cuándo llamar a cada operación ni qué datos prepara. Eso lo hace el servicio. El DAO solo sabe cómo comunicarse con la base de datos. El servicio sabe cuándo y en qué orden llamar a esas operaciones.

El flujo final es: Endpoint(main.py) -> Servicio -> DAO -> Base de datos (Supabase, en la mayoría de DAOs no solo tenemos operaciones CRUD. En el caso de PortoflioDAO también tenemos funciones como:

- obtener_por_usuario: SELECT filtrado por id_usuario, ordenado por created_at.
- eliminar_stocks_por_portfolio y eliminar_relaciones_usuario_portfolio: *DELETEs* manuales en cascada sobre tablas relacionadas (portfolio_activo y usuario_portfolio), como salvaguarda por si el cascade de Supabase falla.
- obtener_stocks_por_ticker: consulta filtrada por *portfolio* y *ticker* a la vez, útil cuando un usuario tiene el mismo activo en multiples entradas.
- obtener_tickers_unicos: método de utilidad que elimina copias redundantes de los *tickers* de un *portfolio* con un *set()*, sin necesidad de lógica extra en el servicio.

Los DAOs encapsulan toda la lógica de acceso a datos, no solo el CRUD básico, sino también consultas filtradas, operaciones en múltiples tablas relacionadas y utilidades de agregación que mantienen los servicios limpios.

6.4 Implementación BBDD

6.4.1 Estructura de Tablas Principales

La base de datos del sistema está alojada en Supabase sobre PostgreSQL y se compone de 14 tablas, donde se diferencian 5 funcionalidades diferentes. Además, hay 13 relaciones de clave foránea entre las tablas.

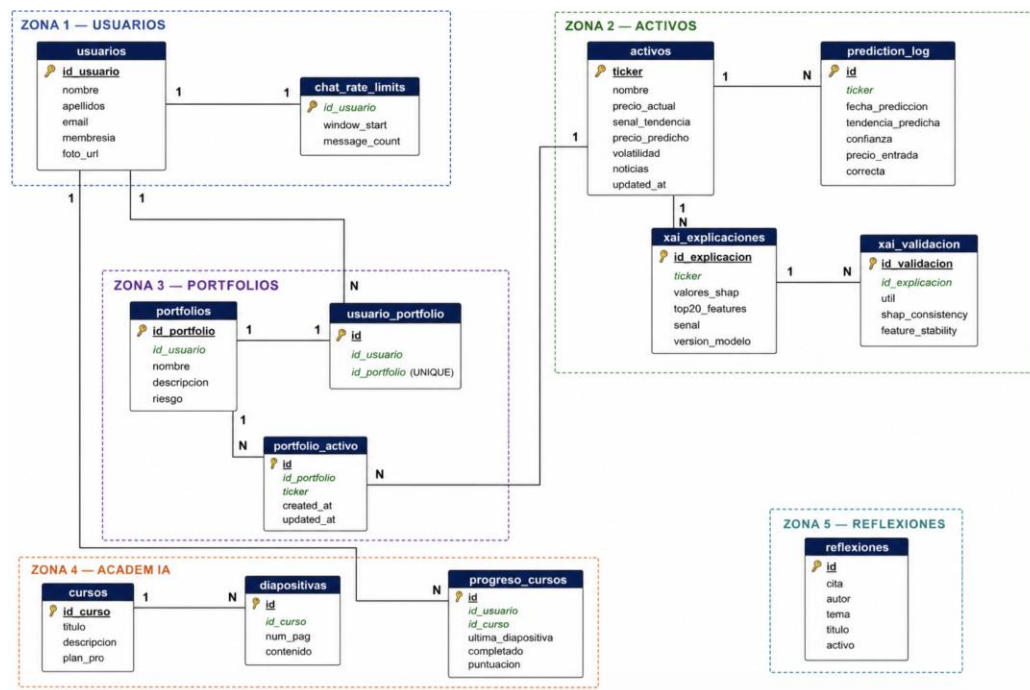


Figura 35- Diagrama Entidad Relación BBDD

6.4.1.1 Usuarios

La tabla “usuarios” almacena los perfiles de los usuarios registrados. Su clave primaria es id_usuario (UUID), que referencia directamente al sistema de autenticación

interno de Supabase (auth.users). Contiene datos personales básicos (nombre, apellidos, email), el nivel de membresía (por defecto 'Gratis') y una URL de foto de perfil. El campo email tiene restricción de unicidad.

La tabla "chat_rate_limits" controla el límite de mensajes que un usuario puede enviar al chat de IA en una ventana temporal. Tiene id_usuario como clave primaria (un registro por usuario), un campo window_start para el inicio de la ventana activa y un contador message_count. La lógica de verificación e incremento atómico se delega a la función PostgreSQL verificar_y_registrar_chat_rate_limit.

La tabla usuarios actúa como entidad raíz del esquema desde la perspectiva del usuario. Cuatro tablas dependen directamente de ella mediante clave foránea sobre id_usuario:

- portfolios: cada *portfolio* pertenece a un usuario registrado. Relación Uno a Muchos (1:N) ya que un usuario puede gestionar varios *portoflios*, pero cada uno de estos pertenece a un solo usuario.
- usuario_portfolio: Implementa claves foráneas bidireccionales que aseguran que la relación entre usuario y portafolio es válida y refuerza la propiedad exclusiva de cada portafolio.
- progreso_cursos: vincula el avance formativo de cada usuario.
- chat_rate_limits: restringe el uso del chat de IA a nivel de usuario individual.

La tabla usuarios se vincula con el sistema de autenticación de Supabase mediante la clave foránea que referencia auth.users(id), asegurando que solo usuarios autenticados tengan registros en la tabla de perfiles.

6.4.1.2 Activos

"Activos" es la tabla central de esta sección. Cada fila representa un activo financiero identificado por su *ticker* (por ejemplo, "AAPL" para Apple), que actúa como clave primaria. Además del nombre y precio actual, almacena información generada

por el modelo de IA: la señal de tendencia (ALCISTA, BAJISTA o LATERAL), el precio predicho, el retorno esperado, la precisión histórica a 30 días (*live_accuracy_30d*), la volatilidad y las probabilidades del modelo XGBoost en formato JSONB. También incluye datos de presentación como el logo del activo y el gráfico de predicción (JSONB).

El campo de noticias en formato JSONB permite almacenar artículos de noticias recientes sobre el activo, facilitando que la Academia proporcione contexto fundamental además de análisis técnico. Finalmente, se mantiene un timestamp de actualización que registra la última modificación del registro, permitiendo detectar cuándo se actualizaron las predicciones.

En cuanto a la tabla “*prediction_log*” registra la fecha de predicción, la fecha objetivo, la tendencia predicha, el nivel de confianza del ensemble y el precio de entrada. Una vez transcurrido el horizonte temporal (campo *dias_horizonte*, por defecto 5), se resuelve la predicción registrando la tendencia real, el precio de salida y si fue correcta. Permite medir el rendimiento del modelo de forma continua.

Por último, en este dominio se encuentran las tablas: “*Xai_explicaciones*” almacena las XAI generadas por SHAP para cada predicción. Contiene los valores SHAP en JSONB, el gráfico en formato texto, las 20 *features* más influyentes y su contribución, junto con la señal y confianza asociadas. Referencia a activos e indica la versión del modelo y la semilla aleatoria utilizada.

“*Xai_validacion*” recoge retroalimentación sobre las explicaciones XAI generadas. Para cada *id_explicacion*, permite registrar si el usuario consideró útil la explicación, un comentario libre, y métricas de calidad técnica: consistencia de los valores SHAP (*shap_consistency*) y estabilidad de las *features* (*feature_stability*). No implementado para esta versión, incluido en trabajo futuro.

Para este dominio, la tabla “*activos*” es la entidad central siendo referenciada por 4 tablas mediante el atributo “*ticker*”. Un único “*ticker*” en la tabla “*activos*” actúa como padre para múltiples registros en “*prediction_log*” y “*xai_explicaciones*”, estableciendo una relación (1:N)

- *portfolio_activo*: asocia activos financieros concretos a *portfolios* de usuarios.

- `prediction_log`: registra cada predicción de tendencia emitida sobre un activo.
- `xai_explicaciones`: vincula las explicaciones XAI generadas con el activo al que corresponden.

6.4.1.3 Portfolios

La tabla “portfolios” representa carteras de inversión que pueden ser creadas por usuarios. Cada portafolio tiene un identificador único, está asociado a un usuario específico mediante UUID, e incluye nombre descriptivo y descripción textual para que el usuario distinga entre múltiples portfolios. Incluye un campo de riesgo (real) que permite clasificar portfolios según su perfil de riesgo: conservador, moderado, o agresivo.

La tabla intermedia “portfolio_activo” implementa la relación muchos-a-muchos entre portfolios y activos. Cada posición en el portfollio tiene un identificador único, vinculándose tanto al portfollio propietario como al activo específico. Esto permite que un portfollio contenga múltiples activos, y un activo específico puede estar presente en múltiples portfolios de diferentes usuarios. Los campos de creación y actualización permiten determinar cuándo fue comprado el activo y cuándo se actualizó la posición.

La tabla “usuario_portfolio” supone una unión entre usuarios y *portfolios*. Aunque *portfolios* ya contiene `id_usuario`, esta tabla existe como relación complementaria (el campo `id_portfolio` tiene restricción UNIQUE, lo que implica que cada *portfolio* pertenece a un único usuario en esta relación).

Este dominio presenta una relación triangular de relaciones entre usuarios, *portfolios* y activos:

- La tabla *portfolios* implementa una clave foránea a usuarios, garantizando que todo *portfolio* pertenece a un usuario válido
- La tabla *portfolio_activo* implementa dos claves foráneas: una a *portfolios* y otra a activos, reforzando la relación muchos-a-muchos (N:M) y asegurando que no se puede añadir un activo inexistente a un portafolio.

6.4.1.4 Academia

La tabla de "cursos" actúa como catálogo de cursos disponibles en la plataforma. Cada curso tiene un título único, una descripción y un indicador booleano `plan_pro` que señala si es exclusivo para usuarios con membresía de pago.

"Diapositivas" es el contenido de cada curso desglosado en diapositivas. La clave primaria es compuesta (`id`, `id_curso`) y cada diapositiva tiene un número de página (`num_pag`) y su contenido en texto.

"Progreso_cursos" registra el avance de cada usuario en cada curso. Almacena la última diapositiva alcanzada, si el curso fue completado y una puntuación obtenida. La clave primaria es tripla (`id`, `id_usuario`, `id_curso`).

En cuanto a las relaciones de este dominio encontramos las siguientes:

- La tabla "diapositivas" se relaciona con "cursos" (vía `id_curso`): cada diapositiva pertenece a un curso concreto.
- La tabla "progreso_cursos" se relaciona con "cursos" (vía `id_curso`) y con "usuarios" (vía `id_usuario`): registra avance de un usuario en un curso determinado, estableciendo una relación N:M, donde un único usuario puede estar inscrito en múltiples programas de formación, y cada curso puede ser cursado por una cantidad ilimitada de alumnos de forma simultánea.

6.4.1.5 Reflexiones

La tabla "reflexiones" contiene los artículos o reflexiones financieras de carácter editorial. Cada registro incluye una cita, el autor, el tema, el título del artículo, el contenido completo, el tiempo estimado de lectura (en minutos) y un array de etiquetas (`tags`). El campo activo permite activar o desactivar la reflexión sin eliminarla. Una función PostgreSQL (`get_reflexion_aleatoria`) permite obtener aleatoriamente una reflexión activa.

6.4.2 Operaciones en Supabase

6.4.2.1 Autenticación

La autenticación de usuarios se delega íntegramente en el módulo Supabase Auth, el cual implementa un sistema de identidad basado en JWT (JSON Web Tokens). En este flujo, el servidor FastAPI valida el *token* firmado criptográficamente en cada petición protegida, verificando simultáneamente la firma, la expiración y el estado del usuario sin necesidad de gestionar una infraestructura de sesiones propia. Para cumplir con el principio de mínimo privilegio, el sistema distingue entre tres niveles de acceso definidos en el módulo `database.py`:

- Cliente Anónimo: Utilizado para operaciones estándar del usuario bajo políticas de Row Level Security (RLS).
- Cliente Administrador: Inicializado con la `service_role_key` para tareas que requieren omitir las restricciones de RLS, como la gestión administrativa de cuentas o la subida de archivos.
- Cliente Supabase: Generado mediante la función `create_supabase_client()`, este cliente se instancia de forma independiente para cada servicio o flujo que lo requiera, evitando así compartir estado de sesión entre distintos módulos del *backend*. Acepta un parámetro `use_service_role` que permite seleccionar en tiempo de ejecución si debe operar con la clave anónima o con la clave de servicio, ofreciendo flexibilidad sin duplicar lógica de inicialización.

6.4.2.2 Supabase Storage

La gestión de los avatares de usuario se realiza a través de Supabase Storage, empleando un *bucket* privado que garantiza el aislamiento de recursos mediante una estructura de rutas vinculada al identificador único del usuario (`user_id`). Para asegurar la privacidad, los archivos no son accesibles mediante URLs públicas directas. En su lugar, el *backend* genera URLs firmadas con un tiempo de vida (TTL) limitado de 3600 segundos, garantizando que el acceso sea temporal y controlado.

6.4.2.3 Funciones Supabase

Se ha delegado parte del procesamiento directamente al motor de base de datos mediante el uso de funciones almacenadas de PostgreSQL. Dentro de Horizon se encuentran las funciones:

- `get_reflexion_aleatoria`: Selecciona y devuelve una reflexión financiera activa de forma aleatoria desde la tabla `reflexiones`. Se usa en `ReflexionDAO` para mostrar contenido editorial al usuario.
- `set_updated_at`: *Trigger* que actualiza automáticamente el campo `updated_at` a la hora actual en cualquier `UPDATE`. No se llama desde el código, sino que actúa de forma transparente en la BD.
- `verificar_y_registrar_chat_rate_limit`: Verifica en una sola transacción atómica si el usuario puede enviar un mensaje según su límite de membresía (1 msg/min) y, si puede, registra el mensaje. Evita condiciones de carrera y se usa en `ChatDAO` antes de procesar cada mensaje.

6.4.3 Restricciones de Integridad

RLS es un mecanismo de seguridad de PostgreSQL que controla qué filas puede ver o modificar cada usuario en una consulta. A diferencia de los controles de acceso tradicionales, que operan a nivel de tabla completa, RLS permite restringir el acceso fila a fila según la identidad del usuario autenticado. Estas restricciones se aplican directamente en el motor de base de datos, por lo que actúan como una capa de seguridad independiente de la aplicación.

El sistema distingue tres roles: *anon* (peticiones sin *token* de autenticación), *authenticated* (usuarios con sesión JWT válida) y *service_role* (cliente administrador del *backend*, que opera con RLS desactivado para tareas de gestión interna).

Las tablas con datos de usuario (`usuarios`, `portfolios`, `portfolio_activo` y `usuario_portfolio`) implementan políticas de aislamiento estricto por propietario, filtrando siempre por `auth.uid()`. En el caso de `portfolio_activo`, dado que no almacena `id_usuario`

directamente, el predicado se resuelve mediante una subconsulta de *join* con la tabla *portfolios*. La tabla *chat_rate_limits* es accesible únicamente por el rol *service_role*, delegando toda la lógica de control en la función PostgreSQL *verificar_y_registrar_chat_rate_limit*.

Las tablas de contenido no personal (activos, cursos, diapositivas, reflexiones) tienen políticas *SELECT* abiertas para los roles *anon* y *authenticated*, mientras que las operaciones de escritura sobre estas tablas quedan reservadas al cliente administrador mediante el rol *service_role*.

6.5 Funcionalidades Principales

6.5.1 Gestión de Usuarios

6.5.1.1 Iniciar sesión

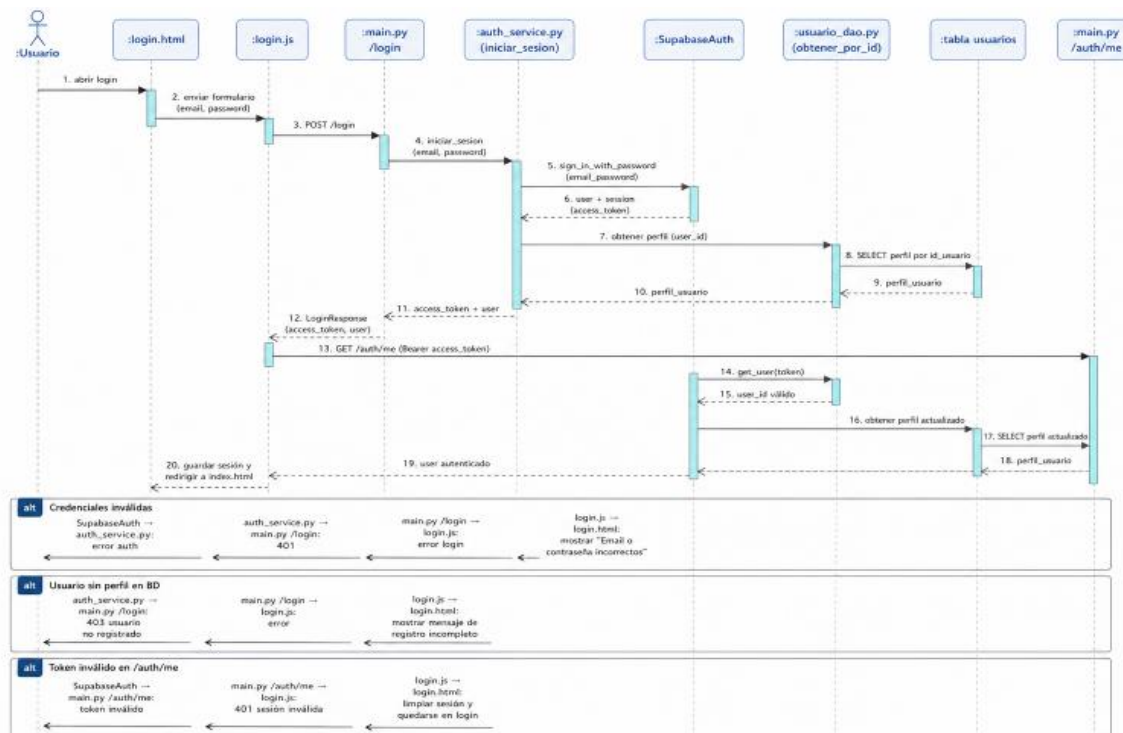


Figura 36 – Diagrama UML de Iniciar Sesión

El usuario inicia la funcionalidad accediendo a la vista de inicio de sesión, representada por `login.html`, donde se muestra el formulario con los campos de correo electrónico y contraseña. A continuación, introduce sus credenciales y pulsa el botón de envío, activando el comportamiento asociado en `login.js`. En ese punto, la interfaz captura los datos del formulario, realiza una validación básica de entrada y prepara la solicitud que será enviada al *backend*. Mientras tanto, el botón queda desactivado y muestra un texto de “autenticando...” en mayúsculas.

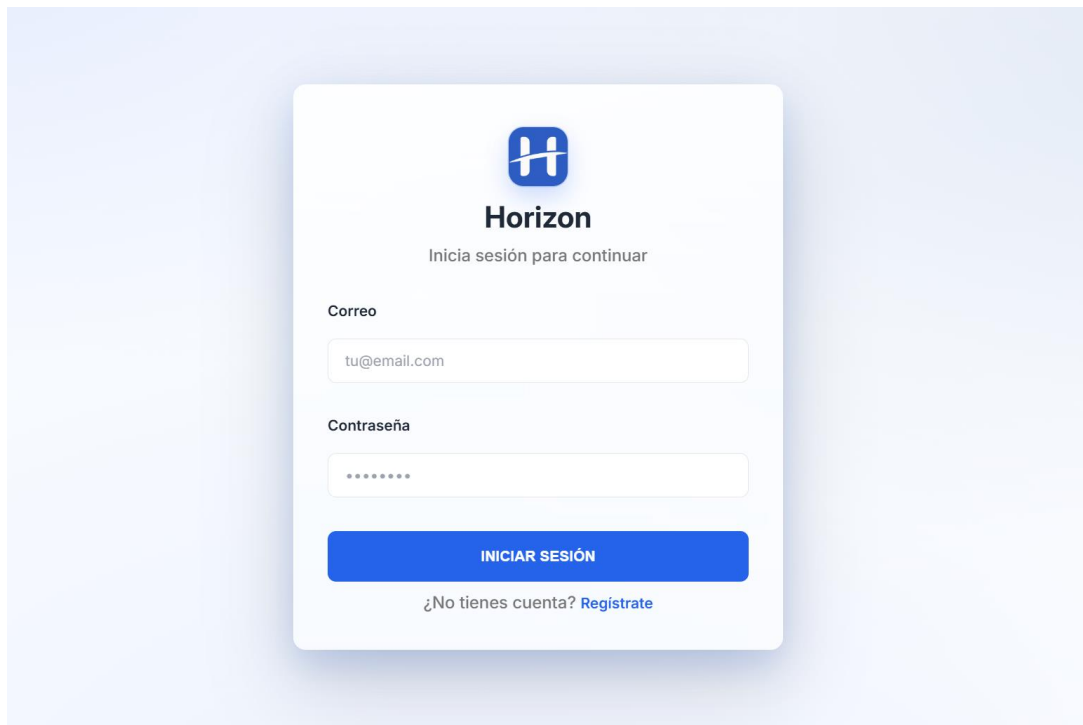
The image shows a login form for a system named "Horizon". At the top is a blue logo consisting of two vertical bars and a horizontal bar, with the word "Horizon" in bold black text below it. Underneath the logo is the text "Inicia sesión para continuar". The form has two input fields: "Correo" (Email) with the placeholder "tu@email.com" and "Contraseña" (Password) with a masked input "*****". Below these fields is a blue button labeled "INICIAR SESIÓN". At the bottom of the form, there is a link that says "¿No tienes cuenta? [Regístrate](#)". The entire form is centered on a light blue background.

Figura 37 - Vista Iniciar Sesión

Tras el envío, el sistema dirige la información al *endpoint* POST `/login` en `main.py`, donde se invoca `auth_service.py` a través de su método `iniciar_sesion()`. El servicio de autenticación crea un cliente Supabase antes de continuar, con el que se comunicará para completar la autenticación. Esto es así para evitar interferencias en futuras acciones que requieran conocer el rol del usuario para ejecutarse, que reutilizando el cliente Supabase se perdería.

Si la autenticación es correcta, el servicio consulta SupabaseAuth, obtiene el *token* de acceso y recupera el perfil del usuario desde `usuario_dao.py` y la tabla `usuarios`. Con esa

respuesta, el *frontend* almacena la sesión, ejecuta la validación posterior mediante GET /auth/me y, si todo es válido, redirige al usuario a la pantalla principal. Si alguna verificación falla, el flujo retorna al formulario mostrando el error correspondiente y sin permitir el acceso a la aplicación.

6.5.1.2 Cerrar sesión

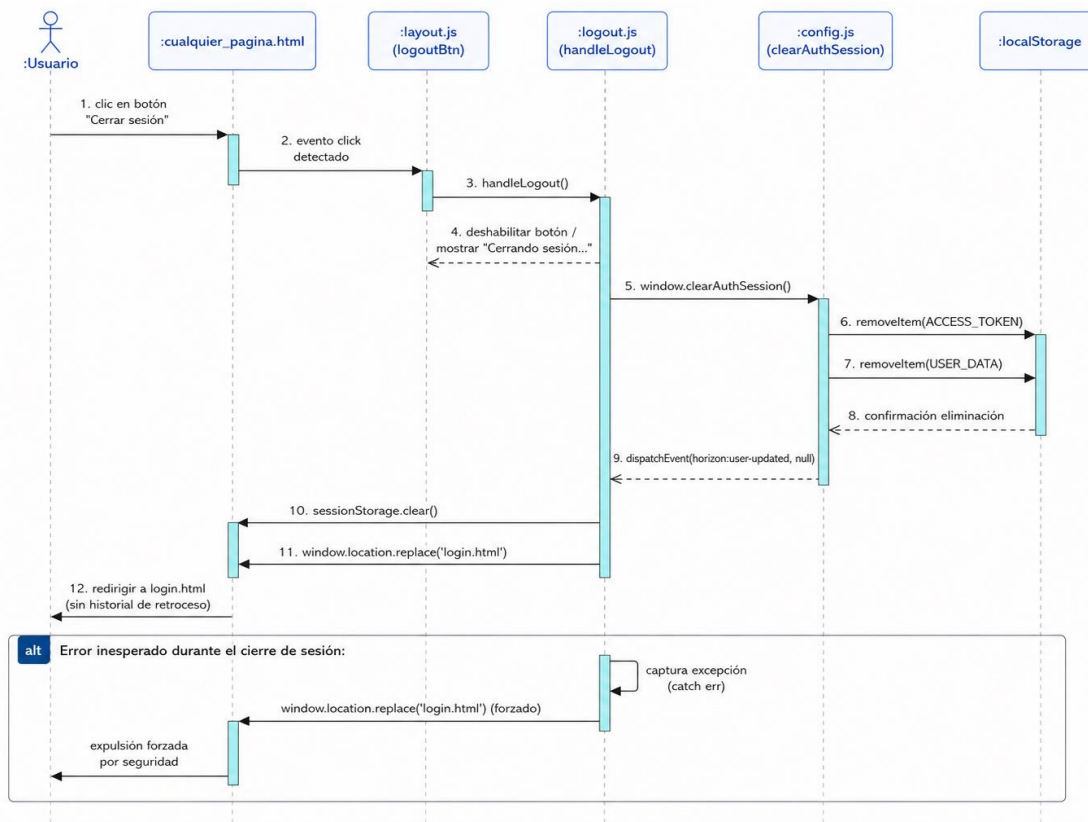


Figura 38 - Diagrama UML de Cerrar Sesión

El usuario activa la funcionalidad desde cualquier pantalla de la aplicación pulsando el botón de "Cerrar sesión", presente en la barra de navegación renderizada por *layout.js*. El evento de clic es capturado por el *listener* registrado en ese mismo componente, que delega inmediatamente la ejecución en *logout.js* a través de la función *handleLogout()*.

Como primer paso de respuesta visual, el botón queda desactivado y muestra el texto "Cerrando sesión...", evitando pulsaciones duplicadas mientras el proceso concluye. A continuación, `handleLogout()` invoca `window.clearAuthSession()`, definida en `config.js`, que es la responsable de destruir el estado de sesión almacenado en el cliente. Esta función ejecuta dos operaciones sobre el `localStorage`: elimina el `token` de acceso (`ACCESS_TOKEN`) y los datos del perfil de usuario (`USER_DATA`). Una vez completada la limpieza, emite el evento interno `horizon:user-updated` con valor nulo, lo que permite que cualquier componente de la interfaz que esté escuchando ese evento actualice su estado de forma reactiva.



Figura 39 - Vista Cerrar Sesión

En `handleLogout()` se limpia también el `sessionStorage` por compatibilidad con flujos alternativos, y finalmente se redirige al usuario a `login.html` mediante `window.location.replace()`. El uso de `replace` en lugar de `assign` es deliberado: elimina

la entrada actual del historial de navegación, impidiendo que el usuario pueda volver atrás con el botón del navegador y acceder a una pantalla protegida sin sesión activa.

6.5.1.3 Registrarse

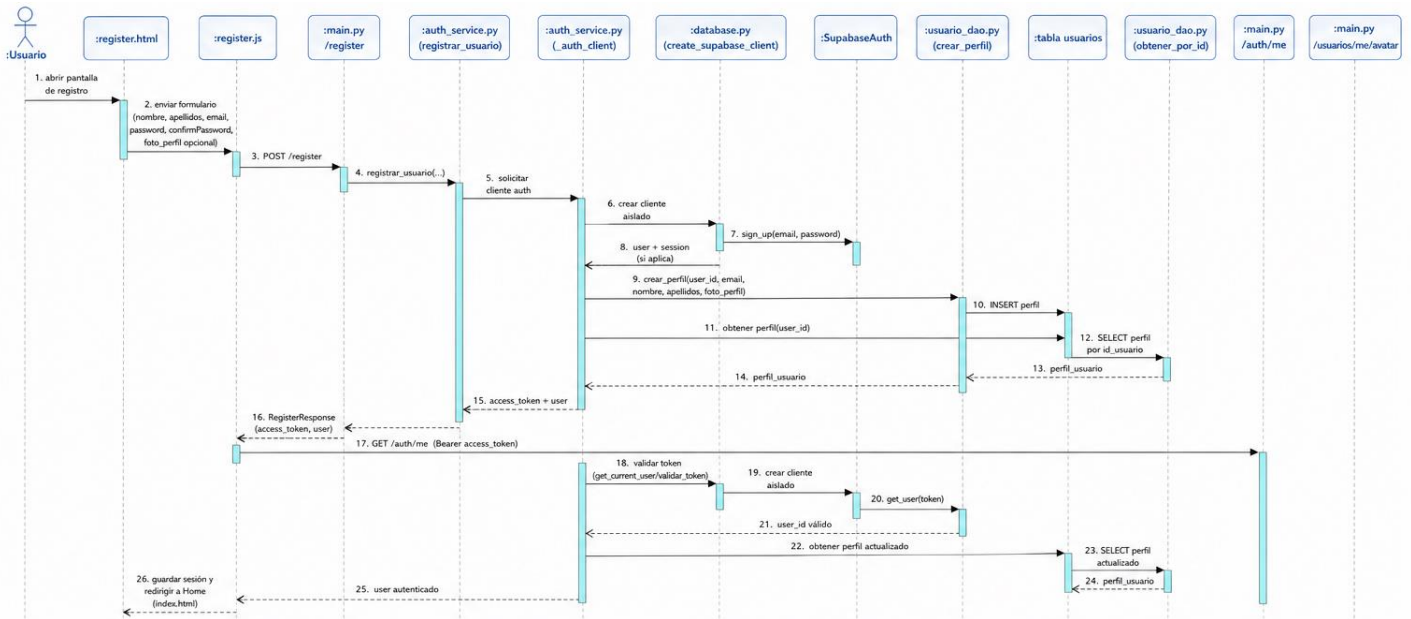


Figura 40 – Diagrama UML de Registro

El proceso de registro comienza cuando el usuario accede a la vista register.html, completa el formulario con sus datos y pulsa el botón de creación de cuenta. Tal como se aprecia en el diagrama de secuencia que generamos, la interacción se inicia en la interfaz de *frontend* y pasa a la lógica de register.js, validando los campos obligatorios, comprueba que las contraseñas coinciden (y son de más de 6 caracteres) y prepara la solicitud hacia el *backend*. En esta fase también puede incluirse la selección de una imagen de perfil, ya sea predeterminada o personalizada, aunque esa parte se resuelve como una extensión posterior del flujo principal.

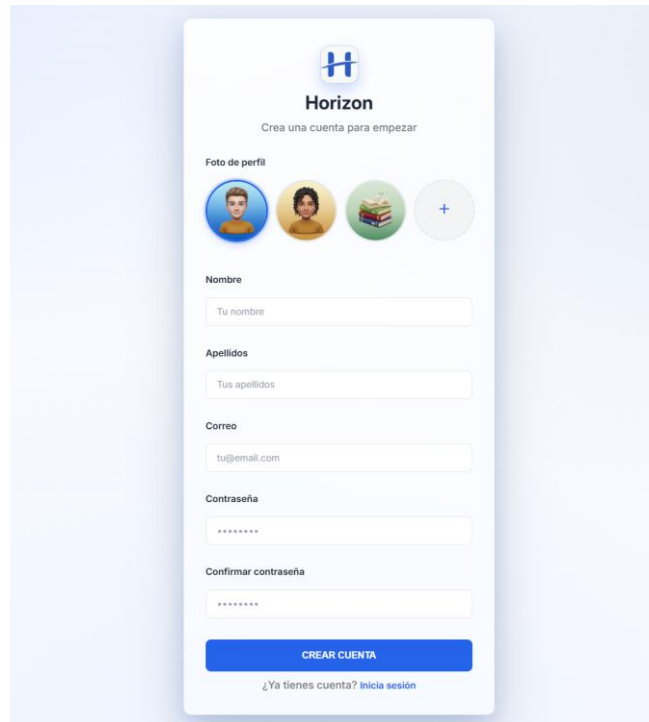


Figura 41 - Vista Registrarse

Después, el *frontend* envía los datos al *endpoint* `main.py`, donde recibe el contrato de `register_dto.py` (`RegisterRequest`) y delega la operación en `auth_service.py`. En el servicio, el sistema crea primero la cuenta en Supabase Auth mediante `auth_service.registrar_usuario()`, y luego registra el perfil del usuario en la tabla pública a través de `usuario_dao.crear_perfil()`. Una vez persistido el perfil, el servicio recupera los datos con `usuario_dao.obtener_por_id()` y devuelve al *endpoint* el token de acceso junto con el perfil creado.

Finalmente, el *backend* responde con el contrato Pydantic de `RegisterResponse`, y el *frontend* guarda el *token* y los datos del usuario en almacenamiento local. A continuación, ejecuta la validación centralizada del *token* contra `main.py`, usando la misma lógica de autenticación que en el *login*, para confirmar que la sesión es válida antes de redirigir al usuario a la pantalla principal. Si la validación falla, el flujo retorna a la interfaz de registro mostrando el error correspondiente y sin completar la autenticación, manejando las excepciones en el propio controlador.

6.5.1.4 Eliminar Perfil

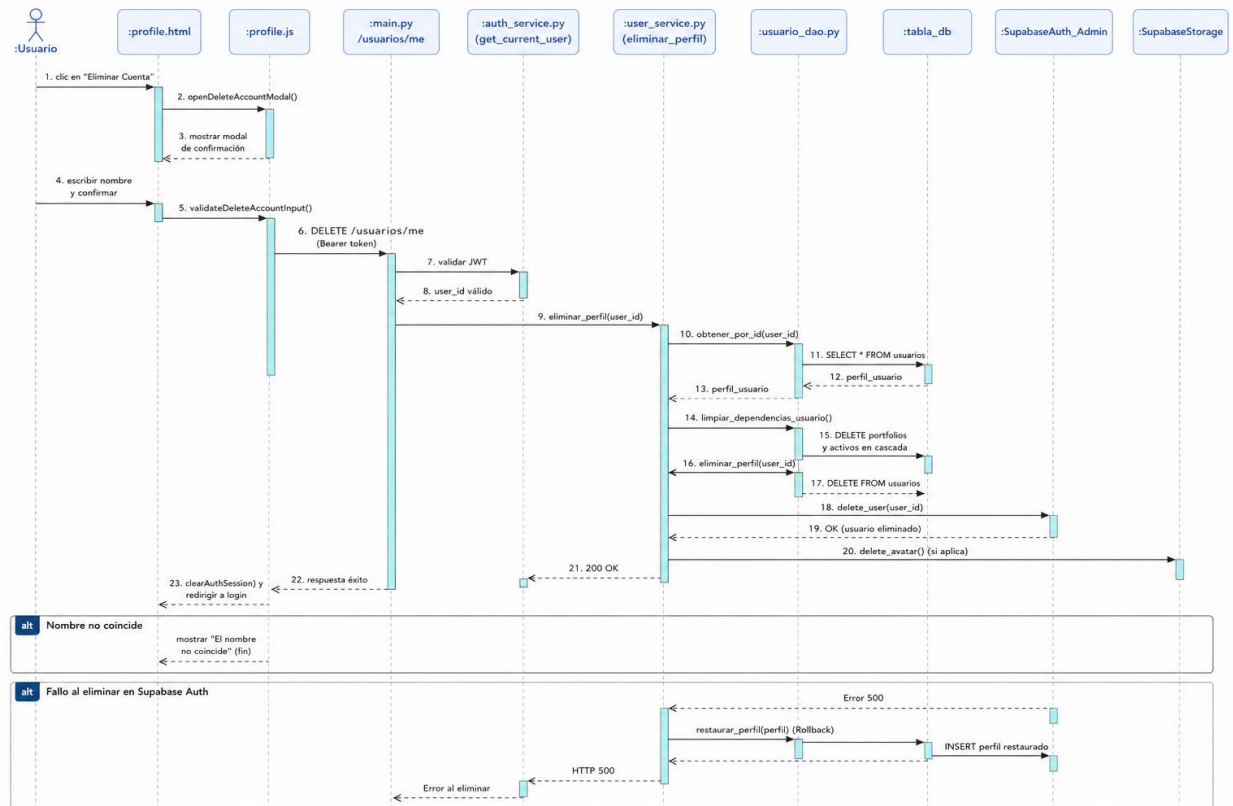


Figura 42 - Diagrama UML de Eliminar Perfil

El proceso de eliminación de perfil se inicia en la vista de perfil del cliente (profile.html y profile.js). Como medida de seguridad frente a borrados accidentales, el usuario debe interactuar con un modal de confirmación donde se requiere introducir manualmente su nombre completo. La función `validateDeleteAccountInput()` evalúa en tiempo real esta entrada y, solo cuando coincide exactamente con los datos introducidos, se habilita el botón de confirmación. Al accionarlo, `confirmDeleteAccount()` lanza una petición HTTP DELETE asíncrona hacia el *endpoint* `/usuarios/me`, inyectando el *token* JWT en las cabeceras mediante `fetchWithAuth`.

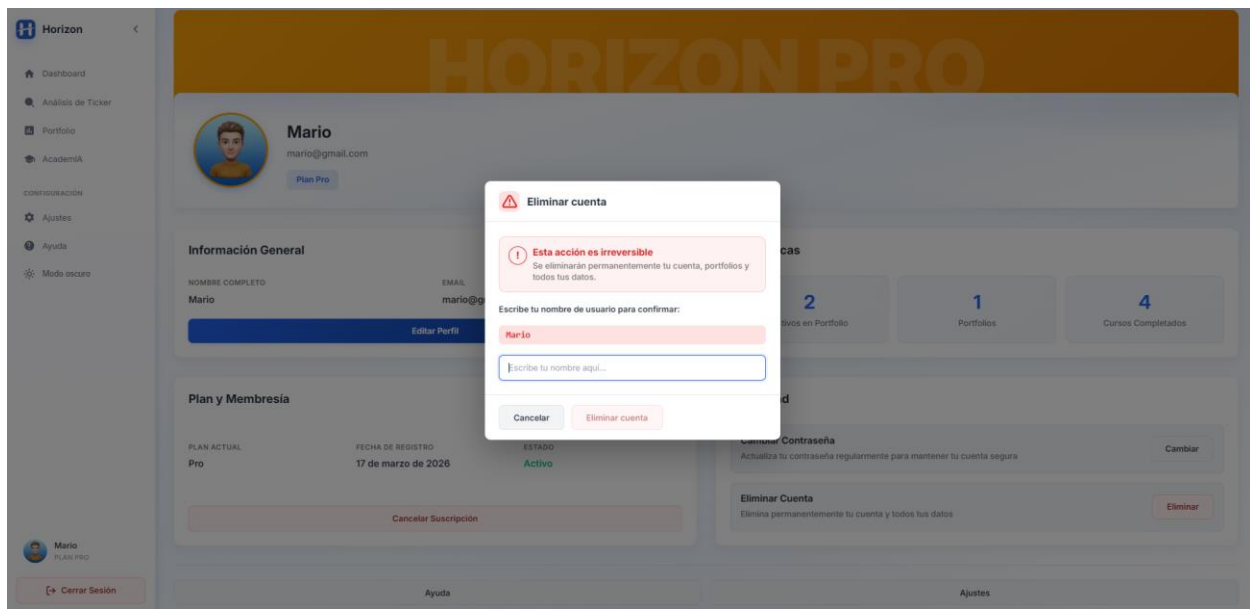


Figura 43 - Vista de Eliminar Perfil

En la capa del servidor (FastAPI), el enrutador de `main.py` intercepta la petición y delega la validación de seguridad a la dependencia `auth_service.get_current_user`. Este servicio extrae y verifica criptográficamente el `token` contra Supabase Auth. Si es válido, se extrae el identificador único del usuario (`user_id`) y se transfiere el control a `UserService.eliminar_perfil()`.

Esta capa de servicio orquesta una transacción compleja que involucra múltiples dominios para evitar inconsistencias y datos huérfanos:

- Limpieza en cascada de dependencias. Se invoca a `UsuarioDAO.limpiar_dependencias_usuario()`, el cual ejecuta sentencias de borrado jerárquicas en la base de datos: primero desvincula y elimina los activos de las carteras (`portfolio_activo`), luego borra las relaciones usuario-cartera (`usuario_portfolio`) y finalmente elimina los registros raíz de `portfolios`.
- Borrado del perfil público: Se elimina la tupla correspondiente al usuario en la tabla `usuarios`.
- Borrado de identidad en Auth: Se invoca al cliente administrador (`supabase_admin.auth.admin.delete_user()`) para eliminar irrevocablemente las credenciales de acceso. En este punto, el sistema implementa un

mecanismo de rollback lógico: si la llamada a Supabase Auth falla (por caída de red o permisos), se ejecuta `UsuarioDAO.restaurar_perfil()` para reinsertar el usuario en la base de datos pública y mantener la consistencia.

- Limpieza de almacenamiento (*Storage*): Si el usuario poseía un avatar personalizado, el servicio solicita a `StorageService` su eliminación definitiva del *bucket* privado.

Finalmente, si toda la orquestación concluye con éxito, el servidor retorna un código HTTP 200 OK. El cliente procesa esta respuesta borrando cualquier rastro de la sesión local (`window.clearAuthSession()`) y redirigiendo al usuario a la pantalla de autenticación.

6.5.1.5 Editar Perfil

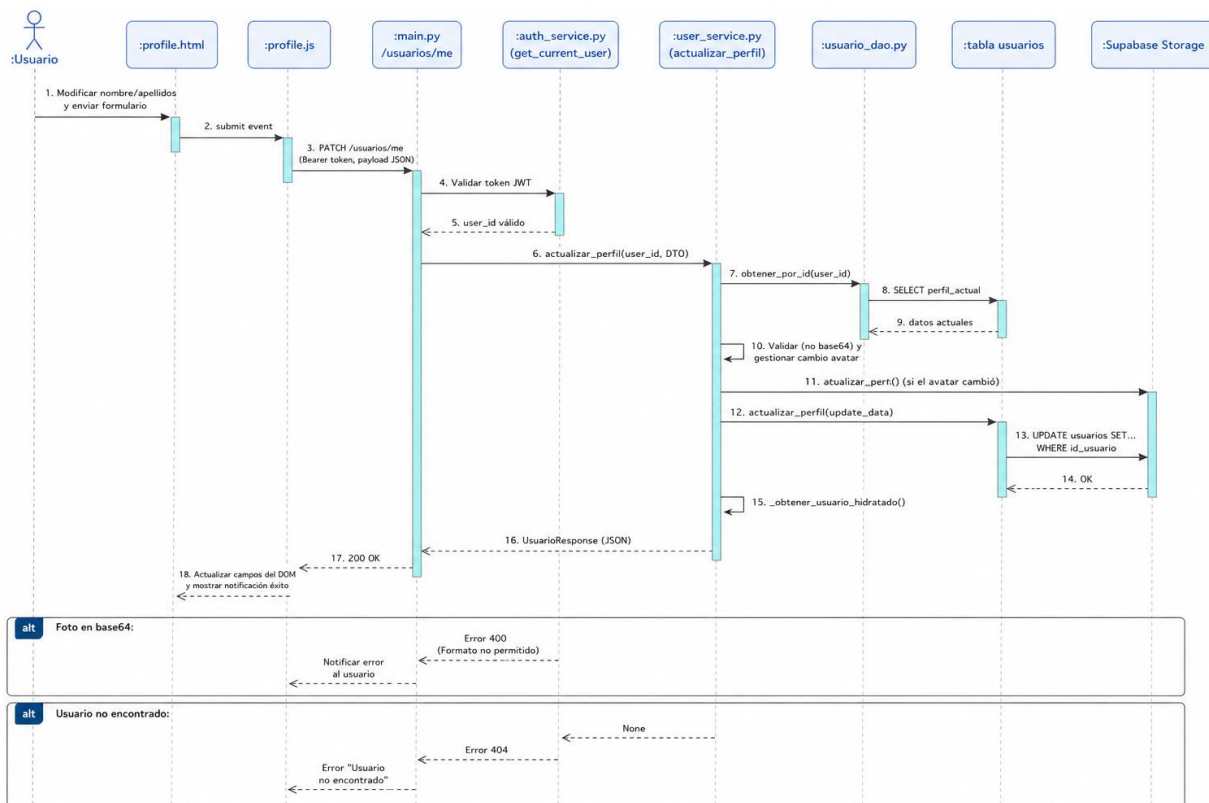


Figura 44 - Diagrama UML de Editar Perfil

El flujo operativo se inicia en el *frontend*, concretamente en *profile.js*, mediante la función *initProfileEvents()*. Al enviar el formulario de edición, se capturan los valores de los campos y se construye un objeto de datos parciales. La comunicación se realiza mediante una petición HTTP PATCH hacia el *endpoint* */usuarios/me*. Se ha optado por el método PATCH en lugar de PUT para permitir actualizaciones parciales, enviando únicamente los campos que el usuario desea modificar y optimizando así el ancho de banda y el procesamiento en el servidor.

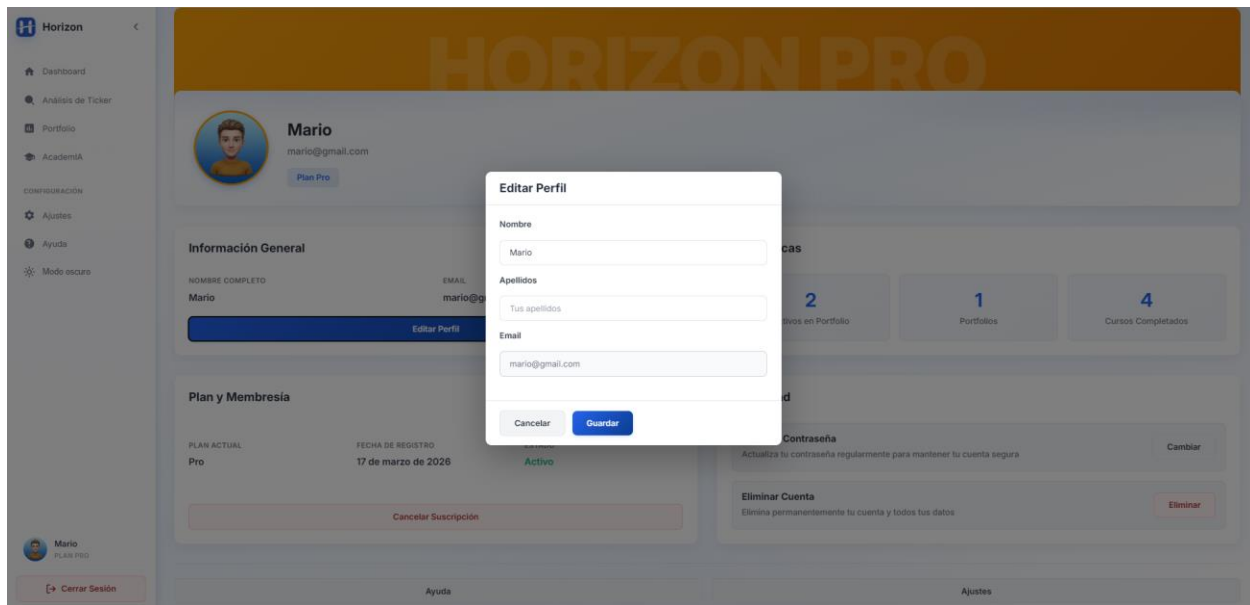


Figura 45 - Vista de Editar Perfil

En el lado del servidor, el proceso se desglosa en las siguientes fases técnicas:

- Validación de Identidad: El *endpoint* en *main.py* utiliza la inyección de dependencias de FastAPI para invocar a *auth_service.get_current_user*. Este componente garantiza que solo el propietario de la cuenta pueda modificar sus datos mediante la verificación del *token* JWT.
- Lógica de Negocio y Seguridad: El control pasa a *user_service.actualizar_perfil()*. En esta capa se aplican reglas de validación críticas: se bloquea cualquier intento de enviar imágenes en formato base64 directamente a la base de datos (exigiendo el uso del *endpoint* específico de *storage*) y se gestiona la limpieza de recursos. Si el usuario cambia su avatar,

el servicio detecta si existía un archivo previo en el *bucket* de Supabase Storage para eliminarlo y evitar la acumulación de archivos huérfanos.

- Actualización: La actualización se ejecuta a través de `UsuarioDAO.actualizar_perfil()`. Tras la actualización, el servicio no devuelve simplemente el registro de la base de datos. Se invoca a `_obtener_usuario_hidratado_or_500()`, que se encarga de resolver las URLs firmadas de los avatares, garantizando que el *frontend* reciba un objeto de usuario listo para ser renderizado con todas sus referencias visuales actualizadas.

Finalmente, el *frontend* recibe un código 200 OK y actualiza el DOM en tiempo real, reflejando los cambios en la cabecera y el panel de información personal sin requerir una recarga completa de la aplicación.

6.5.1.6 Editar Foto de Perfil

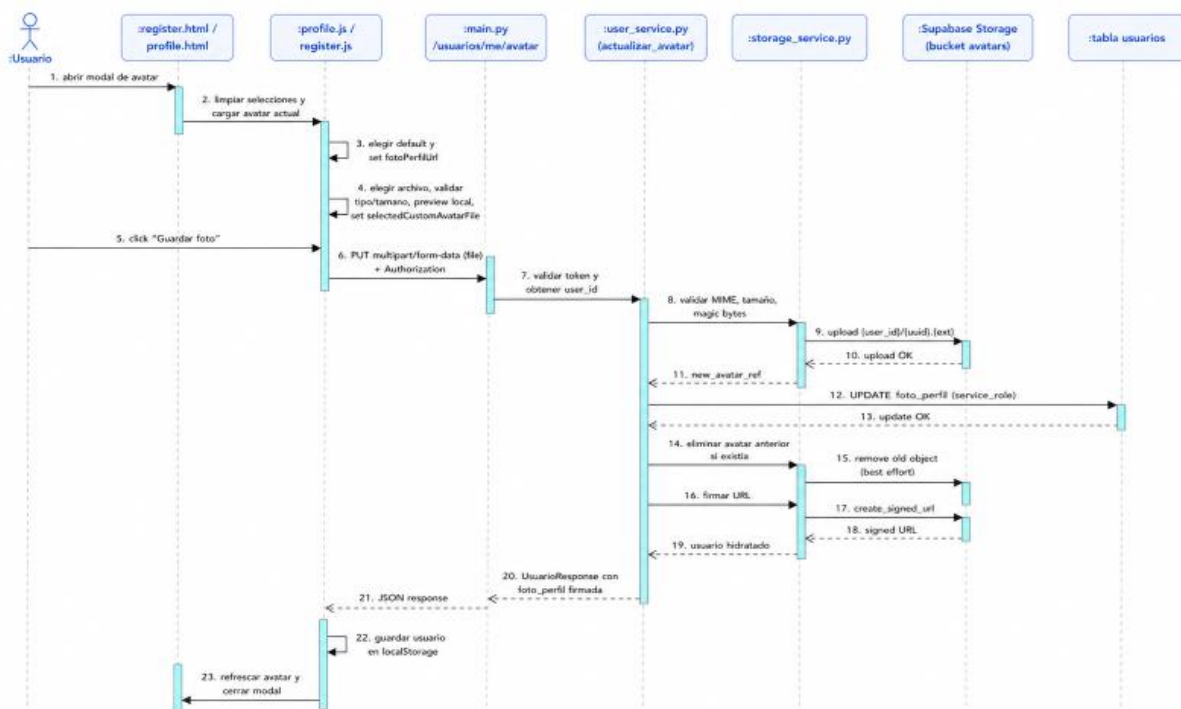


Figura 46 - Diagrama UML de Editar Foto de Perfil

La edición de la foto de perfil comienza cuando el usuario entra en `register.html` o `profile.html` y abre el modal de avatar. Si el usuario elige un default, el *frontend* guarda su URL en `fotoPerfilUrl`, dado que están almacenadas en un *bucket* público, con políticas de visualización pública. Si elige subir archivo, valida tipo y tamaño, guarda el archivo en `"selectedCustomAvatarFile"` y muestra un preview local. Al pulsar "Guardar foto", el *frontend* decide el flujo: para archivo envía un `PUT /usuarios/me/avatar` con `FormData` y `token`; para default envía `PATCH /usuarios/me` con el JSON correspondiente.

En *backend*, `main.py` valida el JWT y resuelve el `user_id`. En el flujo de archivo, `"user_service.actualizar_avatar()"` llama a `storage_service.upload_avatar()`, que valida tamaño máximo y magic bytes antes de subir el objeto al *bucket* privado. Si alguna validación falla, se responde 400 con un mensaje claro (tipo no permitido, imagen demasiado grande o contenido inválido). Si la subida falla por credenciales o error de *storage*, se responde 500 y no se modifica la base de datos.

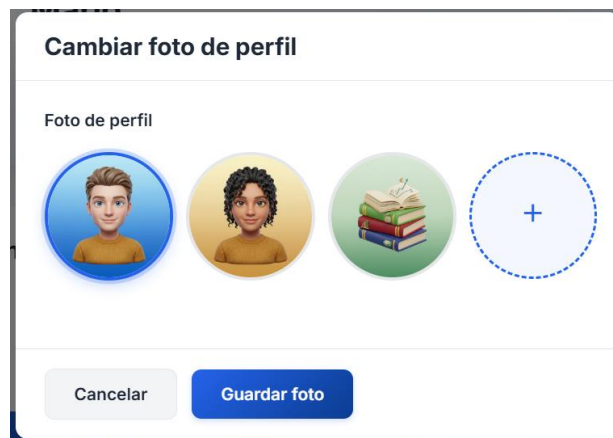


Figura 47 – Vista de Editar Foto Perfil

Si la subida es correcta, el servicio actualiza `usuarios.foto_perfil` usando cliente `admin (service_role)` para no chocar con RLS. Si esta actualización falla, se ejecuta un `rollback best-effort`: se intenta borrar el archivo recién subido para evitar residuos y se devuelve 500. Si todo va bien, se intenta borrar el avatar anterior; este borrado es tolerante a fallos (no detiene el flujo si falla). Finalmente se firma la URL y se devuelve el usuario con `foto_perfil` ya usable por *frontend*. Si la firma fallara, se captura y se devuelve `foto_perfil = null` para no romper el *login* o la edición.

La respuesta vuelve al *frontend*, que actualiza *localStorage* y refresca el avatar en la UI. Si hay error en cualquier punto, el *frontend* muestra un mensaje en el modal y restaura el botón, manteniendo el estado anterior sin dejar la interfaz en blanco. Con esto, el flujo prioriza consistencia en base de datos y evita residuo en *storage*, a la vez que no rompe la sesión si la generación de URL firmada tiene problemas.

6.5.2 Gestión de Activos

6.5.2.1 Añadir activo

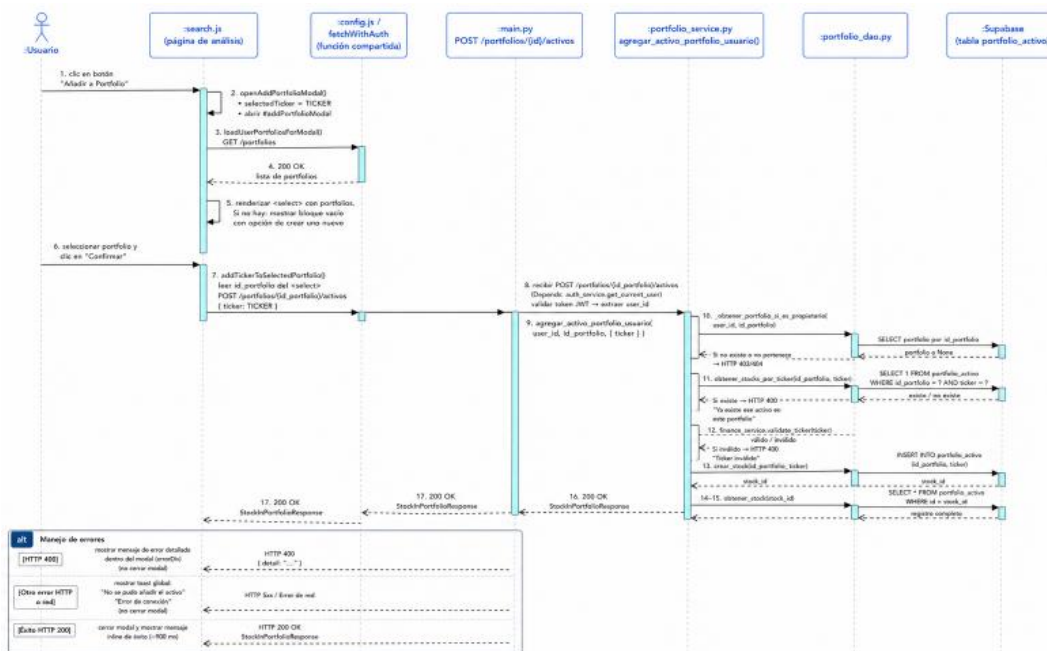


Figura 48 - Diagrama UML añadir activo

Cuando el usuario pulsa "Añadir a Portfolio" sobre un *ticker*, *search.js* guarda el *ticker* seleccionado, abre el modal *#addPortfolioModal* y lanza automáticamente un *GET /portfolios* para obtener los *portfolios* del usuario. Con la respuesta renderiza un

<select> dentro del modal; si no hay *portfolios*, muestra un bloque vacío con opción de crear uno nuevo.

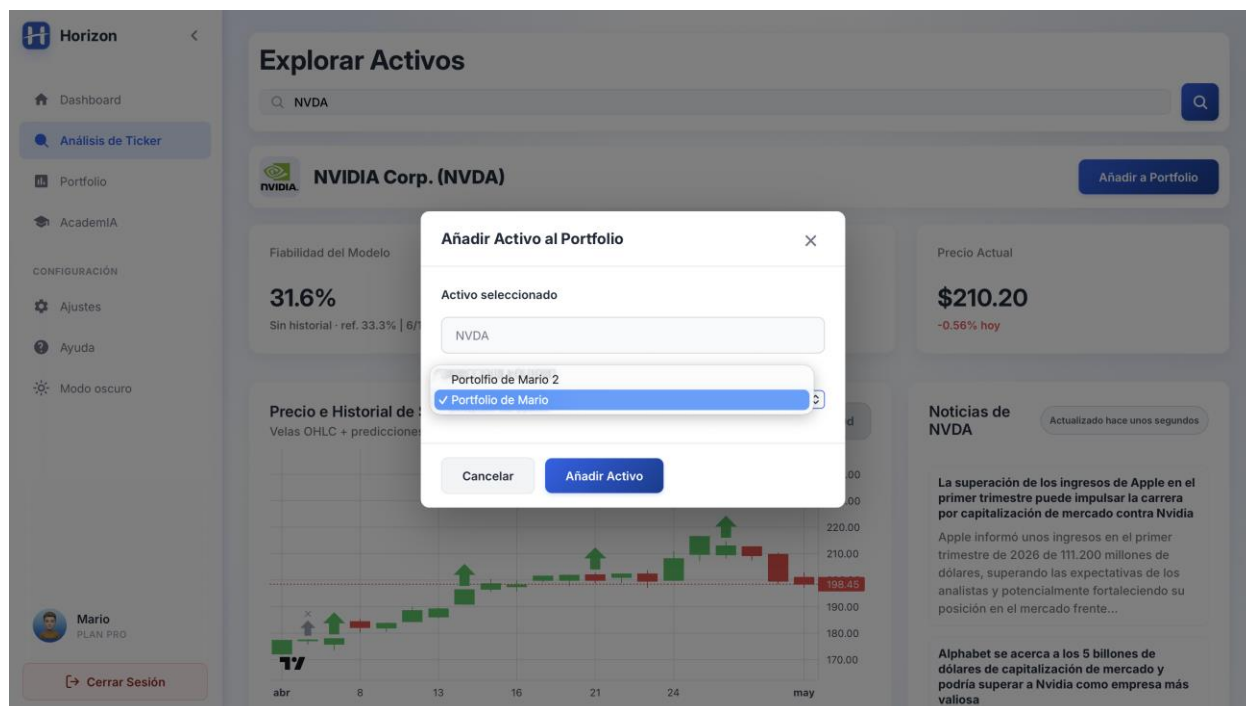


Figura 49 - Añadir activo

Al confirmar la selección, `search.js` ejecuta `addTickerToSelectedPortfolio()` y envía un POST `/portfolios/{id_portfolio}/activos` con el *ticker*. `main.py` valida el JWT y delega en `portfolio_service`, que realiza tres comprobaciones en cadena: verifica que el *portfolio* existe y pertenece al usuario (devuelve 403/404 si no), comprueba que el *ticker* no está ya en ese *portfolio* (400 si existe) y valida el *ticker* en el mercado real (400 si es inválido).

Si todo pasa, el servicio inserta el registro en `portfolio_activo`, recupera el registro completo y devuelve un 200 OK con `StockInPortfolioResponse` hasta el *frontend*, que cierra el modal y muestra un mensaje de éxito inline durante 900 ms. En caso de HTTP 400 se muestra el error detallado dentro del modal sin cerrarlo; cualquier otro error muestra un *toast* global de "No se pudo añadir el activo".

6.5.2.2 Buscar Activo

6.5.2.3 Noticias

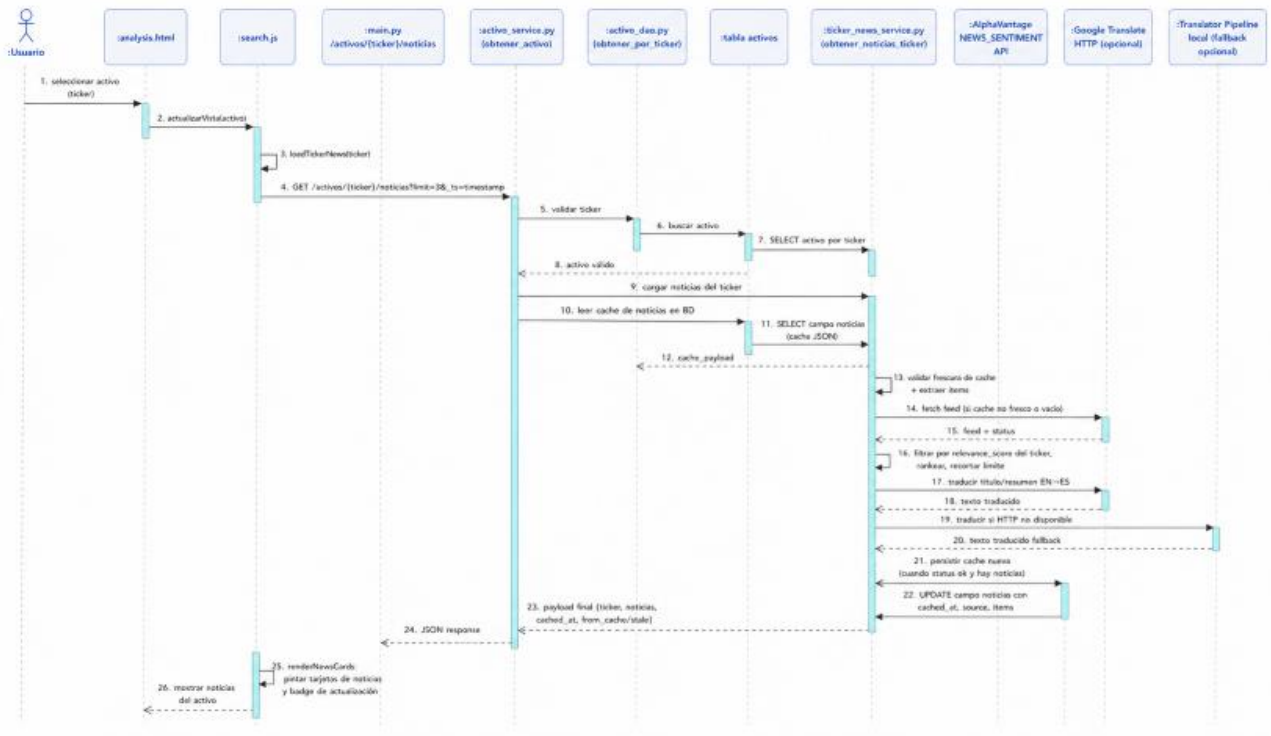


Figura 52 - Diagrama UML de Noticias

El mostrado de noticias comienza cuando el usuario selecciona un activo en la vista de análisis; esa acción activa la carga en `loadTickerNews` en `search.js`, que invoca el endpoint `GET /activos/{ticker}/noticias` en `main.py`.

En *backend*, `main.py` valida primero que el *ticker* exista usando `activo_service.buscar_activos()`. Si no existe, responde 404; si existe, delega en `ticker_news_service.py`.

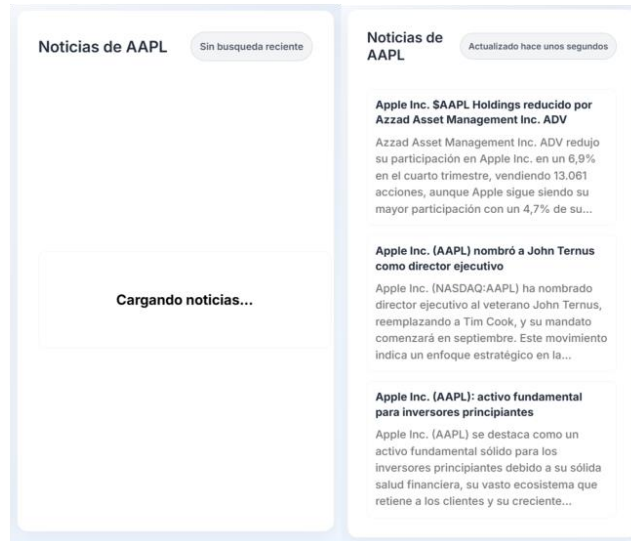


Figura 53 - Vista Noticias

El servicio aplica una estrategia de cache en la base de datos. Lee noticias persistidas con `activo_dao.py`, verifica si hace menos de 1 hora que no se actualiza y, si la cache no es válida, consulta la API de Alpha Vantage para refrescar resultados. Después filtra por relevancia del *ticker*, normaliza/traslada contenido y, cuando procede, persiste la nueva cache con `activo_dao.actualizar_noticias()`. Finalmente devuelve un payload con noticias y metadatos como `cached_at`, `from_cache` o `stale`.

La respuesta vuelve al *frontend*, que renderiza tarjetas con `renderNewsCards()` en `search.js` y actualiza el estado visual de última actualización. En el caso de la figura anterior se ve "Actualizado hace unos segundos". Si no hay datos, muestra un estado vacío controlado en lugar de romper la interfaz. De este modo, el flujo prioriza noticias frescas, pero mantiene continuidad con caché cuando la fuente externa falla o no aporta contenido nuevo. Además, la caché de noticias permite no saturar la API gratuita de Alpha Vantage y reutilizar la búsqueda de un usuario para todos durante una hora.

6.5.2.4 Rentabilidad activo

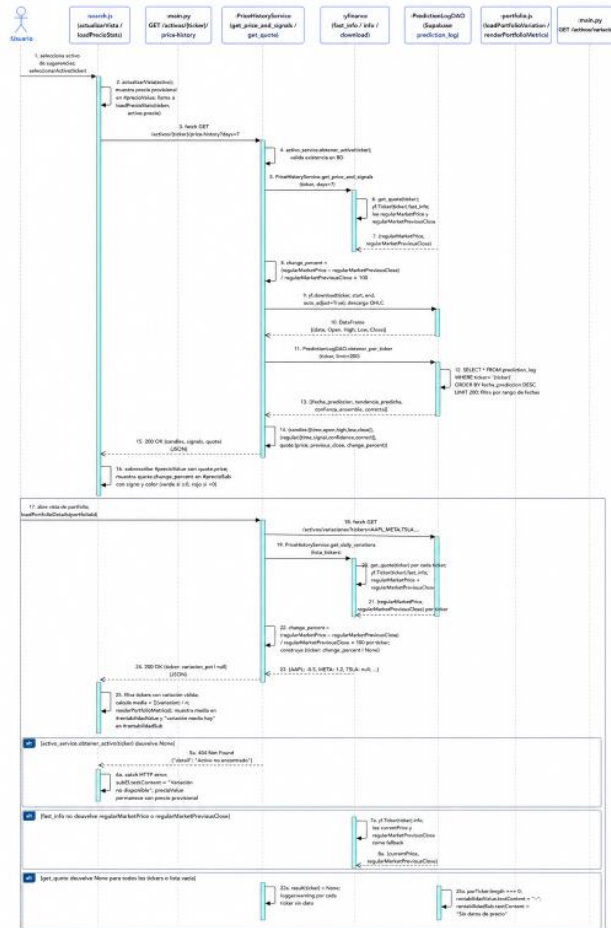


Figura 54 - Diagrama UML rentabilidad activo

Cuando el usuario selecciona un activo, `actualizarVista` normaliza el objeto recibido con `normalizeActivoPayload()` y extrae el precio actual, que pasa a `loadPrecioStats`. Esta función muestra el precio inicial en pantalla y lanza una petición `GET /activos/{ticker}/quote` al *backend*. En el *backend*, `main.py` verifica primero que el activo exista llamando a `activo_service.obtener_activo(ticker)`. Si no se encuentra, devuelve `404 Not Found` y el *frontend* muestra "Variación no disponible", abortando el flujo.

Si el activo existe, `main.py` delega en `PriceHistoryService.get_quote(ticker)`. Este método consulta `yf.Ticker(ticker).fast_info` en tiempo real y obtiene dos valores fundamentales: `regularMarketPrice` (precio actual del mercado) y

regularMarketPreviousClose (cierre oficial del día anterior, sin ajustes por dividendos ni splits, igual que muestra Google Finance). Si fast_info no entrega ambos campos, hay un *fallback* a yf.Ticker(ticker).info con las claves equivalentes (currentPrice, regularMarketPreviousClose). Con esos dos valores calcula el porcentaje de variación del día: $(\text{precio_actual} - \text{cierre_anterior}) / \text{cierre_anterior} \times 100$. La respuesta al *frontend* es un objeto ligero con tres campos: {price, previous_close, change_percent}, sin descargar histórico OHLC ni señales IA, manteniendo la petición eficiente.

De vuelta en el *frontend*, loadPrecioStats usa directamente quote.price para sobrescribir el precio mostrado en pantalla (garantizando que numerador y denominador del porcentaje proceden de la misma foto de mercado) y quote.change_percent para mostrar el % hoy con el color correspondiente (verde si positivo, rojo si negativo). Si por algún motivo el *backend* no devolviera el dato de cotización en vivo, change_percent vendría como *null* y la tarjeta mostraría "Sin datos de variación".

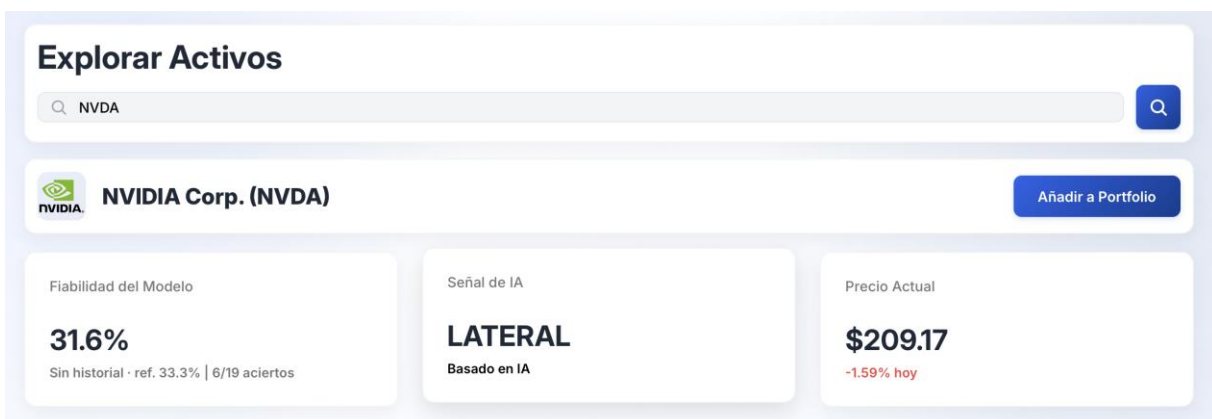


Figura 55 - Rentabilidad Activo

6.5.2.5 Insights IA

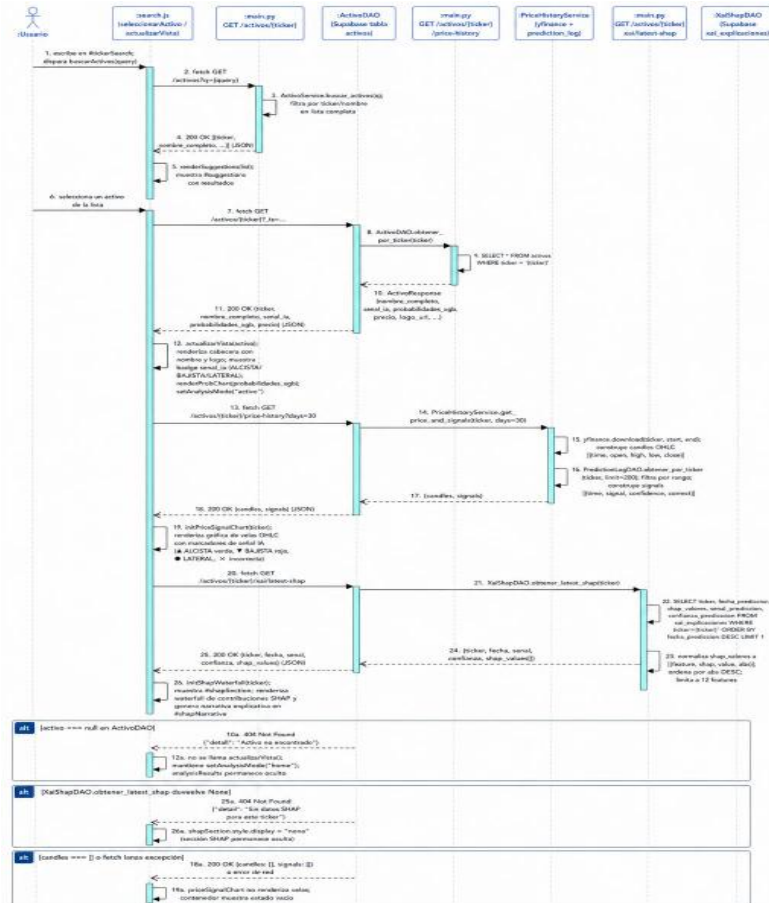


Figura 56 - Diagrama UML insights IA

El flujo de *insights* de IA se inicia en el frontend (search.js): el usuario escribe en #tickersearch y se lanza buscarActivos(query), que consulta GET /activos?q={query} (main.py). El *backend* carga el catálogo desde Supabase (ActivoDAO.buscar_activos) y filtra por ticker/nombre, devolviendo una lista de sugerencias. Al seleccionar un activo, el cliente llama a GET /activos/{ticker} para recuperar el *snapshot* principal (nombre_completo, logo_url, señal_ia, probabilidades_xgb, precio) y actualizar la vista (cabecera + badge de señal + gráfico de probabilidades).

A continuación, la UI pide el contexto temporal y la explicabilidad. Para el histórico, llama a GET /activos/{ticker}/price-history?days=30: PriceHistoryService obtiene

velas OHLC desde yfinance y cruza señales reales desde Supabase (prediction_log) vía PredictionLogDAO.obtener_por_ticker (limit/rango), construyendo marcadores con signal, confidence y correct para pintar el gráfico (colores por señal y marca de acierto/error). Para la explicabilidad, solicita GET /activos/{ticker}/xai/latest-shap: XaiShapDAO.obtener_latest_shap lee xai_explicaciones (última fecha_prediccion), normaliza/ordena shap_values y limita a top features; el frontend renderiza el waterfall SHAP y una narrativa breve asociada.

6.5.3 Gestión de Portfolio

6.5.3.1 Añadir/Eliminar Activo

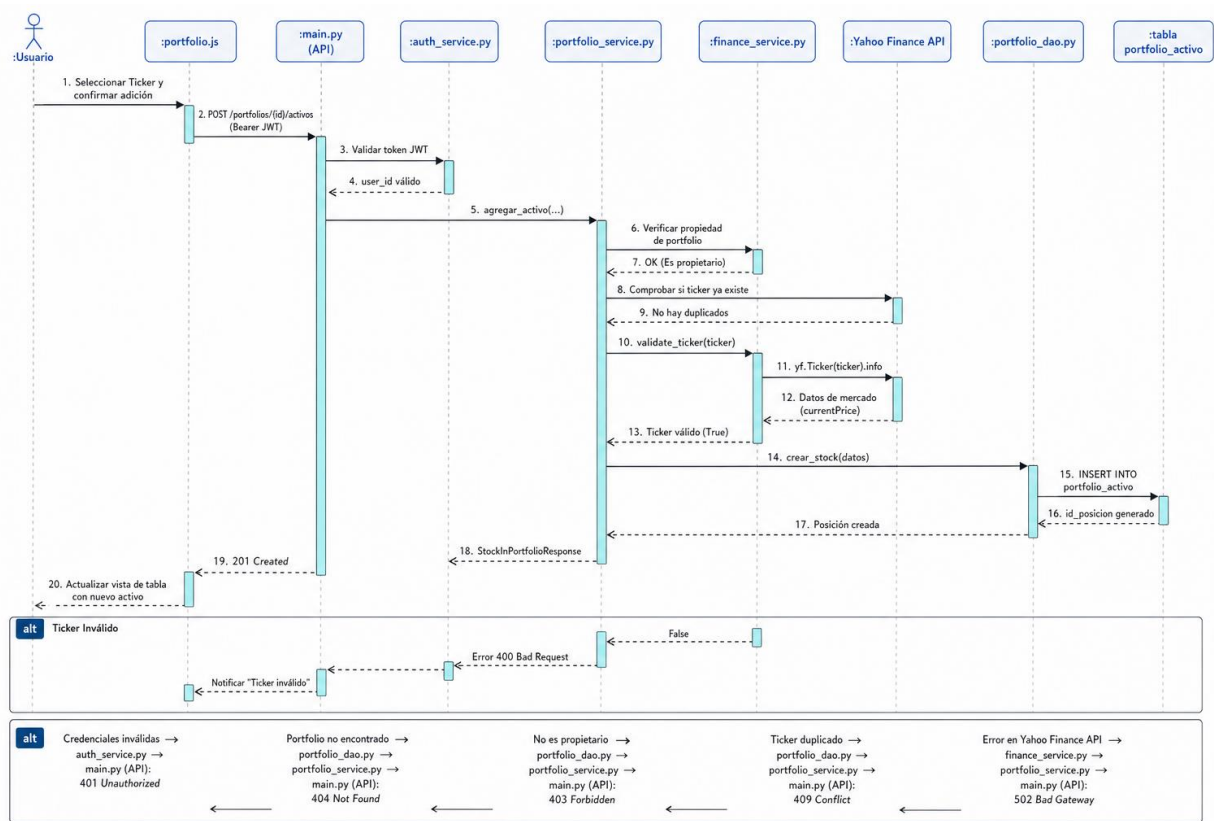


Figura 57 - Diagrama UML de Añadir Activo

El proceso se inicia en el cliente (portfolio.html), donde el usuario interactúa con un modal de búsqueda habilitado por portfolio.js. Al introducir un término, el sistema

realiza peticiones de autocompletado para facilitar la selección. Una vez elegido el activo buscado, la función `addAssetToPortfolio()` construye un objeto `StockInPortfolioRequest` con el *ticker* y emite una petición HTTP POST hacia la ruta `/portfolios/{id_portfolio}/activos`.

En el servidor, el *endpoint* de `main.py` intercepta la llamada y verifica la sesión del usuario mediante la inyección de la dependencia `auth_service.get_current_user`. Superada la validación del *token* JWT, el flujo se delega a `PortfolioService.agregar_activo_portfolio_usuario()`. Esta capa de negocio orquesta una estricta cadena de validaciones:

- Propiedad (Ownership). Se invoca a `_obtener_portfolio_si_es_propietario()` para garantizar que la cartera objetivo pertenece realmente al usuario autenticado, previniendo vulnerabilidades de tipo IDOR (Insecure Direct Object Reference).
- Prevención de duplicados. Se consulta a `PortfolioDAO.obtener_stocks_por_ticker()` para asegurar que el activo no se encuentre ya registrado en esa misma cartera.
- Verificación de mercado. Se delega a `FinanceService.validate_ticker()` la comprobación del *ticker*. Este servicio actúa como un wrapper de la librería `yfinance`, consultando en tiempo real la API de Yahoo Finance. Solo se aprueba la operación si el proveedor externo devuelve información válida de cotización (como `currentPrice` o `regularMarketPrice`).

Si todas las validaciones son satisfactorias, el servicio solicita a `PortfolioDAO.crear_stock()` la inserción del registro en la tabla `portfolio_activo`. El servidor retorna un código HTTP 201 Created junto con la entidad creada (`StockInPortfolioResponse`), y el *frontend* actualiza dinámicamente la tabla de posiciones en la vista del usuario.

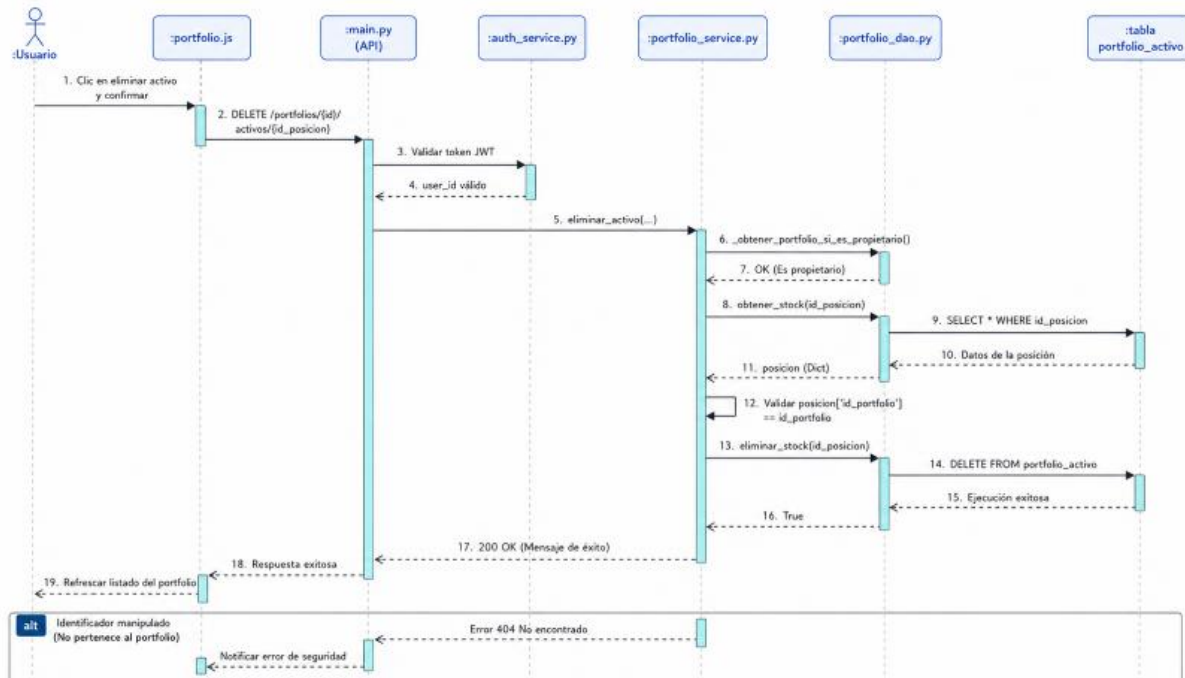


Figura 58 - Diagrama UML de Eliminar Activo

Para quitar un activo de un *portfolio*, el usuario, desde la misma vista de *portfolio*, acciona el botón de borrado en la interfaz, lo que despliega un modal de confirmación para evitar acciones accidentales. Al confirmar, el cliente lanza una petición DELETE a la ruta `/portfolios/{id_portfolio}/activos/{id_posicion}`.

Tras la validación estándar del token JWT, `PortfolioService.eliminar_activo_portfolio_usuario()` toma el control. Además de revalidar la propiedad de la cartera, el servicio añade una capa de seguridad adicional en el acceso a datos: verifica mediante `PortfolioDAO.obtener_stock()` que la posición referenciada por `id_posicion` existe y, fundamentalmente, que pertenece al `id_portfolio` indicado en la ruta. Esto evita que un usuario manipule el identificador de la posición para borrar activos de otras carteras. Finalmente, se ejecuta el borrado físico de la fila en la tabla `portfolio_activo` y se retorna un código HTTP 200 OK, instruyendo al *frontend* a refrescar la vista.

6.5.3.2 Crear/Eliminar Portoflio

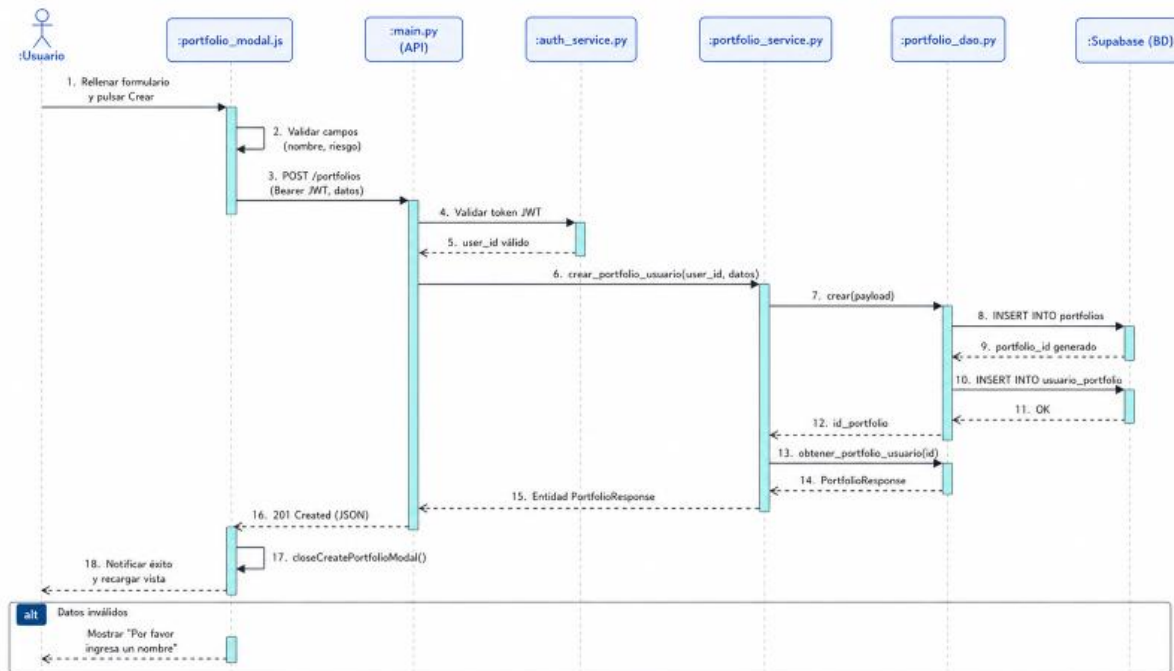


Figura 59 - Diagrama UML de Crear Portoflio

El ciclo de creación se inicia a través de la interfaz de usuario con la invocación de la función `showCreatePortfolioModal()` presente en `portfolio_modal.js`, haciendo clic sobre el botón de "+ Nuevo". El usuario debe proporcionar los metadatos de la nueva cartera: un nombre identificativo, una descripción estratégica y un nivel de tolerancia al riesgo parametrizado (Conservador, Moderado o Agresivo). Tras la validación local de estos campos, la función `createPortfolio()` serializa los datos y emite una petición HTTP POST a la ruta `/portfolios`, adjuntando el `token JWT` de sesión.

En la capa *backend* (`main.py`), la ruta está protegida por la dependencia `auth_service.get_current_user`, que verifica la validez del `token` contra Supabase Auth. Una vez resuelto el `user_id`, el control pasa a `PortfolioService.crear_portfolio_usuario()`. Este servicio instancia el objeto de transferencia de datos (`PortfolioCreateRequest`) y delega la persistencia a `PortfolioDAO.crear()`. A nivel de base de datos, la inserción se realiza de forma transaccional: primero se registra la entidad central en la tabla `portfolios` e, inmediatamente después, se consolida la relación de pertenencia

insertando el registro correspondiente en la tabla intermedia usuario_portfolio. Finalmente, el servicio recupera la entidad completa y el servidor retorna un código HTTP 201 Created, lo que desencadena una actualización asíncrona del DOM en el cliente para mostrar la nueva cartera.

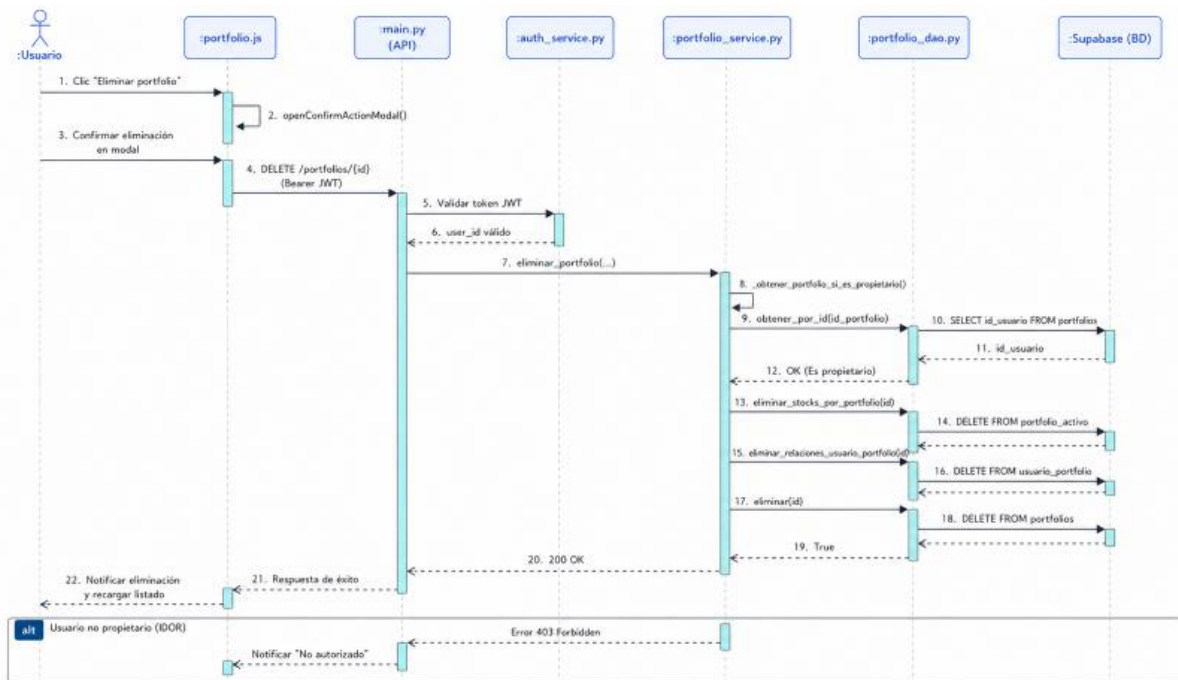


Figura 60 - Diagrama UML de Eliminar Portfolio

La funcionalidad de borrado de un *portfolio* representa una operación crítica, por lo que su diseño incluye medidas preventivas y mecanismos de limpieza en cascada. En el *frontend*, el evento gestionado por `deleteCurrentPortfolio()` en `portfolio.js` despliega un modal de seguridad (`openConfirmActionModal`) para solicitar la confirmación explícita del usuario. Al ratificar la acción, se envía una petición DELETE a `/portfolios/{id_portfolio}`.

Tras la validación del *token* de seguridad, el controlador invoca a `PortfolioService.eliminar_portfolio_usuario()`. La primera y más importante validación es el control de acceso: el servicio ejecuta `_obtener_portfolio_si_es_propietario()` para consultar la base de datos y certificar que el `id_usuario` asociado al *portfolio* coincide unívocamente con el del emisor de la petición. Esta validación anula cualquier intento

de inyección o manipulación de identificadores por parte de terceros (prevención IDOR).

Superada la validación, el servicio orquesta un borrado jerárquico para preservar la integridad referencial y evitar datos huérfanos. A través de PortfolioDAO, se ejecutan tres operaciones secuenciales de eliminación física:

- Se eliminan todas las posiciones financieras asociadas en la tabla `portfolio_activo`.
- Se borran las vinculaciones de propiedad en la tabla `usuario_portfolio`.
- Se elimina el registro raíz de la tabla `portfolios`.

Si el flujo culmina sin excepciones, la API responde con un código HTTP 200 OK y el cliente procede a retirar el *portfolio* del sistema Horizon.

6.5.3.3 Rentabilidad Portfolio

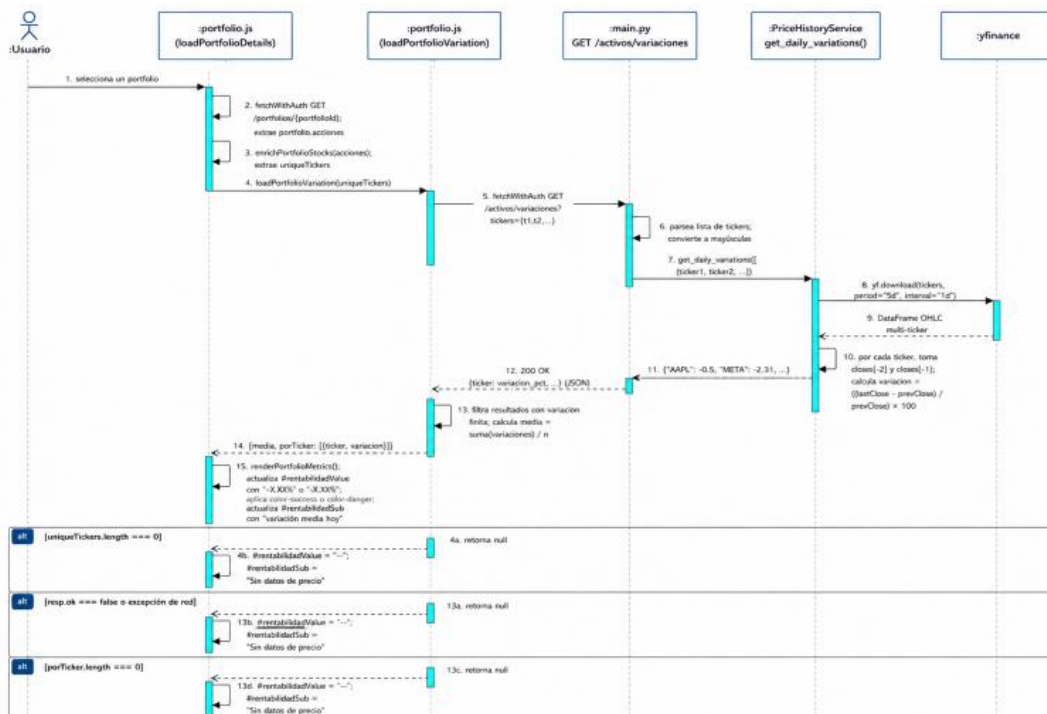


Figura 61 - Diagrama UML Rentabilidad Portfolio

Cuando el usuario selecciona un *portfolio*, *loadPortfolioDetails* obtiene sus acciones del *backend*, enriquece los datos y extrae los *tickers* únicos. Luego delega en *loadPortfolioVariation*, que realiza una única llamada batch a *GET /activos/variaciones?tickers*. El *backend* pasa los *tickers* a *PriceHistoryService.get_daily_variations()*, que descarga 5 días de precios con *yf.download* y calcula por cada *ticker* la variación $((lastClose - prevClose) / prevClose) \times 100$. Devuelve un diccionario {*ticker*: *pct*} al *frontend*, donde se calcula la media igual-ponderada de todos los *tickers* con variación finita, y se renderiza en *#rentabilidadValue* con color verde/rojo.



Figura 62 - Rentabilidad Portfolio

6.5.3.4 Avisos

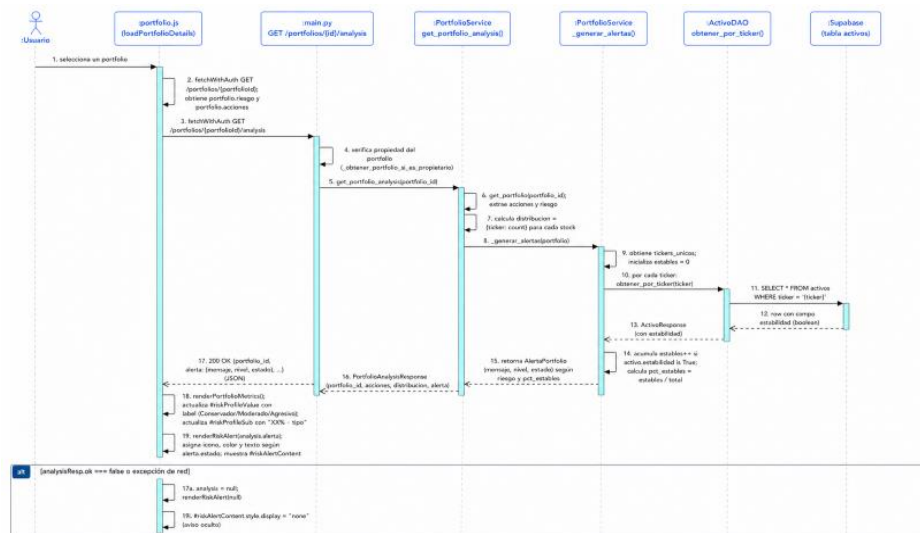


Figura 63 - Diagrama UML Avisos

El diagrama modela el flujo completo que se activa cuando el usuario selecciona un *portfolio*. En primer lugar, *portfolio.js* obtiene los datos del *portfolio* y, en paralelo, solicita su análisis al *endpoint* GET */portfolios/{id}/analysis*. El *backend* valida que el usuario sea propietario del *portfolio* antes de delegar al *PortfolioService*, que extrae las acciones, calcula la distribución por *ticker* y llama internamente a *_generar_alertas*.

_generar_alertas es el núcleo de la lógica de alertas: consulta en Supabase la estabilidad de cada *ticker* único del *portfolio* y calcula el porcentaje de activos estables. Si el *portfolio* está vacío devuelve "Portfolio vacío, añade activos para empezar". Si tiene un solo activo, "Solo 1 activo: considera diversificar" (*warning*). Para *portfolios* con más activos, cruza el valor de riesgo con *pct_estables*: un perfil conservador con ≥ 70 % de activos estables recibe "Cartera alineada con tu perfil conservador" (*ok*), pero si baja del 40 % obtiene "Cartera demasiado arriesgada para perfil conservador" (*danger*). El perfil moderado es correcto entre el 40–70 % de estables ("Buena diversificación para perfil moderado", *ok*) y avisa si se desequilibra hacia volátiles o hacia un perfil conservador. El perfil agresivo considera alineada la cartera mientras los estables sean ≤ 50 %, y emite "Cartera más conservadora que tu perfil agresivo" (*warning*) si los supera.

La respuesta JSON viaja de vuelta a *portfolio.js*, que con *renderRiskAlert* traduce el campo estado (*ok*, *infra*, *sobre*) a un icono, color de fondo y título fijo mientras muestra el mensaje concreto generado por el servicio. Si la llamada falla (*red* o HTTP error), *portfolio* queda a *null* y el bloque de alerta simplemente se oculta, sin romper el resto de la interfaz.

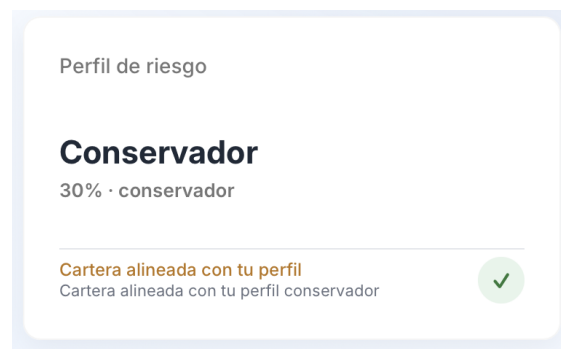


Figura 64 - Avisos

6.5.4 Academia

6.5.4.1 Cursos

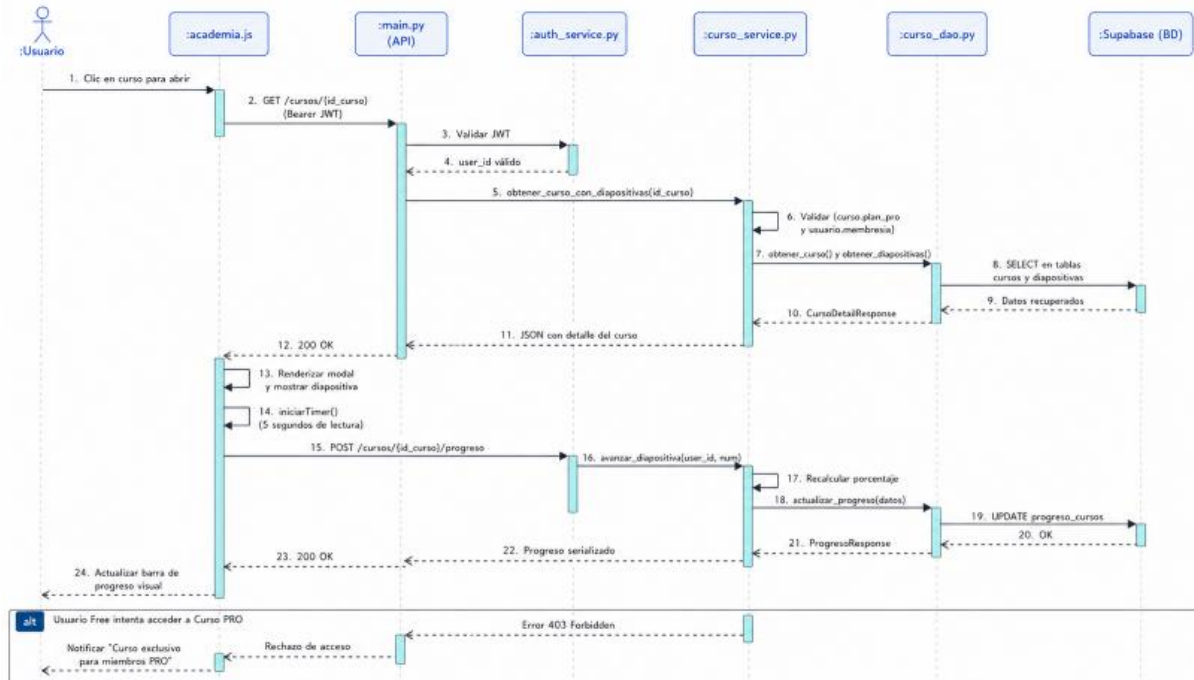


Figura 65 - Diagrama UML de Cursos en Academia

El ciclo de vida del módulo comienza con la carga inicial del panel (`academia.js`), donde, tras la validación inmediata del `token` JWT en la caché local, se lanzan peticiones asíncronas hacia los `endpoints` `GET /cursos` y `GET /cursos/resumen/progreso`. La capa de servicio del `backend` (`CursoService`) organiza llamadas a `CursoDAO` para consultar las tablas `cursos` y `progreso_cursos` en Supabase, retornando el catálogo disponible y el total de cursos completados por el usuario para su renderizado.

El acceso a un curso específico desencadena un flujo estricto de autorización. Al invocar `GET /cursos/{id_curso}`, el controlador no solo recupera los metadatos del curso y sus diapositivas asociadas, sino que aplica una regla de negocio crítica: si el atributo `plan_pro` del curso es verdadero, se verifica en tiempo real que el campo `membreria` del usuario autenticado corresponda a 'Pro'. De no ser así, el servidor aborta la petición devolviendo un código `403 Forbidden`. Paralelamente, se ejecuta `GET /cursos/{id_curso}/progreso`, un `endpoint` diseñado bajo el patrón `get-or-create`: si el

usuario accede por primera vez, `CursoService` inicializa un registro en la tabla `progreso_cursos`; de lo contrario, devuelve el estado guardado, permitiendo al *frontend* reanudar el carrusel en la última diapositiva visitada.

Para la trazabilidad del aprendizaje, el sistema implementa una arquitectura de guardado pasivo orientada a eventos. En el *frontend*, el estado global gestiona una estructura de datos de tipo `Set` que registra las diapositivas ya visualizadas. Cuando el usuario accede a una diapositiva nueva, se activa un temporizador asíncrono de 5 segundos. Transcurrido este umbral de validación de lectura, el cliente dispara automáticamente una petición `POST` a `/cursos/{id_curso}/progreso`.

En el *backend*, el método `CursoService.avanzar_diapositiva()` intercepta esta carga útil, valida que el índice de la diapositiva no exceda la longitud del curso y recalcula dinámicamente el porcentaje completado (`progreso_pct`). Si la petición corresponde a la última diapositiva, el servicio actualiza la tupla en la base de datos marcando el flag booleano `completado = true`. Finalmente, la API devuelve el objeto de progreso actualizado, permitiendo a la interfaz mostrar retroalimentación visual (como barras de progreso y certificaciones de finalización) de manera reactiva.

6.5.4.2 Chat

La funcionalidad del asistente virtual se inicia cuando el usuario, desde la sección *AcademIA*, interactúa con la interfaz ejecutando el método `enviarMensaje()` en `academia.js`, pulsando el botón. En este punto, el *frontend* realiza una validación de seguridad mediante `window.fetchWithAuth()`, que adjunta el *token* `JWT` antes de dirigir la solicitud al *endpoint* `POST /chat` en `main.py`. El *backend* garantiza la integridad del acceso mediante la inyección de la dependencia `get_current_user` en `auth_service.py`, la cual verifica la identidad del usuario con *Supabase*. Cualquier inconsistencia en este nivel resulta en una interrupción del flujo con una respuesta `HTTP 401`.

Tras la autenticación, el sistema gestiona el control de flujo y cuotas a través de `chat_service.py`. El servicio invoca a `chat_dao.verificar_limite_mensajes()`, el cual ejecuta una función `RPC` transaccional en la base de datos para validar el `rate limit`

según la membresía del usuario. Si se excede el límite permitido en la ventana de tiempo definida, el sistema lanza un error capturado por `exception_handlers.py`, devolviendo al cliente un estado HTTP 429. Si la solicitud es admitida, se procede al preprocesamiento en `_preprocesar_mensaje()`, donde se normaliza el texto y se valida (de nuevo, ya que también se valida en el propio componente input html) que su longitud no exceda los 100 caracteres, asegurando así la eficiencia en el consumo de *tokens*.

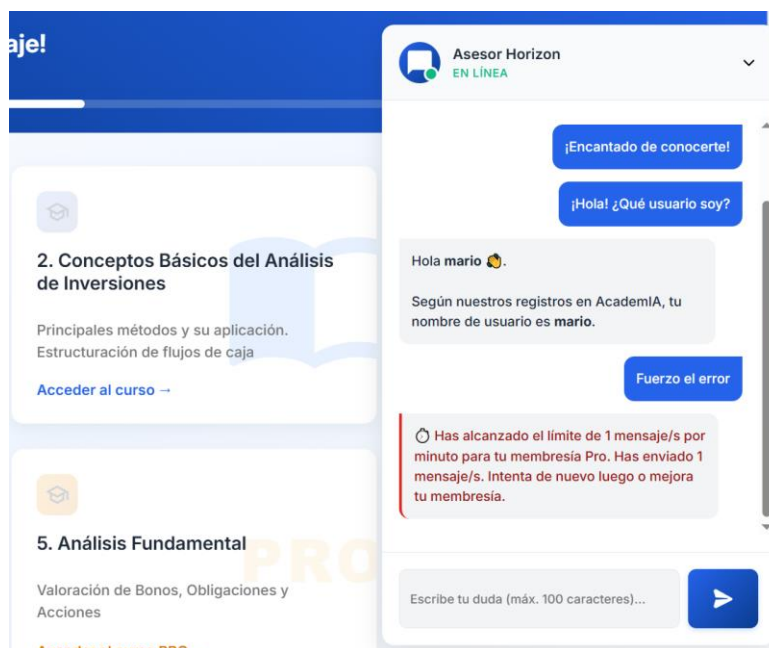


Figura 66 - Vista Chat Academia

Además, el sistema construye un contexto enriquecido mediante el método `_construir_contexto_chat()`. Este componente recopila información desde múltiples DAOs (`curso_dao.py`, `portfolio_dao.py`, `usuario_dao.py`) para estructurar datos sobre el progreso académico y los activos del usuario. La arquitectura implementa una degradación escalada, donde cada consulta está protegida por bloques de control de excepciones; esto garantiza que fallos parciales en la recuperación de datos no detengan la generación de la respuesta. Si no consigue extraer los cursos completados, continúa buscando lo siguiente (*portfolios* del usuario), sacrificando el contexto por una respuesta.

Antes del envío, `gemini_service.py` genera un prompt organizado, serializado y truncado a un máximo de 2500 caracteres, separando por secciones:

- Identidad: quién es “Asesor Horizon” y en qué app está. “Eres un asesor financiero experto ...”
- Reglas de seguridad: Se le indica que no debe asegurar beneficios ni recomendar inversiones concretas, debe redirigir la conversación al ámbito financiero, sin inventar datos, y debe mantener un tono académico.
- Contexto del usuario: Se inyecta el contexto generado por el método comentado. Esto incluye los datos del usuario (sin lo relativo a la autenticación), sus *portfolios*, sus cursos completados junto con los cursos y activos disponibles.
- Mensaje: Se añade el mensaje preprocesado del usuario, normalizando el texto.
- Estilo de respuesta: Concisa, uso de markdown, párrafos, etc.

La generación de la respuesta ocurre en `geminiservice.generar_respuesta()`. El servicio emplea una estrategia de *fallback* robusto, iterando sobre una lista de modelos candidatos (Gemini 2.5 Flash y Gemini 2.0 Flash, incluidos en la API gratuita de Google AI Studio) y aplicando un *backoff* exponencial ante errores transitorios. Si tras los reintentos persiste un fallo de cuota o disponibilidad, el sistema mapea la excepción a códigos HTTP específicos (503 o 429) que informan con precisión al usuario sobre el estado del servicio de inteligencia artificial.

Finalmente, una vez obtenida la respuesta de Gemini, `chat_service.py` la empaqueta en un objeto `ChatMessageResponse` (`chat_dto.py`) y la transfiere al *frontend*. En `academia.js`, el sistema recibe el JSON y, tras validar el éxito de la operación, procesa el texto Markdown para su renderizado en la interfaz. En caso de detectarse un error en cualquier etapa de la cadena (validación, conexión o servidor), la interfaz ejecuta `mostrarError()`, presentando una notificación visual diferenciada por colores e iconos que permite al usuario identificar si el problema reside en sus credenciales, en el límite de mensajes o en la disponibilidad general del sistema, como se puede ver en la figura de Vista Chat Academia.

6.5.5 Dashboard General

6.5.5.1 Artículos Reflexivos



Figura 67 - Vista Reflexiones

La funcionalidad de citas y artículos reflexivos se usa en dos momentos: la previsualización en el *dashboard* y la lectura completa en la vista de artículo. En el *dashboard*, `index.js` ejecuta `cargarReflexion()`, que consume `main.py` mediante `GET /reflexion/aleatoria`. Si la respuesta es válida, actualiza cita y autor en la tarjeta y enlaza al artículo completo con el identificador recibido.

En *backend*, el *endpoint* delega en `reflexion_service.py`, concretamente en `obtener_aleatoria()`. El servicio no consulta SQL directamente: orquesta el acceso a datos a través del DAO `reflexion_dao.py`. Ese DAO llama a Supabase usando una función SQL almacenada (RPC) con `supabase.rpc("get_reflexion_aleatoria").execute()`, definida del lado de la base de datos y diseñada para retornar una reflexión activa de forma aleatoria. Esta capa permite encapsular la lógica aleatoria en SQL y mantener el *backend* limpio.

Cuando el usuario pulsa la tarjeta, se abre la página de detalle y `reflexion.js` extrae el id desde la URL para solicitar `GET /reflexion/{id}` en `main.py`. El *backend* resuelve la consulta con `obtener_por_id()` en `reflexion_service.py` y `reflexion_dao.py`, devolviendo el

payload completo definido en reflexion_dto.py: cita, autor, tema, título, contenido y metadatos. Con ello, el flujo separa claramente descubrimiento aleatorio (RPC SQL en Supabase) y consulta determinista por identificador (SELECT por clave primaria).

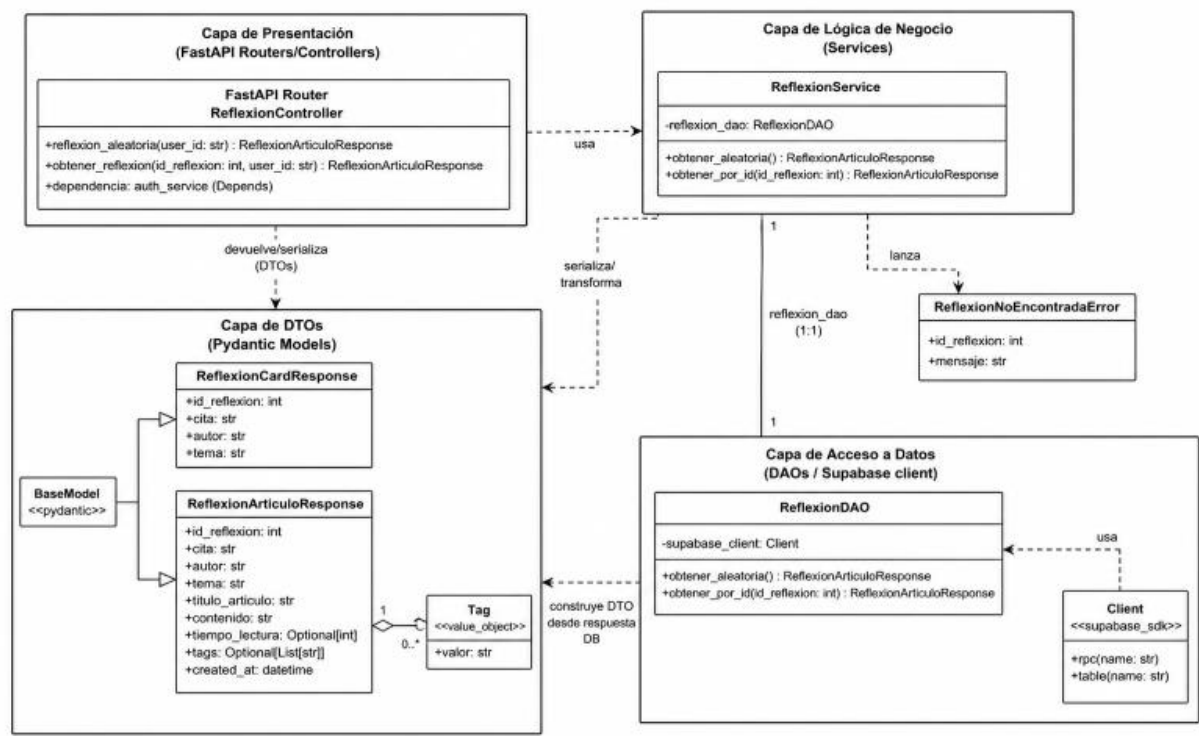


Figura 68 - Diagrama de clases de Reflexiones

6.5.5.2 Modo oscuro

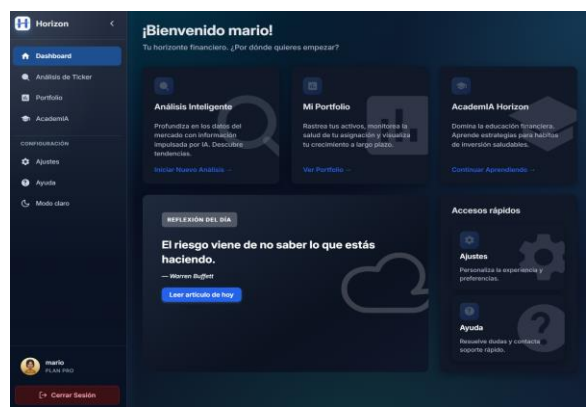


Figura 69 - Vista Dashboard Modo Oscuro

El modo oscuro se activa desde la interfaz, ya sea en el botón de la *sidebar* o desde la pantalla de ajustes. En ambos casos, la acción termina llamando a `setThemePreference()` de `config.js`, que guarda la preferencia en *localStorage* y aplica inmediatamente el tema resolviéndolo con `applyThemePreference()` del mismo archivo. Si el usuario elige “system”, el sistema usa la preferencia del navegador mediante `resolveThemePreference()` y `matchMedia` para decidir entre claro y oscuro.

Una vez aplicado, el tema se materializa en el documento mediante el atributo `data-theme` sobre el HTML, que es lo que consumen el resto de los módulos de la aplicación. En el layout (barra lateral en escritorio), `syncThemeToggleState()` sincroniza el estado visual del selector, cambia la etiqueta del botón y alterna el logo según el tema activo; además, escucha el evento `horizon:theme-changed` para mantener todo actualizado cuando el usuario cambia la preferencia. En ajustes, `getSavedThemePreference()` y `setThemePreference()` permiten leer y guardar el valor seleccionado, en `config.js`.

El efecto se extiende al resto de vistas y gráficos porque varios componentes leen directamente `data-theme` para adaptar su renderizado, como `priceSignalChart.js`, `shapTemporal.js` y `shapWaterfall.js`. El flujo queda centralizado en `config.js`, propagado por evento y reflejado en la interfaz de usuario sin recargar la página.

Además, la hoja de estilos general (`styles.css`) lee la variable `data-theme`, y sobrescribe variables y componentes dentro del bloque de modo oscuro, usando selectores `[data-theme="dark"]` para cambiar colores, fondos, bordes y sombras. Ahí también hay reglas específicas para elementos concretos, por ejemplo, botón de *logout*, *login*, mensajes de error, previews del tema y contenedor de logos de *tickers*.

6.5.5.3 Ayuda

Al cargar `ayuda.html`, accediendo por la barra lateral, la página de inicio, o mi perfil, el *script* `ayuda.js` invoca `window.validateAuthToken()` para verificar que el *token* JWT del usuario sea válido consultando `GET /auth/me`. Si la validación falla, se redirige al *login* mediante `window.location.replace('login.html')`. Si es válida, se almacenan los

datos del usuario globalmente y se ejecuta `initAyudaPage(result.user)` cuando el DOM esté completamente cargado.

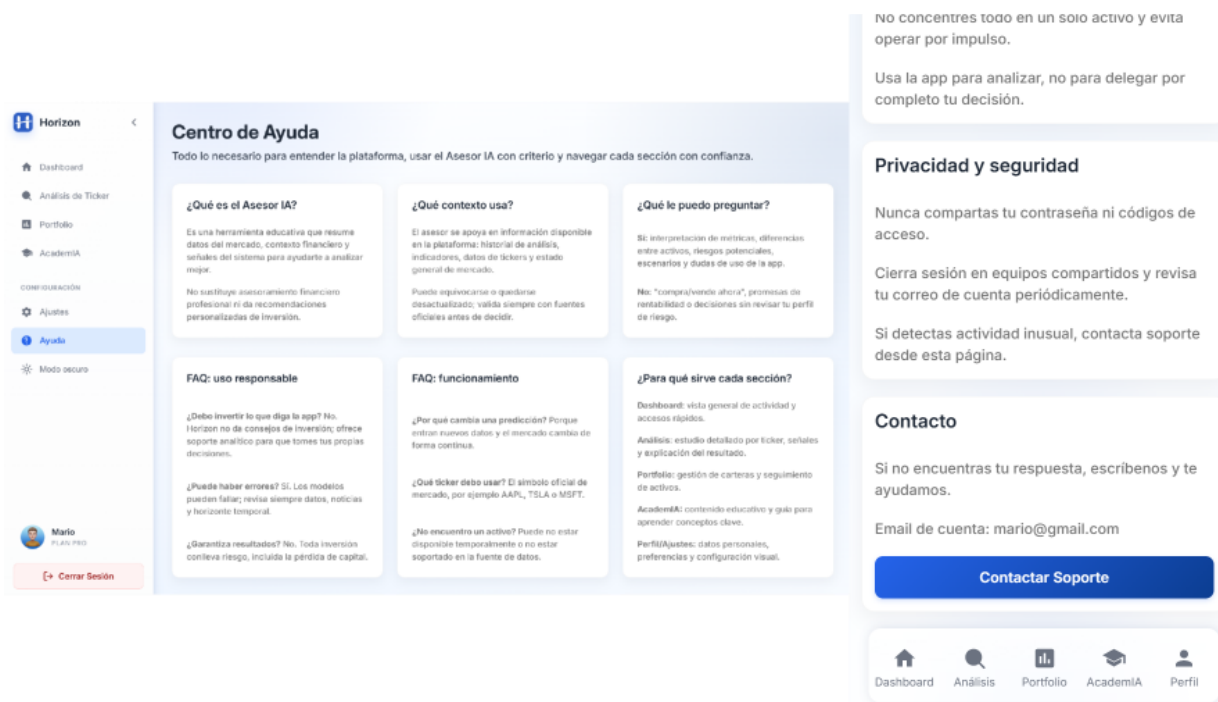


Figura 70 - Vista Ayuda

`ayuda.html` organiza nueve tarjetas (elementos `<article class="card">`) distribuidas en tres secciones temáticas. La primera introduce el Asesor IA: su definición como herramienta educativa no prescriptiva, el contexto que utiliza (datos de la plataforma), y limitaciones sobre qué preguntas responde. La segunda agrupa preguntas frecuentes funcionales: sobre responsabilidad de uso (la app no garantiza rentabilidad, los modelos pueden fallar), funcionamiento técnico (cambios en predicciones por nuevos datos, símbolos de *ticker* requeridos), y descripción breve del propósito de cada sección (*Dashboard*, *Análisis*, *Portfolio*, *AcademiaIA*, *Perfil*). La tercera cubre buenas prácticas de inversión, privacidad y seguridad.

En `ayuda.js`, `initAyudaPage(user)` localiza el elemento `supportUserEmail` y lo actualiza con Email de cuenta: `${user?.email}`, permitiendo al usuario confirmar su identidad. Luego asigna un *listener* al botón `btnContactSupport` que, al ser accionado, abre el cliente de correo predeterminado con `window.location.href =`

<mailto:soporte@horizon.app?subject=Ayuda%20con%20mi%20cuenta%20Horizon>, delegando la gestión de soporte a protocolos nativos del navegador sin requerir lógica *backend*.

6.6 Pipeline ETL y Módulo de Inteligencia Artificial

6.6.1 Arquitectura General del Pipeline de Datos

Horizon se implementa como un *pipeline* de analítica financiera organizado en cuatro capas técnicas: ingesta de datos, transformación cuantitativa, inferencia con explicabilidad y persistencia/servicio. Esta separación permite desacoplar responsabilidades y facilitar mantenimiento, trazabilidad y evolución del sistema.

En la capa de ingesta, el sistema obtiene series temporales de mercado y prepara su disponibilidad para el resto del flujo. La capa de transformación genera variables derivadas a partir de OHLCV para alimentar un modelo supervisado en formato tabular. En la capa de inferencia y entrenamiento, XGBoost produce probabilidades de tendencia, calibración de probabilidades y generación de métricas y el módulo de explicabilidad añade información interpretativa para justificar la predicción. Finalmente, la capa de persistencia y servicio almacena resultados y los expone mediante *backend* FastAPI para su consumo en *frontend*.

Desde el punto de vista operativo, la arquitectura está orientada a ejecución periódica de tareas analíticas, aunque su grado de automatización efectiva depende de la configuración de despliegue.

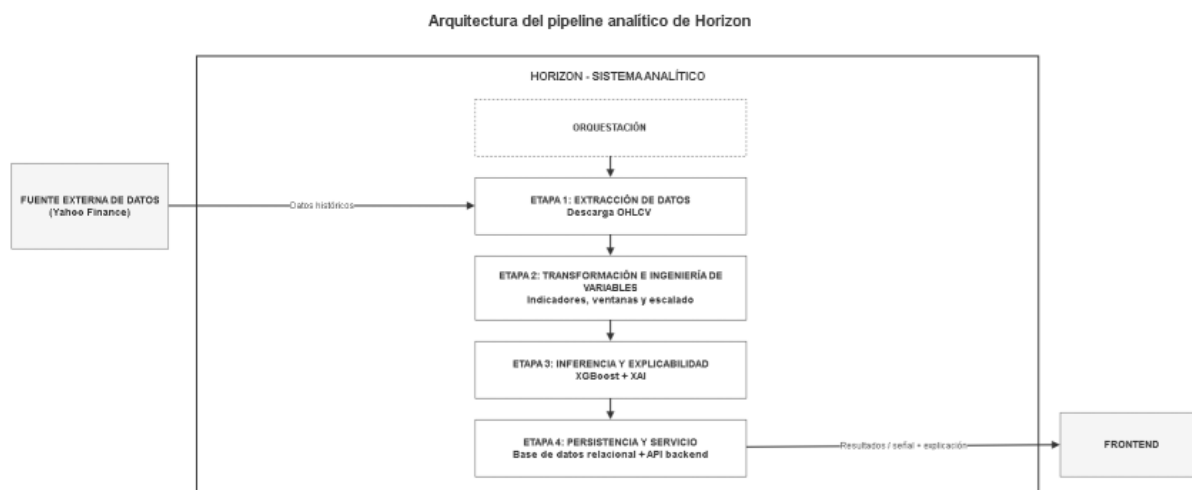


Figura 71 - Diagrama de arquitectura del pipeline

6.6.2 Etapa de Extracción de Datos

La etapa de extracción constituye el punto de entrada de datos en Horizon y su función es abastecer al *pipeline* con series temporales bursátiles actualizadas para entrenamiento e inferencia. Para ello, el proyecto emplea yfinance como interfaz de acceso a Yahoo Finance, una elección alineada con las restricciones de coste propias de un TFG y suficiente para el alcance experimental definido.

Para el entrenamiento del modelo se descarga una ventana histórica inicial que comienza en 2018-01-01 y se extiende hasta la última fecha disponible en Yahoo Finance en el momento de ejecución. No obstante, la ventana efectiva se reduce tras el preprocesado, ya que se eliminan observaciones iniciales por el cálculo de indicadores de ventana larga (p. ej., SMA200) y observaciones finales por la definición del objetivo a 5 días. Sobre el conjunto depurado se aplica una partición cronológica 70/15/15 (entrenamiento/validación/prueba), preservando la consistencia temporal y reduciendo el riesgo de fuga de información.

La ingesta se realiza sobre un conjunto cerrado de activos configurados en el sistema. Para cada activo se obtienen series OHLCV con granularidad diaria y horizonte

histórico amplio, con el fin de cubrir distintos regímenes de mercado y proporcionar una base adecuada para la fase de ingeniería de variables y modelado supervisado.

Desde el punto de vista operativo, la arquitectura contempla ejecución programada en segundo plano para desacoplar la actualización de datos de la interacción del usuario. Dado que la fuente es una API pública gratuita, el proceso incorpora manejo de incidencias de comunicación y controles básicos de robustez, con el objetivo de mantener continuidad en ciclos sucesivos sin asumir disponibilidad ininterrumpida.

6.6.3 Etapa de Transformación e Ingeniería de Variables

Una vez completada la extracción, las series temporales se someten a una etapa de transformación e ingeniería de variables orientada a su uso en modelos tabulares. En Horizon, esta fase condiciona el rendimiento del sistema, ya que XGBoost opera sobre vectores estructurados. A partir de las series OHLCV se calculan indicadores técnicos y de régimen, entre ellos RSI, MACD, ATR y Bandas de Bollinger, junto con métricas de momento y volatilidad, conformando el conjunto base de variables de entrada.

Para incorporar dependencia temporal, las observaciones se organizan en ventanas retrospectivas según el tipo de activo definido en la configuración del proyecto (30 sesiones para activos estables y 60 para activos volátiles). Sobre estas ventanas, el bloque de XGBoost construye una representación tabular multiescala mediante agregaciones last, mean, std y trend aplicadas sobre subventanas corta, media y completa, preservando información temporal relevante en un formato compatible con modelos basados en árboles.

Antes del entrenamiento y la evaluación, el preprocesado aplica normalización Min-Max mediante `MinMaxScaler`, ajustado exclusivamente con el conjunto de entrenamiento y reutilizado en validación, prueba e inferencia para reducir el riesgo de fuga de información y garantizar consistencia. Los artefactos de transformación se almacenan por activo, reforzando la reproducibilidad del *pipeline*.

6.6.4 Etiquetado y Validación Temporal

Una vez construido el vector de características, Horizon formula la predicción como un problema de clasificación ternaria (bajista, lateral y alcista) sobre un horizonte temporal configurable. El etiquetado se realiza mediante umbrales percentilares calibrados por activo (percentiles 33,3 y 66,7), estimados exclusivamente con la porción de entrenamiento y conservados posteriormente como *frozen thresholds*.

Este esquema permite que validación, prueba e inferencia reutilicen los mismos límites de decisión sin recalibración, manteniendo coherencia entre entrenamiento y evaluación fuera de muestra. Además, al definir umbrales por activo, el criterio de clase se adapta al régimen de cada instrumento y evita imponer fronteras absolutas comunes a series con volatilidades diferentes. Se preserva así la trazabilidad del criterio de decisión, ya que cada predicción puede auditarse con los mismos umbrales utilizados durante el entrenamiento.

En el flujo ejecutado mediante `<python -m backend.models.train_xgboost_all>`, la validación se basa en una partición estrictamente cronológica 70/15/15 (entrenamiento/validación/prueba), sin barajado aleatorio, para minimizar el riesgo de fuga de información futura y mantener condiciones de evaluación consistentes con el escenario operativo.

6.6.5 Etapa de Carga y Persistencia

En la etapa de carga, Horizon persiste en PostgreSQL (Supabase) tanto el estado operativo de cada activo como los resultados de predicción y su trazabilidad. Tras el entrenamiento por *ticker* se actualiza principalmente la tabla activos, que funciona como vista consolidada para *backend* y *frontend*.

Para cada *ticker* se almacenan campos como la señal predicha, confianza, precio del día, fecha de actualización de la señal, probabilidades de cada señal predichas (alcista 35%, bajista 25%, lateral 40%), retorno predicho porcentual, precio

predicho, volatilidad a 30 días y, cuando está disponible, acierto a 30d. Esta operación se ejecuta mediante `ActivoDAO.actualizar()`, con *ticker* como clave primaria en activos.

Cada métrica persistida en activos tiene una procedencia operativa definida dentro del flujo de inferencia. En primer lugar, la señal representa la clase final emitida por XGBoost (BAJISTA, LATERAL o ALCISTA), con posible abstención implementada como reasignación a LATERAL cuando la confianza no supera el umbral de decisión. Por su parte, la confianza almacena realmente la confianza de XGBoost, calculada como la probabilidad máxima entre las tres clases.

El campo precio corresponde al último cierre disponible (*Close*) en los datos descargados durante inferencia, mientras que se registra la marca temporal de actualización. Además, `probabilidades_xgb` conserva la distribución de probabilidad por clase (bajista, lateral, alcista) e incluye el indicador de abstención cuando aplica.

A partir de estas probabilidades se derivan métricas de proyección. Concretamente, el retorno predicho porcentual se calcula como esperanza ponderada usando retornos de referencia por clase (+2%, 0%, -2%). A continuación, precio predicho se obtiene aplicando ese retorno esperado al precio actual. Complementariamente, volatilidad a 30 días se estima como la desviación estándar muestral de los retornos logarítmicos recientes (hasta 30 observaciones), expresada en porcentaje.

Finalmente, el acierto a 30 días solo está disponible cuando existe un volumen mínimo de predicciones ya resueltas en la tabla `prediction_log` (es decir, registros para los que ya se conoce el resultado real en la `fecha_objetivo` y se ha podido marcar correcta).

Además, el sistema mantiene tablas específicas para histórico y evaluación. `prediction_log` registra eventos de predicción (fecha de predicción, fecha objetivo, tendencia predicha, confianza, precio de entrada, horizonte y resolución posterior), mientras que `xai_explicaciones` persiste artefactos de interpretabilidad (valores SHAP, variables relevantes y versión de modelo), facilitando auditoría técnica de las decisiones del modelo.

A nivel de integridad, el esquema combina claves primarias/foráneas y restricciones de dominio para etiquetas (ALCISTA, BAJISTA, LATERAL). Adicionalmente, en `prediction_log` se aplica control de duplicados por combinación `ticker` y `fecha_prediccion` como unicidad lógica gestionada por la capa DAO. Esto refuerza la consistencia entre entidades y la trazabilidad del ciclo completo de predicción.

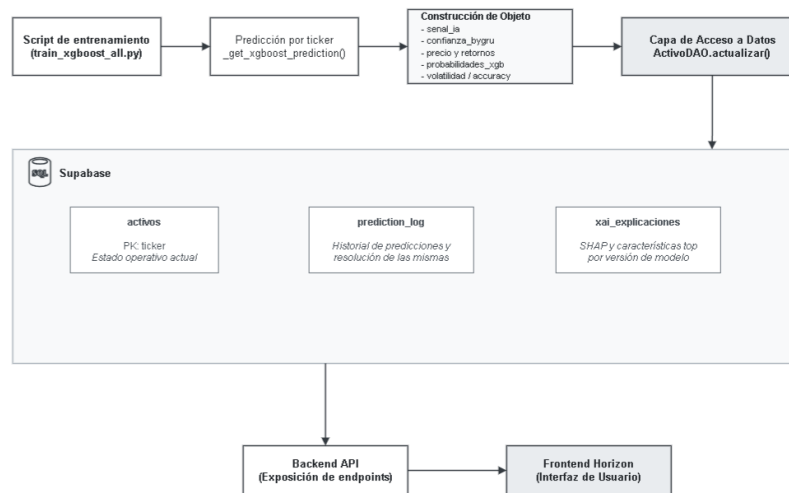


Figura 72 – Diagrama del Flujo de Carga

6.6.6 Modelo Predictivo: eXtreme Gradient Boosting (XGBoost)

El entrenamiento de XGBoost se ejecuta de forma independiente por activo, lo que permite ajustar el modelo a patrones específicos de cada *ticker*. La configuración de hiperparámetros se segmenta por tipo de activo (estable/volátil), en coherencia con el diseño general del *pipeline* y con las diferencias de régimen observadas entre instrumentos. El problema se formula como clasificación multiclase y el modelo se entrena con objetivo `multi:softprob` y función de pérdida `mlogloss`, de modo que produce tanto la clase predicha como la distribución de probabilidades asociada. La entrada al modelo se construye en formato tabular a partir de agregaciones temporales multiescala sobre ventanas retrospectivas, preservando información dinámica relevante en un esquema compatible con árboles.

Para limitar el sobreajuste (*overfitting*), el entrenamiento incorpora *early stopping* sobre validación cronológica. Aunque el etiquetado por activo reduce desequilibrios severos, la distribución de clases no es perfectamente uniforme, por lo que se aplica ponderación de muestras (*sample weights*) según frecuencia de clase para mitigar sesgos hacia la clase mayoritaria y mejorar la estabilidad fuera de muestra. Los artefactos de entrenamiento (modelo por *ticker* y parámetros de decisión asociados, incluidos los umbrales *frozen* por activo) se serializan para su reutilización en inferencia. El rendimiento se reporta principalmente con *balanced accuracy*, adecuada en escenarios multiclase con desbalance residual, y se interpreta frente al *baseline* aleatorio de tres clases (33,3%), priorizando consistencia temporal, reproducibilidad y trazabilidad experimental.

La selección final de XGBoost se adoptó tras una fase comparativa con arquitecturas secuenciales (incluida Bi-GRU) y variantes de preprocesado/etiquetado. En una etapa inicial, sobre Bi-GRU se ensayó además la incorporación de sentimiento de noticias financieras por activo, utilizando un modelo de lenguaje tipo DistilRoBERTa para clasificación de polaridad, con el objetivo de añadir señal exógena al bloque temporal de precios. Aunque este enfoque resultó conceptualmente prometedor, en la práctica introdujo mayor complejidad de integración (alineación temporal noticia-precio, cobertura desigual por *ticker* y mayor coste de inferencia) sin una mejora suficientemente estable en validación. En paralelo, se realizaron iteraciones para superar el *baseline* aleatorio de tres clases (33,3%): ajuste de ventanas por tipo de activo, agregación temporal multiescala (last, mean, std, trend), calibración de umbrales percentilares por *ticker*, ponderación por clase y regla de abstención por confianza. El enfoque tabular con XGBoost concentró el mejor equilibrio entre rendimiento, reproducibilidad, velocidad de ciclo y facilidad de despliegue en producción, por lo que se consolidó como modelo principal del *pipeline*.

6.6.7 Sistema de Etiquetado y Calibración de Umbrales

El aprendizaje supervisado de XGBoost en Horizon se formula como un problema de clasificación ternaria (bajista, lateral, alcista) sobre un horizonte temporal configurable. La variable objetivo-continua se define como el retorno futuro, calculado como variación porcentual entre el precio de cierre actual y el precio de cierre en el horizonte objetivo.

La discretización de ese retorno no se realiza con umbrales absolutos comunes a todos los activos, sino mediante calibración percentilar por *ticker*. En concreto, se estiman los percentiles 33,33 y 66,67 sobre la distribución de retornos del conjunto de entrenamiento. Con ello, los retornos por debajo del primer umbral se etiquetan como bajistas, los superiores al segundo como alcistas y el intervalo intermedio como lateral.

Los umbrales obtenidos se fijan como *frozen thresholds* y se reutilizan sin recalibración en validación, prueba e inferencia. Esta decisión mantiene coherencia entre el criterio de entrenamiento y el criterio de decisión fuera de muestra, y reduce el riesgo de fuga de información temporal al evitar que datos futuros influyan en la definición de clases.

Este enfoque también mejora la comparabilidad entre activos con regímenes distintos de volatilidad: un mismo retorno porcentual puede representar una señal diferente según la dinámica histórica del instrumento. Por ello, la calibración por *ticker* permite un etiquetado más consistente con la estructura estadística de cada serie.

6.6.8 Metodología de Validación y Resultados

La evaluación del modelo se realiza con un protocolo estrictamente temporal, preservando el orden cronológico de las observaciones y evitando barajado aleatorio. En el flujo principal de entrenamiento (`python -m backend.models.train_xgboost_all`), la partición utilizada es 70/15/15 para entrenamiento, validación y prueba, respectivamente, con el objetivo de aproximar el escenario operativo real y reducir fuga de información futura. Durante el ajuste se aplica ponderación de muestras por

frecuencia de clase para compensar desbalance residual, y el rendimiento se resume principalmente mediante *balanced accuracy*, métrica adecuada en clasificación multiclase no perfectamente uniforme.

La interpretación se realiza frente al *baseline* aleatorio de tres clases (33,3%); en los experimentos del proyecto, el desempeño se sitúa por encima de dicho umbral en los múltiples entrenamientos realizados, lo que sugiere señal predictiva útil pero moderada en un entorno financiero ruidoso y no estacionario. El análisis prioriza consistencia temporal, reproducibilidad y trazabilidad experimental sobre mejoras puntuales de métrica.

6.6.9 Inteligencia Artificial Explicable (XAI)

En la fase de inferencia, el modelo XGBoost entrenado por *ticker* procesa datos recientes del activo (ventana de 30 o 60 sesiones, según configuración) y replica el mismo flujo de transformación utilizado en entrenamiento. Para mantener consistencia entre fases, la normalización se realiza con el mismo escalador persistido por activo. Con esta entrada, el modelo genera probabilidades para las tres clases (BAJISTA, LATERAL, ALCISTA).

Sobre esa salida probabilística, Horizon aplica una regla de abstención por confianza. Cuando la probabilidad máxima no supera un umbral mínimo (*confidence_tau*), la señal direccional se reemplaza por LATERAL. Este criterio reduce la emisión de señales con baja convicción en contextos de mayor incertidumbre.

Como capa de transparencia, el sistema integra explicabilidad mediante SHAP, utilizando TreeExplainer para modelos basados en árboles. El objetivo es cuantificar la contribución de cada variable al resultado final y exponer qué factores empujan la predicción en sentido alcista o bajista. Para facilitar la interpretación por parte del usuario, las variables técnicas se presentan con nomenclatura legible y se priorizan las contribuciones más influyentes.

El resultado se materializa en artefactos visuales y metadatos estructurados, como gráficos SHAP (codificados en Base64 para transporte) y contribuciones por variable. Estos elementos se persisten en `xai_explicaciones` junto con información de contexto (*ticker*, señal, confianza, versión de modelo), permitiendo su recuperación desde *backend* y su visualización en *frontend* junto a la señal generada.

De este modo, la predicción no se presenta como una salida opaca, sino como una decisión trazable, auditable e interpretable dentro del flujo operativo del sistema. Toda esta implementación de la IA explicable la vemos reflejada en el apartado de análisis de cada *ticker* con el formato mostrado en la siguiente figura.

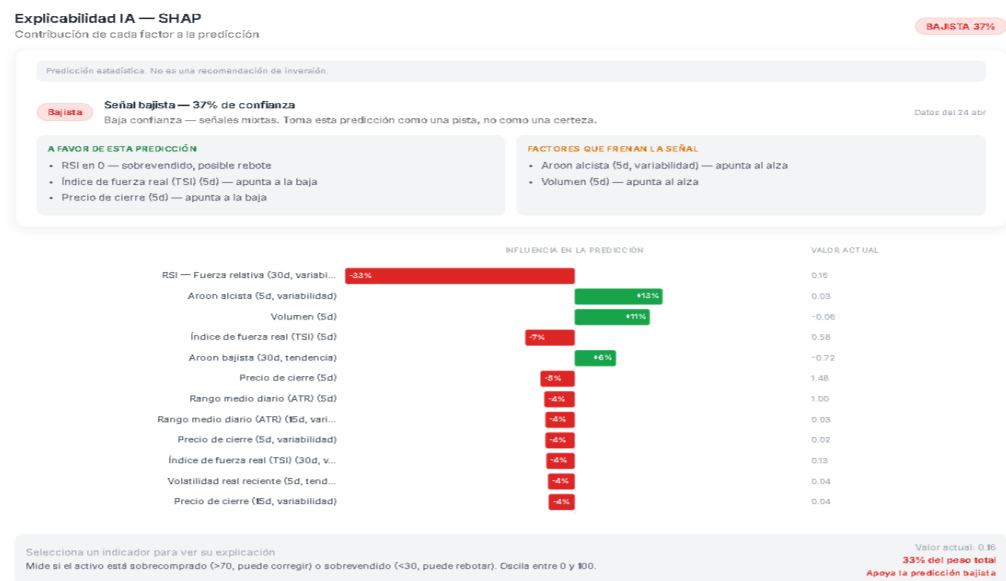


Figura 73- Implementación IA explicable (XAI)

El punto fuerte de este apartado es convertir datos inentendibles para un usuario sin gran formación en finanzas en un formato accesible y explicativo. Para ello se ha optado por una interfaz de usuario pulida que por medio del gráfico facilita el entendimiento del impacto que genera cada métrica en el proceso de predicción del modelo. Se implementa una explicación de carácter técnica pero sencilla de procesar para el usuario acerca que significa cada métrica, reforzando el objetivo del proyecto de aportar herramientas y aprendizaje al usuario inexperto.

6.6.10 Log de Predicciones y Backfill

La tabla `prediction_log` constituye el registro operativo de las predicciones emitidas en producción y de su resultado observado. Su finalidad es separar explícitamente el momento de inferencia del momento de validación real, permitiendo medir desempeño fuera de muestra con coherencia temporal. Cada registro almacena, entre otros campos, el *ticker*, la fecha de predicción, la fecha objetivo en la que se comprueba si la predicción fue correcta, la tendencia predicha, la confianza asociada, el precio de entrada y el horizonte temporal, además del estado de resolución. El *script* que se ejecuta diariamente tras la fecha de cierre para realizar este log de predicciones es el siguiente `< python -m backend.scripts.daily_log run >`, durante el desarrollo del proyecto se ha ejecutado de forma manual cada día.

El ciclo de vida de una predicción se desarrolla en dos etapas. En la primera, al generarse la señal diaria, se inserta una fila con `resuelta = false`, fijando la referencia temporal de predicción y su fecha objetivo. En la segunda, cuando se alcanza dicha fecha y existe dato de mercado suficiente, la predicción se resuelve completando `tendencia_real`, `precio_salida`, `fecha_resolucion` y el campo `correcta`. Este último se calcula comparando la dirección predicha con la dirección efectivamente observada bajo el mismo criterio operativo definido por el sistema.

Para disponer de histórico evaluable desde fases tempranas, se realizó un backfill de `prediction_log` reconstruyendo predicciones pasadas por activo y fecha mediante reglas deterministas. El proceso se diseñó de forma que pudiera reejecutarse sin introducir inconsistencias. En este punto fue clave el control de duplicados por unicidad lógica `ticker + fecha_prediccion` en la capa DAO, evitando inserciones repetidas y priorizando actualización de registros existentes cuando correspondía.

Este registro impacta directamente en las métricas operativas del sistema. En particular, permite calcular el acierto a 30 días sobre predicciones ya resueltas en una ventana móvil reciente, ofreciendo una estimación empírica del rendimiento real en producción. Además, aporta trazabilidad completa del ciclo predicción-resultado, facilita auditoría técnica y permite analizar degradación de comportamiento ante

cambios de régimen de mercado, reforzando la fiabilidad del marco de monitorización del modelo.

Este sistema nos permite mostrar al usuario cuando el modelo ha predicho correctamente la tendencia del mercado de forma gráfica en el análisis de activos. Esto se puede apreciar en la siguiente figura tomada de la web.

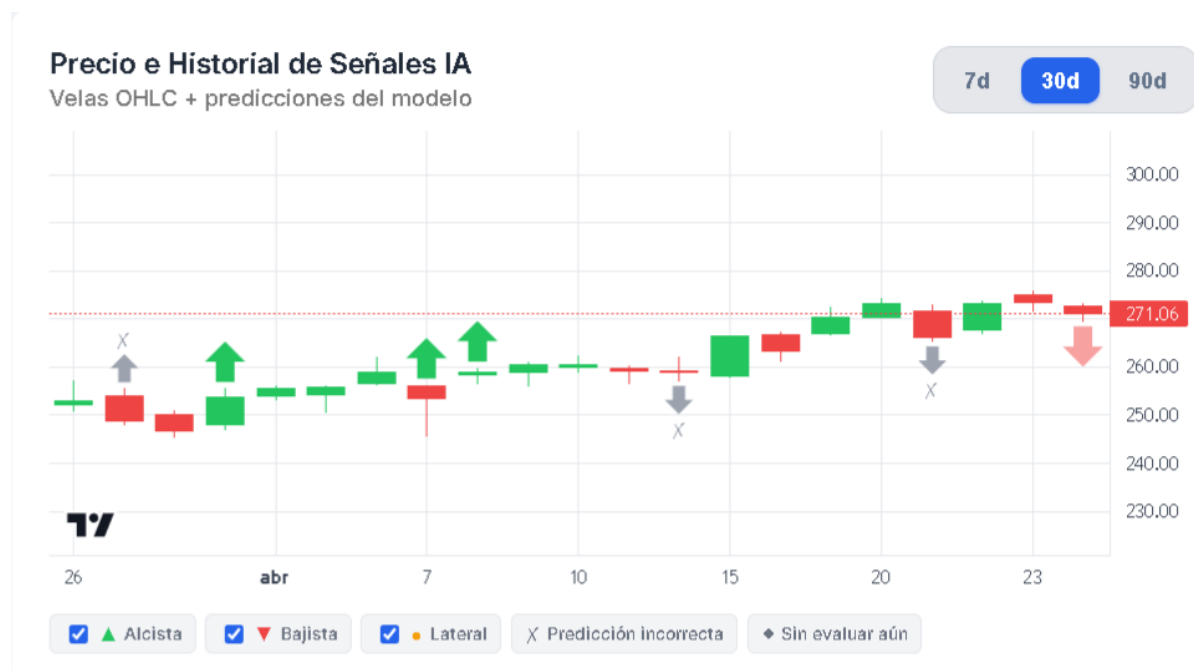


Figura 74- Uso de la Funcionalidad del Log de Predicciones

En esta gráfica integrada en el apartado de análisis de cada *ticker* (en este caso concreto se trata de AAPL) distinguimos entre predicciones correctas de nuestro modelo las flechas ascendentes/descendentes coloreadas, las predicciones incorrectas que son las de color gris con una X y por último las difuminadas que son aquellas que aún no se han confirmado, es decir que aún no se han resuelto siguiendo el método de comprobación mencionado anteriormente.

6.6.11 Limitaciones del Modelo y Líneas Futuras

Aunque el sistema mejora el *baseline* aleatorio de tres clases (33,3%), el rendimiento se mantiene en un rango moderado, en línea con la dificultad intrínseca de la predicción financiera de corto plazo y con los recursos disponibles en el proyecto. La principal restricción metodológica es que el modelo opera sobre un conjunto acotado de variables técnicas derivadas de precio y volumen, lo que limita su capacidad para capturar *shocks* exógenos o cambios estructurales no reflejados de forma inmediata en la serie.

Además, el comportamiento del modelo puede degradarse ante cambios de régimen de mercado (volatilidad, correlaciones o dinámica de tendencia), ya que los patrones aprendidos en entrenamiento no son invariantes en el tiempo. A ello se suma una dependencia directa de la calidad del dato de entrada: discontinuidades, huecos de mercado o diferencias de cobertura entre activos pueden afectar tanto el proceso de inferencia como la estabilidad de las métricas fuera de muestra.

Desde una perspectiva metodológica, se ha priorizado deliberadamente el rigor experimental (particiones temporales estrictas, artefactos congelados, validación trazable y ausencia de ajustes oportunistas) frente a estrategias orientadas a maximizar métricas de forma sesgada. Esta decisión puede producir resultados menos llamativos en términos absolutos, pero aumenta su validez, reproducibilidad y honestidad científica.

Como evolución del sistema, se plantean cuatro líneas prioritarias. En primer lugar, integrar señales exógenas (macroeconómicas, sentimiento de noticias y eventos) mediante protocolos de alineación temporal estrictos, para ampliar la información disponible más allá del bloque técnico. En segundo lugar, estudiar estrategias de recalibración periódica de umbrales por activo y del umbral de abstención, siempre bajo validación temporal estricta, para adaptar el criterio de decisión a cambios de régimen sin introducir fuga de información. En tercer lugar, ampliar el marco de evaluación con métricas orientadas a decisión operativa (no solo clasificación), incorporando análisis de utilidad por señal, estabilidad temporal y sensibilidad al nivel

de confianza. Por último, comparar el enfoque actual con arquitecturas híbridas o ensamblados de modelos, así como con enfoques de *ranking*/priorización de oportunidades, con el objetivo de mejorar la selectividad de señales y la robustez global en producción.

6.7 Testing pre-commit

Todo está configurado en `/.pre-commit-config.yaml`, donde se definen los *hooks* “pylint”, “pytest-unit”, “pytest-integration”, “check-ast” y “check-json”.

En el lintado, el hook “pylint” se dispara en cada *commit* cuando hay cambios en los directorios y archivo principales del *backend* (DAOs, DTOs, services, exceptions y “main.py”). Mide la calidad y consistencia del código Python con reglas de estilo, importaciones y errores comunes, de forma que el control de calidad quede acotado a los componentes críticos. Entre esas acotaciones se incluyen el número máximo de caracteres por línea (120), espacios de indentación, máximo de atributos para clases y argumentos para funciones, código duplicado, entre otros.

En el *testing* unitario, el hook “pytest-unit” se ejecuta siempre en cada *commit*, sin depender de los archivos modificados. Mide el correcto funcionamiento de los *endpoints* y su lógica aislada, validando que las respuestas y validaciones se comporten de manera esperada sin depender de servicios externos. Se validan *endpoints* de autenticación, usuarios, activos y reflexiones, considerando el manejo de errores posibles. Para ello se utilizan *mocks* generados por MagicMock, que simulan los DAOs, SAs y *tokens* (como JWT).

En el *testing* de integración, el hook “pytest-integration” también se ejecuta siempre en cada *commit*. Mide el comportamiento de flujos completos de API, asegurando que las capas principales se integran de forma coherente y que los *endpoints* clave mantienen su contrato de respuesta.

La ejecución de los *tests* usa una animación de carga sencilla tipo *spinner* ASCII para indicar progreso, y cada tipo de prueba tiene su propia subcarpeta con un archivo

markdown de documentación que resume su función: `LINTING_TESTING.md`, `UNIT_TESTING.md` y `INTEGRATION_TESTING.md`.

6.8 Gestión de Dependencias del Proyecto

6.8.1 Organización del Archivo `requirements.txt`

Las dependencias de Python del proyecto se centralizan en el archivo `requirements.txt`, ubicado en la raíz del repositorio. Las librerías están agrupadas temáticamente mediante comentarios, lo que facilita su lectura y mantenimiento a medida que el proyecto crece.

La capa de API y servidor se apoya en `fastapi`[all] y `uvicorn`, que juntos conforman el servidor ASGI encargado de exponer los *endpoints* REST. `pydantic`[email] complementa FastAPI proporcionando validación estricta de datos de entrada y salida, incluyendo formatos de correo electrónico. La comunicación con la base de datos se gestiona mediante la librería `supabase`, mientras que `python-dotenv` permite cargar las credenciales y variables de configuración de forma segura desde un archivo `.env`, sin exponerlas en el código fuente. La integración con los modelos de lenguaje de Google para el asistente de IA se realiza a través de `google-generativeai`.

El aprendizaje automático del modelo concentra el mayor número de dependencias. `torch>=2.0.0` actúa como *framework* principal de *deep learning* sobre el que se construyen los modelos de predicción. `numpy` y `pandas` cubren la manipulación y transformación de series temporales financieras, mientras que `pandas_ta` proporciona el cálculo de indicadores técnicos directamente sobre *DataFrames*. `scikit-learn` aporta herramientas de preprocesamiento como el escalado de datos. `xgboost` y `transformers` amplían el conjunto de algoritmos de predicción con modelos basados en gradient boosting y arquitecturas de atención, respectivamente. `scipy` da soporte a operaciones matemáticas y de optimización avanzadas, y `shap` permite generar explicaciones interpretables de las predicciones (XAI).

Para la obtención de datos financieros, el proyecto depende de `yfinance`, que descarga precios históricos y datos de mercado desde Yahoo Finance, y de `finnhub-python`, que conecta con la API de Finnhub para acceder a datos en tiempo real.

Finalmente, el archivo incluye dependencias orientadas a la calidad del código y las pruebas. `pytest` y `pytest-cov` cubren la ejecución de *tests* unitarios y la medición de cobertura de código. `hypothesis` permite diseñar pruebas basadas en propiedades, generando casos de prueba automáticamente para detectar casos límite. `pylint` realiza análisis estático del código Python, y `pre-commit` automatiza la ejecución de estas comprobaciones antes de cada *commit*, garantizando que el código que entra al repositorio cumple los estándares de calidad establecidos.

Toda la explicación técnica de estas tecnologías aparece más detalla en el [Capítulo 5 – Tecnologías y Herramientas](#).

6.8.2 Reproducibilidad del Ambiente

El archivo `requirements.txt` permite a cualquier desarrollador crear un ambiente de Python aislado e instalar todas las dependencias tras ejecutar `pip install -r requirements.txt`. Esto asegura que todos los desarrolladores trabajan con las mismas versiones, evitando el problema de que tan solo funcione en la máquina local del desarrollador, que es común en desarrollo de software.

Se recomienda usar entornos virtuales (detallado en el [Capítulo 8 – Manual de Usuario](#)) para crear ambientes aislados, evitando conflictos con paquetes globales del sistema.

Capítulo 7 - Evaluación en Usuarios

La fase de evaluación con usuarios reales es un pilar fundamental en el desarrollo de Horizon, ya que nos permite validar si la arquitectura de la información, el diseño de la interfaz y la funcionalidad de la aplicación resuelven verdaderamente las necesidades de nuestro público objetivo.

El objetivo principal de estas pruebas de usabilidad ha sido identificar puntos débiles en los flujos de las funcionalidades principales y evaluar la percepción de valor aportada por el producto.

7.1 Metodología y Usuarios

La investigación se llevó a cabo mediante sesiones de pruebas. Para ejecutar este proceso, el equipo de desarrollo se dividió equitativamente el trabajo de campo, evaluando cada uno a 6 usuarios distintos, alcanzando una muestra total representativa de 24 participantes (N=24).

Para garantizar la validez de los resultados del proyecto, la captación de los participantes se realizó buscando perfiles que se alinearan con nuestro arquetipo de persona primaria: Mario Gómez, el inversor principiante cauteloso. Mario representa a un joven de 22 años (consultor junior en el sector tecnológico, residente en Madrid, con un salario de 24.000€ brutos anuales). Su perfil destaca por ser un usuario intensivo y avanzado en tecnología (acostumbrado a interfaces modernas), pero con un nivel nulo o básico en conocimientos financieros. Busca un entorno seguro donde aprender paso a paso, simular inversiones sin riesgo real y apoyarse en un asistente de inteligencia artificial para resolver dudas en contexto.

Siguiendo este arquetipo, nuestra muestra real se acotó a jóvenes de entre 21 y 28 años, con un alto dominio tecnológico y distintos niveles de experiencia financiera (con un gran peso en perfiles principiantes e intermedios). Las pruebas se desarrollaron principalmente en un entorno de escritorio, simulando la postura soberana de nuestro

usuario objetivo, aunque también, con el uso de las herramientas de desarrollador de Google Chrome, se simuló un entorno de dispositivo móvil con portátiles táctiles. Según la contextualización de Mario, el uso del ordenador en casa proporciona el ambiente de concentración ideal para tareas analíticas o para consumir los módulos de formación. El tiempo medio por sesión rondó los 25-30 minutos, incluyendo una fase previa de presentación y explicación de la dinámica.

7.2 Variables Evaluadas

Para obtener una visión integral del rendimiento del sistema, cada tarea fue evaluada utilizando una combinación de métricas objetivas (cuantitativas) y subjetivas (cualitativas), que se describen a continuación.

Las métricas objetivas incluyen:

- Tasa de Completado: Porcentaje de usuarios que lograron finalizar la tarea con éxito sin ayuda.
- Tiempo Medio: Segundos invertidos desde el inicio de la tarea hasta su resolución, incluyendo el intercambio de comentarios.
- Tasa de Errores: Número de clics erróneos, retrocesos o caminos incorrectos tomados.
- Tasa de Ayuda: Porcentaje de usuarios que requirieron asistencia del moderador para desatascarse.

Por otro lado, las métricas subjetivas, evaluadas del 1-10, contienen:

- Facilidad: Evaluación del esfuerzo requerido para navegar por el flujo.
- Claridad: Evaluación de la carga cognitiva, es decir, si el usuario comprendía los textos, gráficas y la información mostrada.
- Utilidad: Medición del valor real percibido por el usuario respecto a la funcionalidad probada.

- NPS (*Net Promoter Score*): Métrica final recopilada al cierre de la sesión para medir la satisfacción global, calculando el porcentaje de promotores (9-10) frente a detractores (0-6).

A estas métricas se sumó la recolección de comentarios expresados durante la ejecución de las tareas y la redacción de observaciones de experiencia de usuario por parte del evaluador. El guión que se siguió para realizar las entrevistas se encuentra detallado en el [Apéndice B – Guion de entrevistas](#).

7.3 Tareas Analizadas

Durante las sesiones, los participantes se enfrentaron a 10 tareas clave, diseñadas para recorrer las funcionalidades desarrolladas en la aplicación web:

1. *Login*: Iniciar sesión en la plataforma.
2. Registro: Registrarse en la plataforma, pudiendo interactuar con la selección de imágenes.
3. *Dashboard*: Explorar el panel principal y comprender la navegación por iconos.
4. Análisis de Activos: Buscar un activo (ej. AAPL) e interpretar su gráfica de predicción, noticias, etc.
5. *Portfolio*: Crear un *portfolio*, añadir activos simulados, observar rentabilidad acumulada y, posteriormente, eliminarlo.
6. Reflexión del Día: Leer el contenido de artículo de reflexión desde la página principal, y entrando en el detalle.
7. Ajustes: Modificar las preferencias del sistema (cambio a modo oscuro).
8. Ayuda: Buscar soporte e iniciar el proceso de contacto con el equipo.
9. Perfil: Editar datos personales, como foto de perfil y contraseña, y ejecutar el flujo seguro de eliminación de cuenta.
10. AcademIA: Completar un curso interactivo y poner a prueba el asistente de IA contextual de Horizon.

7.4 Conclusiones de los Usuarios

El análisis consolidado de las 24 sesiones de prueba ofrece una validación mayoritariamente positiva del sistema, con un total de 240 tareas evaluadas, a la vez que saca a la luz aspectos negativos puntuales en la interfaz que requieren atención. A nivel global, la propuesta de valor de Horizon como entorno seguro de simulación y aprendizaje ha tenido una recepción excelente, logrando un NPS global del 33%, una cifra notablemente sólida para una primera versión funcional. Para ello se ha calculado restando el porcentaje de detractores, 0 personas sobre 24 que votaron entre un 0 y 6, al porcentaje de promotores, 8 personas sobre 24 que votaron entre un 9 y un 10.

Al analizar el comportamiento de los usuarios durante las pruebas, se observa una clara diferencia en la fluidez de los flujos según la naturaleza de la tarea. Los procesos más estandarizados de la aplicación, que abarcan desde el *Login* y el Registro con selección de imágenes, hasta la exploración del *Dashboard*, la lectura de la Reflexión del Día y la activación del modo oscuro en Ajustes, se completaron con una tasa de éxito del 100%. En todos estos bloques, los participantes se movieron con total autonomía, sin registrar errores ni solicitar asistencia, destacando la limpieza visual y la familiaridad de la interfaz.

El Análisis de Activos y la gestión de *Portfolio* concentran las puntuaciones de utilidad más altas del estudio: 9,75 y 9,50 respectivamente. Sin embargo, ambas tareas presentan los indicadores de fricción más elevados de la evaluación. El Análisis de Activos registró una tasa de ayuda del 12,5% y un promedio de 0,33 errores por usuario, con un tiempo medio de 110,5 segundos. Las observaciones de los evaluadores recogen de forma consistente la existencia de una curva de aprendizaje en la interpretación de la gráfica de predicción, con mayor incidencia en usuarios con experiencia financiera baja o media, mientras que los participantes con experiencia financiera alta resolvieron la tarea con mayor fluidez, tiempos más reducidos y puntuaciones de claridad notablemente superiores. La tarea de *Portfolio*, con una tasa de completado del 83,3% y un tiempo medio de 198,6 segundos, fue la que generó mayor número de errores (0,5 de media), localizados principalmente en la localización del botón de eliminación del *portfolio*. A pesar de ello, su claridad fue valorada con un 9,83 sobre 10, lo que indica

que el problema no es de comprensión sino focalizado en la eliminación de *portfolio* por primera vez.

El módulo de AcademIA fue valorado como uno de los más originales del proyecto gracias a la combinación de los cursos con el asistente de inteligencia artificial, alcanzando una tasa de completado del 100% con cero solicitudes de ayuda. Las pruebas en entornos móviles simulados revelaron comentarios puntuales sobre el tamaño reducido del contenido académico, pero su objetivo inicial se plantea para escritorio. Por el contrario, la tarea de Ayuda concentra los resultados más críticos de toda la evaluación, con una tasa de completado de apenas el 20,8% y un 79,2% de usuarios que requirió asistencia del moderador, al confundirles el correo que se indicaba encima del botón de contacto junto al inesperado despliegue de la aplicación de correo electrónico de Microsoft, que es nativa del ordenador pero que no suelen utilizar. El análisis apunta a una causa única y repetida constantemente, el botón de contacto lanza el cliente de correo del sistema sin especificar la dirección de destino, lo que generó confusión generalizada independientemente del perfil del usuario.

Finalmente, en el flujo de Perfil, el sistema de confirmaciones múltiples para la eliminación de cuenta fue percibido de forma unánime como un mecanismo que transmite seguridad y confianza, siendo uno de los aspectos más valorados de la sesión. El único punto de negativo identificado fue la ausencia de un indicador visual sobre la foto de perfil que comunique su capacidad de edición, lo que obligó a unos pocos participantes a solicitar orientación. No obstante, una vez descubierto lo vieron como una buena solución, comparándola con otras aplicaciones reales que ya usan esa práctica. A continuación, se muestra la utilidad promedio valorada por tarea según los usuarios evaluados.

Tras la evaluación sobre los usuarios, se representa el promedio de utilidad valorada por los mismos de cada tarea, es decir, de las funcionalidades más significativas que han completado en las entrevistas. Esta resume con gran claridad los comentarios generales y satisfacción de usuarios potencialmente reales de la aplicación.

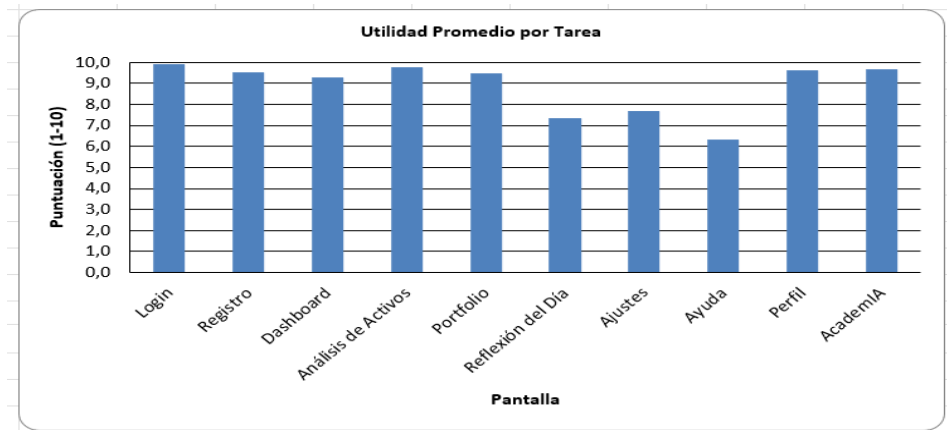


Figura 75 - Utilidad promedio por tarea

Capítulo 8 - Manual de Usuario

Todo el proyecto puede probarse correctamente en local siguiendo este manual. Para levantar el proyecto en local, primero hay que clonar/abrir Horizon en VS Code. En primer lugar, se debe crear el archivo `.env`, siguiendo la ruta y estructura de `.env_example` y copiando sus variables. Las *API keys* se entregarán en el archivo comprimido del trabajo. Después, se debe abrir una terminal dentro de la carpeta `frontend/` y ejecutar el servidor estático con `"python -m http.server 5000"`.

A continuación, desde la raíz del proyecto, se crea el entorno virtual del *backend* con `"python -m venv backend/venv"` e instalan las dependencias con `"pip install -r requirements.txt"`.

Luego se abre una terminal Git Bash, se activa el entorno utilizando el comando `"source backend/venv/Scripts/activate"` y se arranca el *backend* con `"uvicorn backend.main:app --reload"`. Finalmente, se accede desde el navegador a `localhost:5000/index.html`.

Una vez en la aplicación, se puede registrar un usuario nuevo o usar el predeterminado que se ha empleado para pruebas y para representar a la persona primaria. Su correo es mario@gmail.com y contraseña "horizon".

Capítulo 9 - Conclusiones y Trabajo Futuro

9.1 Conclusiones

El principal aprendizaje de este proyecto es que integrar inteligencia artificial en un producto de usuario final exige resolver dos problemas simultáneamente: que el modelo funcione y que el usuario confíe en él. Un modelo técnicamente correcto pero opaco para el usuario no aporta valor real, y una interfaz atractiva sin sustancia predictiva tampoco. La apuesta por XAI mediante valores SHAP fue para hacer que la IA sea interpretable, condición necesaria para que sea útil.

En el plano del modelado, trabajar con series temporales financieras nos ha enseñado que la honestidad metodológica tiene un coste visible en métricas, pero un beneficio real en validez. Aprendimos a valorar el rigor experimental (particiones estrictas, artefactos congelados, ausencia de ajustes oportunistas) precisamente porque los atajos son fáciles de tomar y difíciles de detectar. Concluimos que un modelo moderadamente preciso y metodológicamente sólido es más valioso que uno con métricas infladas sobre datos contaminados.

El desarrollo *full-stack* nos demostró que la complejidad no reside tanto en las tecnologías individuales como en su integración. Conectar el *pipeline* ETL con el modelo, el modelo con el *backend* y con una interfaz que transmita las predicciones de forma comprensible requirió decisiones de diseño que no estaban en ningún tutorial: cómo transportar explicaciones SHAP, cómo representar visualmente la incertidumbre de una señal, cómo hacer que un gráfico *waterfall* sea legible para alguien sin formación técnica. Hemos tratado de aportar valor en cada funcionalidad desarrollada, gestionando la eficiencia para no consumir APIs, el preprocesamiento de los datos enviados a los modelos, o la implementación de métodos de respaldo en todos lados. Esas decisiones son las que más nos han formado como ingenieros.

Finalmente, el proyecto nos ha dejado una conclusión que trasciende lo técnico: la brecha entre el inversor minorista y las herramientas de análisis financiero avanzadas no es solo de acceso, sino de comprensión. Horizon no resuelve esa brecha por

completo, pero demuestra que es posible reducirla combinando predicción, explicabilidad y formación en un mismo entorno, con recursos académicos y sin incurrir en asesoramiento financiero real. Esa posibilidad, y las limitaciones que aún quedan por superar, son el punto de partida del trabajo futuro.

9.2 Trabajo Futuro

Las líneas de trabajo futuro identificadas no son aspiraciones genéricas, sino extensiones directas de las decisiones y limitaciones que este proyecto ha puesto de manifiesto durante su desarrollo.

La incorporación del modelo de optimización de carteras de Markowitz es la evolución natural de la funcionalidad de *portfolios* actual. En su versión presente, Horizon permite al usuario construir y seguir una cartera, pero no le ofrece orientación sobre su composición óptima. Integrar la frontera eficiente de Markowitz permitiría cerrar ese hueco. El sistema pasaría de mostrar rendimiento histórico a sugerir asignaciones de activos que maximicen el retorno esperado para un nivel de riesgo dado, elevando sustancialmente el valor analítico de la plataforma.

La validación de la experiencia de usuario con el módulo XAI es, probablemente, la línea más necesaria desde un punto de vista científico. Horizon asume que las explicaciones SHAP son útiles para el usuario no experto, pero esta hipótesis no ha sido contrastada empíricamente. Aunque se haya hecho una evaluación con usuarios reales, este ha sido general para todas las funcionalidades. Es por ello que diseñar un estudio que mida específicamente si comprenden las contribuciones de las variables, si confían más en el modelo al verlas y si toman decisiones distintas en su presencia permitiría validar o refutar el principal argumento diferenciador del proyecto.

La ampliación del universo de activos cubiertos, junto con la migración a servicios en la nube para el despliegue del *backend* y el *pipeline* de datos, son condiciones necesarias para que Horizon trascienda el entorno académico. El modelo actual opera sobre un conjunto acotado de *tickers*. Escalar a un mayor número de activos,

especialmente mercados emergentes y criptomonedas, incrementaría la relevancia de la plataforma para perfiles de usuario más diversos. Paralelamente, sustituir el despliegue local por infraestructura en la nube (junto con herramientas de observabilidad como Kibana o Snowflake) permitiría monitorizar el comportamiento del modelo en producción real, detectar degradación ante cambios de régimen de mercado y mantener la trazabilidad de métricas a lo largo del tiempo.

En el ámbito del módulo formativo, la línea más ambiciosa es la construcción de un sistema de retos generado por la propia comunidad. Actualmente Academia es un espacio de consumo de contenido, la introducción de retos que cualquier usuario pueda proponer, resolver y puntuar lo transformaría en un espacio de producción y debate. Este cambio es relevante porque la evidencia en plataformas de aprendizaje muestra que la interacción entre pares y la resolución activa de problemas consolidan el aprendizaje de forma más efectiva que la consulta pasiva de contenidos. Un sistema de retos con puntuación, discusión asociada y visibilidad pública de soluciones convertiría a Horizon en una comunidad de inversión más allá de una herramienta.

Por último, se plantea la eliminación de los *logs* generados en cliente para la puesta de producción de la aplicación, limpiando la consola de herramientas de desarrollo para el usuario final de Horizon.

Introduction

Over the last decade, financial markets have undergone a structural transformation driven by digitalisation and the proliferation of retail investment platforms. What was historically a domain reserved for banking institutions and professional operators is today an open ecosystem in which any user with internet access can participate in the buying and selling of assets in real time. This democratisation of access has not been accompanied by an equivalent democratisation of knowledge; the gap between the ease of trading and a genuine understanding of the risk assumed represents one of the primary factors behind losses and disadvantage for the untrained retail investor (Brenner & Meyll, 2020).

In this context, Artificial Intelligence (AI) and machine learning offer an unprecedented opportunity to close this gap. Ensemble algorithms based on gradient-boosted decision trees, in particular XGBoost (Chen & Guestrin, 2016), have demonstrated a remarkable capacity to identify patterns in tabular financial time series, surpassing more complex alternatives in terms of robustness and computational efficiency in high-dimensionality, stochastically noisy scenarios (Sezer, Gudelek & Ozbayoglu, 2020).

Prediction alone is insufficient if the user does not understand the foundations that underpin it. In this regard, Explainable Artificial Intelligence (XAI) emerges as a critical discipline within the financial domain. Through techniques such as SHAP values (SHapley Additive exPlanations), grounded in cooperative game theory (Lundberg & Lee, 2017), it is possible to quantify the marginal contribution of each variable to a specific prediction, transforming the opaque output of a black-box model into an interpretable and auditable explanation. The intersection between XAI and finance constitutes an active line of research, motivated both by the regulatory need for algorithmic transparency and by its pedagogical potential for the non-expert investor (Arieta et al., 2020).

This Bachelor's Thesis arises from the convergence of three areas of computer engineering: the development of high-performance web applications, predictive analysis through machine learning and XAI, and the design of intelligent conversational

interfaces based on Large Language Models (LLMs). The result is Horizon, an educational financial prediction platform structured across four layers — presentation, controllers (FastAPI), services, and data access (DAOs) — articulated around six main modules. The prediction module implements a complete ETL *pipeline* feeding an XGBoost-based engine, trained through *walk-forward* validation. The XAI module applies SHAP values with TreeExplainer to generate interpretable explanations for each prediction, exposing which technical indicators have determined the signal issued. The AcademIA module integrates an LLM-based conversational assistant with enriched user context, acting as an interactive financial tutor. All of this is supported by a centralised authentication system via Supabase and deployed through Docker Compose with Nginx as a reverse proxy.

The document is structured as follows. Chapter 2 reviews the state of the art across the project's converging domains. Chapter 3 presents the software requirements specification. Chapter 4 details the user interface design process. Chapter 5 describes the selected technologies and tools. Chapter 6 covers the complete system implementation. Chapter 7 presents the user manual. Chapter 8 sets out the conclusions and future lines of work.

Our logo and name, "Horizon", aim to encapsulate the project's objective, depicting a horizon as the goal of the beginner investor — rising upward to represent their learning journey, with blue and white colours conveying the technical rigour that defines the application.



Figura 76 – Horizon Logo

Motivation

Access to financial markets has been significantly facilitated by fintech platforms and neobanks such as Revolut or Trade Republic. However, this accessibility entails an information overload that deteriorates the quality of decision-making for the untrained retail investor (Agnew & Szykman, 2005). Added to this is the growing concern over the loss of purchasing power: recent inflationary episodes have eroded the real value of traditional savings (D'Acunto, Prabhala & Rossi, 2019), driving interest in safe-haven assets such as gold (Baur & Lucey, 2010) and in investment more broadly, without most users possessing the financial literacy required to operate with sound judgement.

The lack of tools that explain these phenomena in a didactic manner motivates the creation of Horizon: a system that not only predicts the behaviour of stock market assets, but acts as an intelligent assistant, personalising the experience according to the user's risk profile and reducing the barrier to entry into the world of investment.

Goals

The General Objective of this work is to design, implement and deploy a web application that integrates Artificial Intelligence models to offer personalised financial advisory and real-time, or periodically updated, stock market trend prediction. The ultimate goal of said web application is to become an environment oriented towards predictive analysis and financial education, enabling users to better understand how financial markets work and helping them monitor their desired financial assets through artificial intelligence techniques. Furthermore, the implemented AI must adhere to ethical guidelines and must not encourage users to invest real money or specific assets, but rather guide the beginner investor towards knowledge of sound investment practices, always guaranteeing data security.

Regarding the Specific Objectives, the achievement of the general objective is pursued through the following:

1. State of the Art Study: Analyse current financial prediction techniques, Machine Learning architectures (such as Gradient Boosting Trees and XGBoost), Transformer technology for language processing, and multiclass classification algorithms. Study alternative applications to identify differential aspects.
2. Predictive Model Development: Train and validate an XGBoost-based model capable of classifying and predicting the short-term trend of various stock market *tickers* (equities, cryptocurrencies and commodities) using statistical features extracted over multi-scale temporal windows. Additionally, implement a continuous historical data ingestion *pipeline*.
3. AI Agent: Implement an AI conversational agent, based on LLM language models, within the web application, capable of resolving conceptual queries and explaining financial metrics in a didactic and ethical manner, drawing on specific user data to deliver a personalized and adaptive experience.
4. Web Platform Development: Build an intuitive and usable user interface for the end user, as well as a robust backend that integrates the AI model and manages the database at a speed suitable for concurrent usability.
5. Educational Module: Build an educational content management module that enables users to progress in their learning, expand their knowledge, and develop a reasonably solid grounding in investment and risk.
6. Validation: Evaluate the technical accuracy of the model (through walk-forward methodologies and metrics such as Balanced Accuracy) and the usability of the tool through final testing.
7. Data Security: The core objective will be to ensure the confidentiality of sensitive user data, employing encryption algorithms for passwords and carefully controlling data exposure to the frontend.

Scope and Limitations

The scope of this work is limited to the design and development of a full-stack web platform oriented towards financial asset analysis and user education. In further detail, the scope is specified through the following functionalities and restrictions that delimit the work:

- User and Profile Management Module: Implementation of a secure authentication system and profile management where user data and learning history will be stored, constrained by the capacity of the database employed.
- Data Ingestion and Visualisation Module: Development of a data *pipeline* connecting to external financial APIs (such as Yahoo Finance or Alpha Vantage) for the retrieval of real-time time series and historical data. An interactive *dashboard* with dynamic charts will be included. The scope will be limited to the capabilities of the free-tier APIs used, seeking to mitigate this constraint through caching techniques.
- Predictive Analysis Module (ML): Design, training and deployment of a strict predictive engine based on tree ensemble models (XGBoost), optimised for trend classification problems across a selected universe of stable and volatile assets.
- AcademIA Module: Creation of a structured repository of educational content, made available through a dedicated section of the platform (AcademIA). Additionally, an assistant based on Large Language Models (LLMs) is integrated via an established model API (such as Google Gemini), configured through prompt engineering and message preprocessing to act as a query resolver and concept explainer, considering ethical considerations related to financial matters. As before, the scope is limited to the capabilities of the free-tier model employed.
- Simulation Environment Constraint: Implementation of the web application with informational and analytical views, without involving real investment in any asset, nor constituting legal financial advice. The primary purpose is to enable users to monitor their investments based on the system's predictions and data, without real capital risk and with an accuracy subject to the intrinsic volatility of the market.

- Data Limitations: Given the use of free-tier APIs and limited processing resources, the data update frequency may present a greater lag than optimal, and the available historical data may be constrained.

Work Plan

The work plan followed to achieve the objectives described in the previous section has been grounded in the use of an agile development methodology. This approach has enabled incremental construction of the platform, facilitating early error detection and the continuous integration of the Artificial Intelligence modules with the web interface. The schedule is divided into eight main phases, distributed throughout the execution period of the work:

Fase	Duración
Requirements Specification	23 September to 1 November
User Interface Design	1 November to 22 December
Technologies, Tools and Database	22 December to 20 January
Base <i>Setup</i> and UI Layout	20 January to 20 February
Backend Architecture and Data	20 February to 5 March
Core Functionalities	5 March to 27 April
Artificial Intelligence and ML	20 January to 29 April
Testing and Validation	27 February to 15 May

Tabla 22 - Work Plan

The project begins with the Requirements Specification, a phase spanning the first month and a half of the academic year, during which the functional and non-functional

needs of the system are defined. Throughout this period, alternative applications, academic papers in the same field, and possible solutions were investigated. Use cases and user profiles are identified, establishing the foundations that will guide the remainder of the development to ensure that the final solution addresses the problem posed.

Subsequently, the User Interface Design (UX/UI) phase is addressed, in which theory is translated into visual experience. Continuing into the first semester, this phase was carried out concurrently with the Interactive Systems Design course. Low- and high-fidelity wireframes and prototypes are constructed here, with effort focused on information architecture and usability. The objective, in keeping with the project's core goals, is to ensure that the application is intuitive, accessible to beginners, and that its aesthetic is consistent with current digital design standards.

Once the design is defined, the selection of Technologies, Tools and Database is undertaken. In this phase, the most efficient technology stack is chosen, and the database schema is designed based on the requirements of the use cases. Particular importance was placed on allocating sufficient time to the technology selection process to ensure that the team could develop in the chosen languages and that the system would be scalable to at least the initial objectives.

This technical planning enables the Base Setup and UI Layout phase to be executed successfully, configuring the basic development environment and transforming the prior designs into front-end code components ready to interact with the system's logic.

The technical core is developed across the Backend Architecture and Data and Core Functionalities phases. In the former, the server structure and APIs enabling communication between the client and storage were built, developing all planned endpoints and DAO and DTO entities to ensure correct data transfer and backend architecture.

In the latter, the main logic is implemented. The application's backend functionalities are developed through the definition of services, which establish communication between data entities and other services to handle the data.

Additionally, backend calls are implemented from the dynamic frontend files, adjusting the directives to consume data from the endpoints for display, accounting for any changes that arose during service definition.

In parallel with the preceding technical phases, Artificial Intelligence and ML integration is carried out. While both backend and frontend are being developed, the model for predicting asset trends is progressively refined, with ongoing research into improved solutions. The intelligent conversational assistant is also implemented iteratively, progressing from a basic assistant without rate limiting to one capable of preprocessing messages, injecting user context, and reducing token consumption.

The work plan concludes with the Testing and Validation period. Unit and integration tests are added to verify the absence of errors, the security of the system, and full compliance with the initial requirements defined at the outset of the schedule, including tests conducted with real users. This phase runs in parallel with the technical development of the application as an error containment measure.

Conclusions and future work

The primary takeaway from this project is that integrating artificial intelligence into an end-user product requires solving two problems simultaneously: ensuring the model works and earning the user's trust. A technically correct but opaque model provides no real value to the user, just as an attractive interface lacks value without predictive substance. Our commitment to explainable AI (XAI) using SHAP values was driven by the need to make the AI interpretable, a necessary condition for it to be useful.

On the modeling front, working with financial time series has taught us that methodological honesty carries a visible cost in performance metrics, but yields a real benefit in validity. We learned to value experimental rigor (strict data partitioning, frozen artifacts, and the absence of opportunistic tweaking) precisely because shortcuts are easy to take and difficult to detect. We conclude that a moderately accurate but methodologically sound model is far more valuable than one with inflated metrics based on contaminated data.

The full-stack development process demonstrated that complexity lies less in individual technologies and more in their integration. Connecting the ETL *pipeline* to the model, the model to the backend, and the backend to an interface that conveys predictions comprehensibly required design decisions not found in any tutorial: how to transport SHAP explanations, how to visually represent signal uncertainty, and how to make a waterfall chart legible to someone without technical training. We have strived to add value in every feature we developed—optimizing efficiency to avoid over-consuming APIs, preprocessing data before sending it to the models, and implementing fallback methods across the board. These decisions are the ones that have shaped us the most as engineers.

Finally, the project has left us with a conclusion that transcends the technical realm: the gap between retail investors and advanced financial analysis tools is not merely about access, but about understanding. Horizon does not entirely close this gap, but it demonstrates that it is possible to narrow it by combining prediction, explainability, and education within a single environment—using academic resources and without

crossing the line into actual financial advisory. That possibility, along with the limitations that still need to be overcome, serves as the starting point for our future work.

Future Work

The identified lines of future work are not generic aspirations, but direct extensions of the decisions and limitations that this project has brought to light during its development.

The incorporation of the Markowitz portfolio optimization model is the natural evolution of the current portfolio functionality. In its present version, Horizon allows the user to build and track a portfolio, but it does not offer guidance on its optimal composition. Integrating the Markowitz efficient frontier would allow us to close that gap. The system would transition from showing historical performance to suggesting asset allocations that maximize the expected return for a given level of risk, substantially elevating the analytical value of the platform.

Validating the user experience with the XAI module is, probably, the most necessary line of work from a scientific point of view. Horizon assumes that SHAP explanations are useful for the non-expert user, but this hypothesis has not been empirically tested. Although an evaluation was conducted with real users, it was general for all functionalities. That is why designing a study with real users to measure whether they understand the contributions of the variables, whether they trust the model more when seeing them, and whether they make different decisions in their presence would allow us to validate or refute the main differentiating argument of the project.

The expansion of the universe of covered assets, along with the migration to cloud services for the deployment of the backend and the data *pipeline*, are necessary conditions for Horizon to transcend the academic environment. The current model operates on a limited set of *tickers*. Scaling to a larger number of assets, especially emerging markets and cryptocurrencies, would increase the relevance of the platform for more diverse user profiles. In parallel, replacing the local deployment with cloud

infrastructure (along with observability tools like Kibana or Snowflake) would allow us to monitor the behavior of the model in real production, detect degradation in the face of market regime changes, and maintain the traceability of metrics over time.

In the realm of the educational module, the most ambitious line of work is the construction of a challenge system generated by the community itself. Currently, AcademlA is a space for content consumption; the introduction of challenges that any user can propose, solve, and rate would transform it into a space for production and debate. This change is relevant because evidence in learning platforms shows that peer interaction and active problem-solving consolidate learning more effectively than passive content consultation. A challenge system with scoring, associated discussion, and public visibility of solutions would turn Horizon into an investment community beyond just a tool.

CONTRIBUCIONES PERSONALES

VASIL GEORGIEV KYUCHUKOV

La contribución de este miembro del equipo se reparte entre distintas fases del ciclo de vida del software, distribuyéndose las contribuciones a lo largo de las etapas de investigación previa, diseño lógico de la base de datos, la ingeniería de requisitos y la implementación técnica de la aplicación web.

En las fases iniciales del proyecto, el alumno trabajó de manera conjunta con el resto de los compañeros para investigar otros trabajos académicos y plataformas similares para estudiar el tema, contextualizar el proyecto y delimitar su alcance, tras una puesta en común. Adicionalmente, fue nombrado responsable del modelado de datos inicial. Diseñó la primera versión del modelo lógico de la base de datos de Horizon, modelo que sirvió de base para empezar a implementar el proyecto y que más tarde fue desplegado en Supabase por sus compañeros. Dicho modelo se elaboró priorizando la independencia tecnológica y garantizando el cumplimiento de la Tercera Forma Normal para evitar anomalías y redundancias. Este diseño lógico definió la estructura base de las entidades centrales del sistema, sin embargo, a medida que avanzaba el desarrollo y se resolvían decisiones de implementación clave, se realizaban las modificaciones pertinentes en el modelo de datos según cada miembro del equipo estimaba necesario para la necesidad de su parte del desarrollo. Paralelamente, realizó una breve investigación sobre el uso de tokens (JSON Web Tokens) para la gestión de sesiones sin estado.

En el ámbito de la Ingeniería del Software, el alumno se encargó de elaborar la Especificación de Requisitos Software (SRS) para el sistema Horizon, documento que establece detalladamente los requisitos funcionales y no funcionales del sistema, así como los diagramas de casos de uso que han servido de base para la implementación y desarrollo del proyecto. En su desarrollo se ha seguido el estándar IEEE 830-1998 (práctica recomendada para la Especificación de Requisitos Software), demostrando un criterio correcto y buenas prácticas, mientras que, por otro lado, se han realizado

varias reuniones con los demás miembros del equipo en las cuales se ha realizado un seguimiento y refinamiento de detalles según avanzaba el ciclo de desarrollo.

En el ámbito de desarrollo técnico de la aplicación, el alumno se ha encargado principalmente en la elaboración del módulo de Academia y el perfil de usuario.

En la capa de presentación, implementó el módulo de Academia, desarrollando la interfaz que permite la visualización dinámica del catálogo de cursos, aplicándole la diferenciación visual entre el contenido gratuito y el contenido exclusivo ("Pro"). Implementó también la arquitectura inicial, maquetación y lógica de la vista de Perfil de Usuario que, posteriormente, fue iterada por sus compañeros enriqueciéndola en ciertos aspectos visuales, quedando solamente por realizar tareas de refactorización y limpieza de código css para garantizar la consistencia del diseño con el resto. Implementó también la gestión de la Identidad y la Membresía: por un lado, la lógica de la interfaz para editar el perfil, restricciones lógicas como bloquear el correo electrónico para que no sea editable, el flujo seguro para la eliminación permanente del perfil de usuario y la lógica de cambio de contraseña; por otro lado, el acceso en el cliente basándose en el nivel de membresía (prohibido el acceso a cursos pro para los usuarios con plan gratuito), así como el aspecto diferenciador en el perfil de usuario (*banner pro* y foto con borde indicando membresía pro).

En el lado del *backend*, el miembro materializó la lógica del *frontend* descrita anteriormente y los flujos de seguridad del usuario a través de: implementación del patrón de acceso a datos creando los DAOs, DTOs y SA para la entidad de cursos, modificando también la base de datos según la necesidad de guardar la información en otras nuevas tablas (cursos, diapositivas y progreso_cursos); implementación del control de accesos a cursos pro, emitiendo errores 403 Forbidden ante intentos de acceso no autorizados; y enlazando la funcionalidad de cambio de contraseña con el *endpoint* protegido.

En cuanto a la redacción de la memoria, ha redactado el Capítulo 3: Especificación de Requisitos Software, y el propio documento, incluyéndolo en el Apéndice A de la memoria. Además de ello, ha redactado el punto 6.2: Implementación del *frontend*, detallando la implementación y la arquitectura del

frontend del sistema Horizon. Ha redactado también los puntos 6.5.1.4: Eliminar Perfil y 6.5.1.5 Editar Perfil, donde se describen las dos funcionalidades principales relacionadas con la gestión de la identidad. Ha redactado también los puntos 6.5.1.3: Añadir / Eliminar Activo y 6.5.3.2: Crear / Eliminar *Portfolio* donde, de forma análoga, se describen las funcionalidades correspondientes. Su aportación a la redacción de la memoria finaliza con la redacción de la lógica operativa del módulo de educación financiera, punto 6.5.5.1: Cursos en AcademlA, así como con aportaciones para las conclusiones y el trabajo a futuro.

Finalmente, el estudiante participó en las rondas de revisión general de la memoria junto con el resto del equipo, detectando errores, aportando retroalimentación y aplicando correcciones de contenido y mejoras de formato a lo largo del proceso de elaboración del documento.

DAVID RAMOS DE LUCAS

La aportación de este miembro del equipo abarca desde la concepción y diseño de la base de datos hasta el desarrollo completo de los módulos de análisis de activos y gestión de *portfolios*, tanto en el plano del *frontend* como del *backend*, así como la redacción de varios capítulos clave de la memoria académica.

En la parte inicial del proyecto y en las partes de gestión del proyecto, este alumno participó de forma conjunta con los demás compañeros en la parte de investigación del estado del arte, en la revisión de otros trabajos académicos y en la búsqueda de plataformas similares al proyecto, para poder contextualizar el proyecto y delimitar su alcance. Posteriormente el alumno participó activamente en la definición del modelo de datos, trabajo llevado a cabo de forma conjunta con el compañero encargado del módulo de inteligencia artificial. Ambos evaluaron distintas alternativas de gestión de base de datos con el objetivo de determinar la solución más eficiente para las necesidades del sistema. Fruto de esta investigación, el estudiante fue el responsable de seleccionar Supabase como plataforma de alojamiento de la base de datos en la nube, justificando su elección por las capacidades que ofrece en materia

de autenticación integrada, almacenamiento de archivos y acceso mediante API REST. Una vez tomada esta decisión, creó y configuró el proyecto en Supabase e implementó el esquema de base de datos en su totalidad, desde la versión inicial hasta la versión final desplegada, incluyendo la definición de las tablas principales, sus relaciones y las restricciones de integridad referencial documentadas en el punto 6.4 de la memoria.

En el ámbito del *frontend*, el alumno se responsabilizó del diseño e implementación de las vistas de análisis de activos y gestión de *portfolios*, poniendo especial énfasis en la usabilidad, la coherencia visual con el resto de la plataforma y la intuitividad de la experiencia de usuario. Igualmente, tomó a su cargo la vista de perfil de usuario desde una versión muy temprana del proyecto, realizando una mejora sustancial de la misma hasta lograr un resultado visualmente amigable e intuitivo para el usuario. Además, diseñó e implementó las primeras versiones de las vistas de inicio de sesión, registro y cierre de sesión. Como complemento a estas funcionalidades, desarrolló el sistema de paneles de confirmación para la cancelación de acciones potencialmente destructivas, prestando especial atención al flujo de eliminación de cuenta, que por tratarse de una acción crítica e irreversible fue diseñado con un mecanismo de confirmación reforzado: el usuario debe introducir su nombre de usuario para poder ejecutar la operación, añadiendo una capa de seguridad adicional frente a errores accidentales.

En lo que respecta al *backend*, el alumno implementó la lógica mayoritaria de los módulos de análisis de activos y gestión de *portfolios*, siguiendo en ambos casos el patrón arquitectónico de DAOs, DTOs, capa de Servicios y *Endpoints* establecido para el proyecto. En el módulo de análisis de activos, desarrolló el buscador dinámico de *tickers* con preselección automática del primer resultado, así como la lógica de cálculo de rentabilidad diaria del activo consultado. Implementó también la funcionalidad que permite al usuario añadir activos a un *portfolio* directamente desde la página de análisis, contemplando el caso en que el usuario no disponga todavía de ningún *portfolio* y necesite crearlo en el mismo flujo. En estrecha coordinación con el compañero responsable del modelo de inteligencia artificial, integró los datos predictivos del modelo en la vista de análisis, asegurando su presentación coherente y comprensible

para el usuario final. En el módulo de *portfolios*, desarrolló la lógica de creación y eliminación de *portfolios*, la definición del perfil de riesgo asociado a cada uno y la lógica de avisos que evalúa la coherencia entre los activos incluidos y el perfil de riesgo seleccionado, alertando al usuario cuando se detectan desajustes. Asimismo, implementó el cálculo de la rentabilidad diaria del *portfolio* en función de la variación ponderada de los activos que lo integran. Por otro lado, el estudiante realizó la primera implementación del módulo de noticias de activos, consumiendo la API de Alpha Vantage para la obtención de información de actualidad financiera, funcionalidad que posteriormente fue extendida por otro miembro del equipo mediante un sistema de caché sobre base de datos.

En lo relativo a la redacción de la memoria académica, el alumno se encargó de los apartados 5.3 (Tecnología del Servidor — *Backend*), 5.4 (Base de Datos, Autenticación y Almacenamiento) y 5.5 (Entorno de Desarrollo y Despliegue) del Capítulo 5. En el Capítulo 6, redactó el punto 6.3 sobre la implementación del *backend*, explicando la configuración del servidor FastAPI, los *endpoints*, la capa de servicios y los DAOs, así como la totalidad del punto 6.4, que detalla en profundidad la implementación de la base de datos, su estructura de tablas y sus restricciones de integridad. En el apartado de funcionalidades principales, documentó las siguientes: 6.5.1.3 Cerrar sesión, 6.5.2.1 Añadir activo, 6.5.2.2 Buscar activo, 6.5.2.4 Rentabilidad de activo, 6.5.3.3 Rentabilidad de *portfolio*, 6.5.3.4 Avisos del *portfolio*, y colaboró en la redacción del punto correspondiente a los *insights* de IA. Finalmente, redactó en su integridad el punto 6.7 sobre gestión de dependencias del proyecto.

FRANCISCO JOSÉ SÁNCHEZ DE LEÓN ACEVEDO

La aportación de este miembro del equipo incluye también el desarrollo de cada una de las fases del ciclo de vida del desarrollo del software, con un peso considerable en la dirección técnica del proyecto, definición de la arquitectura, diseño de los interfaces, desarrollo técnico al nivel *full-stack*, puesta en marcha del entorno de infraestructura, redacción y edición de la memoria, entre otros aspectos.

En la parte inicial del proyecto y en las partes de gestión del proyecto, este alumno participó de forma conjunta con los demás compañeros en la parte de investigación del estado del arte, en la revisión de otros trabajos académicos y en la búsqueda de plataformas similares al proyecto, para poder contextualizar el proyecto y delimitar su alcance. Posteriormente, se hizo cargo de una parte en el rol de liderazgo técnico. Gestionó el *stack* tecnológico del proyecto, decidiendo la combinación de FastAPI y Python para la parte del *backend*, y HTML, JS y CSS para el *frontend*, haciendo uso de entornos virtuales (venv) para gestionar dependencias. Gestionó metodológicamente a sus compañeros para levantar el proyecto en local, para realizar pruebas funcionales y para unificar las versiones de los *commits* en GitHub. Gestionó el estado de las tareas llevadas a cabo, controlando la organización y realización de estas mediante sprint semanales.

En el ámbito del diseño de interfaz y experiencia de usuario, el estudiante fue el encargado del Capítulo 4. Llevó a cabo un diseño guiado por objetivos, para lo cual definió a la persona primaria del proyecto a través de la realización de entrevistas no formales. Con esta base investigativa, desarrolló los prototipos de la aplicación, tanto de bajo como de alto nivel, aplicando de manera práctica las herramientas y metodologías de prototipado aprendidas durante la carrera.

A nivel de arquitectura de software, el alumno elaboró la arquitectura completa de la aplicación web y construyó la base de la implementación técnica que sirvió de base para el resto del equipo. En el lado del *backend*, implementó el patrón de acceso a datos estructurando los DAOs, la validación y el paso de objetos DTO utilizando la biblioteca Pydantic, la capa de Servicios y el controlador (main). Desarrolló los primeros *endpoints* (entre los que se incluyen las funciones básicas de inicio de sesión y registro), que sirvieron de ejemplos para que los compañeros desarrollaran más funcionalidades, empleando los primeros servicios de autenticación que establecerían la arquitectura del proyecto. En el lado del *frontend* sentó las bases de las vistas definiendo las etiquetas estructurales principales, como *dashboard_contenido*.

En la capa de presentación, el estudiante estructuró el archivo de estilos CSS de acuerdo con la buena práctica de unificar los estilos (conocimiento aprendido en la

asignatura de Aplicaciones Web). Definió las variables globales de colores, anchuras, sombras, fondos, además de incorporar de manera completa el modo oscuro de la aplicación. Realizó el módulo del *Dashboard* general de inicio y la vista dinámica del explorador de activos, programando las funcionalidades específicas para la barra de búsqueda en la zona central de la pantalla y su transición hacia la parte superior con la finalización de una consulta.

En lo que respecta a la lógica de negocio, la seguridad y todos los elementos del *backend*, el miembro del equipo desarrolló el flujo completo de autenticación y configuró los clientes de Supabase, asegurando la protección de las rutas a través de la validación del *token* de sesión en todas las vistas privadas. Fue el desarrollador del agente conversacional inteligente del módulo de AcademIA, ocupándose del preprocesamiento de datos y de la inyección de contexto del modelo. Asimismo, para complementar esta integración con la API de Google AI Studio, desarrolló un sistema de manejo de excepciones personalizadas, que es fundamental para gestionar los errores devueltos por el *chatbot* y presentarlos al usuario de forma comprensible. Además desarrolló la funcionalidad de noticias de activos, implementando un sistema de caché a través de la base de datos, permitiendo persistir las noticias cada hora entre diferentes usuarios, con la finalidad de ser eficiente y de no consumir rápidamente la capa gratuita de la API de Alpha Vantage. Desarrolló el módulo de las citas de reflexión, desarrollando la interfaz, las correspondientes tablas y sus funciones integradas en Supabase para su aleatoriedad. En cuanto a la gestión del usuario, también desarrolló el complejo flujo relativo al cambio de foto de perfil integrando la solución de Supabase Storage. Éste último desarrollo incluyó la creación y definición de las imágenes predeterminadas, la gestión del cambio de fotos personalizadas con estrategias de recuperación (*fallbacks*) hechas frente a los errores, la eliminación de archivos en cascada y el tratamiento de las URLs generadas por Supabase Storage en tiempo real.

En el ámbito de la calidad del software (QA) y las buenas prácticas de integración continua, el alumno se encargó de desarrollar e implementar un sistema de validación automatizada mediante *hooks* de *pre-commit*. Esta infraestructura abarcó la configuración de herramientas de *linter* para garantizar la legibilidad y el seguimiento

de los estándares de estilo de código, al utilizar un lenguaje como Python, así como la ejecución de pruebas automatizadas, comprendiendo tanto *tests* unitarios como de integración. El objetivo fundamental de esta medida fue establecer una sólida barrera de seguridad que asegurase que ninguna nueva aportación al repositorio introdujera regresiones o corrompiera el funcionamiento global de la aplicación.

En lo que respecta a la infraestructura y despliegue, desarrolló y configuró la infraestructura para levantar un servidor casero, usando Docker y configurando Nginx como proxy inverso para la gestión de las peticiones HTTP y la ocultación de las rutas HTML.

En lo relativo a la documentación y redacción de la memoria, el alumno elaboró la documentación técnica dentro de la carpeta docs/ de los flujos más destacados, así como el Manual de Usuario completo. Dentro de la memoria académica, redactó el apartado 1.4. Plan de trabajo, las primeras iteraciones de los Capítulos 1 - Introducción y 2 - Estado del Arte, y la totalidad del Capítulo 4 - Diseño de interfaz de usuario. En la fase de implementación, redactó el punto 6.1. Estructura y Arquitectura del Proyecto, apoyó en la redacción general de la implementación del *backend* y *frontend*, y detalló las siguientes funcionalidades: 6.5.1.1 Iniciar Sesión, 6.5.1.2. Registrarse, 6.5.1.6. Editar Foto de Perfil, 6.5.2.3 Noticias, 6.5.4.2. Chat y 6.5.5. *Dashboard* general completo, sumado al 6.7. *Testing pre-commit*. Además, estructuró y redactó el contenido del Capítulo 7 – Evaluación de usuarios y Capítulo 8 – Manual de usuario.

SERGIO VINAGRE MARTÍNEZ

La aportación técnica más significativa de este miembro del equipo se concentra en el desarrollo del *pipeline* de datos y del módulo de inteligencia artificial, constituyendo el núcleo funcional del sistema de predicción de Horizon. En lo que respecta al flujo de preparación de datos, el estudiante diseñó e integró el proceso completo de transformación de series temporales a formato tabular mediante el cálculo de indicadores técnicos, así como la generación de características con ventanas retrospectivas adaptadas al perfil de cada activo.

Se implementaron los mecanismos necesarios para garantizar la ausencia de fuga de información entre particiones, aplicando una normalización Min-Max ajustada exclusivamente con el conjunto de entrenamiento y reutilizada de forma consistente en las fases de validación, prueba e inferencia, con persistencia de los artefactos correspondientes para asegurar la reproducibilidad del sistema. En cuanto al sistema de etiquetado, se desarrolló la lógica de clasificación ternaria basada en retornos futuros y la calibración de umbrales percentilares individualizados por *ticker*, mantenidos congelados a lo largo de todo el ciclo de vida del modelo. Adicionalmente, se incorporó la regla de abstención por confianza en la fase de inferencia, mecanismo que reemplaza señales direccionales de baja convicción por la clase lateral cuando la probabilidad máxima del conjunto no supera el umbral establecido. A nivel de persistencia, el estudiante implementó el almacenamiento de resultados y métricas en base de datos, incluyendo el registro de predicciones en la tabla `prediction_log` y el proceso de backfill para la reconstrucción del histórico, con control de duplicados basado en unicidad lógica por *ticker* y fecha de predicción. Asimismo, se llevaron a cabo mejoras sobre las columnas derivadas de la tabla `activos` para consolidar el estado operativo del modelo. En el ámbito de la inteligencia artificial explicable, se desarrolló el módulo de explicabilidad basado en SHAP con TreeExplainer, orientado a cuantificar la contribución de cada variable a la predicción generada. Los resultados se persisten en la tabla `xai_explicaciones` y se exponen al usuario con nomenclatura legible en español, con el objetivo de hacer la salida del modelo auditable e interpretable para perfiles sin formación técnica avanzada.

Complementariamente, el estudiante desarrolló la vista de análisis por activo en el *frontend*, iterando sobre la estructura y los estilos de `analysis.html` para incorporar una línea temporal de precio con señales de IA mediante Lightweight Charts, con selector de rango temporal, leyenda y correcciones de rendimiento. Para alimentar dicha visualización, se implementó el *endpoint* GET `/activos/{ticker}/price-history` y el servicio asociado, que combina el histórico OHLC obtenido de yfinance con las señales almacenadas en `prediction_log`. De igual modo, se integró el gráfico de tipo *waterfall* para la visualización de contribuciones SHAP mediante un módulo específico

desarrollado con D3.js (*shapWaterfall.js*), junto con la capa de acceso a datos correspondiente y el *endpoint* GET */activos/{ticker}/xai/latest-shap*. Por último, se realizaron verificaciones y ajustes sobre la suite de *tests* para asegurar la consistencia y trazabilidad del ciclo completo.

En el ámbito de la redacción de la memoria, la aportación principal de este estudiante correspondió al apartado 6.6, dedicado al *pipeline* ETL y al módulo de inteligencia artificial. En dicho bloque se documentó la arquitectura general del sistema organizada por capas, justificando la separación de responsabilidades como criterio de diseño orientado a la trazabilidad y la evolución del sistema. Se describió con detalle la partición cronológica de los datos en proporciones 70/15/15, argumentando su necesidad para reducir el riesgo de fuga de información y aproximar condiciones operativas realistas, así como la etapa de transformación e ingeniería de variables que constituye la entrada del modelo. Asimismo, se formalizó el sistema de etiquetado y calibración por percentiles con umbrales congelados, se documentó la metodología de evaluación *walk-forward* y se redactó el apartado de inteligencia artificial explicable, explicitando la integración de SHAP como mecanismo de interpretabilidad. El capítulo se cerró con la exposición de las limitaciones detectadas y las principales líneas de trabajo futuro identificadas.

Adicionalmente, el estudiante fue responsable de la redacción del resumen y del *abstract* del trabajo, piezas que sintetizan los objetivos, la metodología y los resultados del proyecto de cara a su presentación formal.

En lo que respecta a otros capítulos, este miembro del equipo participó en la elaboración e iteración de distintas versiones del Capítulo 1 (Introducción) y del Capítulo 2 (Estado del Arte), contribuyendo de manera destacada en la recopilación, selección y estructuración de las fuentes bibliográficas utilizadas, así como en su formateo conforme al estándar APA 7. Se redactaron también los apartados 5.1 y 5.2 del Capítulo 5 (Tecnologías y Herramientas), correspondientes a los lenguajes de programación y desarrollo web base, y al módulo de inteligencia artificial y procesamiento de datos, respectivamente. Desarrolló también el Capítulo 8 de conclusiones y trabajo futuro.

Finalmente, el estudiante participó en las rondas de revisión general de la memoria junto con el resto del equipo, detectando errores, aportando retroalimentación y aplicando correcciones de contenido y mejoras de formato a lo largo del proceso de elaboración del documento.

BIBLIOGRAFÍA

Agnew, J. R., & Szykman, L. R. (2005). Asset allocation and information overload: The influence of information display, asset choice, and investor experience. *Journal of Behavioral Finance*, 6(2), 57–70. https://doi.org/10.1207/s15427579jpfm0602_2

Alcaraz López, V. A. (2022). *Predicción sobre mercado de valores con redes neuronales LSTM* [Trabajo Fin de Grado, Universidad Politécnica de Madrid]. Archivo Digital UPM. <https://oa.upm.es/70562/>

Arrieta, A. B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., & Herrera, F. (2020). Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58, 82–115. <https://doi.org/10.1016/j.inffus.2019.12.012>

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*. <https://arxiv.org/abs/1409.0473>

Baker, T., & Dellaert, B. (2018). Regulating robo advice across the financial services industry. *Iowa Law Review*, 103(2), 713–750.

Baur, D. G., & Lucey, B. M. (2010). Is gold a hedge or a safe haven? An analysis of stocks, bonds and gold. *Financial Review*, 45(2), 217–229. <https://doi.org/10.1111/j.1540-6288.2010.00244.x>

Brenner, L., & Meyll, T. (2020). Robo-advisors: A substitute for human financial advice? *Journal of Behavioral and Experimental Finance*, 25, 100240. <https://doi.org/10.1016/j.jbef.2020.100240>

Cerqueira, V., Torgo, L., & Mozetič, I. (2020). Evaluating time series forecasting models: An empirical study on performance estimation methods. *Machine Learning*, 109(11), 1997–2028. <https://doi.org/10.1007/s10994-020-05910-7>

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.
<https://arxiv.org/abs/1406.1078>

D'Acunto, F., Prabhala, N., & Rossi, A. G. (2019). The promises and pitfalls of robo-advising. *The Review of Financial Studies*, 32(5), 1983–2020.
<https://doi.org/10.1093/rfs/hhz014>

Fisch, J. E., Labouré, M., & Turner, J. A. (2019). The emergence of the robo-advisor. En J. Agnew & O. S. Mitchell (Eds.), *The disruptive impact of FinTech on retirement systems* (pp. 13–37). Oxford University Press.

Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669. <https://doi.org/10.1016/j.ejor.2017.11.054>

Hernández Rodríguez, C. (2020). *Predicción y clasificación de series temporales bursátiles mediante redes neuronales recurrentes* [Trabajo Fin de Grado, Universidad de Valladolid]. UVaDOC. <https://uvadoc.uva.es/handle/10324/44399>

Jung, D., Dörner, V., Glaser, F., & Morana, S. (2018). Robo-advisory: Digitalization and automation of financial advisory. *Business & Information Systems Engineering*, 60(1), 81–86. <https://doi.org/10.1007/s12599-017-0519-3>

Latibex. (2026). Su Cartera LATIBEX.
<https://www.latibex.com/esp/Cotizaciones/Cartera/Nueva>

Li, Y., Wang, S., Ding, H., & Chen, H. (2023). Large language models in finance: A survey. *Proceedings of the Fourth ACM International Conference on AI in Finance*, 374–382. <https://doi.org/10.1145/3604237.3626869>

Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 4765–4774. <https://arxiv.org/abs/1705.07874>

Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., & Lee, S. I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1), 56–67. <https://doi.org/10.1038/s42256-019-0138-9>

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). Statistical and machine learning forecasting methods: Concerns and ways forward. *PLOS ONE*, 13(3), e0194889. <https://doi.org/10.1371/journal.pone.0194889>

Markowitz, H. (1952). Portfolio selection. *The Journal of Finance*, 7(1), 77–91. <https://doi.org/10.1111/j.1540-6261.1952.tb01525.x>

OpenAI. (2026). ChatGPT [Herramienta de software]. <https://chatgpt.com/>

Python Software Foundation. (2024). *Python 3.12 documentation*. <https://docs.python.org/3.12/>

Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11), 2673–2681. <https://doi.org/10.1109/78.650093>

Sezer, O. B., Gudelek, M. U., & Ozbayoglu, A. M. (2020). Financial time series forecasting with deep learning: A systematic study from 2005 to 2019. *Applied Soft Computing*, 90, 106181. <https://doi.org/10.1016/j.asoc.2020.106181>

Traseira Linares, C. (2022). *Predicción de precios de acciones en bolsa a través de redes neuronales* [Trabajo Fin de Grado, Universidad Pontificia Comillas]. Repositorio Comillas. <https://repositorio.comillas.edu/xmlui/handle/11531/56871>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30. <https://arxiv.org/abs/1706.03762>

Yang, H., Liu, X. Y., & Wang, C. D. (2023). FinGPT: Open-source financial large language models. *arXiv preprint arXiv:2306.06031*. <https://arxiv.org/abs/2306.06031>

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., & Wen, J. R. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*.
<https://arxiv.org/abs/2303.18223>

APÉNDICES

Apéndice A - Especificación de Requisitos Software (SRS)

Proyecto: Inteligencia artificial para la gestión de inversiones financieras

Versión: 1.0

Autores:

Vasil Georgiev Kyuchukov

David Ramos de Lucas

Francisco José Sánchez de León Acevedo

Sergio Vinagre Martínez

Universidad Complutense de Madrid

1 de Noviembre de 2025

Madrid, España

1. Introducción

Este documento se trata de una Especificación de Requisitos Software (SRS) para el sistema de gestión de inversiones financieras denominado “Horizon”. Esta especificación ha sido realizada en base a las directrices dadas por el estándar IEEE Práctica Recomendada para Especificaciones de Requisitos Software ANSI/IEEE, 1998.

1.1. Propósito

El propósito de este documento es definir de manera exhaustiva los requisitos funcionales y no funcionales para el desarrollo de Horizon, una aplicación web destinada a la gestión de carteras de inversión simuladas y al análisis de activos financieros mediante técnicas de Inteligencia Artificial (*Machine Learning*).

1.2. Alcance

El sistema software Horizon es una aplicación web de escritorio, cuyas funciones principales incluyen:

- Monitorización de Mercado: Visualización de gráficos de precios en tiempo real o diferido consumiendo APIs externas.
- Gestión de Cartera Básica: Herramientas para crear listas de seguimiento (*portfolios*) y obtener un análisis de su rentabilidad y estabilidad.
- Motor de Predicción (IA): Un módulo de *backend* que procesa datos históricos para predecir la tendencia futura (alcista/bajista) de un activo.
- AcademIA: Módulo didáctico para que el usuario aprenda nociones básicas sobre las inversiones financieras y que aumente su conocimiento para lograr la toma de decisiones de inversión con mejor criterio.
- Explicabilidad IA(XAI): Generación de justificaciones en lenguaje natural sobre las predicciones específicas.

Nota: El sistema funciona estrictamente en un entorno de simulación. No se conecta a brokers bancarios para realizar transacciones reales, ni maneja dinero real.

1.3. Personal involucrado

Autores / Desarrolladores: responsables del desarrollo *full-stack* y del entrenamiento de modelos de ML

Nombre	Vasil Georgiev Kyuchukov
Rol	<i>Full-stack Developer</i>
Categoría profesional	Estudiante Ingeniería Informática + ADE
Responsabilidad	Elaboración y desarrollo de la Especificación de Requisitos Software (SRS, Modelo de Casos de Uso, Modelado UML)
Información de contacto	vasilgeo@ucm.es

Tabla 23 - Desarrollador 1

Nombre	David Ramos de Lucas
Rol	<i>Full-stack Developer</i>
Categoría profesional	Estudiante Ingeniería Informática + ADE
Responsabilidad	Planificación, corrección conjunta y seguimiento
Información de contacto	daramo02@ucm.es

Tabla 24 - Desarrollador 2

Nombre	Francisco José Sánchez de León Acevedo
Rol	<i>Full-stack Developer</i>
Categoría profesional	Estudiante Ingeniería Informática + ADE
Responsabilidad	Planificación, corrección conjunta y seguimiento
Información de contacto	francs10@ucm.es

Tabla 25 - Desarrollador 3

Nombre	Sergio Vinagre García
Rol	<i>Full-stack Developer</i>
Categoría profesional	Estudiante Ingeniería Informática + ADE
Responsabilidad	Planificación, corrección conjunta y seguimiento
Información de contacto	sergvina@ucm.es

Tabla 26 - Desarrollador 4

TFG Tutorizado por:

Nombre	Iván García-Magariño García
Rol	Tutor académico de TFG
Categoría profesional	Doctor por la Universidad Complutense de Madrid y profesor titular de la misma, Dpto. Ingeniería del Software e Inteligencia Artificial

Responsabilidad	Responsable de la validación académica y orientación
Información de contacto	igarciam@ucm.es

Tabla 27 - Tutor

1.4. Definiciones, acrónimos y abreviaturas

CUN: Caso de uso de negocio.

Responsiveness: diseño adaptable de una aplicación web, que hace referencia a su capacidad para ajustarse de forma automática a diferentes dispositivos, tamaños de pantalla y resoluciones, garantizando una experiencia de usuario óptima tanto en ordenadores como en tablets o smartphones.

Ticker: símbolo único que identifica a una acción en bolsa (ej. AAPL para Apple o BTC para Bitcoin).

OHLCV: Acrónimo de *Open, High, Low, Close* y *Volume*. Los cinco datos básicos de una vela financiera.

Inferencia: el proceso por el cual el modelo de IA genera una predicción basada en datos nuevos.

XAI (Explainable AI): Conjunto de técnicas que permiten entender cómo un modelo de caja negra toma decisiones.

Backtesting: Proceso de probar una estrategia o modelo con datos del pasado para verificar su precisión.

1.5. Referencias

Se ha utilizado como referencia el Estándar IEEE 830-1998: *Recommended Practice for Software Requirements Specifications*.

1.6. Resumen

El documento se estructura en tres secciones principales:

- Descripción General. Visión global del producto y los usuarios a los que está destinado.
- Requisitos Específicos, en donde se describen los que son comunes a las interfaces, los funcionales y los no funcionales.
- Apéndice. En él se detalla el informe WAVE sobre los casos de uso para evaluar la usabilidad y accesibilidad web.

2. Descripción General

2.1. Perspectiva del producto

Horizon es un sistema independiente, planteado como aplicación web que sigue una arquitectura Cliente-Servidor. Su objetivo es facilitar la gestión de inversiones financieras, en cuanto a toma de decisiones, ofreciendo a sus usuarios información como la predicción de tendencias del precio de los activos e información relevante como sentimiento de mercado, volatilidad y confianza LSTM.

2.2. Funcionalidad del producto

Las funcionalidades del sistema se agrupan en cuatro bloques principales:

- Gestión de usuarios: Registro, Login y personalización del perfil de usuario.
- Visualización Bursátil: Buscador de activos y renderizado de gráficos interactivos.
- Gestión de Portfolio: Operaciones CRUD (Crear, Leer, Actualizar y Borrar) sobre los activos guardados por el usuario y tendencias simuladas.
- Inteligencia Artificial: Solicitud de predicciones bajo demanda y generación de informes de explicabilidad.

El diagrama de casos de uso elaborado para el sistema Horizon se expone en el [Capítulo 3](#) de este TFG.

2.3. Características de los usuarios

Se explican las características de los usuarios en formato de tabla como se muestra a continuación:

Tipo de usuario	Usuario Registrado
Formación	Conocimientos básicos/medios de finanzas. Alta competencia digital.
Actividades Principales	Consultar gráficos en la app, gestionar sus <i>portfolios</i> , solicitar análisis a la IA, ver los gráficos correspondientes, interactuar con el <i>chatbot</i> y módulo de AcademIA para adquirir conocimientos y resolver dudas.

Tabla 28 - Características de Usuario Registrado

Tipo de usuario	Usuario No Registrado
Formación	Sin formación claramente definida. Cualquier usuario que decida visitar la aplicación web.
Actividades Principales	Dispondrá de acceso a una "Landing Page", cuyo contenido consistirá en una página que informe sobre la herramienta Horizon, dando a conocer su funcionamiento, planes y otras características. El usuario no registrado tendrá la opción de acceder a la página de registro de la herramienta.

Tabla 29 - Características Usuarios No Registrados

2.4. Restricciones

Interfaz: Diseño de *desktop-first* que permitirá compatibilizar de mejor forma la interfaz compleja de gráficos con la pantalla más grande de escritorio/portátil. Aun así, la aplicación será *responsive* para distintos tamaños de pantalla, incluyendo, en específico, la pantalla de un dispositivo móvil.

Datos: Dependencia de la cuota de usos (Rate Limits) de las APIs gratuitas de terceros:

En el caso de Supabase (plan gratuito), utilizado para alojar la base de datos de este proyecto, se encuentra un límite de aproximadamente 500 MB de almacenamiento, un número limitado de conexiones concurrentes, y restricciones en el número de peticiones a la API y uso de funciones *serverless*.

Por su parte, Gemini API (plan gratuito), utilizado para el *chatbot*, también establece límites claros de uso, como un número máximo de solicitudes por minuto (RPM), peticiones diarias (RPD) y cantidad de *tokens* procesados por solicitud y por día.

Latencia de IA: La inferencia del modelo no debe bloquear la interfaz de usuario de ninguna forma y no debe interferir en la velocidad de respuesta del funcionamiento del sistema. Para optimizar el tiempo de respuesta, se ha procedido a guardar datos útiles en la base de datos para que estén disponibles los más recientes cuando sea necesario su uso, en vez de ejecutar el modelo en tiempo real cada vez que el usuario decida acceder al análisis de un activo.

2.5. Suposiciones y dependencias

Conexión a internet: Se asume que el usuario dispone de conexión a internet estable y continua.

Disponibilidad operativa de las APIs: Se asume la disponibilidad operativa de las APIs de proveedores como Yahoo Finance, Gemini API y Supabase que tendrán aplicación directa para el desarrollo de la aplicación web.

2.6. Evolución previsible del sistema

Análisis de Sentimiento de Mercado: Integración de Análisis de Sentimiento de Mercado basado en las noticias y redes sociales (por medio de *web scraping* y NLP). -> Se incorpora al modelo Pdte.

Ampliación de Activos: Ampliación de los activos disponibles dentro de la aplicación. Habrá disponibles no sólo acciones, sino también Bonos, ETFs, Criptomonedas, etc.

Sistemas de alertas: Sistema de alertas por correo electrónico o por notificaciones en la app sobre eventos relevantes para el usuario o directamente derivados de sus preferencias como, por ejemplo, "notifícame cuando el ratio alcance un valor x" o "cuando el precio de acción llegue al valor x".

AcademIA: Incorporación de una IA, al estilo ChatGPT, que permitirá al usuario interactuar de forma natural para resolver dudas relacionadas con el análisis y valoración de activos. Este subsistema, se basará en el entrenamiento de un modelo de IA a través de *Retrieval-Augmented Generation* (RAG), de modo que sea capaz de ofrecer respuestas claras, explicaciones y apoyo en la toma de decisiones dentro de este ámbito específico.

Área de Notificaciones: El área de notificaciones será concebido para mantener informado al usuario sobre distintos eventos dentro de la aplicación. Por un lado, se contemplan notificaciones generales, como actualizaciones del sistema o novedades relevantes. Por otro lado, se prevé la posibilidad de configurar alertas personalizadas, permitiendo al usuario recibir avisos específicos, por ejemplo, cuando el valor de un activo alcance un cierto valor.

3. Requisitos Específicos

3.1. Requisitos comunes de las interfaces

3.1.1 Interfaces de Usuario

Panel de Inicio (*Dashboard*): La página principal presentará un diseño de *desktop-first*, optimizado para pantallas grandes, aunque compatible con distintos tamaños de ventana y dispositivos (*responsiveness*), permitiendo visualizar todos los elementos visuales con claridad. Incluirá enlaces a las distintas funcionalidades de la aplicación, así como acceso a soporte y una sección de reflexiones diarias disponibles para lectura.

Buscador y Gráficos Interactivos: Interfaz dedicada a la búsqueda de activos financieros, por sus *tickers*. Mostrará gráficos de velas en tiempo real o diferido, integrando herramientas de análisis técnico y fundamental en una sola vista. Se mostrarán también datos como la fiabilidad del modelo y el precio actual, además de una selección de noticias recientes sobre el activo seleccionado.

Módulo de Inteligencia Artificial: Una sección específica donde el usuario podrá solicitar predicciones de tendencias (alcista/bajista) sobre activos financieros. Esta interfaz destacará por su componente de Explicabilidad (XAI), mostrando justificaciones en lenguaje natural y las contribuciones de cada factor a la predicción para generar confianza en el usuario.

Panel de Portfolio: Será la página en la que el usuario podrá ver y gestionar (crear, modificar o eliminar) distintos *portfolios*, compuestos por los activos financieros que el usuario desee. En cada *portfolio* se visualizará los activos que lo componen y datos adicionales como rentabilidad diaria, precio, estabilidad y tendencia.

Módulo de Academia: Será la página en la que el usuario tenga acceso a módulos de enseñanza, así como acceso a un *chatbot* con posibilidad de comunicarse, resolver pequeñas dudas y aprender. Los módulos de enseñanza serán esencialmente cursos en forma de diapositivas accesibles para el usuario.

Área de Perfil de Usuario: En esta página, el usuario podrá visualizar la información asociada a su cuenta, incluyendo datos personales y algunas estadísticas básicas relacionadas con su actividad en la aplicación. Además, dispondrá de funcionalidades para editar su perfil, como modificar el nombre, la imagen de perfil, su contraseña u otros datos relevantes, así como la opción de eliminar su cuenta de forma permanente si lo desea. Adicionalmente, habrá disponible una opción de actualizar la membresía del plan, de gratuito a pro.

Área de Ajustes: En esta sección se agrupan las opciones de configuración de la aplicación, permitiendo al usuario personalizar su experiencia. Como función esencial de configuración será disponible el cambio del tema visual entre claro/oscuro. Como funcionalidades previstas, se podrán incluir en posteriores ciclos de desarrollo cambios de tema (en cuanto a colores) y selección de idioma, entre otras.

Área de Ayuda: En esta sección se encontrarán directrices y preguntas frecuentes que ayudarán al usuario a entender todo lo necesario para utilizar el sistema Horizon con confianza. Adicionalmente, habrá un enlace de contacto con soporte para resolver dudas que no queden claras con el contenido de esta sección.

3.1.2. Interfaces de Hardware

Interfaz para Ordenador (Principal): El sistema está diseñado principalmente para su uso en PC o portátiles, aprovechando monitores de mayor resolución para el análisis profundo ("Deep Work") y la lectura de gráficos detallados que se muestran. Se asume el uso de teclado y ratón para una navegación precisa sobre las series temporales.

Interfaz para Móviles (Secundaria): Existirá una versión adaptada para *smartphones* y *tablets* (diseño *responsive*), orientada mayoritariamente a consultas rápidas de estado, revisión del *dashboard* o lectura de notificaciones, dado que el usuario objetivo también demanda acceso en momentos de menor disponibilidad de tiempo.

3.1.3. Interfaces de Software

El sistema es una aplicación web basada en una arquitectura Cliente-Servidor, y debe ser compatible con los navegadores web modernos más utilizados como Google Chrome, Mozilla Firefox, Safari y Microsoft Edge. En el *backend*, el sistema se comunicará mediante peticiones HTTP con servicios financieros de terceros (plataforma financiera de Yahoo Finance) para la obtención de datos de mercado, respetando estrictamente los límites de uso (*Rate Limits*).

Para la gestión del *backend*, se utilizará Supabase como plataforma principal, proporcionando servicios como base de datos relacional (PostgreSQL), autenticación de usuarios, almacenamiento y funciones *serverless*. Supabase actuará como el núcleo del sistema, gestionando la persistencia de datos y la lógica de acceso, además de facilitar la integración con el *frontend* mediante APIs REST automáticas. No obstante, el uso de su plan gratuito implica ciertas limitaciones en almacenamiento, número de peticiones, conexiones concurrentes y transferencia de datos, lo que condiciona el diseño del sistema y obliga a optimizar el uso de recursos para garantizar un rendimiento adecuado.

3.1.4. Interfaces de Comunicación

Toda la comunicación entre el cliente (navegador) y el servidor se realizará a través del protocolo HTTP para garantizar la privacidad de las credenciales y las carteras simuladas de los usuarios.

3.2. Requisitos Funcionales

Los requisitos funcionales y la descripción de los casos de uso se describen en los apartados [3.2 Requisitos Funcionales](#) y [3.3 Diagrama de Casos de Uso](#), respectivamente.

3.3. Requisitos No Funcionales

3.3.1. Requisitos de rendimiento

Latencia de IA: El proceso de inferencia del modelo de Machine Learning y la generación de la explicabilidad (XAI) no debe bloquear o interferir con el hilo principal de la interfaz y, bajo ninguna casuística, debe ralentizar el tiempo máximo respuesta del sistema.

Optimización de Consultas: El sistema debe gestionar eficientemente las llamadas a las APIs de proveedores de datos para evitar alcanzar los Rate Limits gratuitos de forma prematura, implementando sistemas de caché para los gráficos OHLCV más consultados.

3.3.2. Seguridad

Gestión de APIs: Las claves de las APIs externas (ej. Yahoo Finance, Gemini) estarán encriptadas y solo serán accesibles y modificables por el equipo desarrollador detallado al inicio de este documento.

Privacidad del Usuario: Aunque el sistema opera en un entorno de simulación sin dinero real, las contraseñas y datos personales de la generación Z / Millennials (18-30 años) deben ser almacenadas de forma segura, sin permitir ninguna fuga de datos.

3.3.3. Fiabilidad

Transparencia (XAI): Para generar confianza, las explicaciones de la IA deben basarse en datos matemáticos comprobables (ej. niveles de RSI, formaciones de velas) y nunca presentar fallos de cálculo u ofrecer análisis irreales.

3.3.4. Disponibilidad

El sistema estará diseñado para operar 24/7. No obstante, se asume una dependencia directa de la disponibilidad operativa de las APIs de proveedores de

mercado. Si el proveedor cae, el sistema debe mostrar un mensaje de error amigable ("Modo degradado") en lugar de colapsar.

3.3.5. Mantenibilidad

Desacoplamiento: El motor de predicción de IA (*Backend*) estará programado y modularizado de forma independiente al *frontend*, facilitando el reentrenamiento continuo de los modelos predictivos por parte del equipo desarrollador sin requerir la caída de la aplicación web.

3.3.6. Portabilidad

Al ser una arquitectura web *desktop-first*, el sistema no requiere instalación de software de escritorio. Se ejecutará fluidamente en entornos Windows, macOS y Linux a través del navegador web.

4. Informe WAVE

4.1. Introducción

Para seleccionar los diferentes casos de uso que tendrá nuestra aplicación hemos realizado un informe WAVE, donde analizamos cada uno de los casos de uso por separado. Las letras del informe hacen referencia a cada una de las cuestiones a analizar:

- W: Hace referencia a "qué" va a hacer dicho caso de uso.
- A: Hace referencia a los "actores" que van a interactuar en el caso de uso.
- V: indica el "valor" que genera este caso de uso para los usuarios.
- E: indica el "entorno" y cuál es la salida producida por el caso de uso.

4.2. Informe WAVE

CUN01: Iniciar Sesión

- W: Autenticar al usuario validando sus credenciales para otorgarle acceso a su cuenta en la aplicación web.
- A: Usuario Registrado.
- V: Garantiza la seguridad de los datos del usuario y le permite acceder de forma personalizada a sus herramientas de análisis y carteras.
- E: Entorno web (*Frontend*) con validación en *Backend*. La salida producida es un *token* de sesión válido y la redirección al panel principal (*Dashboard*).

CUN02: Cerrar Sesión

- W: Finalizar la sesión activa del usuario de forma segura.
- A: Usuario Registrado.
- V: Protege la privacidad y la información financiera simulada del usuario, especialmente en dispositivos compartidos, evitando accesos no autorizados.
- E: Entorno web. La salida es la invalidación y destrucción del *token* de sesión actual, con su correspondiente redirección a la pantalla de inicio o registro.

CUN03: Registrarse (Crear Cuenta)

- W: Capturar los datos personales de un visitante para crear y dar de alta una nueva cuenta en el sistema.
- A: Usuario No Registrado.
- V: Permite a nuevos usuarios integrarse en el ecosistema de Horizon para comenzar a utilizar sus herramientas financieras y funcionalidades basadas en IA.
- E: Entorno web conectado a la base de datos (Supabase). La salida producida es la creación del registro del usuario, un *token* de inicio de sesión y la redirección al *Dashboard*.

CUN04: Buscar y visualizar activo financiero

- W: Procesar la búsqueda de un *Ticker* y renderizar su gráfico histórico de evolución.
- A: Usuario Registrado (y comunicación con API externa Yahoo Finance).
- V: Proporciona al usuario información de mercado fundamental y técnica, en formato visual e interactivo, facilitando su análisis previo a la toma de decisiones.
- E: Entorno web interactuando con API de terceros. La salida producida es la interfaz de visualización renderizada con el gráfico interactivo, precio, tendencia y fiabilidad del activo.

CUN05: Visualizar predicción de IA y Explicabilidad (XAI)

- W: Extraer y mostrar la predicción de tendencia futura de un activo y la justificación textual del modelo (Explicabilidad).
- A: Usuario Registrado.
- V: Otorga al usuario información predictiva de alto valor y fomenta la confianza y el aprendizaje al explicar, de forma transparente, qué factores han motivado la predicción de la IA.
- E: Entorno web interconectado con el motor de Machine Learning. La salida es un gráfico con el historial de señales IA (7, 30 y 90 días) y un informe de justificación en lenguaje natural.

CUN06: Añadir Activo a Portfolio

- W: Registrar un activo financiero específico dentro de una lista de seguimiento creada por el usuario.
- A: Usuario Registrado.
- V: Permite al usuario organizar, agrupar y hacer un seguimiento rápido de los activos que le interesan sin necesidad de buscarlos repetidamente.
- E: Entorno web con persistencia en base de datos. La salida es la actualización exitosa de la relación del *portfolio* en la base de datos y su reflejo visual en la interfaz.

CUN07: Eliminar Activo de Portfolio

- W: Suprimir la relación de seguimiento entre un activo financiero y un *portfolio* del usuario.
- A: Usuario Registrado.
- V: Facilita la gestión dinámica de las inversiones simuladas, permitiendo al usuario descartar activos que ya no encajan en su estrategia de seguimiento.
- E: Entorno web y base de datos. La salida es la actualización de la lista de seguimiento, eliminando dicho registro del sistema.

CUN08: Crear Portfoli

- W: Instanciar una nueva lista de seguimiento personalizada con nombre, descripción y nivel de riesgo.
- A: Usuario Registrado.
- V: Dota al usuario de una herramienta de segmentación para separar y evaluar distintas estrategias de inversión simulada de manera independiente.
- E: Entorno web y base de datos. La salida producida es un nuevo registro de *portfolio* asignado al usuario y su disponibilidad inmediata en la vista de carteras.

CUN09: Eliminar Portfolio

- W: Borrar permanentemente una lista de seguimiento completa y todas las referencias a los activos que contenga.
- A: Usuario Registrado.
- V: Mantiene la interfaz de trabajo limpia y ordenada, dando libertad al usuario para rehacer sus estrategias desde cero.
- E: Entorno web y base de datos. La salida es la eliminación en cascada de los datos del *portfolio* seleccionado y la actualización visual de la lista del usuario.

CUN10: Consultar Portfolio

- W: Cargar y mostrar los detalles, componentes y métricas de rendimiento de una cartera guardada.
- A: Usuario Registrado.

- V: Permite al usuario monitorizar de un solo vistazo el estado global de su estrategia de inversión, su rentabilidad y estabilidad.
- E: Entorno web. La salida producida es la renderización de la vista detalle del *portfolio*, estructurando los activos que lo componen en la interfaz.

CUN11: Editar Perfil

- W: Modificar la información personal asociada a la cuenta (como el nombre o apellidos).
- A: Usuario Registrado.
- V: Ofrece al usuario el control total sobre su identidad y datos dentro de la plataforma, asegurando que la información sea correcta y actual.
- E: Entorno web y base de datos. La salida es la actualización y persistencia de los nuevos atributos del usuario en el sistema.

CUN12: Cambiar contraseña

- W: Actualizar la credencial de seguridad empleada para el inicio de sesión.
- A: Usuario Registrado.
- V: Garantiza la seguridad y autonomía de la cuenta, permitiendo al usuario reaccionar ante posibles brechas o simplemente renovar su seguridad proactivamente.
- E: Entorno web y servicio de autenticación. La salida es el cambio efectivo de la contraseña cifrada dentro del sistema y la confirmación en la interfaz.

CUN13: Mensaje al chatbot

- W: Enviar una consulta escrita en lenguaje natural y procesar una respuesta automatizada de inteligencia artificial.
- A: Usuario Registrado (y comunicación con API Gemini).
- V: Acelera el aprendizaje financiero resolviendo dudas puntuales e interactuando como un tutor virtual personalizado (RAG) para la formación del usuario.
- E: Entorno web conectado al modelo de lenguaje (Gemini API). La salida es una respuesta conversacional renderizada dentro del bloque de chat de la interfaz.

CUN14: Acceder a curso didáctico

- W: Cargar el material de aprendizaje visual (diapositivas) asociado a una lección específica del módulo de Academia.
- A: Usuario Registrado.
- V: Fomenta la adquisición de competencias y nociones básicas financieras, elevando el criterio técnico del usuario frente a los mercados.
- E: Entorno web. La salida es la apertura del visualizador interactivo mostrando las diapositivas correspondientes al curso seleccionado.

CUN15: Leer reflexión del día

- W: Mostrar un artículo corto o contenido reflexivo destacado en la pantalla inicial de la aplicación.
- A: Usuario Registrado.
- V: Potencia el engagement diario del usuario y ofrece píldoras rápidas de conocimiento y perspectiva de mercado sin necesidad de búsquedas profundas.
- E: Entorno web (*Dashboard*). La salida producida es la apertura de la vista completa del artículo para permitir la lectura de este.

CUN16: Alternar entre tema oscuro y claro

- W: Modificar las variables de estilo global de la interfaz de usuario para invertir la paleta de colores.
- A: Usuario Registrado.
- V: Mejora la ergonomía visual y accesibilidad del sistema, permitiendo adaptar la aplicación a las preferencias personales o a la fatiga visual del entorno del usuario.
- E: Entorno web (*Frontend*). La salida es el cambio dinámico e inmediato de la hoja de estilos y aspecto visual de toda la aplicación.

CUN17: Leer noticias relacionadas con un activo

- W: Extraer y mostrar recortes de prensa o noticias económicas vinculadas directamente al *Ticker* analizado.
- A: Usuario Registrado.

- V: Proporciona contexto de "Análisis Fundamental", permitiendo al usuario correlacionar eventos sociopolíticos o empresariales con el movimiento del gráfico.
- E: Entorno web. La salida es la apertura de los enlaces a las noticias en una nueva pestaña o ventana del navegador para su lectura completa.

CUN18: Eliminar Cuenta

- W: Iniciar el proceso de borrado permanente e irreversible del perfil y los registros asociados al usuario.
- A: Usuario Registrado.
- V: Garantiza el respeto a los derechos del usuario y el cumplimiento de las normativas de protección de datos (Derecho al olvido).
- E: Entorno web y base de datos relacional. La salida producida es la eliminación de las credenciales, *portfolios* e historial del usuario del sistema Horizon.

CUN19: Actualizar suscripción

- W: Gestionar el nivel de membresía del usuario, alterando su plan entre versiones Gratuitas y Pro.
- A: Usuario Registrado.
- V: Permite al usuario tener autonomía económica sobre la plataforma, desbloqueando funciones avanzadas si lo requiere o volviendo al plan básico si desea reducir el gasto.
- E: La salida producida es la modificación de los permisos (roles) de la cuenta en base de datos y la activación/desactivación de los límites en la interfaz.

Apéndice B - Guion de entrevistas

Para la llevar a cabo la evaluación de los usuarios se han realizado diversas entrevistas con pruebas reales sobre la aplicación. Estas han sido guiadas por el guion que se describe a continuación:

1. Introducción y Bienvenida

"Hola, muchas gracias por dedicar tu tiempo a participar en esta sesión. Mi nombre es [Nombre del Entrevistador] y formo parte del equipo de desarrollo de Horizon.

Hoy te hemos invitado para probar nuestra plataforma financiera. Quiero dejar muy claro que estamos evaluando la aplicación, no a ti. No hay respuestas correctas ni incorrectas, cualquier problema que encuentres es culpa del diseño y nos ayudará muchísimo a mejorarlo.

Durante la prueba, te pediré que realices una serie de tareas. Te agradecería mucho que pensaras en voz alta todo lo que haces, cuéntame qué estás buscando, qué esperas que haga un botón antes de pulsarlo, o si algo te sorprende o te frustra.

Se tomarán notas sobre la sesión únicamente con fines académicos para extraer métricas. Tus datos serán totalmente anónimos. ¿Tienes alguna pregunta antes de empezar?"

2. Cuestionario de Perfilado

P1: ¿Cuál es tu edad? *(Rango objetivo 21-28 años)*.

P2: ¿Cómo describirías tu nivel de experiencia en inversiones?

- Baja (Principiante): No invierto o tengo conocimientos muy básicos.
- Media (Intermedio): Conozco conceptos básicos, tengo algún *portfolio* o sigo el mercado.
- Alta (Avanzado): Opero con frecuencia y analizo gráficas o datos técnicos.

3. Ejecución de Tareas

- No guiar al usuario a menos que se quede bloqueado (registrar si se pide ayuda).
- Cronometrar el tiempo desde que se lee la tarea hasta que el usuario la da por concluida.
- Registrar número de errores (clics equivocados, retrocesos en la navegación).

- Tarea 1: Iniciar Sesión

Escenario: "Ya tienes una cuenta creada en Horizon. Por favor, accede a la plataforma con las credenciales de Mario."

- Tarea 2: Registro Personalizado

Escenario: "Imagina que eres un usuario nuevo. Entra a la sección de registro, completa el formulario y, muy importante, configura tu perfil con la foto que quieras."

- Tarea 3: *Dashboard*

Escenario: "Acabas de entrar a tu cuenta. Tómate un momento para explorar esta pantalla principal. Dime qué ves, qué crees que puedes hacer aquí y qué te parece la distribución usando la barra de iconos. (inferior en móvil, lateral en escritorio)"

- Tarea 4: Análisis de Activos

Escenario: "Quieres buscar información sobre una empresa concreta, por ejemplo, Apple. Búscala en el sistema y revisa la gráfica de predicción. Cuéntame en voz alta qué entiendes de los datos que te muestra."

- Tarea 5: Gestión de *Portfolio*

Escenario: "Vamos a gestionar tus inversiones. Crea un nuevo *portfolio*, añádele un par de activos de tu elección. Una vez hecho, decide que ya no lo quieres y elimínalo por completo."

- Tarea 6: Reflexión del Día

Escenario: "Horizon tiene un módulo de reflexiones y citas financieras. Ve al *Dashboard*, revisa la tarjeta de resumen y lee la cita de hoy. Dime qué te parece esta herramienta."

- Tarea 7: Ajustes del Sistema

Escenario: "Sientes que la pantalla tiene mucho brillo y prefieres la interfaz oscura. Ve a los ajustes de tu cuenta y activa el modo oscuro."

- Tarea 8: Ayuda y Soporte

Escenario: "Te has encontrado con un problema técnico y necesitas hablar con el equipo de Horizon. Localiza la sección de ayuda e intenta iniciar el proceso de contacto con nosotros."

- Tarea 9: Edición de Perfil y Borrado

Escenario: "Quieres actualizar tu información. Ve a tu perfil, intenta cambiar tu foto, tu nombre, y tu contraseña. Una vez guardado, inicia el proceso para eliminar tu cuenta definitivamente, no llegues a eliminarla."

- Tarea 10: AcademIA (Educación + IA)

Escenario: "Quieres aprender más sobre un concepto financiero. Entra en un curso de AcademIA, avanza por algunas diapositivas e interactúa con el asistente IA haciéndole alguna pregunta sobre lo que quieras de la aplicación."

4. Evaluación Post-Tarea (Realizar tras completar CADA tarea individual)

El moderador lanza estas 4 preguntas rápidas inmediatamente después de cada una de las 10 tareas anteriores.

- Facilidad (1-10): "¿Cómo de fácil te ha resultado completar esta tarea a nivel de navegación y clics? (1 = Muy difícil, 10 = Muy fácil)".

- Claridad (1-10): "¿Cómo de clara era la información que estabas viendo? ¿Entendías bien los textos y gráficos? (1 = Muy confuso, 10 = Todo clarísimo)".
- Utilidad (1-10): "Pensando en tu día a día, ¿cuánto valor le ves a esta funcionalidad? (1 = Inútil, 10 = Imprescindible)".

5. Cierre y Evaluación Global (NPS)

"Basándote en tu experiencia de hoy probando todas estas herramientas, en una escala del 0 al 10, ¿con qué probabilidad recomendarías la aplicación Horizon a un amigo?"

"Con esto concluimos la prueba. Muchas gracias por tu ayuda."